整书阅读版

《C++ 语言》

从 C 到 C++ · 核心与常用新特性 · 真编译真运行

单元 20 个 · 生成于构建期

全书目录

工具链与第一个 C++ 程序：g++、std::print、编译链接
引用、const 与命名空间：C++ 的地基
类与 RAII：构造、析构、拷贝构造
拷贝与移动：值类别、std::move 与 Rule of Five
智能指针与所有权：unique_ptr、shared_ptr、weak_ptr
继承与虚函数：动态多态、对象切片
多态的设计代价：虚析构、接口类、组合优于继承
模板入门：函数模板、类模板、实例化
concepts 与约束：从 SFINAE 到 requires
可变参数模板与完美转发：forward、折叠表达式
序列容器：vector、string、deque、list 的取舍
关联容器与哈希：map、set、unordered_*
迭代器、算法与 ranges：lambda、views、zip / enumerate
auto、decltype 与结构化绑定
lambda 与函数对象
异常、noexcept 与异常安全
optional、variant 与 expected
constexpr 与 consteval
文件、格式化与生成器
多文件工程、ODR 与收尾

第 1 章

起步：从 C 到 C++

本章 2 个单元 · 页眉从这里之后的页面开始出现

1. 工具链与第一个 C++ 程序：g++、std::print、编译链接 全书第 1 单元
2. 引用、const 与命名空间：C++ 的地基 零基础 · 第 2 单元 · 真编译真运行

工具链与第一个 C++ 程序：g++、std::print、编译链接

C 分册的第一个程序用 `gcc` 编译，C++ 分册的第一个程序换成 `g++`。这一步不只是「换一个命令」：C++ 多了一层标准库、多了名字修饰与链接约定，也多了 `std::print` 这类在 C 里没有的输出方式。本单元先把「源文件怎样变成一个能跑的程序」重新走一遍，再用三个最小的输出例子把 `std::cout`、`std::format`、`std::print` 的差别钉住。

◎ 学习目标 · 看完这一单元，你应该能

- 用一条 `g++` 命令把单个源文件编成可执行文件，并说出 `-std=c++23` 在做什么；
- 说出预处理、编译、汇编、链接四个阶段各产出什么，以及 C 与 C++ 在这一步的相同与不同；
- 分别用 `std::cout`、`std::format`、`std::print` 输出同一组值，并解释三者的差别；
- 解释本机 `<print>` 为什么需要 `libs` 里的 `-lstdc++exp`、运行时为什么还要固定 PATH；
- 建起一个最小的多文件 C++ 工程（`include/` + 两个 `src/`），说清声明与定义的分工。

1.1 先看工程长什么样

第 1 章仍然允许单文件工程：六个例子都在 `examples/ch01/` 下，前五个是单文件，最后一个是多文件小工程。第 2 章起才强制每个清单至少两个 `.cpp` + 一个被 `#include` 的 `.h`；这一章先把「一个源文件也能跑」这件事看清楚。

目录树 **工程：examples/ch01/ 的六个最小例子**

第 1 步 先建三个输出方式的对照

```
examples/
└── ch01/
    ├── hello_cout/ ← 本课新增
    ├── hello_format/ ← 本课新增
    └── hello_print/ ← 本课新增
```

第 2 步 加上失败对照与类型对照

```
examples/
└── ch01/
    ├── 前三个例子 ： 同上
    ├── print_no_lib/ ← 本课新增
    └── hello_types/ ← 本课新增
```

第 3 步 最后加上第一个多文件工程

```
examples/
└── ch01/
    ├── 前五个例子 ： 同上
    └── hello_multi/ ← 本课新增
```

第 1 帧的三个例子都只做一件事：打印一行固定文本，但分别走 `std::cout`、`std::format`、`std::print` 三条路。第 2 帧的 `print_no_lib/` 故意不加 `-lstdc++exp`，让链接期失败成为可复现的证据；`hello_types/` 用同一组不同类型的值验证格式化不会把类型弄错。第 3 帧的 `hello_multi/` 第一次把 `include/` 与 `src/` 分开：`hello_multi.h` 只放声明，`hello_multi.cpp` 放定义，`main.cpp` 只调用。

这六个例子各自回答一个问题：`hello_cout` 证明「C++ 的流式输出能用」；`hello_format` 证明「格式化可以先得到一个字符串」；`hello_print` 证明「直接打印需要额外的链接与运行环境」；`print_no_lib` 把「少了那一步」变成可复现的链接错误；`hello_types` 证明「同一组值不会因为格式串写错而读错内存」；`hello_multi` 证明「声明与定义可以分家」。它们都不是「语法演示」，而是六个可以被闸门逐字对账的实验。

第 1 章允许单文件工程，所以前五个例子只有一个 `src/main.cpp`；第 2 章起硬规定③生效，每个清单至少两个 `.cpp` + 一个被 `#include` 的 `.h`。这不是说单文件「不专业」，而是说：从第二个项目开始，你要习惯「接口放头文件、实现放源文件、多个翻译单元一起链接」的工作方式。`hello_multi` 是这条路的起点。

1.2 从源文件到可执行文件：g++ 做了什么

C 分册已经讲过四阶段：预处理、编译、汇编、链接。C++ 走同一条流水线，只是驱动程序换成 `g++`，并且默认把 C++ 标准库接进来。最小的命令是：

编译命令是 `g++ -std=c++23 -Wall -Wextra -Wpedantic -O0 -g hello.cpp -o hello.exe`。这条命令既是编译也是链接：`g++` 会在背后调用 `cc1plus`、`as`、`collect2` / `ld`。

- `-std=c++23` 选语言标准。C 分册用 `-std=c17`，C++ 分册用 `-std=c++23`；两者不是同一套标准。
- `-Wall -Wextra -Wpedantic` 是本书的基础警告档；`-O0 -g` 让调试信息完整、优化关掉，便于对照。
- `-o hello.exe` 指定目标文件名；不写时 Windows 默认是 `a.exe`。

先看最短的 `std::cout` 版本：

cpp **hello_cout: C++ 的流式输出** examples/ch01/hello_cout/src/main.cpp 第 1–6 行

```cpp
#include <iostream>

int main() {
    std::cout << "hello, cout\n";
    return 0;
}
```

终端输出 终端输出 hello-cout-out

```
hello, cout
```

`#include <iostream>` 把 `std::cout` 的声明带进来；`<<` 是流插入运算符，把右边的字符串送进标准输出。`\n` 是换行字符，`std::cout` 不保证立刻刷新，但程序正常退出时会刷新。这个例子和 C 的 `printf("hello\n")` 输出一样，区别在于 `<<` 的重载由类型决定，不需要你写格式串。

四个阶段可以画成一条链：

图 / 搜索过程 **一个 .cpp 怎样变成 .exe**

第 1 步 预处理

hello.cpp ：你写的源文件 → **hello.ii ：展开 #include 与宏** ← 本步新增

hello.cpp → hello.ii g++ -E

第 2 步 编译

hello.s ：这台机器的汇编 ← 本步新增

hello.ii → hello.s cc1plus

第 3 步 汇编

hello.o ：机器码 + 符号表 ← 本步新增

hello.s → hello.o as

第 4 步 链接

hello.exe ：接上 C++ 标准库与启动代码 ← 本步新增

hello.o → hello.exe collect2 / ld

第 1 帧的 `.ii` 是预处理结果；第 2 帧的 `.s` 是汇编；第 3 帧的 `.o` 是目标文件，还缺 `std::cout` 的实现；第 4 帧把这些实现接上，才有可执行文件。C 与 C++ 的差别主要在第 4 帧：C++ 的标准库更大、名字经过修饰（mangling），链接器要按修饰后的名字找实现。**「编译过了」不等于「链接过了」**，后面的 `std::print` 例子就会在第 4 帧失败。

把四个阶段拆开看，每一段都有独立的失败方式：

- **预处理**只做文本替换：展开 `#include`、展开宏、处理条件编译。它不认识 C++ 语法，也不做类型检查；`#include <iostream>` 在这一步只是把几千行声明搬进你的源文件。预处理失败的典型原因是找不到头文件（`fatal error: iostream: No such file or directory`）或宏写错。

- **编译**把预处理结果翻成汇编或直接生成目标码。C++ 的类型检查、模板实例化、重载决议都在这一步；`error:` 与 `warning:` 大多来自这里。`-std=c++23` 会影响这一步接受哪些语法。
- **汇编**把汇编翻成机器码，产出 `.o`。这一步通常最安静；失败往往是内联汇编写错或目标架构不匹配。
- **链接**把多个 `.o` 与库接起来，解析符号。C++ 的函数名会被修饰成包含参数类型的名字（ABI 约定，不是 C++ 标准规定的具体拼法），所以 C 与 C++ 混编时要用 `extern "C"` 关掉修饰；标准库的实现也要在这一步接上。

这四段对应四种不同的错误：找不到头文件、语法/类型错误、汇编错误、`undefined reference`。**先判断错误发生在哪一段，再决定去看什么**；把链接错误当成语法错误去改源文件，是最常见的无效排查。

C 分册用 `gcc`，C++ 分册用 `g++`，这不是「换个名字」：`gcc` 也能识别 `.cpp` 并调用 C++ 前端，但它**不会自动链接 C++ 标准库**；`g++` 会把 `.cpp` 当 C++ 编译，并在链接时自动加上 `-lstdc++` 一类的参数。反过来，`g++` 会把 `.c` 也当 C++ 编译，所以 C 代码要用 `gcc` 编、C++ 代码要用 `g++` 编，别拿 `gcc` 去链接 C++ 程序。C 与 C++ 是两种语言：C 的 `void*` 可以隐式转成任意指针，C++ 不行；C 的 designated initializer 规则与 C++ 不同；这些差异会在你以后读「看起来像 C 的 C++」时冒出来。

C++ 与 C 混编时还要处理名字修饰：C++ 编译器会把 `greeting` 修饰成包含参数类型的内部名字，C 编译器不会。如果 C++ 要调用一个 C 库函数，C 头文件通常用 `#ifdef __cplusplus extern "C" { ... } #endif` 把声明包起来，关掉修饰。**修饰规则是 ABI 约定，不是 C++ 标准规定的具体拼法**；同一份源码在不同编译器/平台上修饰后的名字可能不同，所以不要手写修饰名，交给编译器。

1.3 三种输出方式：`cout`、`format`、`print`

- **`std::cout`：流式，类型安全，但拼接冗长**

`std::cout << a << b << c` 的每个 `<<` 都是一次重载调用，类型由编译器选；它不会像 `printf` 那样因为 `%d` 和 `double` 不匹配而读错内存。代价是格式化能力弱：对齐、进制、精度要配合 `std::setw` / `std::hex` / `std::setprecision` 等操纵符，代码容易变长。

`std::cout` 还有几个容易忽略的细节：`<<` 返回流本身，所以可以链式写；`std::endl` 在换行的同时强制刷新缓冲区，`\n` 只换行不刷新；程序正常退出时标准输出会刷新，但崩溃或 `std::_Exit` 不会。**日志/崩溃前需要留下证据时，显式刷新（`std::flush`）比迷信 `std::endl` 更可靠**；平时输出用 `\n` 就够了，不必每行都刷新。`std::cout` 与 C 的 `printf` 默认共享同一套缓冲区，混用时要注意刷新顺序，但这属于工程细节，本单元不展开。

- **`std::format`：先拼成字符串，再决定往哪写**

`std::format("format: {} {}\n", 42, "ok")` 返回一个 `std::string`，你可以把它交给 `std::cout`、写进文件、放进日志。`{}` 是占位符，类型由实参推导；它不需要你手写 `%d` / `%s`，也不会因为格式串与参数不匹配而读错内存。

cpp **hello_format: std::format 返回字符串** examples/ch01/hello_format/src/main.cpp 第 1–7 行

```cpp
#include <format>
#include <iostream>

int main() {
    std::cout << std::format("format: {} {}\n", 42, "ok");
    return 0;
}
```

终端输出 终端输出 hello-format-out

```
format: 42 ok
```

`std::format` 的 `{}` 不只是「占位」：你可以在花括号里写格式说明，例如 `{:>6}` 右对齐、`{:.2f}` 保留两位小数、`{:#x}` 十六进制带前缀。它返回一个 `std::string`，这意味着你可以先拼好再决定输出目标——写文件、进日志、放进异常消息都可以。代价是这次格式化会真的构造一个字符串对象；如果只是往终端打一行，`std::print` 可以少一次字符串构造。

`std::format` 的格式串在编译期会被检查（`std::format_string` 的 consteval 构造），参数数量/类型不匹配通常是编译错误；如果你把格式串作为运行期字符串传进去，就会走 `std::vformat` 那条路，检查退到运行期并可能抛 `std::format_error`。**「编译期检查」不是无条件保证，取决于你用哪个入口。**

- **`std::print`：直接写到标准输出，链接要额外库**

`std::println("hello, print: {}", 42)` 直接输出并换行，不用先建字符串、也不用 `std::cout <<`。它是 C++23 的 `<print>` 设施；在本机 MSYS2 UCRT64 的 g++ 上，**只写 `#include <print>` 能编译，但链接会失败**，必须加 `-lstdc++exp`。

cpp **hello_print: std::println 直接输出** examples/ch01/hello_print/src/main.cpp 第 1–6 行

```cpp
#include <print>

int main() {
    std::println("hello, print: {}", 42);
    return 0;
}
```

终端输出 终端输出 hello-print-out

```
hello, print: 42
```

清单里因此多了 `libs` 与 `env` 两个字段：`libs` 写 `["-lstdc++exp"]`，`env` 把 PATH 钉到 `C:/msys64/ucrt64/bin;C:/msys64/usr/bin;C:/Windows/System32`。`libs` 让闸门把库放在**源文件之后**再 `-o`，静态库顺序才正确；`env` 把 PATH 钉到干净的 MSYS2 路径，运行时才能加载到配套的 `libstdc++` DLL。两件事缺一不可：只加库、不钉 PATH，本机会以 3221225785 退出（加载到别的 DLL）；只钉 PATH、不加库，链接期就过不去。

把「不加库」的失败做成可复现的反例：

cpp **print_no_lib: 同一个程序，清单里故意不写 libs** examples/ch01/print_no_lib/src/main.cpp 第 1–6 行

```cpp
#include <print>

int main() {
    std::println("this will not link without -lstdc++exp");
    return 0;
}
```

编译诊断 本机真实 stderr：<print> 少了 -lstdc++exp print-no-lib-diag

```
C:/msys64/ucrt64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe:
C:\Users\Administrator\Documents\Study\dev-notes\dist\work\cpp01-print-no-lib\cpp01-print-no-lib-main.o: in
function `std::vprint_unicode(_iobuf*, std::basic_string_view<char, std::char_traits<char> >,
std::basic_format_args<std::basic_format_context<std::__format::_Sink_iter<char>, char> >)':
C:/msys64/ucrt64/include/c++/16.1.0/bits/print.h:310:
(.text$_ZSt14vprint_unicodeP6_iobufSt17basic_string_viewIcSt11char_traitsIcEESt17basic_format_argsISt20basic_
format_contextINSt8__format10_Sink_iterIcEEcEE[_ZSt14vprint_unicodeP6_iobufSt17basic_string_viewIcSt11char_tr
aitsIcEESt17basic_format_argsISt20basic_format_contextINSt8__format10_Sink_iterIcEEcEE]+0x192): undefined
reference to `std::__open_terminal(_iobuf*)'
C:/msys64/ucrt64/bin/../lib/gcc/x86_64-w64-mingw32/16.1.0/../../../../x86_64-w64-mingw32/bin/ld.exe:
C:/msys64/ucrt64/include/c++/16.1.0/bits/print.h:315:
(.text$_ZSt14vprint_unicodeP6_iobufSt17basic_string_viewIcSt11char_traitsIcEESt17basic_format_argsISt20basic_
format_contextINSt8__format10_Sink_iterIcEEcEE[_ZSt14vprint_unicodeP6_iobufSt17basic_string_viewIcSt11char_tr
aitsIcEESt17basic_format_argsISt20basic_format_contextINSt8__format10_Sink_iterIcEEcEE]+0x1f9): undefined
reference to `std::__write_to_terminal(void*, std::span<char, 18446744073709551615ull>)'
collect2.exe: error: ld returned 1 exit status
```

这段诊断有三个关键信息：`ld.exe` 报的是 `undefined reference`，缺的符号是 `std::__open_terminal` 与 `std::__write_to_terminal`，最后 `collect2` 把退出码转述成一行。**它不是拼写错误，也不是标准库没装**——头文件在，声明在，缺的是 `<print>` 背后的终端支持库。`libs: ["-lstdc++exp"]` 补的就是这一层。

`std::print` / `std::println` 是 C++23 才进入标准库的设施；它和 `std::format` 共享格式化语法，但输出目标是标准输出（或指定的 `FILE*`）。`println` 在末尾自动换行，`print` 不换行；两者都不保证每次调用都刷新缓冲区，需要立刻可见时仍要 `std::flush`。它在本机需要 `-lstdc++exp`，是因为 GCC 把终端相关的实现放在了实验库里；这是**本机工具链的现状**，不是「所有 C++23 实现都必须这样链接」。换一个编译器/标准库，链接参数可能不同——所以清单里的 `libs` 是实验条件，不是语言规则。

编译器视角 · 编译器视角：为什么要看得懂链接错误

编译错误会告诉你「哪一行、哪个语法不对」；链接错误只会告诉你「哪个符号没找到」。`undefined reference to 'std::__open_terminal(_iobuf*)'` 里的名字虽然长，但它指向一个具体的实现，不是你的代码拼错了。看到 `ld.exe` / `collect2` 字样，先分清楚这是**链接期**，再去查「这个符号应该由哪个库提供」。C 分册的 `nm` 与 `ld` 那一节讲的就是这套排查方法；C++ 只是把符号名换成了修饰后的形式。

实验记录 · 本机实测：三种输出与 `<print>` 的两个条件

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

例子	读数
`hello_cout`	`hello, cout`
`hello_format`	`format: 42 ok`
`hello_print`（`libs` + `env`）	`hello, print: 42`
`print_no_lib`（无 `libs`）	链接失败：`undefined reference to std::__open_terminal` / `std::__write_to_terminal`
`hello_print`（有 `libs`、默认 PATH）	退出 3221225785（DLL 入口点），不是程序逻辑错误

前四行是本机真实输出/诊断；最后一行是「库找对了、运行环境没钉住」的反例。**`libs` 与 `env` 是实验条件的一部分，不是可选项。**

- **三种输出怎么选**

方式	产出	格式能力	额外条件	适合
`std::cout`	直接写标准输出，返回流	基础，复杂格式靠操纵符	无	简单拼接、已有流对象
`std::format`	返回 `std::string`	强，`{}` + 格式说明	无	先拼字符串、写文件/日志
`std::print`	直接写标准输出	强，与 `format` 同语法	本机要 `-lstdc++exp` + PATH	终端输出、少一次字符串构造

这张表不是「谁更好」的排名，而是「你的输出目标是什么」的选择：要字符串就 `format`，要终端就 `print`，已经在用流就 `cout`。三种方式都依赖同一套 `{}` 格式化语法（`cout` 除外），学会一种，另外两种的格式说明可以直接迁移。

格式说明也不必背：`{:>6}` 右对齐到 6 列、`{:<6}` 左对齐、`{:^6}` 居中、`{:.2f}` 两位小数、`{:#x}` 十六进制带 `0x`、`{:08.2f}` 补零到 8 列。C 的 `printf` 用 `%6d` / `%-6d` / `%.2f` 表达同一组意图，但 C++ 的写法把「哪个参数」交给位置，格式说明只描述「这个参数怎么显示」。**参数顺序由 `{}` 的顺序决定，不再靠格式串里 `%d` 的个数去对齐实参**；这也是它更不容易错位的原因。

1.4 第一个多文件工程：声明与定义分家

单文件跑通之后，把 `greeting()` 拆到另一个 `.cpp`：

`cpp` **头文件只放声明** `examples/ch01/hello_multi/include/hello_multi.h` 第 1–8 行

```cpp
#ifndef HELLO_MULTI_H
#define HELLO_MULTI_H

#include <string>

std::string greeting();

#endif
```

cpp **定义放在 .cpp** examples/ch01/hello_multi/src/hello_multi.cpp 第 1–5 行

```cpp
#include "hello_multi.h"

std::string greeting() {
    return "hello from another .cpp";
}
```

cpp **main 只调用，不定义** examples/ch01/hello_multi/src/main.cpp 第 1–8 行

```cpp
#include "hello_multi.h"

#include <iostream>

int main() {
    std::cout << greeting() << "\n";
    return 0;
}
```

终端输出 终端输出 hello-multi-out

```
hello from another .cpp
```

编译命令要把两个 `.cpp` 一起交给 g++：`g++ -std=c++23 -Wall -Wextra -Wpedantic -O0 -g -Iinclude src/main.cpp src/hello_multi.cpp -o app.exe`。`-Iinclude` 告诉编译器去哪里找 `"hello_multi.h"`。两个 `.cpp` 各自编译成一个 `.o`，链接器把 `main.o` 里对 `greeting()` 的调用接到 `hello_multi.o` 的定义上。**声明让编译器知道「这个名字长什么样」，定义让链接器找到「它的实现在哪」；两者分开，才有「改一个文件、只重编一个文件」的可能。**

头文件里的 `#ifndef HELLO_MULTI_H / #define / #endif` 是 include guard：同一个翻译单元里第二次 `#include` 同一份头文件时，宏已经定义，内容会被跳过去。没有 guard，一个 `.cpp` 里两次间接包含同一个头文件，就可能把同一个类/函数声明定义两遍。`#pragma once` 是常见替代，语义更简单，但它是编译器扩展，不是标准；本书统一用 guard，原因是 C 与 C++ 共用同一套头文件纪律，也方便你以后读别人的代码。

头文件里能放什么、不能放什么，也有边界：声明（函数、类、模板、extern 变量）可以放；`inline` 函数、`constexpr` 变量、类内定义的成员函数可以放；普通函数定义、普通全局变量定义不能放（会违反 ODR，链接期重复定义）。**头文件会被多个翻译单元各包含一次，任何「只能有一份」的东西都不能直接写进去。** 这条规则在第 9 单元会展开；本单元先记住 `greeting()` 的声明在 `.h`、定义在 `.cpp`，这是最常用也最不容易错的分工。

分开编译还可以用 `-c` 单独产出目标文件：`g++ -c src/main.cpp -o main.o` 与 `g++ -c src/hello_multi.cpp -o hello_multi.o`，最后 `g++ main.o hello_multi.o -o app.exe`。这正是构建脚本在做的事：**编译是一份文件一份文件地做，链接是把所有 `.o` 与库接起来。**改 `hello_multi.cpp` 只需重编它自己，`main.cpp` 的目标文件可以复用。代价是你要维护「哪些文件参与链接」这份清单，漏一个就 `undefined reference`，多一个就可能重复定义。

`build.sh` 的最小形态可以只有三行：删旧产物、逐个编译、链接。真正工程化的构建脚本还会加依赖检查、增量编译、错误即停（`set -e`）、清理目标等；本机没有 make/cmake，所以本书用脚本 + 清单固定命令，不引入额外构建系统。**脚本的价值不是「少打几个字」，而是让「同一份源码怎样变成同一个可执行文件」可复现**；这对后面讲 ODR、多模块、测试都是地基。

谁能看见谁 **👁 两个 .cpp 都 include 同一个头文件：谁看得见什么**

第 1 步 头文件里的声明

hello_multi.h ： greeting 的声明

main.cpp → hello_multi.h #include
hello_multi.cpp → hello_multi.h #include

第 2 步 定义只在一处

hello_multi.cpp ： greeting 的定义 ← 本步新增　main.cpp ： 只调用，不定义

第 1 帧：两个 `.cpp` 都看见了声明，所以编译期都知道 `greeting` 的名字和签名。第 2 帧：定义只出现在 `hello_multi.cpp` 里，链接器把 `main.cpp` 的调用接过去。如果头文件里写函数**定义**（而不是声明），两个翻译单元

就会各生成一份，链接期报重复定义——这是 C++ 分册第 9 单元（ODR）要展开的事。

1.5 类型安全：格式化不会把 `double` 读成 `int`

C 的 `printf("%d", 3.5)` 是未定义行为：格式串说「读一个 int」，实参却是一个 `double`，调用约定对不上。C++ 的 `std::format` / `std::print` 用 `{}` 占位符，类型由实参推导，不会出现「格式串说一套、实参是另一套」的错位。

cpp **hello_types：同一组值，三种类型** examples/ch01/hello_types/src/main.cpp 第 1–6 行

```cpp
#include <print>

int main() {
    std::println("int={} double={} text={}", 7, 3.5, "ok");
    return 0;
}
```

终端输出 终端输出 hello-types-out

```
int=7 double=3.5 text=ok
```

本机输出 `int=7 double=3.5 text=ok`。`7` 是 `int`，`3.5` 是 `double`，`"ok"` 是字符串字面量；占位符按顺序绑定，编译器在编译期检查参数数量与格式串。**这不是「C++ 比 C 聪明」，而是两套接口的契约不同**：`printf` 把类型信息放在运行期的格式串里，`std::format` / `std::print` 把类型信息放在编译期的重载/模板里。

C 的 `printf("%d", 3.5)` 是未定义行为：格式串要求一个 `int`，实参却是 `double`，可变参数调用不会帮你转换，函数会按 `int` 的读法去取那 8 个字节的一部分，结果不可预测。编译器可能会给 `-Wformat` 警告，但它不是语言层面的类型检查；关掉警告或换一个调用点，程序照样能编出来。`std::format` / `std::print` 的 `{}` 不需要你声明类型，类型由实参推导；格式串写错（参数数量不对、占位符语法不对）通常在编译期就报错。**这不是「C++ 更安全」的空话，是「类型信息放在哪一层」的直接结果。**

但 `std::format` 也不是万能的：格式串里仍然可以写 `{:d}` 要求整数，传一个 `std::string` 就会编译失败；运行期格式串（`std::vformat`）会把检查推迟到运行期，可能抛 `std::format_error`。所以「用了 `std::format` 就绝不会格式错」是过度承诺；准确的说法是：**在格式串是编译期常量、参数类型匹配的前提下，错误会在编译期暴露。**

编译警告也值得单独说：`-Wall` 不是「所有警告」，`-Wextra` 补一批，`-Wpedantic` 再要求严格符合标准；三者都打开，仍然有警告不在其中。本单元不靠「记住每个开关」来写代码，而是靠闸门把基础档的真实输出钉住：同一份源码，警告要么出现、要么不出现，正文不能凭印象写。**看到警告先读它指向的行与条件，不要用 `-Wno-*` 把它关掉**；本书的清单也不允许关警告。

{} 语法规则 · 本单元的编译命令

- 单个源文件：`g++ -std=c++23 -Wall -Wextra -Wpedantic -O0 -g hello.cpp -o hello.exe`
- 多个源文件：`g++ ... -Iinclude src/main.cpp src/hello_multi.cpp -o app.exe`
- 需要 `<print>`：在清单 `libs` 里写 `["-lstdc++exp"]`，并用 `env` 固定 MSYS2 PATH
- 只看预处理：`g++ -std=c++23 -E hello.cpp -o hello.ii`

⚠ 陷阱 · `<print>` 不是「写了 include 就能跑」

新手最容易把「编译通过」当成「程序没问题」。`#include <print>` 只保证声明可见；`std::println` 的实现体在 `libstdc++exp` 里，链接期才需要它。本机的两个条件——`-lstdc++exp` 与干净 PATH——缺一个都跑不起来，而且报错位置完全不同：缺库是链接期 `undefined reference`，缺 PATH 是运行期 3221225785。**看到 `undefined reference` 先查库；看到程序一启动就退出，先查运行环境。**

工程惯例 · 工程惯例：把编译命令固定下来

单文件时手敲 g++ 没问题；多文件时命令会越来越长，参数顺序、`-I` 路径、库名都容易漏。工程上的做法是把命令写进 `build.sh` 或清单，让「怎么编」成为可复现的事实，而不是某次终端里的记忆。C 分册第 7 章已经用过 `build.sh`；C++ 分册第 9 单元会把多文件、ODR 与构建脚本一起收口。

1.6 编译错误与链接错误怎么读

编译错误的形状是 `文件:行:列: error: 说明`，下面通常跟着一行源码和 `^` 指到具体位置。读它的顺序是：先看**第一个** error（后面的错误经常是它的连锁反应），再看文件/行号是不是你改的那一处，最后看 `error:` 后面的动词（`expected`、`no matching function`、`cannot convert`）——它比整句英文更直接。修完一处就重编，不要一次改十处；否则新的错误会混进旧的噪音里。

链接错误的形状是 `undefined reference to '符号'` 或 `multiple definition of '符号'`。它没有行号，因为问题不在某一行，而在「所有目标文件与库的集合」。先看符号属于谁：`greeting` 是你自己的函数，就查「定义它的 `.cpp` 有没有参与链接」；`std::__open_terminal` 是库函数，就查「清单有没有写对应的 `libs`」。`collect2.exe: error: ld returned N exit status` 只是转述，不是根因；真正的根因在它上面的 `undefined reference` / `multiple definition` 行。

C 分册讲过：本机默认 PATH 下，`ld` 的诊断可能只剩 `collect2` 一行。C++ 的 `<print>` 例子用干净 PATH 把完整诊断钉住了；你以后遇到「只有一行 collect2」时，换干净 MSYS2 PATH 或直接看 `nm`，不要猜。

遇到看不懂的编译/链接错误，做一个**最小复现**：把出问题的代码删到只剩一个文件、一个函数、一个最小调用，确认错误还在；再逐项加回选项（`-std`、`-Wall`、`-I`、`libs`、`env`），看哪一项让错误出现/消失。最小复现不是「浪费时间删代码」，而是把「环境、选项、源码」三个变量分开；本单元 `print_no_lib` 就是这样一个最小复现：同一份源码，只差清单里的 `libs`。

1.7 本机工具链边界：标准、实现与实验条件

本书的 C++ 工具链是 MSYS2 UCRT64 的 g++ 16.1.0，标准 `C++23`。本机探测结果里有三条边界，写作者和读者都要知道：

- `<print>` 需要 `libs: ["-lstdc++exp"]`，运行时还要用 `env` 固定干净的 MSYS2 PATH；这是**本机实现**的链接/运行条件，不是 C++ 标准的要求；
- `<modules>` 在本机不可用（`#include <modules>` 直接报找不到头文件），所以本分册不教 modules，也不写「理论上可以」；
- `<stacktrace>` 能链接（同样需要 `-lstdc++exp`）但本机只给 4 帧、首帧是 `register_frame_ctor`，符号质量不足以教学，只在工具链事实里记录，不放进正文例子。

标准要求 / 本机实测 / 工程惯例三分在这里第一次出现：`std::print` 是 C++23 标准设施（标准要求）；本机要加库、钉 PATH（本机实测）；把编译命令写进清单/脚本、头文件用 include guard（工程惯例）。三者不要混写成一句「C++ 就是这样」。

1.8 把第一个程序放进工程

单文件程序可以手敲命令，但工程里要固定下来。至少做三件事：

1. **命令进脚本**：把 `g++ -std=c++23 -Wall -Wextra -Wpedantic -O0 -g ...` 写进 `build.sh` 或清单，不要依赖终端历史；
2. **头文件纪律**：声明放 `.h`，定义放 `.cpp`；头文件用 include guard；不要在头文件里写 `using namespace std;`，也不要用 `#include <bits/stdc++.h>` 这种非标准汇总头；
3. **入口简单**：`main` 返回 `int`，正常结束写 `return 0;`（也可以省略，标准会隐式返回 0）；需要命令行参数时再用 `int main(int argc, char** argv)`。

`#include <bits/stdc++.h>` 在竞赛环境常见，但它不是标准头，换一个标准库就可能没有；本书不用它。`using namespace std;` 在 `.cpp` 里偶尔用可以，在头文件里会污染所有包含它的翻译单元，所以头文件里禁止。**这些不是语法错误，而是「让别人能接手」的工程惯例**；第 9 单元会把它们和 ODR 一起收口。

`build.sh` 的骨架可以写成：`set -e`（任何一步失败就停）、`rm -f app.exe`、`g++ -std=c++23 ... src/main.cpp src/hello_multi.cpp -o app.exe`。命令里的每个参数都有理由：`-std` 固定标准，`-Wall -Wextra -Wpedantic` 固定警告档，`-Iinclude` 固定头文件搜索路径，`-O0 -g` 固定调试档。**不要用「上次能编过」当理由省略参数**；换一台机器、换一个终端，默认值就可能不同。

1.9 下一单元从哪继续

这一单元只解决了「程序能编、能链、能输出」。下一单元 `1.2 引用、const 与命名空间` 会把注意力从工具链移到语言本身：C++ 的引用为什么必须初始化、`const T&` 为什么能接临时对象、命名空间怎样避免名字冲突。它们看起来是三个独立话题，其实都在回答同一个问题：**一个名字在 C++ 里到底绑定到哪块存储、哪个作用域、哪份定义。**

小结 · 这一节要带走的五件事

- C++ 与 C 共用预处理/编译/汇编/链接四阶段，但驱动换成 `g++`，链接期要接 C++ 标准库。
- `std::cout` 是流式输出；`std::format` 先返回字符串；`std::print` 直接写标准输出。
- 本机 `<print>` 需要 `libs: ["-lstdc++exp"]`，运行时还要用 `env` 固定干净的 MSYS2 PATH。
- 多文件工程把声明放头文件、定义放 `.cpp`，`-Iinclude` 与「两个 `.cpp` 一起编」缺一不可。
- `std::format` / `std::print` 的 `{}` 按实参推导类型，不会出现 `printf` 的格式串错位。

自测题 · 自测题（先自己做，再对答案）

1. `g++ -std=c++23 hello.cpp -o hello.exe` 这条命令里，编译与链接分别由谁完成？
2. `std::cout`、`std::format`、`std::print` 三者的返回值/输出目标有什么不同？
3. 本机 `<print>` 只写 `#include <print>` 时，编译能过吗？链接能过吗？为什么？
4. `print_no_lib` 的 `undefined reference` 与「拼写错误」有什么区别？看到它应该先查什么？
5. `hello_multi` 为什么要同时编译 `src/main.cpp` 与 `src/hello_multi.cpp`？只编 `main.cpp` 会怎样？

注 · 自测答案

1. `g++` 是驱动程序：它先调用编译器前端/汇编器生成目标文件，再调用链接器把目标文件与 C++ 标准库接起来。你只敲一条命令，但两个阶段都发生了。
2. `std::cout << ...` 直接写标准输出，返回流对象；`std::format(...)` 返回一个 `std::string`，不直接输出；`std::print` / `std::println` 直接写标准输出，返回 `void`（或流状态，随标准库实现）。
3. 编译能过（头文件提供了声明），链接不能过：`std::println` 的实现需要 `-lstdc++exp`，否则报 `undefined reference to std::__open_terminal` / `std::__write_to_terminal`。再加上运行时的干净 PATH，才完整。
4. 拼写错误是编译期「这个名字不存在」；`undefined reference` 是链接期「名字存在，但找不到实现」。先查「这个符号应该由哪个库提供」，而不是改源文件的拼写。
5. 因为 `greeting()` 的定义在 `hello_multi.cpp` 里；只编 `main.cpp` 会得到「声明可见、定义缺失」的链接错误。两个 `.cpp` 一起交给 g++，链接器才能把调用接上。

引用、const 与命名空间：C++ 的地基

C 分册里，函数想改调用者的变量，只能把地址传进去；C++ 多了一条更直接的路：引用。引用看上去像"自动解引用的指针"，但两者的规则并不一样——引用必须绑定、绑定后不能换、语言里没有"空引用"这个值。围绕引用，`const` 又引出一串新问题：`const int*` 和 `int* const` 锁住的到底是谁？`const T&` 为什么能接一个临时对象？临时对象能活多久？最后，这些名字都住在哪个作用域里，由命名空间回答。

这一单元不碰类，也不碰模板；它只把 C++ 最底层的四件事钉住：**引用怎么绑定、`const` 锁住谁、临时对象活多久、名字在哪个作用域里找。**这四件事在后面每一个单元都会出现，所以例子都尽量小，每个结论都用真实输出或真实诊断顶着。

学习目标 · 看完这一单元，你应该能

- 说出引用与指针的三条硬差别，并用 `sizeof` 与 `&r == &a` 的读数证明引用不是"另一个指针变量"；
- 解释 `const T&` 的只读契约，并说清临时对象在"变量初始化"和"函数传参/返回"两种场景下活得一样不一样；
- 分清 `const int*`、`int* const`、`const int* const` 各自锁住什么，能判断一行赋值该不该编译；
- 解释返回局部变量引用的程序为什么在 C++23 下直接编译失败，并给出安全的替代写法；
- 用命名空间与 `using` 声明组织同名函数，并说清头文件里为什么不该写 `using namespace`。

2.1 先看工程长什么样

这一单元仍然在 `examples/ch01/` 里加例子。第 1 章允许单文件工程，所以六个例子都只有一个 `src/main.cpp`；到第 2 章，硬规定③才会要求每个清单至少两个 `.cpp` 加一个被 `#include` 的 `.h`。这里先把"一个源文件也能把规则讲清"这件事做扎实。

目录树 **工程：examples/ch01/ 这一单元新增的六个例子**

第 1 步 先建引用传参的对照

```
examples/
└── ch01/
    ├── 上一单元 ：hello_cout 等六个例子
    └── ref_basics/ ← 本课新增
```

第 2 步 加上 const 引用、const 指针与悬空返回

```
examples/
└── ch01/
    ├── ref_basics/
    ├── 上一单元 ：同上
    ├── const_ref/ ← 本课新增
    ├── const_ptr/ ← 本课新增
    └── ref_return/ ← 本课新增
```

第 3 步 最后加上命名空间与 using 作用域

```
examples/
└── ch01/
    ├── 前四个例子 ：同上
    ├── namespace_demo/ ← 本课新增
    └── using_scope/ ← 本课新增
```

第 1 帧只有 `ref_basics/`，它把"引用传参"和"指针传参"放在同一个程序里对照：两者都能改到实参，但调用点的写法与可空性不同。第 2 帧加上 `const_ref/`（`const T&` 接临时对象）、`const_ptr/`（两种 `const` 位置）与 `ref_return/`（返回局部变量的引用，故意编不过）。第 3 帧加上 `namespace_demo/`（两个命名空间的同名函数）与 `using_scope/`（`using` 声明的作用域到哪里结束）。

这六个例子不是"语法清单"，而是六次对照实验：`ref_basics` 用地址比较回答"引用是不是独立变量"；`const_ref` 用构造/析构顺序回答"临时对象活多久"；`const_ptr` 用两行合法赋值回答"const 锁住谁"；`ref_return` 用真实诊断回答"为什么不能返回局部引用"；`namespace_demo` 与 `using_scope` 用两个同名函数和一个块作用域回答"名字在哪里找"。

2.2 引用不是指针：必须绑定，不能换绑

先看引用传参与指针传参的并排对照：

cpp **两个函数：一个收引用，一个收指针** examples/ch01/ref_basics/src/main.cpp 第 3–9 行

```cpp
void add_one_ref(int& value) {
    ++value;
}

void add_one_ptr(int* value) {
    ++(*value);
}
```

cpp **main：两种传参都改到实参，再比较地址与大小** examples/ch01/ref_basics/src/main.cpp 第 11–25 行

```cpp
int main() {
    int a = 10;
    int b = 20;
    add_one_ref(a);
    add_one_ptr(&b);

    int& r = a;
    r = 99;
```

```
19
20      std::cout << "a=" << a << " b=" << b << "\n";
21      std::cout << "same=" << (&r == &a ? "yes" : "no") << "\n";
22      std::cout << "sizeof_ref=" << sizeof(int&)
23                << " sizeof_ptr=" << sizeof(int*) << "\n";
24      return 0;
25  }
```

终端输出 终端输出 ref-basics-out

```
a=99 b=21
same=yes
sizeof_ref=4 sizeof_ptr=8
```

三行输出各回答一个问题。`a=99 b=21` 说明引用传参和指针传参都改到了调用者的变量；`same=yes` 说明 `&r` 取到的是 `a` 的地址，`r` 没有自己的存储；`sizeof_ref=4 sizeof_ptr=8` 说明 `sizeof(int&)` 量的是"被引用的 `int`"，不是"一个指针变量"。

引用在语言层面不是对象，是已有对象的另一个名字。 因此：

- 声明引用时必须初始化：`int& r;` 直接编译失败；指针可以先声明后赋值；
- 引用绑定后不能换绑：`r = b;` 不是"让 `r` 改指 `b`"，而是"把 `b` 的值写进 `r` 绑定的那个对象"，也就是写进 `a`；
- 没有"空引用"这个值：引用必须绑定到一个对象或函数；指针可以是 `nullptr`。

这三条差别决定了接口设计。收引用 `int&` 的函数默认假设"你一定会给我一个对象"，所以函数里不需要判空；收指针 `int*` 的函数可以接受 `nullptr`，所以函数里必须先判空。反过来说，调用点看到 `add_one_ref(a)` 时，看不出 `a` 会不会被改；看到 `add_one_ptr(&b)` 时，`&` 明确提示"地址被交出去了"。**引用更简洁，指针更显眼；选哪个，取决于你想让调用者看到什么契约。**

图 / 搜索过程 **变量、引用、指针：三种绑定关系**

第 1 步 **变量 a 有自己的存储**

a : int, 值 10

第 2 步 **引用 r 是 a 的另一个名字**

a : int, 值 99　r : int&, 绑定 a

r → a 同一块存储

第 3 步 **指针 p 自己也是一个变量**

a : int, 值 99　p : int*, 存 &a

p → a 指向，p 可以改指

第 2 帧的 `r` 不是"指向 a 的箭头"，而是"a 的别名"；`&r == &a` 是它最直接的证据。第 3 帧的 `p` 才是一个独立变量：它自己占 8 个字节，里面装 `a` 的地址，所以 `p` 可以被重新赋值去指别人。把这两帧摆在一起看，就不会再说"引用就是自动解引用的指针"——**语法上像，语义上不是。**

内存形态 **引用与实参共享同一块存储**

第 1 步 **main 里的 a**

a = 10 : 栈上一块 4 字节

第 2 步 **add_one_ref 的形参 value**

a = 11 : 同一块 4 字节　value : int&, 没有独立存储

a ⇔ value &a == &value

第 2 帧里 `value` 没有新增一个盒子，它只是 `a` 的另一个名字；所以 `++value` 改的就是 `a`。这也是引用传参比指针传参少一次"解引用"动作的原因——编译器在背后仍然传地址，但语言把它包装成"直接使用对象"。

标准原文 · **标准原文：引用必须初始化，且不能换绑**

> A variable whose declared type is "reference to T" shall be initialized. ... A reference cannot be changed to refer to another object after initialization.

大意：声明为"T 的引用"的变量必须初始化；初始化之后，引用不能再改成指向另一个对象。给引用赋值，改的是被引用的那个对象。

出处：N4950 §9.4.4「References」（[dcl.init.ref]）。

陷阱 · **把 `r = b` 当成"让 r 改指 b"**

你会看到：`int& r = a; r = b;` 之后，`a` 变成了 `b` 的值，`r` 仍然和 `a` 绑定。新手会以为 `r` 现在指向 `b`，于是后面通过 `r` 修改时，意外改到了 `a`。真相是：引用初始化后不能换绑；这行赋值等价于 `a = b;`。要让"指向谁"可变，用指针；要让"这个名字永远指这个对象"，用引用。

2.3 `const T&`：只读契约与临时对象的寿命

`const T&` 是 C++ 里最常见的参数类型。它同时表达两件事：函数不会通过这个引用修改对象；它可以绑定临时对象。

cpp **Tracker 与 longer：一个会打印构造/析构，一个返回 const 引用** examples/ch01/const_ref/src/main.cpp 第 5–19 行

```cpp
struct Tracker {
    std::string name;

    explicit Tracker(std::string n) : name(std::move(n)) {
        std::cout << "ctor " << name << "\n";
    }

    ~Tracker() {
        std::cout << "dtor " << name << "\n";
    }
};

const std::string& longer(const std::string& a, const std::string& b) {
    return a.size() >= b.size() ? a : b;
}
```

cpp **main：一个绑定临时对象，一个把临时对象传进函数** examples/ch01/const_ref/src/main.cpp 第 21–26 行

```cpp
int main() {
    const Tracker& bound = Tracker("bound");
    std::cout << "name=" << bound.name << "\n";
    std::cout << "longer=" << longer("hi", "longer") << "\n";
    return 0;
}
```

终端输出 终端输出 const-ref-out

```
ctor bound
name=bound
longer=longer
dtor bound
```

`const Tracker& bound = Tracker("bound");` 里，`Tracker("bound")` 先构造一个临时对象；`const Tracker&` 绑定它之后，这个临时对象的寿命被延长到 `bound` 的作用域结束。所以 `name=bound` 能读到名字，`dtor bound` 直到 `main` 结束才出现。如果写成非 const 的 `Tracker& bound = Tracker("bound");`，编译就过不去——非 const 引用不能绑定临时对象。

`longer("hi", "longer")` 更微妙：两个字符串字面量先各自构造一个临时 `std::string`，绑定到两个 `const std::string&` 形参；`longer` 返回其中一个的引用。这个引用在函数返回后仍然指向那个临时对象，而临时对象要活到"包含这次调用的完整表达式"结束——也就是这一整条 `std::cout << ... << "\n";` 执行完。**在同一个表达式里立刻用掉是安全的；一旦把它存进变量、留到下一行，就是悬空引用。**

标准原文 · 标准原文：函数参数的临时对象活到完整表达式结束

> A temporary object bound to a reference parameter in a function call persists until the completion of the full-expression containing the call. ... The lifetime of a temporary bound to the returned value in a function return statement is not extended; the temporary is destroyed at the end of the full-expression in the return statement.

大意：绑到函数引用参数上的临时对象，活到"包含这次调用的完整表达式"结束；绑到返回引用上的临时对象，寿命不会延长，返回语句所在的完整表达式一结束就销毁。

出处：N4950 §6.7.7「Temporary objects」（[class.temporary]）第 6.9、6.11 条。

这段标准原文解释了两件事。第一，`longer("hi", "longer")` 的两个临时实参在整条输出语句结束前都活着，所以 `longer` 返回的引用在这一刻有效。第二，如果 `longer` 返回的是"函数内部创建的临时对象"的引用，那个临时对象在 `return` 语句结束时就销毁了，调用者拿到的是悬空引用。**"const 引用能接临时对象"和"临时对象能活很久"是两件事；前者是绑定规则，后者要看绑定发生在哪里。**

内存形态 **const 引用绑定临时对象的两种寿命**

第 1 步 变量初始化，寿命延长

`bound : const Tracker&` `temp : Tracker("bound"), 活到 bound 作用域结束`

bound → temp 延长

第 2 步 函数参数，活到完整表达式结束

`call : longer("hi", "longer")` `temp_a : std::string("hi")` `temp_b : std::string("longer")`

temp_a ↔ call 表达式结束即销毁
temp_b ↔ call 表达式结束即销毁

第 1 帧的延长规则只适用于"用临时对象初始化一个引用变量"；第 2 帧的形参绑定只保证到完整表达式结束，不保证到函数返回之后。把这两帧分清，就不会写出"函数返回 `const T&`，调用者长期保存"的悬空代码。

`const T&` 的"只读"也不是装饰：它让函数体里对参数的修改直接编译失败，调用者不用读函数实现就能知道"这个参数不会被改"。代价是它可能绑定一个临时对象（多一次构造），也可能绑定一个长期对象（零拷贝）；函数内部无法假设参数一定指向长期存储。**接口写 `const T&`，是在说"我只看，不拥有，也不保证你传进来的是长期对象"。**

2.4 顶层 `const` 与底层 `const`：谁被锁住

`const` 放在 `*` 的哪一边，锁住的东西完全不同：

cpp **const_ptr：一个指向 const，一个自身 const** examples/ch01/const_ptr/src/main.cpp 第 3–16 行

```cpp
int main() {
    int a = 1;
    int b = 2;

    const int* p = &a;
    int* const q = &a;

    p = &b;
    *q = 5;

    std::cout << "*p=" << *p << " *q=" << *q << "\n";
    std::cout << "a=" << a << " b=" << b << "\n";
    return 0;
}
```

终端输出 终端输出 const-ptr-out

```
*p=2 *q=5
a=5 b=2
```

- `const int* p`：`p` 是"指向 const int 的指针"。被锁住的是 `*p`——不能通过 `p` 写 `a`；但 `p` 自己可以改指 `b`。所以 `p = &b;` 合法，输出 `*p=2`。
- `int* const q`：`q` 是"const 指针，指向 int"。被锁住的是 `q` 自己——不能改指别人；但可以通过 `q` 写 `a`。所以 `*q = 5;` 合法，输出 `*q=5`、`a=5`。

用"谁不能变"来判断最稳：`const` 紧挨着 `int`，锁的是被指对象；`const` 紧挨着变量名（在 `*` 右边），锁的是指针变量本身。两者都锁就是 `const int* const p`：既不能改指，也不能通过它写对象。读声明时从变量名往外读：`q` 是 const 的指针，指向 int；`p` 是指针，指向 const int。

顶层 `const` / 底层 `const` 是术语，记不住也没关系，记住"锁谁"就行。 顶层 const 修饰对象本身（如 `int* const q` 里的 `q`），底层 const 修饰指针/引用所指的类型（如 `const int* p` 里的 `int`）。函数重载、模板推导、`auto` 推导时，顶层 const 常常被丢掉，底层 const 会保留——这个差别在第 14 单元讲 `auto` 时会再出现。

大白话 · 大白话：引用与 const 的"门牌"

把对象想成一间房，引用就是"这间房的另一个门牌"，钉上去之后不能换到别的房；指针是一张写着房间号的纸条，纸条可以换成别的号，也可以空着。`const int*` 是"纸条上的房间你不能改里面的东西"，`int* const` 是"纸条本身被胶水粘死了，不能换号"。两种限制互不替代；写代码前先问：我要锁的是房间，还是纸条？

2.5 返回引用：悬空不是"可能"，而是"什么时候爆"

看一个故意写错的例子：

cpp `ref_return`：返回局部变量的引用 examples/ch01/ref_return/src/main.cpp 第 3–12 行

```cpp
int& bad_reference() {
    int local = 7;
    return local;
}

int main() {
    int& r = bad_reference();
    std::cout << r << "\n";
    return 0;
}
```

这份源码在本机 C++23 下**直接编译失败**：

编译诊断 本机真实 stderr：C++23 把返回局部引用判成错误 ref-return-diag

```
src/main.cpp: In function 'int& bad_reference()':
src/main.cpp:5:12: error: cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'
    5 |     return local;
      |            ^~~~~
```

诊断说的不是"局部变量不能返回"，而是"`int&` 不能绑定到一个右值"。C++23 的隐式移动规则要求：`return` 的表达式如果是一个局部变量，先按右值处理；返回类型是非 const 左值引用 `int&` 时，右值绑不上左值引用，于是编译失败。**这条规则把一类悬空引用从"运行期才炸"提前到了编译期**，是 C++23 相对早期标准的一个实际改进（规则见 N4950 §11.9.6，[class.copy.elision]）。

即使把返回类型改成 `const int&` 让编译通过，问题也没有消失：函数返回后 `local` 已经销毁，引用指向一块失效的栈内存；读它属于未定义行为。编译器可能给 `-Wreturn-local-addr` 警告，也可能什么都不说。**判据不是"编过了"，而是"引用指向的对象在调用者使用时还活着吗"。**

安全的返回方式只有三种：返回值（拷贝或移动，让调用者拥有自己的对象）；返回引用，但引用指向调用者传入的对象、全局对象或生命周期明确更长的对象；返回指针并明确约定所有权与空值。函数返回 `const std::string&` 时，如果它返回的是某个形参的引用，调用者只能在"包含这次调用的完整表达式"里用——这就是 2.3 节 `longer` 的边界。

编译器视角 · 编译器视角：读“不能绑定”这类诊断

`cannot bind non-const lvalue reference of type 'int&' to an rvalue of type 'int'` 这句话可以拆成三段：`int&` 是函数声明的返回类型；`local` 在 `return` 里被当成右值；右值不能初始化非 const 左值引用。它没有说“local 生命周期结束”，因为编译器在类型检查阶段就拦住了，根本走不到生命周期分析。**看到 `cannot bind` 先看两边的类型与值类别，不要先去改生命周期。**

2.6 命名空间：同名函数怎样不打架

两个模块都想提供一个叫 `code` 的函数，一个把颜色编号翻成十六进制，一个把状态编号翻成 HTTP 码。直接放在全局作用域会重定义；放进两个命名空间就各自独立：

cpp **namespace_demo：两个命名空间，两个 code** examples/ch01/namespace_demo/src/main.cpp 第 3–21 行

```cpp
namespace color {
int code(int index) {
    return index == 1 ? 0xF00 : 0x0F0;
}
}

namespace status {
int code(int index) {
    return index == 1 ? 200 : 404;
}
}

int main() {
    using color::code;

    std::cout << "color=" << code(1) << "\n";
    std::cout << "status=" << status::code(1) << "\n";
    return 0;
}
```

终端输出 终端输出 namespace-demo-out

```
color=3840
status=200
```

`color::code` 与 `status::code` 是两个不同的全名；`using color::code;` 只是把 `color::code` 这一个名字引进 `main` 的作用域，所以后面写 `code(1)` 等价于 `color::code(1)`。`status::code(1)` 仍然要写全名，因为 `status::code` 没有被引进来的话，`code` 在 `main` 里只指 `color::code` 这一个候选。

`std::cout` 也是同样的道理：`std` 是一个命名空间，`cout` 是里面的一个名字，`<iostream>` 把声明放进去。你写 `std::cout` 是“用全名”，写 `using std::cout;` 是“把这一个名字引进当前作用域”，写 `using namespace std;` 是“把 std 里的所有名字都放进查找范围”。三者不是一回事。

`using` 声明有明确的作用域边界：

cpp **using_scope：块作用域结束后，引进来的名字也消失** examples/ch01/using_scope/src/main.cpp 第 3–19 行

```cpp
namespace cfg {
int retries = 3;
int timeout = 30;
}

int main() {
    std::cout << "qualified=" << cfg::retries << "\n";

    {
        using cfg::retries;
        std::cout << "using=" << retries << "\n";
    }

    int retries = 7;
    std::cout << "local=" << retries << "\n";
```

```
    return 0;
}
```

终端输出 终端输出 using-scope-out

```
qualified=3
using=3
local=7
```

块内的 `using cfg::retries;` 只在这个块里有效；出了块，`retries` 这个名字不再指向 `cfg::retries`，所以后面可以声明一个局部变量 `int retries = 7;`。如果 `using` 放在 `main` 的作用域（块外），再在同一个作用域声明同名局部变量就会冲突。**`using` 不是"把名字复制过来"，而是"在当前作用域里增加一条查找路径"；路径随作用域结束而失效。**

图 / 搜索过程 **同一个 code 名字的三条查找路径**

第 1 步 两个命名空间各自持有 code

color : 命名空间　status : 命名空间　color_code : color::code　status_code : status::code

color → color_code 声明
status → status_code 声明

第 2 步 using 只引进 color::code

main : 函数作用域　color_code : using color::code 引入　status_code : 仍需写全名

main → color_code code(1) 找到它
main → status_code status::code(1)

第 2 帧的 `using` 只加了一条从 `main` 到 `color::code` 的查找路径，没有把 `status::code` 也拉进来；所以 `code(1)` 没有歧义。若同时 `using color::code; using status::code;`，再写 `code(1)`，查找会同时命中两个声明，编译期报歧义——这不是"运行时选一个"，而是"编译期拒绝猜"。

工程惯例 · **工程惯例：头文件里不写 `using namespace`**

`using namespace std;` 放在 `.cpp` 的函数里，影响范围有限；放在头文件的全局作用域，会污染所有包含它的翻译单元，别人写 `count`、`distance`、`size` 时可能撞上 `std` 里的同名模板。**这是惯例，不是标准禁令**：标准没有禁止你写，但工程上通常禁止，因为头文件会被很多文件包含，影响面不可控。本书的规矩是：头文件里只用全名（`std::string`），源文件里可以在函数内用 `using std::cout;` 这类窄范围声明。

2.7 本机实测：四个读数

实验记录 · **本机实测：引用、const 与命名空间**

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `sizeof(int&)` 输出 `4`，`sizeof(int*)` 输出 `8`：引用不是独立的指针变量；
- `&r == &a` 输出 `same=yes`：引用与实参共享同一地址；
- `const Tracker& bound = Tracker("bound");` 的输出顺序是 `ctor bound` → `name=bound` → `dtor bound`：临时对象被延长到作用域结束；
- `ref_return` 编译退出码非 0，诊断是 `cannot bind non-const lvalue reference of type 'int&' to an rvalue`：C++23 在编译期拒绝返回局部变量的非 const 引用。

2.8 把规则变成工程习惯

引用、`const`、命名空间这三件事，落到工程上就是几条可执行的约定：

1. **只读参数用 `const T&`**：大对象不复制；函数体不能改；能接临时对象。小标量（`int`、`double`、指针）直接按值传，引用不会更快，反而多一层间接。
2. **要改实参用 `T&`，要可空用 `T*`**：`T&` 的契约是"一定有对象"，`T*` 的契约是"可能为空"。不要把 `T*` 一律换成 `T&`，也不要用 `T&` 接一个可能不存在的对象。

3. **不要返回局部变量或临时对象的引用**：返回值让调用者拥有对象；返回引用就要求对象活得比调用长；两者不能兼得时，选返回值。
4. **头文件里用全名，源文件里窄范围** `using`：头文件不写 `using namespace`；`using std::string;` 放在 `.cpp` 的函数里，影响面最小。
5. **命名空间按模块分，不按"工具"分**：`color::`、`status::` 这种按领域分比 `utils::` 更耐放；名字冲突时，先想"这个函数属于哪个概念"，再想"要不要改个名"。

`const` 与 `constexpr` 的区别这里只点到：`const int n = 5;` 表示"n 不可修改"，但 `n` 不一定能在编译期求值；`constexpr int n = 5;` 表示"n 是常量表达式"，可以用在 `static_assert`、数组长度等需要编译期常量的地方。第 18 单元会把 `constexpr` / `consteval` 单独讲透；现在只要不把两者当成同义词。

2.9 下一单元从哪继续

这一单元解决的是"名字绑定到哪、const 锁住谁、临时对象活多久"。下一单元 `2.1 类与 RAII` 会把这三件事放进一个类里：构造函数建立不变量、析构函数确定性地释放资源、拷贝构造决定"复制一个对象"到底复制什么。引用与 `const` 会以成员函数、`const` 成员函数、构造参数的形式再次出现；命名空间则用来给类与工厂函数分家。

小结 · 这一节要带走的五件事

- 引用必须初始化、不能换绑、没有空值；`&r == &a` 与 `sizeof(int&) == sizeof(int)` 证明它没有独立存储。
- `const T&` 是只读契约；用临时对象初始化引用变量会延长寿命，函数参数的临时对象只活到完整表达式结束。
- `const int*` 锁住被指对象，`int* const` 锁住指针本身；读声明时看 `const` 紧挨着谁。
- 返回局部变量的非 const 引用在 C++23 下直接编译失败；安全的返回方式是返回值，或返回生命周期更长的对象的引用。
- 命名空间给名字分组；`using` 声明只把指定名字引进当前作用域，头文件里不写 `using namespace`。

自测题 · 自测题（先自己做，再对答案）

1. `int& r = a; r = b;` 之后，`a` 和 `r` 分别变成什么？`r` 会改绑到 `b` 吗？
2. 为什么本机 `sizeof(int&)` 是 4 而不是 8？这个读数说明了什么？
3. `const Tracker& bound = Tracker("bound");` 中临时对象的析构发生在什么时候？如果换成函数参数 `f(Tracker("bound"))`，析构又发生在什么时候？
4. `const int* p` 与 `int* const q` 各自能不能改指、能不能通过它改写对象？用两行合法赋值证明。
5. 返回 `int&` 指向局部变量的函数，为什么在本机 C++23 下编不过？如果改成 `const int&` 让编译通过，运行时还有没有问题？

注 · 自测答案

1. `r` 仍是 `a` 的别名，这行等价于 `a = b;`；所以 `a` 变成 `b` 的值，`r` 读到的也是这个新值。引用初始化后不能换绑。
2. `sizeof` 作用于引用时，量的是被引用类型的大小；`int` 是 4 字节，所以结果是 4。指针变量本身是 8 字节（本机 64 位），但引用不是独立变量，没有自己的 8 字节。
3. 绑定到引用变量的临时对象活到该引用的作用域结束，所以 `bound` 离开作用域时才析构；作为函数实参的临时对象活到包含这次调用的完整表达式结束，通常就是这条语句末尾。
4. `const int* p = &a; p = &b;` 合法（改指），但 `*p = 5;` 非法；`int* const q = &a; *q = 5;` 合法（改写对象），但 `q = &b;` 非法。前者锁对象，后者锁指针。
5. C++23 的隐式移动规则把 `return local;` 里的局部变量按右值处理，`int&` 不能绑定右值，所以编译失败。改成 `const int&` 后编译可能通过，但返回的引用指向已销毁的局部变量，运行时读它是未定义行为。

第 2 章

对象生命周期

本章 3 个单元 · 页眉从这里之后的页面开始出现

1. 类与 RAII：构造、析构、拷贝构造 零基础 · 第 3 单元 · 真编译真运行
2. 拷贝与移动：值类别、std::move 与 Rule of Five 全书第 4 单元；C++ 试点
3. 智能指针与所有权：unique_ptr、shared_ptr、weak_ptr 零基础 · 第 5 单元 · 真编译真运行

类与 RAII：构造、析构、拷贝构造

C 的结构体只是一组字段；C++ 的类在字段之外还带一组函数，其中两个最关键：**构造函数**负责把对象带到"可用"的状态，**析构函数**负责在对象消失时把资源还回去。把"资源的生命周期"绑到"对象的生命周期"上，这套做法叫 RAII（Resource Acquisition Is Initialization）。它不是语法糖，而是 C++ 管理内存、文件、锁、网络连接的默认方式。

这一单元从最小的 RAII 包装开始，依次看清四件事：构造/析构什么时候执行、成员按什么顺序初始化、默认拷贝为什么会把资源复制成两份指针、以及什么时候必须手写拷贝构造与拷贝赋值。最后用 `explicit` 把"隐式转换"这道门关上。

◎ 学习目标 · 看完这一单元，你应该能

- 写出一个 RAII 类，让构造函数获取资源、析构函数释放资源，并说清"确定性释放"发生在哪一刻；
- 解释成员初始化顺序由声明顺序决定，而不是由初始化列表的书写顺序决定；
- 说出默认拷贝构造对"拥有裸指针"的类做了什么，并用退出码证明双重释放；
- 按 Rule of Three 写出拷贝构造、拷贝赋值与析构，并用 copy-and-swap 处理自赋值；
- 解释 `explicit` 阻止了哪种隐式转换，并读懂对应的编译诊断。

3.1 先看工程长什么样

第 2 章起，硬规定③生效：每个清单至少两个 `.cpp`，并且至少一个 `.h` 被源文件 `#include`。这一单元的六个例子都按这个形状组织：`include/<例名>.h` 放声明，`src/<例名>.cpp` 放实现，`src/main.cpp` 只做驱动与打印，`<例名>.json` 固定编译命令。

目录树 **工程：examples/ch02/ 这一单元新增的六个多文件例子**

第 1 步 先建一个 RAII 文件句柄

```
examples/
└── ch02/
    ├── 上一单元 ： cpp04 试点的八个例子
    └── raii_file/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── raii_file.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── raii_file.cpp ← 本课新增
        └── raii_file.json ← 本课新增
```

第 2 步 加上构造顺序、浅拷贝与深拷贝

```
examples/
└── ch02/
    ├── raii_file/ ： 同上
    ├── ctor_order/ ← 本课新增
    ├── copy_shallow/ ← 本课新增
    └── copy_deep/ ← 本课新增
```

第 3 步 最后加上返回值省略与 explicit

```
examples/
└── ch02/
    ├── 前四个例子 ： 同上
    ├── copy_elision_basic/ ← 本课新增
    └── explicit_ctor/ ← 本课新增
```

第 1 帧只有 `raii_file/`：`FileHandle` 的构造函数接收一个 `FILE*`，析构函数负责 `fclose`；`main.cpp` 只负责拿临时文件、写一行、读回来。第 2 帧加上 `ctor_order/`（成员初始化顺序）、`copy_shallow/`（默认拷贝导致双重释放）与 `copy_deep/`（手写深拷贝）。第 3 帧加上 `copy_elision_basic/`（返回局部对象时的拷贝/移动计数）与 `explicit_ctor/`（隐式转换被拒绝）。

注意每个目录都把接口放在 `include/`、实现放在 `src/`：`FileHandle` 的用户只看得到头文件里的声明，不需要知道析构函数里调用了 `fclose`；`Buffer` 的用户只看得到 `data()` / `set()`，不需要知道内部用 `new char[]`。**这不是为了“看起来工程化”，而是为了让“谁能看见什么”变成可检查的事实**——头文件里没有定义，链接期就必须由某个 `.cpp` 补上。

3.2 类、构造与析构：对象的一生

`class` 与 `struct` 在 C++ 里只有一个语法差别：默认访问权限。`struct` 的成员默认 `public`，`class` 默认 `private`。本书的惯例是：**只装数据的聚合用 `struct`，有不变量的类型用 `class`**；这只是惯例，不是标准要求。`FileHandle` 有“必须持有有效文件、析构时必须关闭”的不变量，所以用 `class`，把 `file_` 藏起来，只暴露 `get()`。

构造函数在对象创建时自动调用，析构函数在对象销毁时自动调用。所谓“自动”，指编译器在作用域结束时插入调用，不需要你手写 `close()`：

cpp **FileHandle：一个最小的 RAII 类** examples/ch02/raii_file/include/raii_file.h 第 6–18 行

```cpp
class FileHandle {
public:
    explicit FileHandle(std::FILE* file);
    ~FileHandle();

    FileHandle(const FileHandle&) = delete;
    FileHandle& operator=(const FileHandle&) = delete;

    std::FILE* get() const;

private:
    std::FILE* file_;
};
```

cpp **构造拿资源，析构还资源** examples/ch02/raii_file/src/raii_file.cpp 第 5–22 行

```cpp
FileHandle::FileHandle(std::FILE* file) : file_(file) {
    if (file_ == nullptr) {
        std::cout << "open failed\n";
        return;
    }
    std::cout << "open\n";
}

FileHandle::~FileHandle() {
    if (file_ != nullptr) {
        std::fclose(file_);
        std::cout << "close\n";
    }
}

std::FILE* FileHandle::get() const {
    return file_;
}
```

cpp **main：不写 close()，析构也会执行** examples/ch02/raii_file/src/main.cpp 第 6–21 行

```cpp
int main() {
    FileHandle file(std::tmpfile());
    if (file.get() == nullptr) {
        return 1;
    }

    std::fputs("hello\n", file.get());
    std::fflush(file.get());
    std::rewind(file.get());

    char buffer[16] = {};
    if (std::fgets(buffer, sizeof(buffer), file.get()) != nullptr) {
```

```
18          std::cout << "read=" << buffer;
19      }
20      return 0;
21  }
```

终端输出 终端输出 raii-file-out

```
open
read=hello
close
```

输出顺序是 `open` → `read=hello` → `close`。`close` 出现在 `main` 返回之后、`file` 离开作用域时，由析构函数打印。**这就是确定性释放：对象的生命周期一结束，资源立刻归还，不依赖垃圾回收，也不依赖你记得写 `fclose`。** 如果 `main` 中途 `return 1`，`file` 仍然会被析构，因为栈上的对象在函数返回时按构造的逆序销毁。

`~FileHandle()` 里判断 `file_ != nullptr`，是因为构造函数允许传入空指针；如果构造失败，对象仍然被构造出来（只是处于"没有资源"的状态），析构函数必须能处理它。更严格的写法是在构造函数里抛异常，让对象根本构造不出来；这属于第 16 单元"异常与异常安全"的内容，本单元先用"空指针 + 析构判空"的简化模型。

调用树 **FileHandle 的一生：构造一次，析构一次**

第 1 步 **main 里构造 file**

main() ：第 7 行，准备打开临时文件
main() → FileHandle :FileHandle(FILE*)： 构造，打印 open

第 2 步 **file 离开作用域，析构自动执行**

main() ：准备 return
main() → FileHandle :~FileHandle()： 析构，打印 close
FileHandle::~FileHandle() → fclose() 真正关闭文件

第 1 帧的构造调用发生在 `FileHandle file(std::tmpfile());` 这一行；第 2 帧的析构调用不需要你写，编译器在 `file` 离开作用域时插入。两帧之间没有任何"手动关闭"的代码，`fclose` 只出现在析构函数体内——这就是 RAII 的调用链。

`FileHandle(const FileHandle&) = delete;` 与 `FileHandle& operator=(const FileHandle&) = delete;` 把拷贝构造与拷贝赋值删掉了。原因很直接：如果允许拷贝，两个 `FileHandle` 会持有同一个 `FILE*`，两个析构函数各 `fclose` 一次，第二次就是重复关闭。**"拥有资源"的类型默认不应该可拷贝；要么禁止拷贝，要么写深拷贝。** 这一条会在 3.5 与 3.6 节用两个例子分别展示"禁止"和"深拷贝"。

大白话 · **大白话：RAII 就是"借东西的人负责还"**

去图书馆借书，借的时候登记（构造函数），还的时候注销（析构函数）。你不需要每天想着"我借过书"，只要书还在你手上，登记就有效；你一离开（对象销毁），系统自动注销。RAII 把"还书"这件事绑在"你离开"这个事件上，而不是绑在"你记得"上。C++ 的析构函数就是那个"离开时自动执行"的钩子。

3.3 成员初始化：顺序看声明，不看列表

构造函数的初始化列表写在参数列表后面，用冒号开头。很多人以为"列表里先写的先初始化"，实际规则是：**非静态数据成员按它们在类里的声明顺序初始化，跟列表顺序无关。**

cpp **Widget：声明顺序是 first_ 再 second_** examples/ch02/ctor_order/include/ctor_order.h 第 11–21 行

```
11  class Widget {
12  public:
13      Widget();
14
15      const std::string& first() const;
16      const std::string& second() const;
17
18  private:
19      Part first_;
20      Part second_;
```

```
};
```

cpp 构造列表反着写，执行顺序仍是声明顺序 examples/ch02/ctor_order/src/ctor_order.cpp 第 6–10 行

```cpp
Part::Part(std::string n) : name(std::move(n)) {
    std::cout << "init " << name << "\n";
}

Widget::Widget() : second_("second"), first_("first") {}
```

终端输出 终端输出 ctor-order-out

```
init first
init second
first=first second=second
```

输出先 `init first` 再 `init second`，尽管初始化列表里写的是 `second_` 在前。原因：编译器按声明顺序（`first_`、`second_`）逐个初始化成员，初始化列表只负责提供"用哪个值初始化"，不负责排序。**如果你在列表里写 `b_(a_)`，而 `b_` 声明在 `a_` 前面，那么 `b_` 会先初始化，此时 `a_` 还没初始化，读它就是未定义行为。** 编译器通常给 `-Wreorder` 警告，但警告不阻止编译，运行时可能"看起来正常"，换一个优化档就变。

`-Wreorder` 的诊断（本机编译 `ctor_order` 时真实出现）会指出"`second_` 将先于 `first_` 初始化"，并给出声明与初始化列表的位置。看到它不要关警告，改列表顺序或改声明顺序，让两者一致。**初始化列表的最佳实践是：按声明顺序写，能在列表里初始化的就不要放进函数体。** 放进函数体意味着成员先被默认构造一次，再被赋值一次，多一次构造/析构；对 `const` 成员与引用成员，函数体赋值甚至不合法，因为它们必须在初始化时绑定。

编译器视角 · 编译器视角：`-Wreorder` 在说什么

这份代码在本机编译时，编译器给出 `-Wreorder`：它发现 `Widget::second_` 会在 `Widget::first_` 之前初始化（因为声明顺序），但初始化列表写的是 `second_` 先出现，容易让读者误判。诊断不是"代码错了"，而是"你的书写顺序与执行顺序不一致"。修法只有两种：把列表改成 `first_("first"), second_("second")`，或把成员声明改成 `second_` 在前。**先让书写顺序与执行顺序一致，再谈其他优化。**

3.4 默认拷贝：逐成员复制，指针只复制地址

如果类没有写拷贝构造，编译器会生成一个默认拷贝构造：逐个成员复制。成员是 `int`、`std::string` 这类值类型时，逐成员复制是对的；成员是裸指针时，逐成员复制只复制地址，两个对象指向同一块内存。

cpp Shallow：拥有一个 new 出来的 int examples/ch02/copy_shallow/include/copy_shallow.h 第 4–13 行

```cpp
class Shallow {
public:
    explicit Shallow(int value);
    ~Shallow();

    int value() const;

private:
    int* data_;
};
```

cpp 构造分配，析构释放 examples/ch02/copy_shallow/src/copy_shallow.cpp 第 3–7 行

```cpp
Shallow::Shallow(int value) : data_(new int(value)) {}

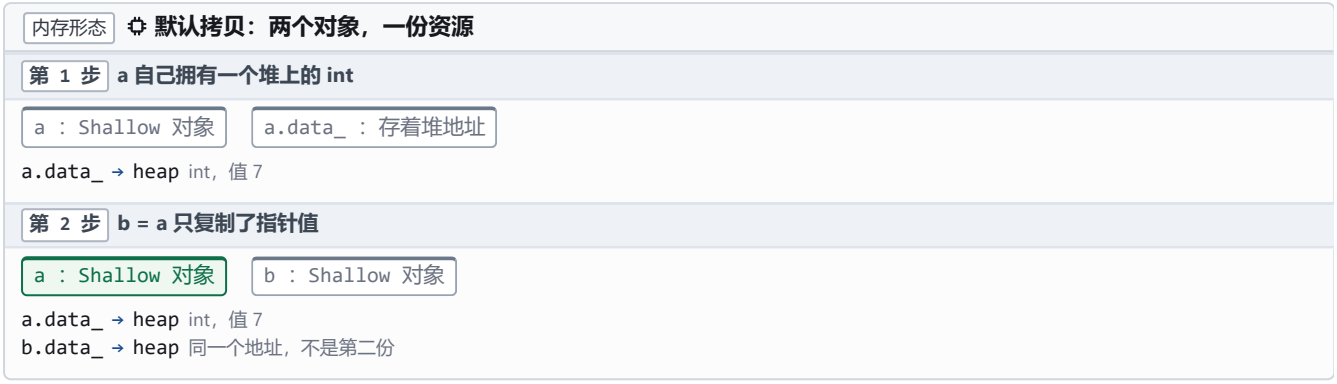
Shallow::~Shallow() {
    delete data_;
}
```

cpp main：默认拷贝后两个对象指向同一块堆内存 examples/ch02/copy_shallow/src/main.cpp 第 3–7 行

```cpp
int main() {
    Shallow a(7);
    Shallow b = a;
    return a.value() + b.value() == 14 ? 0 : 1;
```

```
}
```

这份程序编译通过、没有警告；运行时的退出码是 **3221226356**（Windows 的堆损坏/双重释放异常码，本机 3/3 稳定），stderr 为空，stdout 也为空。原因是：`Shallow b = a;` 走默认拷贝构造，`b.data_` 与 `a.data_` 存的是同一个地址；`main` 返回时先析构 `b`，`delete data_` 释放了那块内存；再析构 `a`，对同一地址再 `delete` 一次，堆管理器发现重复释放，进程终止。



第 1 帧的堆盒子只有一个；第 2 帧的 `b.data_` 没有新建盒子，只是把 `a.data_` 里的地址又抄了一份。所以"两个对象"在语言层面成立，"两份资源"却不成立——析构函数各删一次，第二次就删到了同一块已经归还的内存。

这个例子没有输出，所以正文用退出码作为证据。**"编译通过"只说明类型正确，不说明所有权正确。** 默认拷贝构造不知道你的 `data_` 是"拥有的资源"还是"借来的观察指针"；它一律逐成员复制。要让它正确，只有两条路：禁止拷贝（像 `FileHandle` 那样 `= delete`），或手写深拷贝（像下一节的 `Buffer`）。

> ⚠ **陷阱** · **默认拷贝把"拥有"复制成"共享"**
>
> 你会看到：一个持有 `new` 指针的类，按值传给函数、放进 `vector`、或写 `T b = a;` 之后，程序偶尔崩溃、偶尔"看起来正常"。你会误判成"`new` 用错了"或"编译器 bug"。真相是：默认拷贝构造只复制指针值，两个对象都以为自己拥有同一块内存；谁先析构谁释放，后析构的那个就是双重释放。**判断标准不是"有没有写 `new`"，而是"这个类有没有复制一份资源的能力"。**

3.5 深拷贝与 Rule of Three

要允许拷贝，就必须让新对象拥有自己的一份资源。`Buffer` 用 `new char[]` 保存字符串，手写拷贝构造分配新数组并逐字节复制；拷贝赋值用 copy-and-swap：先按源对象构造一个临时副本，再与自身交换，临时对象析构时释放旧资源。

cpp **Buffer：拷贝构造、拷贝赋值、析构三个都要写** examples/ch02/copy_deep/include/copy_deep.h 第 6–22 行

```cpp
class Buffer {
public:
    explicit Buffer(const char* text);
    Buffer(const Buffer& other);
    Buffer& operator=(const Buffer& other);
    ~Buffer();

    void set(std::size_t index, char value);
    const char* data() const;
    std::size_t size() const;

private:
    char* data_;
    std::size_t size_;

    void swap(Buffer& other) noexcept;
};
```

cpp **深拷贝与 copy-and-swap** examples/ch02/copy_deep/src/copy_deep.cpp 第 12–23 行

```cpp
Buffer::Buffer(const Buffer& other) : data_(nullptr), size_(other.size_) {
    data_ = new char[size_ + 1];
```

```
    std::memcpy(data_, other.data_, size_ + 1);
}

Buffer& Buffer::operator=(const Buffer& other) {
    if (this != &other) {
        Buffer copy(other);
        swap(copy);
    }
    return *this;
}
```

cpp **main：改副本不影响原对象** examples/ch02/copy_deep/src/main.cpp 第 5–11 行

```
int main() {
    Buffer a("alpha");
    Buffer b = a;
    b.set(0, 'A');
    std::cout << "a=" << a.data() << " b=" << b.data() << "\n";
    return 0;
}
```

终端输出 终端输出 copy-deep-out

```
a=alpha b=Alpha
```

`a=alpha b=Alpha` 证明两个对象各有自己的字符数组：改 `b` 的第一个字符，`a` 不变。拷贝构造里 `data_(nullptr)` 先占位，再在函数体分配；如果 `new` 抛异常，对象构造失败，析构函数不会执行，所以不会误删未分配的指针。拷贝赋值里的 `if (this != &other)` 处理自赋值；`Buffer copy(other); swap(copy);` 让旧资源随 `copy` 的析构释放，交换本身只交换两个指针与大小，不抛异常。

Rule of Three（三法则）：如果一个类需要手写析构函数、拷贝构造、拷贝赋值中的任何一个，通常三个都需要手写。原因是它们共同管理同一份资源：析构负责释放，拷贝构造负责复制，拷贝赋值负责"先释放旧的、再复制新的"。只写其中一个，另外两个会退化成编译器生成的默认版本，所有权模型就断了。C++11 之后还有移动构造/移动赋值，扩展成 Rule of Five，第 4 单元会讲。

`swap` 标了 `noexcept`，因为它只交换指针与整数，不会失败。copy-and-swap 的代价是多一次临时对象的构造与析构；收益是拷贝赋值只需要写一次资源释放逻辑（析构函数），并且天然处理自赋值与异常安全。**这不是唯一写法，但它是"先正确、再优化"的稳妥写法。**

3.6 返回值省略：为什么计数是 0

cpp **Counter：只数拷贝与移动** examples/ch02/copy_elision_basic/include/copy_elision_basic.h 第 4–15 行

```
class Counter {
public:
    Counter();
    Counter(const Counter& other);
    Counter(Counter&& other) noexcept;
    ~Counter();

    static int copies;
    static int moves;
};

Counter make_counter();
```

cpp **返回一个局部对象** examples/ch02/copy_elision_basic/src/copy_elision_basic.cpp 第 18–21 行

```
Counter make_counter() {
    Counter local;
    return local;
}
```

终端输出 终端输出 copy-elision-basic-out

```
copies=0 moves=0
```

`make_counter()` 里明明有一个局部对象 `local`，`main` 里又有一个 `result`，为什么拷贝和移动都是 0？因为编译器做了**返回值优化**（NRVO，Named Return Value Optimization）：它直接在 `result` 的存储里构造 `local`，省掉了"构造局部对象 → 拷贝/移动到返回值 → 析构局部对象"这一串。C++17 起，返回纯右值（如 `return Counter{};`）的拷贝省略是**强制**的；NRVO（返回有名字的局部变量）仍然是**可选**的，只是主流编译器都会做。本机 g++ 在这个例子中做了 NRVO，所以计数为 0。

计数为 0 不代表"拷贝构造没写对"，也不代表"移动构造没用"。换一个编译器、换一个优化档、或在 `return` 里写 `std::move(local)`，计数就可能变成 1。第 4 单元会用 `-fno-elide-constructors` 把这条优化关掉，对照 `copies` / `moves` 的真实变化。**现在的结论只有一条：返回值不一定发生拷贝；"计数为 0"是优化结果，不是语言保证。**

3.7 `explicit`：关掉隐式转换的门

只有一个参数的构造函数，默认允许隐式转换：`Meter distance = 42;` 会把 `42` 转成一个 `Meter`。如果这个转换在语义上不合理（42 什么单位？米还是厘米？），就应该用 `explicit` 禁止它。

cpp **explicit_ctor: 这行赋值应当被拒绝** examples/ch02/explicit_ctor/src/main.cpp 第 1–6 行

```cpp
#include "explicit_ctor.h"

int main() {
    Meter distance = 42;
    return distance.value();
}
```

编译诊断 本机真实 stderr：explicit 构造阻止隐式转换 explicit-ctor-diag

```
src/main.cpp: In function 'int main()':
src/main.cpp:4:22: error: conversion from 'int' to non-scalar type 'Meter' requested
    4 |     Meter distance = 42;
      |                      ^~
In file included from src/main.cpp:1:
include/explicit_ctor.h:6:14: note: explicit conversion function was not considered
    6 |     explicit Meter(int value);
      |              ^~~~~
```

诊断的关键在 `explicit conversion function was not considered`：编译器看到了 `Meter(int)`，但因为它是 `explicit`，不参与这次隐式转换。把 `Meter` 的构造函数改成非 `explicit`，这行就能编过；但代价是任何 `int` 都能悄悄变成 `Meter`，包括函数传参、容器初始化、返回值。**`explicit` 不是"更严格所以更好"，而是"这个转换是否应该由程序员显式写出来"的设计选择。** 单参数构造函数默认加 `explicit`，需要隐式转换时再放开，是常见的工程惯例。

3.8 本机实测：三个读数

实验记录 · 本机实测：构造、析构与双重释放

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `raii_file` 的真实 stdout 是 `open` / `read=hello` / `close`，`close` 在 `main` 返回后由析构函数打印；
- `ctor_order` 的真实 stdout 是 `init first` / `init second`，与初始化列表的书写顺序相反，与声明顺序一致；编译器同时给出 `-Wreorder` 警告；
- `copy_shallow` 运行退出码 **3221226356**，stderr 与 stdout 都为空，连续 3 次一致：默认拷贝后两个对象释放同一块堆内存；
- `copy_elision_basic` 的真实 stdout 是 `copies=0 moves=0`：本机 g++ 在 `-O0` 下也做了 NRVO。

3.9 把 RAII 变成工程习惯

1. **资源用对象管**：文件、内存、锁、socket 都用 RAII 包装，构造获取、析构释放；不要让"打开"和"关闭"分散在两个函数里。
2. **拥有资源的类先想拷贝**：禁止拷贝（`= delete`）或写深拷贝；不要依赖默认拷贝。只读观察用引用/指针，不拥有就不负责释放。

3. **三法则/五法则**：写了析构、拷贝构造、拷贝赋值中的任何一个，就把另外两个一起检查；第 4 单元再加上移动构造/移动赋值。
4. **单参数构造函数默认 `explicit`**：除非你确实想要隐式转换（例如 `std::string` 从 `const char*` 构造是标准库的设计选择）。
5. **成员按声明顺序初始化**：列表与声明保持一致；能在列表里初始化就不要放进函数体。

`std::unique_ptr` 与 `std::shared_ptr` 会把"禁止拷贝/引用计数"做进标准库，下一单元会用它们替代手写 `new` / `delete`。但 RAII 的思想不变：**所有权跟着对象走，析构负责归还。**

3.10 下一单元从哪继续

下一单元 `3.2 拷贝与移动` 会把这一单元的"拷贝"扩展到"移动"：值类别、`std::move`、Rule of Five、`noexcept` 移动构造与 `vector` 扩容。本单元已经出现的 `Counter`、`Buffer` 与 `FileHandle` 会在那里以"可移动版本"再次出现；先确保你能独立写出 `Buffer` 的拷贝构造、拷贝赋值与析构，再往下走。

小结 · 这一节要带走的五件事

- 构造函数建立不变量，析构函数确定性释放资源；对象离开作用域时按构造的逆序析构。
- 成员按声明顺序初始化，初始化列表的顺序不决定执行顺序；顺序不一致时 `-Wreorder` 会警告。
- 默认拷贝构造逐成员复制；成员是裸指针时只复制地址，两个对象会重复释放同一块内存。
- Rule of Three：析构、拷贝构造、拷贝赋值共同管理资源，写一个就要检查另外两个；copy-and-swap 是稳妥的拷贝赋值写法。
- 单参数构造函数默认允许隐式转换，`explicit` 关掉它；`explicit` 的选择是接口设计，不是"越严格越好"。

自测题 · 自测题（先自己做，再对答案）

1. `FileHandle` 的 `close` 输出为什么出现在 `read=hello` 之后，而不是 `main` 里显式调用 `fclose` 时？
2. `Widget::Widget() : second_("second"), first_("first") {}` 的执行顺序是什么？为什么？
3. `Shallow b = a;` 之后，两个对象的 `data_` 有什么关系？程序为什么以 3221226356 退出？
4. `Buffer` 的拷贝赋值里，`Buffer copy(other); swap(copy);` 各自解决了什么问题？为什么不用先 `delete[] data_` 再 `new`？
5. `Meter distance = 42;` 为什么会编译失败？如果去掉 `explicit`，这行会发生什么？

注 · 自测答案

1. `FileHandle` 的析构函数在 `file` 离开 `main` 作用域时自动调用，`fclose` 与 `close` 输出都在那之后；程序没有显式调用 `fclose`，`close` 是 RAII 的确定性释放。
2. 先 `init first`，再 `init second`：成员按声明顺序初始化（`first_` 在 `second_` 前），初始化列表只提供初值，不决定顺序。
3. `b.data_` 与 `a.data_` 存的是同一个地址（默认拷贝只复制指针）。`b` 先析构释放该地址，`a` 再析构时对同一地址重复 `delete`，触发堆损坏，退出码 3221226356。
4. `Buffer copy(other);` 先构造一份独立副本，失败时不会改动 `this`；`swap(copy);` 把旧资源换到临时对象里，随其析构释放。先 `delete[]` 再 `new` 的话，如果 `new` 抛异常，`this` 就留下一个已释放的指针；copy-and-swap 保证要么成功，要么原对象不变。
5. `Meter(int)` 是 `explicit`，不参与隐式转换，所以 `42` 不能自动变成 `Meter`。去掉 `explicit` 后这行合法，`42` 会被隐式转换成一个 `Meter` 对象。

拷贝与移动：值类别、std::move 与 Rule of Five

这是 C++ 分册的第一个「对象生命周期」单元，也是全书第 4 单元。它要回答一个 C 语言里没有的问题：**一个对象被复制、被搬走、被销毁时，到底发生了什么？** C 的 `struct` 赋值是逐字节复制；C++ 允许你在复制之外再定义一条「搬走」的路——移动。移动不是语法糖，它决定了资源（堆内存、文件句柄）归谁所有。

◎ 学习目标 · 看完这一单元，你应该能

- 说出拷贝构造与拷贝赋值各在什么时候调用，并用计数证明；
- 解释 `std::move` 只是 `static_cast<T&&>`，并指出真正搬资源的是哪个函数；
- 说出 moved-from 对象的标准状态，并给出一个「本机读数是空、但标准不保证」的例子；
- 用 `-Wpessimizing-move` 解释 `return std::move(local)` 为什么反而阻止优化；
- 按 Rule of Five 写出一个资源类的五个特殊成员函数，并说明 `noexcept` 移动构造对 `vector` 扩容的影响。

C 分册里，结构体赋值就是逐字节复制：`b = a;` 之后两个结构体各有一份字节，谁也不欠谁。这个模型在成员都是值、没有堆资源时够用；一旦成员是一个「拥有堆内存的指针」，逐字节复制就只复制了地址，两个对象会指向同一块内存。C++ 把「复制」与「搬走」拆成两个可编程的入口：拷贝构造 / 拷贝赋值负责「再要一份」，移动构造 / 移动赋值负责「把这一份交出去」。这一单元不追求把 C++ 的拷贝语义讲全，而是用八个可运行的对照实验，把「什么时候复制、什么时候搬、搬完源对象是什么状态」这三件事钉死。

4.1 先看工程长什么样

本单元的例子仍然在 `examples/ch02/` 下。为了把「拷贝」与「移动」的每个侧面都单独验证，这里有 **8 个多文件小工程**，每个目录都是 `include/<例名>.h` + `src/main.cpp` + `src/<例名>.cpp` + `<例名>.json`。第 2 章起，硬规定③生效：不允许单文件工程——每个清单至少两个 `.cpp`，并且至少一个 `.h` 被某个源文件 `#include` 进来。

目录树 **工程：examples/ch02/ 从零到八个例子**

第 1 步 先建第一个例子，验证拷贝计数

```
examples/
└── ch02/
    └── copy_basics/ ← 本课新增
```

第 2 步 再加上移动语义的四个侧面

```
examples/
└── ch02/
    ├── copy_basics/
    ├── 前一个例子 : 同上
    ├── move_is_cast/ ← 本课新增
    ├── moved_from/ ← 本课新增
    ├── rvo_counter/ ← 本课新增
    └── pessimizing_move/ ← 本课新增
```

第 3 步 最后补上资源管理、浅拷贝与容器扩容

```
examples/
└── ch02/
    ├── 前五个例子 : 同上
    ├── rule_of_five/ ← 本课新增
    ├── shallow_copy_bug/ ← 本课新增
    └── vector_realloc/ ← 本课新增
```

第 1 帧只有 `copy_basics/`，它用计数告诉你「拷贝构造」和「拷贝赋值」是两次不同的事件。第 2 帧加上 `move_is_cast/`（`std::move` 只是类型转换）、`moved_from/`（被搬走之后还剩什么）、`rvo_counter/`（返回值优化）与 `pessimizing_move/`（一个会让编译器警告的坏习惯）。第 3 帧加上 `rule_of_five/`（五个特殊成员函数）、`shallow_copy_bug/`（默认拷贝导致双重释放）与 `vector_realloc/`（`noexcept` 移动构造如何影响容器扩容）。每个目录里的 `include/` 与 `src/` 分家方式与 C 分册相同：声明放头文件，定义放源文件。

把这八个目录当成八次对照实验，每次只改变一个变量：

- `copy_basics/`：只数拷贝构造与拷贝赋值的次数，证明它们是两个入口；
- `move_is_cast/`：用两个 `describe` 重载证明 `std::move` 只改变值类别，不搬对象；
- `moved_from/`：把 `string` 与 `vector` 搬走，观察源对象在本机还剩什么；
- `rvo_counter/`：同一份源码，默认与 `-fno-elide-constructors` 两份清单，对照 RVO/NRVO 的计数；
- `pessimizing_move/`：一个 `return std::move(local)`，让 `-Werror=pessimizing-move` 把它变成编译失败；

- `rule_of_five/`：一个真正拥有 `new char[]` 的类，五个特殊成员函数各走一遍并计数；
- `shallow_copy_bug/`：故意不写拷贝构造，让默认浅拷贝把程序送进双重释放；
- `vector_realloc/`：`noexcept` 移动构造与不带 `noexcept` 的移动构造，对照 `vector` 扩容时选移动还是拷贝。

这八个例子都不靠「看代码猜结论」：每个结论都有一条真实输出或退出码在后面顶着。第 2 章起不允许单文件工程，所以每个例子都有 `main.cpp` 与一个实现 `.cpp`，头文件只放声明；这样你也能顺带看到「声明与定义分家」在 C++ 里是怎么落地的。

4.2 拷贝构造与拷贝赋值：两次不同的事件

先看一个最小的计数类。它只有一个 `std::string` 成员，不拥有堆资源，因此默认的拷贝行为是对的；我们只是把「拷贝构造」和「拷贝赋值」各自数出来。

{} 语法规则 · 拷贝与移动的五个签名

- 拷贝构造：`T(const T&)`
- 移动构造：`T(T&&) noexcept`（`noexcept` 不是语法要求，但资源类通常这么写）
- 拷贝赋值：`T& operator=(const T&)`
- 移动赋值：`T& operator=(T&&) noexcept`
- 析构：`~T()`

cpp **Tracker：两个拷贝入口分别计数** examples/ch02/copy_basics/src/copy_basics.cpp 第 9–28 行

```cpp
struct Tracker {
    static int copies;
    static int assignments;

    std::string name;

    explicit Tracker(std::string n) : name(std::move(n)) {}

    Tracker(const Tracker& other) : name(other.name) {
        ++copies;
    }

    Tracker& operator=(const Tracker& other) {
        if (this != &other) {
            name = other.name;
            ++assignments;
        }
        return *this;
    }
};
```

`Tracker b = a;` 走的是**拷贝构造**：`b` 此时还不存在，构造函数负责把 `b` 建出来，所以 `copies` 加一。`c = a;` 走的是**拷贝赋值**：`c` 已经存在，赋值运算符负责把 `a` 的状态写进 `c`，所以 `assignments` 加一。两者签名不同、调用时机不同、要做的事也不同——拷贝赋值必须先处理「自己已经有资源」这件事（这里是 `std::string` 自己处理），否则就会泄漏或重复释放。

cpp **main 里只有两行，两个入口各命中一次** examples/ch02/copy_basics/src/copy_basics.cpp 第 35–39 行

```cpp
void run_copy_basics() {
    Tracker a("alpha");
    Tracker b = a;              // 拷贝构造
    Tracker c("gamma");
    c = a;                      // 拷贝赋值
```

真实输出：

终端输出 终端输出 copy-basics-out

```
a=alpha b=alpha c=alpha
copies=1 assignments=1
```

`a=alpha b=alpha c=alpha` 说明三个对象各自有一份名字；`copies=1 assignments=1` 说明两个入口各走了一次。注意 `Tracker` 的 `name` 是 `std::string`，它自己有深拷贝；如果成员是一个裸指针，默认拷贝就只复制指针值——这正是 4.7 节要看的 bug。

拷贝构造最常见的触发点有四个：用同类型对象初始化新对象（`Tracker b = a;` 或 `Tracker b(a);`）、按值传参、按值返回（没有被省略时）、把对象放进容器（`vector` 扩容或 `push_back`）。它们的共同点是「新对象还没有自己的资源」，所以拷贝构造必须把源对象的资源复制一份。拷贝赋值不同：目标对象已经存在，可能已经握着一块资源，所以它必须先处理「旧资源怎么办」，再复制新资源——顺序反了就会泄漏，忘了处理自赋值就会把自己的资源释放掉再读。

一个常见的写法是 copy-and-swap：先按源对象构造一个临时对象（复用拷贝构造），再把自己和临时对象交换，临时对象在析构时释放旧资源。这样拷贝赋值只需要写一次资源释放逻辑（析构函数），而且天然处理自赋值与异常安全——交换不会抛异常。代价是多了一次临时对象的构造与析构，本单元 4.6 节的 `copy_ctors=2` 就是这笔代价的读数。

为什么拷贝构造的参数是 `const T&`，不是 `T`？如果写成 `T(T other)`，为了把实参传进来，编译器又要调用一次拷贝构造，递归没有出口；`const T&` 直接绑定源对象，不复制。为什么加 `const`？因为拷贝不应该修改源对象，而且 `const T&` 能绑定临时对象（比如 `Tracker b = Tracker("x");`），非 const 引用绑定不了。拷贝赋值返回 `T&` 而不是 `void`，是为了支持 `a = b = c` 这样的链式赋值；返回 `*this` 是惯例，不是标准强制。

把「按值传参」单独拎出来看：`void take(Tracker t)` 调用时，实参会被复制进形参，触发一次拷贝构造；函数返回 `Tracker` 时，如果编译器不做省略，还会再触发一次拷贝或移动。这就是为什么大对象通常按 `const T&` 传参——不是为了「语法正确」，而是为了省掉一次真实的对象构造。反过来说，如果函数本来就要一份独立的副本（例如它要在内部修改），按值传参反而是更清晰的接口：调用者不用关心内部怎么改，函数自己拿到一份副本。**传参方式的选择，取决于「你要不要一份自己的副本」，不是「引用看起来更高级」。**

拷贝构造与移动构造的签名只差一个 `&`：`T(const T&)` 与 `T(T&&)`。这个 `&&` 不是「两个地址」，而是右值引用类型；它让重载决议在实参是右值时优先选移动构造。拷贝赋值与移动赋值的签名同理，只是返回类型通常是 `T&`。注意：移动构造 / 移动赋值通常标 `noexcept`，拷贝构造 / 拷贝赋值通常不标——因为拷贝要分配资源，可能抛；移动只接管资源，通常不抛。这个差异在 4.8 节会变成 `vector` 的实际选择。

`emplace_back` 与 `push_back` 的区别也常和拷贝/移动一起问：`v.push_back(T("x"))` 先构造一个临时 `T`，再把这个临时对象移动/拷贝进容器；`v.emplace_back("x")` 把参数直接传给容器里的构造，**在目标位置直接构造**，少了一次临时对象与一次移动。少掉的那一次不是语法糖，是真实的构造/析构次数差异。但 `emplace_back` 不保证一定更快：如果容器扩容，已有元素仍然要被移动/拷贝；如果参数类型与构造函数不匹配，编译错误也会更难读。**先写清楚语义，再谈省一次构造。**

计数实验本身也有一个设计点：为什么用 `static int` 计数，而不是在每个构造函数里直接 `std::cout`？因为输出会与构造/析构的顺序纠缠在一起，读起来像一锅粥；把计数留到最后打印，得到的是一张干净的对照表。代价是你看不到「第几个对象、在什么时候」构造的——要看顺序，就换成在构造函数里打印指针/名字。**证据的形式要跟着问题走：问次数用计数，问顺序用日志，问所有权用地址。**

4.3 `std::move` 只是 `static_cast<T&&>`：它不搬任何东西

还要区分「拷贝构造」与「拷贝赋值」的语法位置：`T b = a;` 看起来像赋值，其实是**初始化**，调用拷贝构造；只有 `b = a;`（`b` 已经声明过）才是拷贝赋值。这个区别在写计数实验时最容易搞混——把 `T b = a;` 当成赋值，计数就会对不上。

4.3 `std::move` 只是 `static_cast<T&&>`：它不搬任何东西

新手最常见的误解是「`std::move(x)` 把 x 搬走了」。没有。`std::move` 的实现本质上就是一次类型转换：把左值表达式转换成右值引用，让重载决议有机会选中「移动」那条路。真正搬资源的是**移动构造 / 移动赋值**，不是 `std::move`。

cpp **两个 describe 重载：左值走 const&，右值走 &&** examples/ch02/move_is_cast/src/move_is_cast.cpp 第 10–31 行

```cpp
void describe(const std::string&) {
    std::cout << "lvalue overload\n";
}

void describe(std::string&&) {
```

```cpp
    std::cout << "rvalue overload\n";
}

}  // namespace

void run_move_is_cast() {
    std::string s = "abc";

    describe(s);                // 左值：匹配 const string&
    describe(std::move(s));     // 右值：匹配 string&&

    using T = std::string;
    T&& r = static_cast<T&&>(s);

    std::cout << std::boolalpha
              << "same address: " << (std::addressof(r) == std::addressof(s))
              << "\n";
```

终端输出 终端输出 move-is-cast-out

```
lvalue overload
rvalue overload
same address: true
```

输出里 `lvalue overload` 与 `rvalue overload` 各出现一次，说明重载决议确实按值类别选了不同的函数；`same address: true` 说明 `r` 和 `s` 指向**同一个对象**——`std::move` 没有创建新对象，也没有搬走字符。把这张图记住：

图 / 搜索过程 **值类别与重载选择：describe 为什么两次走不同分支**

第 1 步 **s 是左值**

s ：有名字，能取地址

s → describe(const string&) 左值匹配 const&

第 2 步 **std::move(s) 是右值**

std::move(s) ：static_cast<string&&>，还是同一个对象 ← 本步新增

*std::move(s) → describe(string&&) 右值匹配 &&

第 1 帧的 `s` 是左值；第 2 帧的 `std::move(s)` 是右值引用表达式，但它仍然指向 `s` 那块存储。**右值引用是「允许被搬」的标签，不是「已经搬走」的事实。** `std::move` 本身不改变 `s`；只有真正发生移动构造 / 移动赋值后，源对象才进入「有效但未指定」状态。

值类别可以先用三句话记：**左值**有名字、能取地址；**纯右值**是临时结果、没有名字；**将亡值**是「有身份但马上要被搬走」的对象，`std::move(x)` 产生的就是它。`T&&` 是**类型**，不是值类别；一个声明出来的 `T&& r = ...;` 变量本身是左值（它有名字），所以把 `r` 再传给函数时，不写 `std::move(r)` 仍然按左值匹配。这就是为什么"右值引用"和"右值"不能混着说。

几个常见表达式的值类别可以列成一张小表：

表达式	值类别	说明
`x`	左值	有名字，能取地址
`std::move(x)`	将亡值（右值）	还是 `x` 那个对象，只是被标成「可搬」
`T{}`	纯右值	临时对象，没有名字
`x++`	纯右值	返回旧值的副本
`++x`	左值	返回 `x` 本身

这张表说明：`std::move(x)` 并没有让 `x` 变成另一个对象，它只是改变了表达式的值类别。重载决议看的是**表达式**的值类别，不是变量声明时的类型。`std::forward<T>` 也是做类似的事，但它会按模板参数 `T` 保留原来的值类别——这是第 8 单元「完美转发」的内容，本单元只要记住：`std::move` 是**无条件**转成右值，`std::forward` 是**有条件**保留。

`std::move` 最常见的四个误用：

- 对 `const` 对象调用：`const std::string s = "x";` 之后 `std::move(s)` 得到 `const std::string&&`，移动构造要的是 `std::string&&`，匹配不上，最后还是拷贝；
- 以为搬完源对象就空了：见 4.4，标准只保证有效但未指定；
- 对指针调用：`std::move(p)` 只把指针本身当右值，指针指向的堆内存不会跟着搬；要让所有权转移，得移动拥有它的对象（例如 `unique_ptr`），不是移动裸指针；
- 以为写了 `std::move` 就一定走移动构造：如果类型没有移动构造，或者移动构造被删除，重载决议会退回拷贝构造，`std::move` 只是白写。

所以判断「有没有搬」的正确方式不是看代码里有没有 `std::move`，而是看类型有没有可用的移动操作，以及计数 / 地址 / 输出给出的证据。本单元的 `move_is_cast` 用 `same address: true` 证明没有搬对象；4.5 的 `rvo_counter` 用计数证明移动发生了几次；4.8 的 `vector_realloc` 用 `noexcept` 对照证明容器会不会选移动。

4.4 moved-from：有效但未指定，不是「空」

`std::string t = std::move(s);` 之后，`s` 还在，类型还是 `std::string`，你仍然可以对它调用 `size()`、`empty()`、`clear()`，甚至重新赋值。但标准不保证它现在是空串：它只保证「有效但未指定」（valid but unspecified）。本机 libstdc++ 对短串（SSO）移动时复制内部缓冲区并把源串长度置 0；长串才可能接管堆缓冲区。所以 `s` 看起来是空的；换一个实现，`s` 可能保留一小段内容。

cpp **moved-from 的 string 与 vector：本机读数是空，但不能当保证** examples/ch02/moved_from/src/moved_from.cpp 第 8–20 行

```cpp
void run_moved_from() {
    std::string s = "hello";
    std::string t = std::move(s);

    std::cout << "t=" << t << " s.size=" << s.size()
              << " s.empty=" << std::boolalpha << s.empty() << "\n";

    std::vector<int> a{1, 2, 3};
    std::vector<int> b = std::move(a);

    std::cout << "b.size=" << b.size() << " a.size=" << a.size()
              << " a.empty=" << a.empty() << "\n";
}
```

终端输出 终端输出 moved-from-out

```
t=hello s.size=0 s.empty=true
b.size=3 a.size=0 a.empty=true
```

本机输出 `t=hello s.size=0 s.empty=true`、`b.size=3 a.size=0 a.empty=true`。**这是本机 libstdc++ 的一次读数，不是标准保证。** 标准库对 moved-from 对象的总则写在 `[lib.types.movedfrom]`：除非另有规定，被移动后的对象处于有效但未指定的状态。你可以对它做「不依赖具体值」的操作（析构、赋值、`clear()`），但不能假设它一定是空、一定保留原值、或者 `size()` 一定为 0。

把 moved-from 想成一个「已经交出家具有没有搬完不知道」的房间：房间本身还在，门锁、灯、地板都还能用（析构、赋值、`clear()` 都是安全的），但你不能根据房间里还剩什么做判断。**「有效」保证的是「不会一碰就炸」，「未指定」说明「别读具体值」。** 这也解释了为什么标准不规定成「一定为空」：不同实现有不同的优化空间（短串优化、缓冲区复用），强行规定为空反而会限制实现。

实际写代码时，moved-from 只适合做三件事：让它自然析构、给它赋新值、把它 `clear()` / `swap()` 掉。不要拿它当查询结果、不要把它当容器的键、不要在日志里打印它的内容并指望每次都一样。**能读**和**能依赖读到的值**是两回事。

标准原文 · 标准原文：moved-from 不是「可以随便读」

> C++ 标准对标准库对象的 moved-from 状态有一句总则：除非另有规定，被移动后的对象处于**有效但未指定**的状态（`[lib.types.movedfrom]`，N4950 §16.4.6.15）。注意：moved-from 对象的生命周期**没有结束**；`[basic.life]`（N4950 §6.7.3）管的是生命周期结束后的存储使用，和 moved-from 不是同一件事。两句话合起来：**moved-from 不是空，也不是坏对象；它是「还能安全使用，但别猜内容」的对象。**

moved-from 的规则不只影响 `string` / `vector`，它决定了「移动之后还能不能用源对象」这个接口问题。一个函数如果接受 `T&&` 并把参数搬走，调用者就必须知道「传进去的对象之后不能再用具体值」；如果函数只是借用（比如 `const T&`），就不该搬。这也是为什么标准库的 `std::move` 算法（`std::move(first, last, out)`）名字里虽然有 move，它做的是「把每个元素移动到输出区间」，源区间之后同样处于有效但未指定状态。**移动是所有权转移，不是「读取一下再复制」；接口设计要把这件事说清楚。**

如果确实需要保留源对象的值，就不要移动，老老实实拷贝；拷贝的代价是可见的、可预测的，而「移动之后偷偷读源对象」的 bug 往往在换实现或换优化档后才暴露。C++ 的移动语义把「我还要不要这个值」变成了一个必须由调用者回答的问题。

标准库各类型对 moved-from 的具体保证并不一样：`std::unique_ptr` 被移动后保证为空（它的规范明确写了 `nullptr`），`std::string` / `std::vector` 只给「有效但未指定」。**「有效但未指定」是默认规则，不是所有类型的统一读数。** 查一个类型的 moved-from 状态，要看它自己的条款，不要拿 `string` 的本机读数去套 `unique_ptr`。这也是为什么本单元把 `moved_from` 的读数标成「本机 libstdc++」，而不是「C++ 保证」。

调试时不要用 `printf` / `cout` 打印 moved-from 的值来判断「移动有没有成功」：打印本身是合法的，但打印出来的值没有规范意义，换一版标准库就可能变。判断移动是否发生，应该看**目标对象**（它拿到了什么）与**计数/地址**（移动构造调用了几次、两个对象是否指向同一块），而不是看源对象「看起来空不空」。**源对象的状态是结果，不是证据；证据在目标和计数上。**

4.5 RVO 与 NRVO：为什么 `return std::move(x)` 反而更慢

先看一个返回局部对象的函数。`make_counted()` 里有一个局部变量 `local`，返回它时会发生什么？

cpp **移动构造 + make_counted + 计数** examples/ch02/rvo_counter/src/rvo_counter.cpp 第 17–32 行

```cpp
Counted::Counted(Counted&&) noexcept {
    ++moves;
}

Counted make_counted() {
    Counted local;
    return local;       // NRVO；关掉省略时退化为移动
}

void run_rvo_counter() {
    Counted x = make_counted();
    std::cout << "ctors=" << Counted::ctors
              << " copies=" << Counted::copies
              << " moves=" << Counted::moves << "\n";
    (void)x;
}
```

`make_counted()` 返回的是一个纯右值（prvalue），`Counted x = make_counted();` 这一句从 C++17 起受**强制拷贝省略**保护：调用点不会为返回值再构造一个临时对象。至于函数内部的 `local` 是否直接构造在 `x` 的存储里，那是 **NRVO（具名返回值优化）**——标准允许但不强制。本机 GCC 在这里做了 NRVO，所以默认清单的输出是：

终端输出 终端输出 rvo-default-out

```
ctors=1 copies=0 moves=0
```

把同一份源码加上 `-fno-elide-constructors`（清单 `rvo_counter_noelide.json` 的唯一区别）再跑一次：

终端输出 终端输出 rvo-noelide-out

```
ctors=1 copies=0 moves=1
```

默认 `copy=0 move=0`（本机 GCC 做了 NRVO）；关掉省略后 `copy=0 move=1`。**移动不是零成本，省略才是零成本；NRVO 是编译器给你的礼物，不是标准欠你的保证。** 这也解释了一个常见的反向优化：

这里要把三个名字分开：**RVO**（返回值优化）通常指返回一个临时对象时，直接在调用者的存储里构造；**NRVO**（具名返回值优化）指返回一个具名局部变量时做同样的事；**强制拷贝省略**是 C++17 起对「用同类型纯右值初始化对象」的硬性规定——`Counted x = make_counted();` 的调用点不会再多构造一个临时对象。NRVO 不在强制之列，`-fno-`

`elide-constructors` 关掉的正是这类**可选**省略。所以本机默认 `move=0` 是 GCC 的选择，不是标准欠你的；换一个不做 NRVO 的编译器，默认也可能 `move=1`。三种情况都不要和「`std::move` 一定会移动」混为一谈。

编译器怎么做到省略？它不需要「先构造一个对象再搬过去」，而是把返回槽（调用者为返回值预留的那块存储）的地址传给被调函数，让被调函数直接在那里构造 `local`。这不是运行期的优化技巧，而是编译器在生成代码时就把「返回对象」和「调用者的接收对象」当成同一块存储。C++17 的强制省略把这件事写进了语言规则：只要初始化表达式是**同类型的纯右值**，就不允许再产生一次拷贝/移动。NRVO 则取决于编译器是否愿意做这个分析——标准允许，但不强制。**这也解释了为什么 `-fno-elide-constructors` 只影响可选省略，而 `std::move` 的误用会主动破坏它：你让返回表达式不再是「一个名字」，编译器就没有 NRVO 的入口了。**

`cpp` **一个看起来更努力、实际更慢的 `return`** examples/ch02/pessimizing_move/src/pessimizing_move.cpp 第 5–8 行

```cpp
std::string make_text() {
    std::string local = "abc";
    return std::move(local);    // -Wpessimizing-move: 阻止 NRVO
}
```

`return std::move(local);` 把 `local` 显式变成右值，编译器因此不能再对它做 NRVO（具名返回值优化要求返回的是一个名字，而不是一个转换后的表达式）。在本机默认档，结果是从 0 次构造退化成 1 次移动；换一个本来就不做 NRVO 的实现，默认也可能是一次移动。GCC 会在 `-Wall` 里直接报 `-Wpessimizing-move`；本书的清单再加 `-Werror=pessimizing-move`，让它变成编译失败，诊断原文由闸门从真实 stderr 回写：

编译诊断 本机真实 stderr：-Werror=pessimizing-move 让编译失败 pessimizing-move-diag

```
src/pessimizing_move.cpp: In function 'std::string make_text()':
src/pessimizing_move.cpp:7:21: error: moving a local object in a return statement prevents copy elision [-
Werror=pessimizing-move]
    7 |     return std::move(local);    // -Wpessimizing-move: 阻止 NRVO
      |            ~~~~~~~~~^~~~~~
src/pessimizing_move.cpp:7:21: note: remove 'std::move' call
cc1plus.exe: some warnings being treated as errors
```

编译器视角 · **编译器视角：这条警告在说什么**

`-Wpessimizing-move` 的触发条件是「在 return 语句里对一个局部对象调用 `std::move`」。编译器知道这个局部对象马上就要销毁，本来可以直接在调用者的存储里构造（RVO/NRVO），而 `std::move` 把这条零成本的路堵上了。诊断里的 `moving a local object in a return statement prevents copy elision` 就是字面意思：**你阻止了拷贝省略。**它不是「你写错了类型」，而是「你多写了一次移动」。

注意这个例子的清单模式是 `compile_fail`，不是 `run`：程序根本编不出来，所以没有运行时计数。证据就是编译器真实吐出的 stderr（上面那个块），而不是「我觉得它会慢」。这也是本书用 `-Werror=pessimizing-move` 的原因——把一条容易被忽略的警告升级成必须处理的编译失败，让例子在闸门里稳定复现。**警告不是噪音，它是编译器把「你多写了一次移动」这件事直接说给你听。**

调用树 **RVO 与移动：make_counted 返回时到底调用了谁**

第 1 步 **默认（本机 GCC 做了 NRVO）**

```
make_counted() : GCC 把 local 直接构造在返回槽（NRVO，可选优化）
Counted x = make_counted() : 调用点的纯右值初始化强制省略，copy=0 move=0 ← 本步新增
```

第 2 步 **-fno-elide-constructors**

```
make_counted() : 返回时把 local 当成右值 ← 本步新增
Counted x = ... : 移动构造一次，copy=0 move=1 ← 本步新增
```

第 1 帧是默认情况：本机 GCC 做了 NRVO，`local` 的构造发生在返回槽里，调用点又受强制省略保护，所以没有第二次构造。第 2 帧是关掉可选省略后的情况：编译器仍然先构造 `local`，返回时再移动构造一次。两张图只差一个开关，计数就从 `move=0` 变成 `move=1`——这就是「可选省略」与「移动」的区别。

RVO/NRVO 也有边界：如果 `return` 的表达式类型与函数返回类型不同（例如局部是 `std::unique_ptr<Derived>`，返回 `std::unique_ptr<Base>`），编译器不能直接省略，只能先完成类型转换，再移动/复制；如果返回的是成员变量、全局变量或函数参数，NRVO 也不适用，因为它们不是「函数内部的具名局部对象」。**不要把 NRVO 当成「只要**

return 局部变量就零拷贝」的保证；它是一条允许编译器优化的路径，不是语言强制。真正稳定的是「返回纯右值时调用点的强制省略」，以及「不要用 `std::move` 去堵 NRVO」。

那什么时候**应该**写 `std::move`？答案是「你确实想把所有权交出去」的时候：

- 把一个即将销毁的对象塞进容器：`v.push_back(std::move(s));`，之后不再用 `s`；
- 移动赋值：`dst = std::move(src);`，`src` 之后只做析构或重新赋值；
- 实现移动构造 / 移动赋值时，对成员用 `std::move`（如果成员是资源拥有者）；
- 泛型代码里转发参数用 `std::forward<T>`，不是 `std::move`（4.10 之外的第 8 章会展开）。

反过来，**不要**在 `return` 一个局部变量时写 `std::move`，不要对 `const` 对象写，不要在移动之后还依赖源对象的值。`std::move` 是「我确定不再需要这个值」的声明，不是「让代码变快」的咒语。

4.6 Rule of Five：资源类必须把五个特殊成员函数想清楚

`Buffer` 拥有 `new char[]`。它必须自己定义五个特殊成员函数：析构、拷贝构造、移动构造、拷贝赋值、移动赋值。**不要假设「少写一个，编译器就会生成你想要的默认版本」**：一旦你声明了析构、拷贝构造或拷贝赋值，移动构造 / 移动赋值就不会被隐式声明；隐式拷贝对裸指针只做浅拷贝，后果见 4.7。Rule of Five 的要点是「五个一起想清楚」，不是「五个都必须手写」。

标准原文 · **标准原文：五个特殊成员函数**

> C++ 标准把拷贝构造、移动构造放在 `[class.copy.ctor]`（N4950 §11.4.5.3），把拷贝赋值、移动赋值放在 `[class.copy.assign]`（N4950 §11.4.6）。它们不是五个孤立的函数：编译器在**没有被抑制**的前提下生成默认版本，默认版本对每个成员做逐成员拷贝/移动；一旦你声明了析构、拷贝构造或拷贝赋值，移动操作就不再隐式生成。**「Rule of Five」是工程惯例的总结，不是标准里的一个条款**——标准只规定这五个函数各自的行为，不会强迫你全写。

工程惯例 · **工程惯例：Rule of Five**

「五个一起想清楚」是工程惯例，不是标准条款。标准不要求你手写五个；它只规定这五个函数各自的行为。真正的纪律是：资源类一旦自己管理资源，就把五个都检查一遍——缺哪个、为什么缺、缺了以后默认行为是什么，要能说出来。

cpp **Buffer 的五个特殊成员函数（声明）** examples/ch02/rule_of_five/include/rule_of_five.h 第 6–25 行

```cpp
class Buffer {
public:
    static int ctors;
    static int copy_ctors;
    static int move_ctors;
    static int copy_assigns;
    static int move_assigns;
    static int dtors;

    explicit Buffer(std::size_t n);
    Buffer(const Buffer& other);
    Buffer(Buffer&& other) noexcept;
    Buffer& operator=(const Buffer& other);
    Buffer& operator=(Buffer&& other) noexcept;
    ~Buffer();

private:
    char* data_;
    std::size_t size_;
};
```

cpp **移动构造：接管指针，把源对象置空** examples/ch02/rule_of_five/src/rule_of_five.cpp 第 24–29 行

```cpp
Buffer::Buffer(Buffer&& other) noexcept
    : data_(other.data_), size_(other.size_) {
    other.data_ = nullptr;
    other.size_ = 0;
```

```
    ++move_ctors;
}
```

cpp **析构与一组真实调用** examples/ch02/rule_of_five/src/rule_of_five.cpp 第 53–69 行

```cpp
Buffer::~Buffer() {
    delete[] data_;
    ++dtors;
}

void run_rule_of_five() {
    {
        Buffer a(4);
        Buffer b = a;                 // 拷贝构造
        Buffer c = std::move(a);      // 移动构造
        Buffer d(2);
        d = b;                        // 拷贝赋值
        Buffer e(2);
        e = std::move(c);             // 移动赋值
        (void)d;
        (void)e;
    }
```

终端输出 终端输出 rule-of-five-out

```
ctors=3 copy_ctors=2 move_ctors=1 copy_assigns=1 move_assigns=1 dtors=6
```

先别急着看计数，先看 `Buffer` 的五个函数各自做了什么：

- **析构**：`delete[] data_`，只释放自己拥有的那块；
- **拷贝构造**：新分配一块，逐字节复制，源对象不动；
- **移动构造**：直接接管 `other.data_`，然后把 `other` 置空，保证它的析构不会重复释放；
- **拷贝赋值**：先构造临时对象（copy-and-swap），再交换，异常安全；
- **移动赋值**：先释放自己原有的块，再接管对方的块，把对方置空。

本机输出是 `ctors=3 copy_ctors=2 move_ctors=1 copy_assigns=1 move_assigns=1 dtors=6`。为什么 `copy_ctors` 是 2 而不是 1？因为拷贝赋值 `d = b;` 内部构造了一个临时 `Buffer tmp(other)`，它也是一次拷贝构造。**计数表里多出来的那一次，不是 bug，而是 copy-and-swap 的代价。** 这也说明「五个函数」不是五个孤立的函数：拷贝赋值可以复用拷贝构造，移动赋值要处理自赋值，析构要能接住所有路径。

把五个函数按「谁分配、谁释放、谁接管」重读一遍，逻辑就清楚了：析构函数只负责释放**自己当前拥有**的块，所以移动构造 / 移动赋值必须把源对象的指针置空，否则源对象析构时会再释放一次；拷贝构造负责**新分配 + 复制**，它不碰源对象；拷贝赋值要先处理「自己原来那块」，再复制对方的；移动赋值要先释放自己那块，再接管对方那块，最后把对方置空。五个函数的共同不变量是：**每一个非空指针恰好有一个所有者。**只要这个不变量成立，就不会 double free，也不会泄漏。

这也解释了「少写一个会怎样」：只写析构而不写拷贝构造，默认拷贝会复制指针，两个对象都以为自己拥有那块内存；只写拷贝构造而不写移动构造，`std::move` 会退回复制（因为移动构造没有被隐式声明时，重载决议选拷贝）；写了移动构造而不写移动赋值，移动赋值会退回复制赋值。**编译器不会替你补齐你想要的那一个**，它只会按语言规则生成或抑制某些默认版本。

自赋值是另一个容易漏掉的边界：`a = a;` 时，如果拷贝赋值先 `delete[] data_` 再读 `other.data_`，源对象已经被自己释放了。`Buffer` 用 `if (this != &other)` 挡住自赋值；copy-and-swap 的写法则天然安全，因为临时对象先把源内容复制走，再交换。两种写法都可以，关键是你要意识到「源对象可能就是自己」。

如果资源已经由 `std::string`、`std::vector`、`std::unique_ptr` 这类 RAII 成员管理，通常不需要手写五个函数——编译器生成的默认版本会正确调用成员的拷贝/移动，这叫 **Rule of Zero**。Rule of Five 是「你自己管理裸资源」时的纪律；能交给 RAII 成员，就优先交给它们。

移动构造的成员初始化列表也有讲究：`Buffer(Buffer&& other) noexcept : data_(other.data_), size_(other.size_) { other.data_ = nullptr; other.size_ = 0; }`——**初始化列表负责接管，函数体负责把源对象置空。**这两步不能反过来：如果先置空源对象，初始化列表就拿不到指针了；如果忘了置空，源对象析构时会重复释

放。移动赋值则是「先释放自己、再接管、再置空」，顺序同样不能错。拷贝构造没有「置空源对象」这一步，因为它不改变源对象。

移动赋值还要处理「自己移动给自己」：`a = std::move(a);` 在移动赋值里如果先 `delete[] data_`，再读 `other.data_`，而 `other` 就是 `*this`，读到的就是已经释放的指针。`if (this != &other)` 这一行看起来是防御性代码，实际上是正确性的一部分。拷贝赋值的自赋值同样危险：先释放自己的资源再读源对象，源对象已经被释放了。**自赋值不是「没人会这么写」的边角料，它是赋值运算符必须成立的边界条件。**

拷贝赋值有两种常见写法，各有代价：

- **释放 + 复制**：先 `delete[] data_`，再按源对象大小分配、复制。少一次临时对象，但如果复制过程中 `new` 抛异常，对象已经处于「旧资源没了、新资源没建好」的坏状态；
- **copy-and-swap**：先按源对象构造临时对象（复用拷贝构造），再交换，临时对象析构时释放旧资源。多一次构造/析构，但交换不抛异常，异常安全更好，而且天然处理自赋值。

`Buffer` 选了 copy-and-swap，所以计数表里 `copy_ctors=2`。这不是唯一的正确写法，而是一次有意的取舍：**用一次临时构造换异常安全与自赋值安全。** 如果你的资源释放不会抛、复制逻辑简单，释放+复制也可以；关键是你要能说出「异常发生时对象处于什么状态」。这就是「异常安全」的三个档位：不泄漏（基本保证）、操作要么全成要么全不成（强保证）、承诺不抛（`noexcept`）。本单元的 `Buffer` 的目标是强保证。

析构函数什么时候该是虚的？本单元不展开多态，但先留一句：如果你会通过基类指针 `delete` 一个派生对象，基类析构必须是虚的，否则派生部分的析构不会被调用。`Buffer` 不是基类，所以它的析构不需要虚；第 6 单元会把这条规则和虚函数一起讲。**不要因为「看到类就加 virtual」——虚析构有运行期开销，只有多态删除的场景才需要。**



第 1 帧：`a` 的 `data_` 指向一块堆内存。第 2 帧：`b` 拷贝构造后指向**另一块**内存，两个对象互不影响。第 3 帧：`c` 移动构造后直接接管 `a` 的指针，`a.data_` 被置空；此后 `a` 的析构不会 `delete[]` 任何东西。**本例的移动构造是「接管指针 + 源指针置空」；这是 `Buffer` 自己的约定。** 标准库只承诺 moved-from 有效但未指定，用户类型的 moved-from 状态由它自己的移动操作规定。移动不是「把字节复制过去再删掉」。

为什么移动构造标了 `noexcept`，而拷贝构造没标？因为移动构造只做指针赋值与置空，不会分配内存，天然不会抛；拷贝构造要 `new char[]`，可能抛 `std::bad_alloc`。这个标注意味着「移动一定成功」，`vector` 扩容时才敢放心地移动；4.8 节会用 `CopyFallback` 演示不标 `noexcept` 的后果。

4.7 浅拷贝：默认拷贝构造会把两个对象绑在同一块内存上

把 `Buffer` 的五个函数全部删掉，只留下构造函数与析构函数，会得到什么？默认拷贝构造会逐成员复制——对 `char* data_` 来说，复制的是**指针值**，不是它指向的字符。于是两个 `Buffer` 的 `data_` 指向同一块内存，两个析构函数各 `delete[]` 一次。

cpp 默认拷贝构造：两个对象，一次 new，两次 delete examples/ch02/shallow_copy_bug/src/shallow_copy_bug.cpp 第 15–25 行

```cpp
void run_shallow_copy_bug() {
    std::cout << "before copy\n" << std::flush;

    {
        Shallow a(7);
        Shallow b = a;      // 默认拷贝构造：两个对象共用一个 int
        std::cout << "b=" << b.value() << "\n" << std::flush;
```

```
    }                       // 两个析构函数各 delete 一次

    std::cout << "after scope\n";
}
```

终端输出 终端输出 shallow-bug-out

```
before copy
b=7
```

本机退出码是 **3221226356**（Windows 的堆损坏），stderr 为空；程序在第二个 `delete` 时被杀掉，所以 `after scope` 不会打印。这是**未定义行为**：标准不保证它一定崩、也不保证崩在哪一行。你能依赖的只有「默认拷贝构造对裸拥有指针是浅拷贝」这一条语言规则，以及「两个析构函数都释放同一块内存」这个因果链。

为什么 `before copy` 和 `b=7` 能留下来？因为代码在这两行后面显式 `std::flush` 了；如果只写 `\n`，标准输出可能还在缓冲区里，进程被终止时整段丢失。**崩溃程序的 stdout 不是可靠的证据**，这也是本单元把退出码与输出分开记录的原因：退出码证明「它死了」，输出证明「死之前走到了哪一步」，两者不能互相替代。

修这个 bug 有三条路：一是按 Rule of Five 手写深拷贝（本单元 `rule_of_five`）；二是把 `int*` 换成 `std::unique_ptr<int>`，让所有权不可复制（第 5 单元）；三是明确禁止拷贝（`Shallow(const Shallow&) = delete;`），只允许移动。三条路的共同点是「让所有权关系在类型里显式化」，而不是靠调用者记得「这个对象不能复制」。默认拷贝构造不会替你思考所有权，它只会逐成员复制。

这个 bug 在容器里更隐蔽：`std::vector<Shallow> v; v.push_back(a);` 会拷贝构造元素；`v` 扩容时又会拷贝/移动已有元素。如果 `Shallow` 的拷贝是浅拷贝，容器里就会出现多个「拥有同一块内存」的对象，任何一个先析构都会让其余的变成悬空指针。**容器不会替你修拷贝语义，它只是按类型提供的拷贝/移动操作行事。** 所以在把自定义类型放进容器之前，先问一句：它的拷贝构造和移动构造到底做了什么？

接口设计也受这条规则影响：一个函数如果只是「看一眼」对象，用 `const T&`；如果它要保留一份自己的副本，用按值传参；如果它要接管调用者的资源，用 `T&&`（并在文档/命名里说清楚「调用后源对象不再使用」）。**参数类型是所有权契约的一部分，不是随便挑一个能编过的写法。** 你把参数写成 `T&&`，调用者就会以为可以 `std::move` 进来；你写成 `const T&`，调用者就知道你只是借用。C++ 的移动语义把「所有权」放到了签名里，读签名的人应该能看出这件事。

Rule of Three 是 Rule of Five 的前身：如果你需要自定义析构、拷贝构造、拷贝赋值中的任何一个，通常三个都要一起考虑；C++11 之后多了移动构造与移动赋值，变成 Rule of Five。Rule of Zero 则是「资源都交给 RAII 成员」的写法。三条规则不是三个教条，而是同一个问题的三种答案：**谁拥有资源、谁负责释放、复制/移动时所有权怎么变。**

> ⚠ **陷阱** · **`std::move` 不移动，moved-from 也不是空**
>
> 两句话都反直觉，但都重要：
>
> 1. **`std::move(x)` 不移动任何东西。**它只是把 `x` 转成右值引用；如果目标类型没有移动构造，或者重载决议选了拷贝构造，`std::move` 之后仍然是一次拷贝。判断「有没有移动」要看计数，不要看 `std::move` 这三个字。
> 2. **moved-from 不是空。**本机 `std::string` 被移动后 `size()==0`，所以很多人以为「移动 = 清空」；标准只说「有效但未指定」。换一个标准库实现，moved-from 的 `string` 可能保留内容。**不要依赖读到的值；读本身合法，值未指定。** 要给对象赋新值之前，别拿旧值做判断。

4.8 `noexcept` 与 vector 扩容：为什么移动构造要标 `noexcept`

`std::vector` 扩容时要把旧元素搬到新缓冲区。标准给它的异常安全承诺有一个条件：如果搬移过程中抛异常，旧缓冲区要保持原样——但对**不可拷贝**且移动可能抛异常的类型，这个承诺不成立（没有安全退路）。libstdc++ 的做法是 `std::move_if_noexcept`：移动构造 `noexcept` 就移动；否则如果有拷贝构造就拷贝；如果只能移动，就只能移动。`std::move_if_noexcept` 是实现手法，不是标准强制。

cpp **MoveOnly 的移动构造是 noexcept：扩容走移动** examples/ch02/vector_realloc/src/vector_realloc.cpp 第 11–21 行

```
void run_vector_realloc() {
    {
        std::vector<MoveOnly> v;
        v.reserve(1);
        v.emplace_back();
```

```
        for (int i = 0; i < 4; ++i) {
            v.emplace_back();
        }
        std::cout << "grow: moves=" << MoveOnly::moves
                  << " copies=" << MoveOnly::copies << "\n";
    }
```

cpp **CopyFallback 的移动构造没有 noexcept：扩容退化成拷贝** examples/ch02/vector_realloc/src/vector_realloc.cpp 第 36–45 行

```
    {
        std::vector<CopyFallback> v;
        v.reserve(1);
        v.emplace_back();
        for (int i = 0; i < 4; ++i) {
            v.emplace_back();
        }
        std::cout << "fallback: moves=" << CopyFallback::moves
                  << " copies=" << CopyFallback::copies << "\n";
    }
```

终端输出 终端输出 vector-realloc-out

```
grow: moves=7 copies=0
reserve: moves=0 copies=0
fallback: moves=0 copies=7
```

本机输出三行：`grow: moves=7 copies=0`、`reserve: moves=0 copies=0`、`fallback: moves=0 copies=7`。本机 libstdc++ 按倍增扩容，`grow` 的 7 次移动来自容量 1→2→4→8 的三次扩容（1+2+4）；标准只保证 `capacity() >= size()`，换实现扩容次数可能不同。`reserve` 提前把容量留到 8，所以一次都不用搬；`fallback` 的移动构造没有 `noexcept`，`vector` 选择了拷贝，7 次移动变成 7 次拷贝。**`noexcept` 不是装饰，它直接改变容器选择哪条路。**

这里要小心不要把「`noexcept` 移动构造」当成无条件的好事。`noexcept` 是一个**承诺**：函数不会让异常逃出去。如果你的移动构造真的可能抛（例如它要分配内存、调用可能抛的函数），却硬标 `noexcept`，异常会直接触发 `std::terminate`，程序当场死掉——比「不标 `noexcept`、让 `vector` 选择拷贝」更糟。标准库容器的 `noexcept` 移动构造（`std::string`、`std::vector`、`std::unique_ptr`）之所以敢标，是因为它们内部只做指针/缓冲区的接管，不分配新资源。**先保证移动构造真的不抛，再标 `noexcept`。**

`reserve` 的对照也值得记住：`grow` 的 7 次移动不是因为「`vector` 喜欢搬」，而是因为容量不够、必须扩容。你提前 `reserve(8)`，`vector` 一次都不用搬。**能预估大小时先 `reserve`，比事后怪移动构造更有效。** 当然，`reserve` 也可能一次分配很大块内存，预估过大同样浪费；这是工程取舍，不是语法规则。

最后要分清「计数」与「性能」：本单元数的是**构造/移动/拷贝调用了多少次**，不是耗时。7 次移动可能比 7 次拷贝便宜，也可能因为对象很小而差不多；耗时随机器、负载、缓存状态变化，计数才是稳定、可复现的证据。**要谈性能，先固定测量口径；本书的 `experiment` 只报计数、退出码、地址这类可复现读数，不写「快了多少倍」这种没有对照的结论。** 这也是「标准要求 / 本机实测 / 工程惯例」三分里，本机实测只对自己这台机器负责的原因。

扩容还会让**迭代器、指针、引用失效**：`vector` 换了一块缓冲区，原来的迭代器指向旧内存，继续用就是悬空。`reserve` 只保证「在容量范围内不再重新分配」，不保证「已有的迭代器永远有效」；`shrink_to_fit` 则可能触发一次收缩，同样让迭代器失效。C 分册讲 `realloc` 时说过「地址可能变」，C++ 的 `vector` 是同一个道理，只是它把「搬元素」包装成了移动构造。**任何可能导致扩容的操作之后，都不要假设旧迭代器还指着原来的元素。**

那 `noexcept` 移动构造是不是越多越好？不是。它是「我保证不抛」的承诺，不是「写了就更快」的优化开关。只有当你确定移动构造不会抛（只做指针/句柄接管），才标 `noexcept`；不确定就不标，让 `vector` 走拷贝退路。**把可能抛的移动硬标 `noexcept`，等于把异常变成 `std::terminate`，比拷贝更危险。**

`shrink_to_fit()` 也值得提一句：它是「请求」而不是「命令」，实现可以忽略；即使实现接受，它也可能重新分配一块更小的缓冲区，让所有迭代器、指针、引用失效。**不要因为「容量看起来太大」就随手 `shrink_to_fit`，先确认没有人在用旧迭代器，再确认这次收缩真的值得。** 这是 C 分册 `realloc` 的同一个教训：地址变了，所有指向旧地址的东西都作废。

`resize` 与 `reserve` 也经常被混用：`resize(n)` 改变的是 **size**，会构造/析构元素，之后 `v[n-1]` 是合法元素；`reserve(n)` 改变的是 **capacity**，不构造任何元素，之后 `v.size()` 可能还是 0。**要「提前留空间」用 `reserve`，要**

「真的有 n 个元素」用 `resize`。 这两个操作都可能触发重新分配（`resize` 超过当前 capacity 时），所以迭代器失效的规则同样适用。

实验记录 · 本机实测：计数表

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

场景	读数
`Tracker` 一次拷贝构造 + 一次拷贝赋值	`copies=1 assignments=1`
`rvo_counter` 默认	`ctors=1 copies=0 moves=0`
`rvo_counter` + `-fno-elide-constructors`	`ctors=1 copies=0 moves=1`
`Buffer` 的五个函数都走到（拷贝构造两次）	`ctors=3 copy_ctors=2 move_ctors=1 copy_assigns=1 move_assigns=1 dtors=6`
`vector<MoveOnly>` 从 reserve(1) 长到 5 个	`moves=7 copies=0`
`vector<MoveOnly>` 先 reserve(8)	`moves=0 copies=0`
`vector<CopyFallback>`（移动构造无 noexcept）	`moves=0 copies=7`

这些数字是这台机器、这一版标准库的读数；换实现可能不同。能当结论的是**关系**：在本机默认档，NRVO 让计数为 0；关掉可选省略后多一次移动。对可拷贝类型，`noexcept` 移动构造让 `vector` 敢移动。

4.9 两个 `.cpp` 都 include 同一个头文件：谁看得见什么

`copy_basics` 的 `main.cpp` 与 `copy_basics.cpp` 都 `#include "copy_basics.h"`。头文件里只有 `run_copy_basics()` 的**声明**，定义在 `copy_basics.cpp` 里。两个翻译单元各自编译，链接器把 `main.cpp` 里的调用接到 `copy_basics.cpp` 的定义上。



第 1 帧：两个 `.cpp` 都看见了声明，所以编译期都知道 `run_copy_basics` 这个名字和签名。第 2 帧：定义只出现在 `copy_basics.cpp` 里，链接器把 `main.cpp` 的调用接到它上面。如果头文件里写函数**定义**（而不是声明），两个翻译单元就会各生成一份，链接期报重复定义——这是 C++ 分册第 20 单元（ODR）要展开的事。本单元只要求你记住：**头文件放声明，定义放 `.cpp`；模板是例外，它的定义必须让使用它的翻译单元看见。**

为什么模板是例外？因为模板本身不是「一份可链接的代码」，它要等实参类型确定后才实例化；编译器在使用点必须看得见完整定义，才能生成那一份实例。C 分册的「声明放头文件、定义放 `.cpp`」在这里仍然成立，只是模板把「定义」也推到了头文件里。这个差异在第 8 单元（模板入门）会再用真实链接错误讲一次。

头文件里除了声明，还可以放 `inline` 函数、`constexpr` 变量、类定义、模板定义；不能放普通函数定义、普通全局变量定义（会违反 ODR，链接期重复定义）。C++17 起可以用 `inline` 变量在头文件里定义全局变量，这是第 9 单元（多文件工程与 ODR）的内容。本单元只要求你记住一条：**头文件会被多个翻译单元各包含一次，任何「只能有一份」的东西都要小心。** 这也是为什么 `Buffer` 的成员函数定义放在 `.cpp`，头文件里只有类定义与函数声明。

`inline` 在 C++ 里的含义和 C 不一样：C 的 `inline` 只是「建议内联」，C++ 的 `inline` 还附带「允许在多个翻译单元里出现同一份定义」的链接语义。所以头文件里的 `inline` 函数不会重复定义，普通函数会。这个区别在第 9 单元会

展开；本单元只要记住：**看到头文件里的函数定义，先问它是不是 `inline` / 模板 / 类内定义，否则链接期可能炸。**

4.10 大白话：为什么 `return std::move(x)` 反而更慢

大白话 · 用大白话说：搬家与打包

把返回局部对象想成搬家。默认的 RVO/NRVO 是「新房就是按你的旧房布局盖的，家具直接在新房摆好」——一次搬运都没有。`std::move(local)` 则是「先把家具打包，再搬到新房」——多了一次打包和拆包。编译器本来能直接在新房摆好，你非要在中间加一道打包工序，于是它只好移动一次。

再换个角度：`std::move` 不是「搬家公司的卡车」，它更像「在箱子上贴一张『可搬』的标签」。真正搬箱子的是移动构造。你在 `return` 里贴标签，等于告诉编译器「这个箱子可以搬」；可编译器本来就能把整个房间直接盖在新地址上，你贴标签反而让它只能搬箱子。**标签不是搬运，别把手段当目的。**

更反直觉的是：**`std::move` 不保证「搬」**。如果目标类型没有移动构造，`std::move` 之后仍然调用拷贝构造；如果编译器做了省略，`std::move` 甚至会让省略失效。判断有没有搬，不要看代码里写没写 `std::move`，要看计数（本单元的 `rvo_counter` 与 `vector_realloc` 就是干这个的）。**标签贴得再多，也代替不了搬家公司的车；而 C++ 的「车」是移动构造，不是 `std::move`。**

4.11 写自己的资源类时的检查清单

这一节没有新例子，只有一张可以带走的检查表。下次你写一个「拥有资源」的类，按顺序问自己：

1. **它真的拥有资源吗？** 如果成员是 `std::string`、`std::vector`、`std::unique_ptr` 这类 RAII 类型，默认的拷贝/移动会正确调用成员的操作，通常不需要手写五个函数（Rule of Zero）。只有裸指针、文件描述符、锁这类「析构时必须手动释放」的成员，才需要你接管。
2. **默认拷贝做了什么？** 对裸拥有指针，默认拷贝是浅拷贝：两个对象共用一个资源，析构两次就 double free。先写出这个反例，再决定是深拷贝、禁拷贝还是换 RAII 成员。
3. **五个函数是否一致？** 析构释放资源；拷贝构造分配并复制；移动构造接管资源并把源对象置空；拷贝赋值处理旧资源与自赋值；移动赋值释放旧资源、接管、置空。五个函数共享一个不变量：**每个非空资源只有一个所有者。**
4. **自赋值和异常安全怎么办？** `a = a;` 与 `a = std::move(a);` 都必须安全。copy-and-swap 用一次临时对象换取异常安全；手写释放/接管则要保证「先释放再读源对象」的顺序不会踩到自赋值。
5. **`noexcept` 是真实的吗？** 只有移动构造确实不抛（只接管指针/句柄）时才标 `noexcept`；标了却抛，异常会变成 `std::terminate`。不标 `noexcept` 的移动构造会让 `vector` 在可拷贝时退回拷贝，这是安全的选择，不是失败。
6. **怎么验证？** 用计数（拷贝/移动/析构各几次）、地址（两个对象是否指向同一块）、退出码（double free 的本机退出码）和调试器（gdb 的 `bt`）拿证据。本机没有 ASan/UBSan，所以「我觉得不会泄漏」不算证据，能复现的计数/退出码才算。

这张表也可以反过来用：读别人的资源类时，按这六条逐一对照，缺哪条、为什么缺，通常就是 bug 或未定义行为的藏身处。

移动也不是默认选择：需要保留原值、源对象是 `const`、类型没有移动构造、移动构造其实会抛却标了 `noexcept`、或者你根本不确定所有权归谁——这些情况下拷贝或引用更安全。**移动是优化，不是义务；先保证语义正确，再谈少一次构造。**

小结 · 这一节要带走的五件事

- 拷贝构造与拷贝赋值是两个不同入口：前者建新对象，后者改写已有对象。
- `std::move` 只是 `static_cast<T&&>`，不搬任何东西；真正搬资源的是移动构造 / 移动赋值。
- moved-from 是「有效但未指定」，不是空；本机读数不能当标准保证。
- 从 C++17 起，调用点的纯右值初始化有强制拷贝省略；NRVO 仍是可选优化。`return std::move(local)` 会阻止 NRVO，反而多一次移动，`-Wpessimizing-move` 会点名。
- 资源类要按 Rule of Five 想清楚五个特殊成员函数；对可拷贝类型，移动构造标 `noexcept` 会让 `vector` 扩容选择移动而不是拷贝（只能移动的类型没有退路）。

自测题 · 自测题（先自己做，再对答案）

1. `Tracker b = a;` 与 `c = a;` 分别调用哪个函数？各把哪个计数加一？
2. `std::move(s)` 之后，`s` 和 `std::move(s)` 是同一个对象吗？`std::move` 到底做了什么？
3. 本机 `std::string s = "hello"; std::string t = std::move(s);` 之后 `s.empty()` 为 `true`；能不能据此断言「所有实现的 moved-from string 都是空」？标准怎么规定的？
4. 为什么 `Buffer` 的 `copy_ctors` 在「一次拷贝构造 + 一次拷贝赋值」的场景里是 2 而不是 1？
5. `vector<CopyFallback>` 扩容时为什么选择拷贝而不是移动？把 `CopyFallback` 的移动构造加上 `noexcept` 后，你预期计数怎么变？

注 · 自测答案

1. `Tracker b = a;` 调用拷贝构造，`copies` 加一；`c = a;` 调用拷贝赋值，`assignments` 加一。前者建新对象，后者改写已有对象。注意 `T b = a;` 虽然长得像赋值，语法上是**初始化**，所以走拷贝构造，不走拷贝赋值。
2. 是同一个对象。`std::move(s)` 只是 `static_cast<std::string&&>(s)`，产生一个右值引用表达式；它不构造新对象、不搬字符。`same address: true` 就是证据。`describe(std::move(s))` 之后 `s` 仍然可以是 `"abc"`，因为 `describe` 只是接收了右值引用，并没有真正移动它。
3. 不能。标准只说 moved-from 的标准库对象处于「有效但未指定」状态（`[lib.types.movedfrom]`）；本机 libstdc++ 恰好把短串清空，换实现或换长串都可能不同。能依赖的是「还能安全析构 / 赋值 / `clear()`」，不是具体值。`std::unique_ptr` 是另一种情况：它的规范明确保证 moved-from 为 `nullptr`——所以要查具体类型，不要拿 `string` 的读数套所有类型。
4. 拷贝赋值 `d = b;` 内部用 copy-and-swap 构造了临时 `Buffer tmp(other)`，这次临时对象的构造也是一次拷贝构造，所以 `copy_ctors` 是 2；多出来的一次是异常安全实现的代价，不是计数错误。如果改成「先释放旧资源、再逐成员复制」的写法，拷贝构造次数会变少，但自赋值和异常安全要另外处理。
5. 因为 `CopyFallback` 的移动构造没有 `noexcept`，而它有拷贝构造；对可拷贝类型，`vector` 为守住异常安全承诺，用 `std::move_if_noexcept` 选择了拷贝。把移动构造标成 `noexcept` 后，扩容应改成移动，预期 `moves=7 copies=0`（与 `MoveOnly` 相同）；只能移动的类型没有拷贝退路，不会因为移动可能抛异常就改拷贝。标 `noexcept` 之前要确认移动真的不会抛，否则异常会变成 `std::terminate`。

智能指针与所有权：unique_ptr、shared_ptr、weak_ptr

上一单元的 `FileHandle` 与 `Buffer` 都在手写 `new` / `delete`。手写没有错，但它把"谁负责释放"藏在注释与约定里：拷贝一下就多一个释放者，传个指针就不知道对方会不会删。智能指针把所有权写进类型：`unique_ptr` 独占、`shared_ptr` 共享、`weak_ptr` 只观察不拥有。类型说清了所有权，析构函数就能按规则自动释放。

这一单元不教并发。智能指针常被当作"线程安全的万能药"，但 `shared_ptr` 只保证引用计数的更新是安全的，不保证它指向的对象线程安全；并发不在本书范围内，这里只把边界说清，不展开。

学习目标 · 看完这一单元，你应该能

- 说出独占、共享、观察三种所有权各自由哪个类型表达，以及它们各自能不能拷贝；
- 用 `unique_ptr` 管理一个对象，解释 `reset()`、`get()`、移动与 `make_unique` 各做什么；
- 用 `use_count()` 的读数证明 `shared_ptr` 的共享计数，并说清控制块与对象是两件事；
- 画出一个 `shared_ptr` 循环引用，解释为什么析构计数不归零，并用 `weak_ptr::lock` 修好；
- 为一个 C 风格资源写自定义删除器，并说出 `shared_ptr` 不该被当成默认选择的原因。

5.1 先看工程长什么样

这一单元继续在 `examples/ch02/` 里加六个多文件小工程。每个目录仍然是 `include/<例名>.h` + `src/<例名>.cpp` + `src/main.cpp` + `<例名>.json`；第 2 章起硬规定③要求每个清单至少两个 `.cpp`，且至少一个 `.h` 被 `#include`。

目录树 **工程：examples/ch02/ 这一单元新增的六个例子**

第 1 步 先建 unique_ptr 的基本用法

```
examples/
└── ch02/
    ├── 上一单元 ： cpp03 的六个例子
    └── unique_basic/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── unique_basic.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── unique_basic.cpp ← 本课新增
        └── unique_basic.json ← 本课新增
```

第 2 步 加上“不能拷贝”的反例与引用计数

```
examples/
└── ch02/
    ├── unique_basic/ ： 同上
    ├── unique_move/ ← 本课新增
    └── shared_count/ ← 本课新增
```

第 3 步 最后加上循环引用、工厂与删除器

```
examples/
└── ch02/
    ├── 前三个例子 ： 同上
    ├── weak_cycle/ ← 本课新增
    ├── factory_unique/ ← 本课新增
    └── custom_deleter/ ← 本课新增
```

第 1 帧的 `unique_basic/` 用 `make_unique` 创建一个 `Widget`，`reset()` 之后立刻看到析构输出。第 2 帧的 `unique_move/` 故意拷贝一个 `unique_ptr`，让编译失败成为“独占”的证据；`shared_count/` 用 `use_count()` 展示共享计数。第 3 帧的 `weak_cycle/` 先造一个循环引用（计数不归零），再用 `weak_ptr` 修好；`factory_unique/` 用工厂函数返回 `unique_ptr` 并把它移进 `vector`；`custom_deleter/` 用 `malloc` / `free` 展示删除器。

这六个例子各自回答一个问题：`unique_basic` 回答“独占指针什么时候释放”；`unique_move` 回答“为什么不能拷贝”；`shared_count` 回答“共享计数什么时候加减”；`weak_cycle` 回答“循环引用为什么泄漏、`weak_ptr` 怎么破”；`factory_unique` 回答“工厂怎样交出所有权”；`custom_deleter` 回答“不是 `new` 出来的资源怎么交给智能指针”。

5.2 所有权模型：独占、共享、观察

先看三种所有权在类型上的区别：

所有权	类型	能否拷贝	谁负责释放	典型场景
独占	`std::unique_ptr<T>`	不能，只能移动	指针的持有者	工厂返回值、容器里的对象、RAII 成员
共享	`std::shared_ptr<T>`	能，计数加一	最后一个持有者	多处共享同一对象
观察	`std::weak_ptr<T>`	能，不计数	不负责释放	打破循环、缓存、观察者

`unique_ptr` 的“独占”不是口头约定，而是类型系统强制的：它删除了拷贝构造与拷贝赋值，只保留移动。你把 `unique_ptr` 按值传给函数，必须写 `std::move`，所有权在代码里看得见。

cpp **unique_basic: 创建、使用、reset** examples/ch02/unique_basic/src/main.cpp 第 5–12 行

```cpp
int main() {
    auto widget = make_widget("w");
    std::cout << "name=" << widget->name() << "\n";

    widget.reset();
    std::cout << "empty=" << (widget ? "no" : "yes") << "\n";
    return 0;
}
```

终端输出 终端输出 unique-basic-out

```
ctor w
name=w
dtor w
empty=yes
```

`make_widget` 内部用 `std::make_unique<Widget>("w")` 创建对象；`widget` 独占它，`widget->name()` 通过 `operator->` 访问成员。`widget.reset()` 释放当前对象（打印 `dtor w`），把指针置空；之后 `widget` 在布尔判断里为假，输出 `empty=yes`。**`reset()` 是“立刻释放并接管新对象或置空”，不是“重置对象的值”。** 如果你只想清空指针、保留对象，应该 `release()`，但 `release()` 会把释放责任交回给你，通常不是好主意。

`get()` 返回裸指针，用来传给只观察、不拥有的旧接口；它不转移所有权，也不延长生命周期。**`get()` 的返回值只在你持有 `unique_ptr` 期间有效**；把 `get()` 存起来，等 `unique_ptr` 析构后再用，就是悬空指针。

{} 语法规则 · 智能指针的最小语法

- 创建：`auto p = std::make_unique<T>(args...);` / `auto s = std::make_shared<T>(args...);`
- 访问：`p->member`、`*p`、`p.get()`（裸指针，不转移所有权）
- 释放：`p.reset()`（立刻析构并置空）、`p = nullptr;`（等价于 `reset()`）
- 转移：`auto q = std::move(p);`（`p` 变为空，`q` 独占）
- 观察：`std::weak_ptr<T> w = s;`、`w.lock()`（成功返回 `shared_ptr`，失败返回空）

5.3 为什么不能拷贝：独占的代价与保证

cpp **unique_move: 这一行拷贝必须被拒绝** examples/ch02/unique_move/src/main.cpp 第 5–9 行

```cpp
int main() {
    std::unique_ptr<int> a = make_value();
    std::unique_ptr<int> b = a;
    return *b;
}
```

编译诊断 本机真实 stderr：unique_ptr 的拷贝构造被删除 unique-move-diag

```
src/main.cpp: In function 'int main()':
src/main.cpp:7:30: error: use of deleted function 'std::unique_ptr<_Tp, _Dp>::unique_ptr(const
std::unique_ptr<_Tp, _Dp>&) [with _Tp = int; _Dp = std::default_delete<int>]'
    7 |     std::unique_ptr<int> b = a;
      |                              ^
In file included from C:/msys64/ucrt64/include/c++/16.1.0/memory:80,
                 from include/unique_move.h:4,
                 from src/main.cpp:1:
C:/msys64/ucrt64/include/c++/16.1.0/bits/unique_ptr.h:524:7: note: declared here
  524 |       unique_ptr(const unique_ptr&) = delete;
      |       ^~~~~~~~~~
src/main.cpp:7:30: note: use '-fdiagnostics-all-candidates' to display considered candidates
    7 |     std::unique_ptr<int> b = a;
      |                              ^
```

诊断的核心是 `use of deleted function`：`unique_ptr` 的拷贝构造被显式删除了。为什么删除？如果允许拷贝，两个 `unique_ptr` 会持有同一个指针，两个析构函数都会 `delete` 它，结果就是我们上一单元看到的双重释放。**“独占”不是性能优化，而是正确性约束：一个资源只能有一个释放者。** 要转移所有权，写 `std::unique_ptr<int> b = std::move(a);`：移动之后 `a` 变为空，`b` 接管。

`unique_ptr` 的移动后状态是“空”，这一点与 `std::string` / `std::vector` 的“有效但未指定”不同——`unique_ptr` 不拥有任何东西时，布尔判断为假，`get()` 返回 `nullptr`。`a` 仍然是一个有效的 `unique_ptr` 对象，你可以对它赋值、`reset()`，或重新让它接管新对象；但不要在没有重新赋值前解引用它。

`unique_ptr` 还有一个数组特化 `std::unique_ptr<T[]>`，它会对数组调用 `delete[]`。用 `std::make_unique<int[]>(n)` 创建，不能对单个对象用数组特化，也不能混用 `delete` / `delete[]`。本书的例子不需要数组版本，但你应该知道它存在：**删除器必须与分配方式匹配**，这是自定义删除器要解决的问题。

所有权也应该写在函数签名上。**要接管所有权，按值收** `std::unique_ptr<T>`：调用者必须 `std::move`，转移意图明确；函数内部负责释放。**只观察不接管，收** `T*` **或** `T&`，不要收 `const unique_ptr<T>&` 再从中 `get()`——那会让接口看起来"拥有"，实际只是借用。**要共享所有权，按值收** `std::shared_ptr<T>`，计数会加一；如果只是短期观察，收 `const std::shared_ptr<T>&` 可以避免计数增减，但不会延长对象寿命。

`std::make_shared<T>` 通常把对象与控制块放在一次分配里，比 `std::shared_ptr<T>(new T)` 少一次分配，也避免"对象构造成功、控制块分配失败"的异常安全问题。**优先用** `make_shared`；只有需要自定义删除器、或对象已经存在时，才用 `shared_ptr` 的其它构造函数。`shared_ptr` 的引用计数是线程安全的，但指向的对象本身不是线程安全的；多线程读写同一个对象仍要加锁。

`enable_shared_from_this<T>` 让对象在"已经由 `shared_ptr` 管理"时，安全地产生指向自己的 `shared_ptr`。**不要在构造函数里调用** `shared_from_this()`：那时对象还没交给 `shared_ptr`，控制块没建立，会抛 `std::bad_weak_ptr`。常见用法是在成员函数里返回 `shared_from_this()`，把 `this` 交给异步任务；如果对象不是由 `shared_ptr` 管理，应该用 `weak_from_this()` 检查，而不是假设。

5.4 `shared_ptr`：共享所有权与引用计数

`shared_ptr` 允许多个指针共享同一个对象。它内部有两个指针：一个指向对象，一个指向**控制块**；控制块里存引用计数与删除器。拷贝一个 `shared_ptr` 让计数加一，销毁一个让计数减一；计数归零时，控制块调用删除器销毁对象。

cpp **shared_count：拷贝让计数加一，离开作用域让计数减一** examples/ch02/shared_count/src/main.cpp 第 5–18 行

```cpp
int main() {
    auto a = make_item("shared");
    std::cout << "a=" << a.use_count() << "\n";

    auto b = a;
    std::cout << "b=" << b.use_count() << "\n";

    {
        auto c = b;
        std::cout << "c=" << c.use_count() << "\n";
    }
    std::cout << "after=" << a.use_count() << "\n";
    return 0;
}
```

终端输出 终端输出 shared-count-out

```
ctor shared
a=1
b=2
c=3
after=2
dtor shared
```

`a=1` 是唯一持有者；`auto b = a;` 之后 `b=2`；块内 `auto c = b;` 之后 `c=3`；块结束，`c` 析构，`after=2`；`main` 结束，`b` 与 `a` 依次析构，计数归零，对象销毁，打印 `dtor shared`。**`use_count()` 是观察读数，不是同步原语；不要用它写"计数等于 1 就安全"的逻辑，因为别的线程/别的持有者随时可能改变它。**

内存形态 **shared_ptr 的两个部分：对象与控制块**

第 1 步 **一个 shared_ptr 持有对象**

a : shared_ptr<Item> | a.ptr : 指向 Item | a.ctrl : 控制块，强引用计数 = 1

a.ptr → item Item("shared")

第 2 步 **拷贝后两个 shared_ptr 共享控制块**

a : shared_ptr<Item> | b : shared_ptr<Item>

a.ctrl ↔ b.ctrl 同一个控制块，计数 = 2
a.ptr → item 同一个对象
b.ptr → item 同一个对象

第 1 帧里对象与控制块是两个不同的内存块；第 2 帧的 `b` 复制了 `a` 的两个指针，计数从 1 变 2。**“共享对象”不等于“共享指针变量”**：`a` 与 `b` 各自有一个 `shared_ptr` 对象，它们只是都指向同一个控制块与同一个资源。控制块记录“还有几个强引用”，最后一个强引用离开时，对象析构；弱引用不增加强引用计数，但会让控制块多活一会儿（`weak_ptr` 要用它判断对象是否还在）。

`std::make_shared<T>` 通常把对象与控制块分配在同一块内存里，少一次分配；`std::shared_ptr<T>(new T)` 则是两次分配。这个差别是**实现与性能**层面的，不影响语义；本书的例子统一用 `make_shared`。`shared_ptr` 也可以接管 `new` 出来的裸指针，但不要用同一个裸指针构造两个独立的 `shared_ptr`——那会得到两个控制块，计数各算各的，最后还是双重释放。

5.5 循环引用：计数为什么不归零

`shared_ptr` 的计数只在“强引用”消失时减一。如果两个对象互相持有对方的 `shared_ptr`，它们就互为强引用：即使外部所有指针都离开作用域，两个对象的计数仍然各为 1，永远不会归零。

cpp **Node：一个强引用 next，一个弱引用 weak_next** examples/ch02/weak_cycle/include/weak_cycle.h 第 7–20 行

```cpp
class Node {
public:
    explicit Node(std::string name);
    ~Node();

    const std::string& name() const;

    std::shared_ptr<Node> next;
    std::weak_ptr<Node> weak_next;
    static int alive;

private:
    std::string name_;
};
```

cpp **第 1 块：两个对象互相强引用** examples/ch02/weak_cycle/src/main.cpp 第 6–16 行

```cpp
int main() {
    std::cout << "start=" << Node::alive << "\n";

    {
        auto a = std::make_shared<Node>("a");
        auto b = std::make_shared<Node>("b");
        a->next = b;
        b->next = a;
        std::cout << "cycle_alive=" << Node::alive << "\n";
    }
    std::cout << "after_cycle=" << Node::alive << "\n";
```

前半段先制造循环：两个 `shared_ptr` 成员互相指着对方。下面是同一份 `main` 的后半段，把强引用的那一条边换成 `weak_next`，其余结构不变。

cpp **（续）第 2 块：用 weak_ptr 打破循环** examples/ch02/weak_cycle/src/main.cpp 第 18–33 行

```cpp
    std::weak_ptr<Node> observer;
    {
        auto a = std::make_shared<Node>("c");
        auto b = std::make_shared<Node>("d");
        a->weak_next = b;
        b->weak_next = a;
        if (auto locked = a->weak_next.lock()) {
            std::cout << "lock=" << locked->name() << "\n";
        }
        std::cout << "weak_alive=" << Node::alive << "\n";
        observer = a->weak_next;
    }
    std::cout << "after_weak=" << Node::alive
              << " expired=" << observer.expired() << "\n";
    return 0;
```

```
33  }
```

终端输出 终端输出 weak-cycle-out

```
start=0
cycle_alive=2
after_cycle=2
lock=d
weak_alive=4
after_weak=2 expired=1
```

第一段：`a->next = b; b->next = a;` 之后，`a` 与 `b` 的强引用计数各为 2（外部一个 + 对方一个）。块结束时，外部两个 `shared_ptr` 析构，计数各降到 1，两个对象仍然活着，所以 `after_cycle=2` ——**这就是内存泄漏：没有指针能再访问它们，但它们的析构函数不会执行。**

第二段：`weak_next` 是 `weak_ptr`，赋值不增加强引用计数。块内 `a->weak_next.lock()` 成功，拿到一个临时的 `shared_ptr`，可以读 `name()`；块结束时 `a`、`b` 的强引用计数归零，两个对象析构，`after_weak` 回到 2（只剩第一段泄漏的两个），`observer.expired()` 为 1。**`weak_ptr` 不延长对象寿命；它只是“知道对象在哪，需要时尝试升级为 `shared_ptr`”。**

`weak_ptr` 的典型用途就是打破循环、实现缓存与观察者：父节点持有子节点的 `shared_ptr`，子节点持有父节点的 `weak_ptr`；缓存持有对象的 `weak_ptr`，对象被释放后缓存项自动失效。**`weak_ptr` 不能直接解引用**，必须先 `lock()` 或 `shared_ptr` 构造，因为对象可能已经不存在。

> ⚠ **陷阱** · **`shared_ptr` 能解决一切所有权问题**
>
> 你会看到：只要把成员都改成 `shared_ptr`，代码就能编过，析构也“看起来正常”。你会误判成“共享指针是更安全的指针”，于是所有对象都用 `shared_ptr`。真相是：共享所有权把“谁负责释放”变成了“最后一个离开的人负责”，循环引用时没有人是最后一个；而且共享计数带来额外的控制块与原子操作。默认应该用 `unique_ptr` 或值语义；只有确实需要多处共享生命周期时，才用 `shared_ptr`，并想清楚谁持有谁、哪里需要 `weak_ptr`。

5.6 工厂与所有权转移

工厂函数返回 `unique_ptr` 是 C++ 的常见写法：调用者拿到独占所有权，不需要知道对象是怎么创建的，也不会忘记释放。

cpp **factory_unique: 从工厂拿到 unique_ptr，再移进 vector** examples/ch02/factory_unique/src/main.cpp 第 7–17 行

```cpp
int main() {
    auto message = make_message("hello");
    std::cout << "text=" << message->text() << "\n";

    std::vector<std::unique_ptr<Message>> box;
    box.push_back(std::move(message));

    std::cout << "size=" << box.size()
              << " moved=" << (message == nullptr ? "yes" : "no") << "\n";
    return 0;
}
```

终端输出 终端输出 factory-unique-out

```
make hello
text=hello
size=1 moved=yes
drop hello
```

`make_message` 返回 `std::unique_ptr<Message>`，`main` 用 `auto` 接住；`box.push_back(std::move(message))` 把所有权移进 `vector`，`message` 变为空，所以 `moved=yes`。`vector` 销毁时，它持有的 `unique_ptr` 析构，`Message` 打印 `drop hello`。**容器里放 `unique_ptr` 而不是裸指针，容器的析构就自动负责释放所有元素；容器拷贝被禁止，但移动/`emplace_back` 仍然可用。**

图 / 搜索过程 **unique_ptr 的所有权转移**

第 1 步 **工厂创建对象，message 独占**

factory : make_message　message : unique_ptr<Message>

factory → message 返回所有权

message → object 独占

第 2 步 **move 之后，所有权到 vector**

message : 空 unique_ptr　box : vector<unique_ptr<Message>>

message → box std::move

box → object 独占

第 1 帧的所有权箭头从工厂指向 `message`；第 2 帧 `message` 变空，箭头改从 `box` 指向对象。**移动不是“复制指针”，而是“把释放责任交给另一个对象，并让原对象置空”**；这正是 `unique_ptr` 移动构造的语义。

5.7 自定义删除器：不是所有资源都来自 `new`

`unique_ptr` 的第二个模板参数是删除器。默认删除器 `std::default_delete<T>` 调用 `delete`；如果资源来自 `malloc`，就必须用 `free`，否则是未定义行为。

cpp **FreeDeleter: 把 free 包装成函数对象** examples/ch02/custom_deleter/include/custom_deleter.h 第 7–12 行

```cpp
struct FreeDeleter {
    int* freed;
    void operator()(int* value) const;
};

std::unique_ptr<int, FreeDeleter> make_malloc_int(int value, int* freed);
```

cpp **删除器里调用 free，并记一次数** examples/ch02/custom_deleter/src/custom_deleter.cpp 第 6–21 行

```cpp
void FreeDeleter::operator()(int* value) const {
    if (value != nullptr) {
        std::cout << "free " << *value << "\n";
        std::free(value);
        ++(*freed);
    }
}

std::unique_ptr<int, FreeDeleter> make_malloc_int(int value, int* freed) {
    int* raw = static_cast<int*>(std::malloc(sizeof(int)));
    if (raw == nullptr) {
        return std::unique_ptr<int, FreeDeleter>(nullptr, FreeDeleter{freed});
    }
    *raw = value;
    return std::unique_ptr<int, FreeDeleter>(raw, FreeDeleter{freed});
}
```

cpp **main: 离开作用域时删除器执行** examples/ch02/custom_deleter/src/main.cpp 第 5–13 行

```cpp
int main() {
    int freed = 0;
    {
        auto value = make_malloc_int(21, &freed);
        std::cout << "value=" << *value << " freed=" << freed << "\n";
    }
    std::cout << "after=" << freed << "\n";
    return 0;
}
```

终端输出 终端输出 custom-deleter-out

```
value=21 freed=0
free 21
after=1
```

`make_malloc_int` 用 `std::malloc` 分配，返回的 `unique_ptr` 带着 `FreeDeleter{freed}`；离开作用域时删除器调用 `free`，`freed` 变成 1。**删除器是 `unique_ptr` 类型的一部分**：`std::unique_ptr<int, FreeDeleter>` 与 `std::unique_ptr<int>` 是两种不同的类型，不能互相赋值。这既是约束，也是好处——删除方式在编译期就固定了，不会出现"忘了用对应释放函数"的运行期错误。

`shared_ptr` 也接受自定义删除器，但删除器存在控制块里，类型擦除掉了；代价是控制块更大，且删除器的类型不影响 `shared_ptr` 的类型。`make_shared` 不支持自定义删除器，因为它要把对象与控制块一起分配。**选择原则**：资源来自 `new` 就用 `make_unique` / `make_shared`；来自 C API 就写删除器；不需要共享就用 `unique_ptr`。

5.8 边界：不要把所有东西都包成智能指针

智能指针解决"动态分配对象的生命周期"，不是"所有变量的默认容器"。下面这些情况不该用智能指针：

- 对象是局部值、成员值、容器元素：直接用值语义，栈上分配，析构自动发生；
- 函数只是观察参数：用 `T&` 或 `const T&`，不要为了"安全"传 `shared_ptr`；
- 所有权明确且唯一：用 `unique_ptr`，不要用 `shared_ptr` 增加计数开销；
- 需要多态容器：可以用 `vector<unique_ptr<Base>>`，但这是第 6 单元继承之后的话题。

接口里接收 `shared_ptr`，就等于向调用方宣告"我要参与共享所有权"；如果函数只是用完就走，应该改成 `T&` 或 `const T&`，否则调用方会被迫改变自己的生命周期管理方式。**所有权语义要写在签名上，而不是藏在实现里**，这样读接口的人不必猜。

`shared_ptr` 的线程安全边界也要说清：标准保证多个线程可以同时操作**不同的** `shared_ptr` 对象，引用计数的更新是安全的；但它不保证同一个对象的内容线程安全，也不保证"先检查 `use_count()` 再使用"是原子的。**并发不在本书范围内**；这里只要求你不把 `shared_ptr` 当成"线程安全"的同义词。

还有一个常见的所有权误用：把 `this` 直接交给两个 `shared_ptr` 构造，会建立两个独立的控制块，最后双重释放。**让对象自己产生 `shared_ptr` 时，用 `enable_shared_from_this` 或工厂函数**；不要写 `std::shared_ptr<T>(this)`。所有权模型的价值就在于"释放者唯一且可追踪"，任何绕过控制块的写法都要先怀疑。

标准原文 · 标准原文：unique_ptr 独占，shared_ptr 共享

> A unique pointer is an object that owns another object and manages that other object through a pointer.
> ... each such U is Cpp17MoveConstructible and Cpp17MoveAssignable, but is not Cpp17CopyConstructible nor Cpp17CopyAssignable.
>
> The shared_ptr class template stores a pointer ... shared_ptr implements semantics of shared ownership; the last remaining owner of the pointer is responsible for destroying the object.

大意：`unique_ptr` 独占一个对象，只可移动、不可拷贝；`shared_ptr` 实现共享所有权，最后一个持有者负责销毁对象。

出处：N4950 §20.3.1.1「General」（[unique.ptr.general]）与 §20.3.2.2.1「General」（[util.smartptr.shared.general]）。

`shared_ptr` 还有一个别名构造：它可以让新的 `shared_ptr` 指向对象的某个成员，同时与原对象共享同一个控制块。这在把成员地址交给外部、又要求宿主对象保持存活时很有用；但读代码的人容易误以为新指针拥有成员本身。**所有权看控制块，不看指针指向哪里**，别名指针不是独立的资源所有者。本书不展开别名构造的细节，只提醒一点：看到 `shared_ptr` 时，先问"控制块是谁创建的"，再问"指针指向什么"；两个问题分开，所有权就不会看错。

5.9 本机实测：四个读数

实验记录 · 本机实测：独占、共享与循环引用

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `unique_basic` 的 stdout 是 `ctor w` / `name=w` / `dtor w` / `empty=yes`：`reset()` 立刻析构；
- `unique_move` 编译失败，诊断是 `use of deleted function 'std::unique_ptr<...>::unique_ptr(const std::unique_ptr<...>&)'`；
- `shared_count` 的 `use_count()` 依次是 `1`、`2`、`3`、`2`，对象在最后一个持有者析构时打印 `dtor shared`；
- `weak_cycle` 的真实读数是 `cycle_alive=2`、`after_cycle=2`、`after_weak=2 expired=1`：循环引用留下两个对象，`weak_ptr` 不延长寿命；
- `custom_deleter` 的输出是 `value=21 freed=0` → `free 21` → `after=1`：删除器在作用域结束时执行。

5.10 下一单元从哪继续

下一章 `6.1 继承与虚函数` 会把"所有权"扩展到"多态"：基类指针指向派生对象，虚函数决定调用哪一份实现，虚析构决定删除时走哪条析构链。`unique_ptr<Base>` 持有派生对象时，基类的析构函数必须是虚的，否则删除只走基类析构，派生部分泄漏——这是第 6 单元的第一个坑。

小结 · 这一节要带走的五件事

- `unique_ptr` 独占、只能移动；`shared_ptr` 共享、计数归零时释放；`weak_ptr` 观察、不延长寿命。
- `unique_ptr` 的拷贝被删除，所有权转移必须写 `std::move`；移动后原指针为空。
- `shared_ptr` 的对象与控制块是两件事；`use_count()` 是读数，不是同步工具。
- 循环引用会让强引用计数永不归零；把环中的一条边改成 `weak_ptr`，并用 `lock()` 临时提升。
- 资源来自 `new` 用 `make_unique` / `make_shared`；来自 C API 写删除器；默认优先 `unique_ptr`，不要把所有对象都共享。

自测题 · 自测题（先自己做，再对答案）

1. `unique_ptr` 为什么禁止拷贝？如果允许拷贝，会发生什么？
2. `widget.reset()` 与 `widget.release()` 的区别是什么？各自之后 `widget` 是什么状态？
3. `shared_ptr` 的 `use_count()` 从 3 降到 2 时，对象会析构吗？什么时候才会析构？
4. 两个对象互相持有 `shared_ptr`，为什么块结束后 `alive` 仍然是 2？把其中一条边改成 `weak_ptr` 后，`expired()` 为什么是 1？
5. 为什么 `malloc` 出来的内存不能直接交给默认的 `unique_ptr<int>`？自定义删除器怎样解决？

注 · 自测答案

1. 如果允许拷贝，两个 `unique_ptr` 会持有同一个指针，两个析构函数都会释放它，造成双重释放。独占语义要求"一个资源只有一个释放者"，所以拷贝被删除，只能移动。
2. `reset()` 释放当前对象并把指针置空；`release()` 放弃所有权并返回裸指针，`unique_ptr` 本身变为空，但对象不会被释放，释放责任交回调用者。
3. 不会。`use_count()` 只表示还有几个强引用；计数降到 0 时，最后一个持有者离开，控制块调用删除器销毁对象。
4. 因为两个对象互为强引用，外部指针离开后计数仍各为 1，析构不会发生。把一条边改成 `weak_ptr` 后，强引用计数能归零，对象析构；观察该对象的 `weak_ptr` 随即 `expired()` 为真。
5. 默认删除器调用 `delete`，而 `malloc` 必须用 `free` 释放，混用是未定义行为。写一个调用 `free` 的删除器，并把它作为 `unique_ptr` 的第二个模板参数。

第 3 章

继承与多态

本章 2 个单元 · 页眉从这里之后的页面开始出现

1. 继承与虚函数：动态多态、对象切片 零基础 · 第 6 单元 · 真编译真运行
2. 多态的设计代价：虚析构、接口类、组合优于继承 零基础 · 第 7 单元 · 真编译真运行

继承与虚函数：动态多态、对象切片

C 的"复用"主要靠组合：一个结构体里放另一个结构体。C++ 多了继承：`class Derived : public Base` 表示"派生类是一种基类"，派生对象里包含一个基类子对象。继承本身不产生多态；**只有虚函数 + 通过基类引用/指针调用**，才会在运行期选到派生类的实现。这一单元把这两件事分开：先看继承的对象布局，再看虚函数怎样把"调用哪份实现"推迟到运行期。

多态不是免费的：对象里多一个虚表指针，调用要经过一次间接跳转，析构函数如果不是虚的，通过基类指针删除派生对象就是未定义行为。这一单元先用六个例子把"怎么用"讲清，下一单元再算"值不值得用"。

◎ 学习目标 · 看完这一单元，你应该能

- 写出一个基类 + 两个派生类，用 `override` 标注重写，并通过 `unique_ptr<Base>` 容器调用虚函数；
- 解释对象切片：按值传递基类为什么丢掉派生部分，`sizeof` 的读数说明什么；
- 说出虚析构函数解决什么问题，并解释"非虚析构 + 基类指针删除"为什么是未定义行为；
- 读懂 `marked 'override', but does not override` 的诊断，知道签名差一个 `const` 会怎样；
- 说出构造与析构的顺序：基类 → 成员 → 派生类，销毁时反过来。

6.1 先看工程长什么样

这一单元在 `examples/ch03/` 下建六个多文件例子。每个例子仍然是 `include/<例名>.h` 放类声明、`src/<例名>.cpp` 放成员函数实现、`src/main.cpp` 只做驱动与打印；第 2 章起的硬规定③继续生效。

目录树 **工程：examples/ch03/ 这一单元的六个例子**

第 1 步 先建一个基类 + 两个派生类

```
examples/
└── ch03/
    └── virtual_basic/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── virtual_basic.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── virtual_basic.cpp ← 本课新增
        └── virtual_basic.json ← 本课新增
```

第 2 步 加上切片与虚析构的对照

```
examples/
└── ch03/
    ├── virtual_basic/ : 同上
    ├── slicing/ ← 本课新增
    └── virtual_dtor/ ← 本课新增
```

第 3 步 最后加上签名错误、抽象类与构造顺序

```
examples/
└── ch03/
    ├── 前三个例子 : 同上
    ├── override_typo/ ← 本课新增
    ├── abstract_shape/ ← 本课新增
    └── ctor_dtor_order/ ← 本课新增
```

第 1 帧的 `virtual_basic/` 定义抽象基类 `Shape` 与两个派生类 `Circle` / `Rect`，用 `vector<unique_ptr<Shape>>` 装起来，循环调用虚函数。第 2 帧的 `slicing/` 按值传基类，观察派生部分被切掉；`virtual_dtor/` 对照"非虚析构"与"虚析构"两种基类。第 3 帧的 `override_typo/` 故意把 `draw() const` 写成 `draw()`，让 `override` 报错；`abstract_shape/` 用纯虚接口 + 工厂；`ctor_dtor_order/` 打印基类、成员、派生类的构造/析构顺序。

6.2 继承与虚函数：同一句调用，不同的实现

cpp **抽象基类：只规定接口** examples/ch03/virtual_basic/include/virtual_basic.h 第 8–13 行

```cpp
class Shape {
public:
    virtual ~Shape() = default;
    virtual std::string name() const = 0;
    virtual double area() const = 0;
};
```

cpp **两个派生类：用 override 标注重写** examples/ch03/virtual_basic/include/virtual_basic.h 第 15–34 行

```cpp
class Circle : public Shape {
public:
    explicit Circle(double radius);
    std::string name() const override;
    double area() const override;

private:
    double radius_;
};

class Rect : public Shape {
public:
    Rect(double width, double height);
    std::string name() const override;
    double area() const override;

private:
    double width_;
    double height_;
};
```

cpp **main：基类指针容器，调用的是派生实现** examples/ch03/virtual_basic/src/main.cpp 第 5–12 行

```cpp
int main() {
    auto shapes = make_shapes();
    for (const auto& shape : shapes) {
        std::cout << shape->name() << "=" << shape->area() << "\n";
    }
    std::cout << "count=" << shapes.size() << "\n";
    return 0;
}
```

终端输出 终端输出 virtual-basic-out

```
circle=12.56
rect=12
count=2
```

`shapes` 的元素类型是 `unique_ptr<Shape>`，但循环里 `shape->name()` 与 `shape->area()` 分别调到了 `Circle` 与 `Rect` 的实现，输出 `circle=12.56`、`rect=12`。这就是**动态绑定**：编译器在调用点只知道"这是一个 `Shape`"，具体调用哪份函数由对象的动态类型决定。

`virtual` 只写在基类；派生类的 `override` 不是必须的（不写也能重写），但它是**让编译器帮你检查签名**的开关。`override` 的意思是"我声称这个函数重写基类的虚函数"；如果签名对不上（参数、`const`、返回类型不兼容），编译器直接报错，而不是悄悄生成一个同名的新函数。`final` 则相反：它告诉编译器"这个虚函数到此为止，派生类不能再重写"；写在类名后面表示"这个类不能被继承"。

标准原文 · 标准原文：虚函数与重写

> A non-static member function is a virtual function if it is first declared with the keyword virtual or if it overrides a virtual member function declared in a base class. ... Virtual functions support dynamic binding and object-oriented programming.
>
> If a virtual member function F is declared in a class B, and, in a class D derived (directly or indirectly) from B, a declaration of a member function G corresponds to a declaration of F, ... then G overrides F.

大意：虚函数可以用 `virtual` 首次声明，也可以重写基类的虚函数；虚函数支持动态绑定。派生类中"对应"的函数声明会重写基类的虚函数，对应关系包含参数列表与 cv 限定符。

出处：N4950 §11.7.3「Virtual functions」（[class.virtual]）。

`= 0` 是纯虚函数：基类只声明接口，不提供实现，派生类必须重写，否则派生类也是抽象的。含纯虚函数的类叫抽象类，不能直接创建对象。`Shape` 的析构函数写成 `virtual ~Shape() = default;`，原因在 6.5 节；现在只要记住：**只要一个类准备被继承并通过基类指针删除，析构函数就该是虚的。**

调用树 **虚调用：循环里的一次 shape->area()**

第 1 步 循环拿到第一个元素

```
main() ：shapes[0] 的动态类型是 Circle
main() → Shape :area()：静态类型是 Shape
```

第 2 步 虚表把调用转到 Circle::area

```
Circle::area() ：返回 3.14 * 2 * 2
Circle::area() ⇒ main() 12.56
```

第 1 帧的调用点写的是 `Shape::area()`，编译期只能确认"`Shape` 有这个名字"；第 2 帧由对象的虚表指针找到 `Circle::area()`，返回 12.56。**虚函数不是"运行时查字符串"，而是通过对象里的虚表指针做一次间接调用**；具体布局由 ABI 决定，C++ 标准只规定行为，不规定虚表长什么样。

带虚函数的类，对象里通常多一个"虚表指针"，所以 `sizeof` 会比同字段的非多态类大；多继承时可能有多份虚表指针，布局更复杂。**这些是实现细节，不是标准保证**；但选型时值得知道：多态不是零成本，每个对象都可能多一次间接寻址与一个指针的空间。下一单元会把成本展开。

`final` 有两个用法：写在虚函数后面表示"派生类不能再重写"，写在类名后面表示"不能被继承"。它不仅是文档，也能让编译器在确定动态类型时做去虚化（devirtualization）；**但不要把 `final` 当成性能开关**，它的主要价值是表达设计意图：这个接口到此为止。滥用 `final` 会让本来可扩展的类变得封闭。

构造/析构期间调用虚函数不会派发到派生类：基类构造时派生部分还没构造，基类析构时派生部分已经销毁，虚调用只能落到当前构造/析构的类。**在构造函数里调用纯虚函数是未定义行为**（通常会终止）；需要"构造后初始化"，用两阶段构造或工厂函数，而不是在构造函数里调用虚函数。这条规则与 6.7 的构造顺序一起记，能解释很多"为什么多态对象在构造阶段不听话"的疑问。

6.3 对象切片：按值传基类会切掉派生部分

cpp **基类与派生类，以及一个按值收基类的函数** examples/ch03/slicing/include/slicing.h 第 6–20 行

```cpp
class Shape {
public:
    virtual ~Shape() = default;
    virtual std::string name() const;
};

class Derived : public Shape {
public:
    std::string name() const override;

private:
    double extra_ = 3.14;
```

```
18  };
19
20  void show_by_value(Shape shape);
```

cpp **main: 直接调用是 derived，按值传进去变成 base** examples/ch03/slicing/src/main.cpp 第 5–12 行

```
5   int main() {
6       Derived d;
7       std::cout << "direct=" << d.name() << "\n";
8       show_by_value(d);
9       std::cout << "sizeof_base=" << sizeof(Shape)
10                << " sizeof_derived=" << sizeof(Derived) << "\n";
11      return 0;
12  }
```

终端输出 终端输出 slicing-out

```
direct=derived
copy=base
sizeof_base=8 sizeof_derived=16
```

`d.name()` 直接调用，输出 `direct=derived`；`show_by_value(d)` 按值接收 `Shape`，编译器只复制 `d` 的基类子对象，派生类的 `extra_` 与派生类的虚表指针都没有被复制过来，函数里的 `shape` 是一个真正的 `Shape` 对象，所以输出 `copy=base`。`sizeof_base=8 sizeof_derived=16` 进一步说明：基类对象只有一个虚表指针（8 字节），派生类对象在它后面还有 `double`（8 字节），但按值传参只复制了前 8 字节。

对象切片不是“类型转换失败”，而是“派生部分被静默丢弃”。 编译器不会为此报警（有些编译器会对“按值传多态对象”给出提示，但不是标准要求）。避免切片的方法：传 `const Shape&` 或 `Shape*`，让函数通过引用/指针看动态类型；容器用 `unique_ptr<Shape>` 或 `Shape&`，不要用 `vector<Shape>` 装派生对象。

内存形态 **切片前后的对象内容**

第 1 步 派生对象里有基类子对象 + 派生成员

Derived 对象 ：Shape 子对象 + extra_ | Shape 子对象 ：vptr

vptr → Derived 虚表 name 指向 Derived::name

第 2 步 按值复制成基类对象

Shape 副本 ：只复制 Shape 子对象 | extra_ ：没有进入副本

Shape 副本 → Shape 虚表 只剩 Shape::name

第 1 帧的派生对象由“基类子对象 + 派生成员”拼成；第 2 帧的 `Shape 副本` 只包含基类子对象，`extra_` 没有位置可放，虚表指针也换成了 `Shape` 的。**切片是“按基类的大小复制”，不是“把派生对象转成基类引用”**；引用/指针不会创建新对象，所以不会切。

切片还有更隐蔽的版本：把派生对象赋给基类变量、把派生对象放进 `vector<Base>`、从函数按值返回 `Base`。它们的共同点是“新对象是基类类型”，派生部分没有容身之处。**判断标准：这个位置会不会创建一个 `Base` 类型的独立对象？会，就切片；只是绑定一个名字/指针，就不会。**

6.4 `override` 与签名：差一个 `const` 就不是重写

cpp **基类：draw() 带 const** examples/ch03/override_typo/include/override_typo.h 第 4–10 行

```
4   class Base {
5   public:
6       virtual ~Base() = default;
7       virtual void draw() const;
8   };
9
10  #endif
```

cpp **派生类：漏了 const，却写了 override** examples/ch03/override_typo/src/main.cpp 第 3–6 行

```
3   class Derived : public Base {
```

```
    public:
        void draw() override;
    };
```

编译诊断 本机真实 stderr：override 签名不匹配 override-typo-diag

```
src/main.cpp:5:10: error: 'void Derived::draw()' marked 'override', but does not override
    5 |     void draw() override;
      |          ^~~~
In file included from src/main.cpp:1:
include/override_typo.h:7:18: warning: 'virtual void Base::draw() const' was hidden [-Woverloaded-virtual=]
    7 |     virtual void draw() const;
      |                  ^~~~
src/main.cpp:5:10: note:   by 'void Derived::draw()'
    5 |     void draw() override;
      |          ^~~~
```

`Base::draw() const` 带 `const`，`Derived::draw()` 不带，两个函数的签名不同，所以前者没有被重写。`override` 的作用在这里体现：编译器直接报 `marked 'override', but does not override`，而不是让你得到一个"看起来能调用、实际上没有多态"的派生函数。旁边的 `-Woverloaded-virtual` 警告还指出：派生类的 `draw()` 隐藏了基类的 `draw() const`，通过基类指针调用时不会走到它。

签名包含参数列表与 cv 限定符；返回类型要兼容（协变返回类型除外）。 常见错误有：漏 `const`、参数类型写成引用/值不一致、把 `noexcept` 当成签名一部分（C++17 起 `noexcept` 是函数类型的一部分，重写时也要匹配）、把 `virtual` 写在派生类却漏了 `override`。写 `override` 的成本很低，收益是让编译器替你抓这些错误；本书建议所有重写都写 `override`，不重写就不写 `virtual`。

⚠ 陷阱 · 以为"同名函数就是重写"

你会看到：派生类写了一个和基类同名、参数略有不同的函数，编译通过，调用却走到基类版本。你会误判成"虚函数失效了"或"编译器没看见派生类"。真相是：参数列表/ `const` 不同就不是重写，而是**隐藏**——派生类的新函数把基类的同名函数挡住了，基类指针调用仍然按基类的虚表走。`override` 会让这种"假重写"直接编译失败；不写 `override` 时，编译器可能只给一条 `-Woverloaded-virtual` 警告，甚至什么都不说。

6.5 虚析构：通过基类指针删除派生对象

cpp **第 1 组：析构函数不是虚的** examples/ch03/virtual_dtor/include/virtual_dtor.h 第 4–15 行

```
class BadBase {
public:
    ~BadBase();
};

class BadDerived : public BadBase {
public:
    BadDerived();
    ~BadDerived();

    static int destroyed;
};
```

下面是同一份头文件里的第二组：类的形状不变，只把基类析构改成 `virtual`。

cpp **（续）第 2 组：析构函数是虚的** examples/ch03/virtual_dtor/include/virtual_dtor.h 第 17–28 行

```
class GoodBase {
public:
    virtual ~GoodBase();
};

class GoodDerived : public GoodBase {
public:
    GoodDerived();
    ~GoodDerived() override;

    static int destroyed;
```

```
28  };
```

cpp **main: 同样的 delete，不同的析构链** examples/ch03/virtual_dtor/src/main.cpp 第 5-14 行

```cpp
5   int main() {
6       BadBase* bad = new BadDerived();
7       delete bad;
8       std::cout << "bad_destroyed=" << BadDerived::destroyed << "\n";
9
10      GoodBase* good = new GoodDerived();
11      delete good;
12      std::cout << "good_destroyed=" << GoodDerived::destroyed << "\n";
13      return 0;
14  }
```

终端输出 终端输出 virtual-dtor-out

```
BadDerived
~BadBase
bad_destroyed=0
GoodDerived
~GoodDerived
~GoodBase
good_destroyed=1
```

`bad` 的静态类型是 `BadBase*`，动态类型是 `BadDerived`；`delete bad;` 只调用了 `~BadBase`，`~BadDerived` 没有执行，所以 `bad_destroyed=0`。`good` 的基类析构是 `virtual`，`delete good;` 先调用 `~GoodDerived`，再自动调用 `~GoodBase`，所以 `good_destroyed=1`。

通过基类指针删除派生对象，而基类析构不是虚的，是未定义行为。 本机可观察到的结果是"派生析构没执行、派生类的资源泄漏"，但这只是这一套 ABI/运行库下的表现；标准不保证你一定看到这个结果，也不保证不会崩溃。规则很简单：**如果一个类准备被继承，并且可能通过基类指针删除，就把析构函数写成 `virtual`。** 如果一个类不准备被继承，可以用 `final` 明确禁止继承，避免别人误用。

虚析构的代价是对象里多一个虚表指针（如果还没有其他虚函数），以及一次间接调用；对"多态基类"来说，这是必须付的成本。`Shape` 的 `virtual ~Shape() = default;` 就是这条规则的落地：它没有资源要释放，但它是多态基类，必须有虚析构，否则 `unique_ptr<Shape>` 删除派生对象时只走基类析构。

6.6 抽象类与接口：只规定"能做什么"

cpp **纯虚接口 + 工厂 + 只观察的函数** examples/ch03/abstract_shape/include/abstract_shape.h 第 6-22 行

```cpp
6   class Shape {
7   public:
8       virtual ~Shape() = default;
9       virtual double area() const = 0;
10  };
11
12  class Square : public Shape {
13  public:
14      explicit Square(double side);
15      double area() const override;
16
17  private:
18      double side_;
19  };
20
21  std::unique_ptr<Shape> make_square(double side);
22  double total_area(const Shape& shape);
```

终端输出 终端输出 abstract-shape-out

```
area=9
```

`Shape` 只有纯虚函数与虚析构，没有任何数据成员；这种类常被叫作**接口类**。它不能直接实例化（`Shape s;` 编译失败），但可以定义 `Shape&` / `Shape*` / `unique_ptr<Shape>`，让调用者只依赖"能算面积"这个能力。`make_square` 返

回 `unique_ptr<Shape>`，调用者不需要知道具体是 `Square`；`total_area(const Shape&)` 只读地使用接口，不拥有对象。**“面向接口编程”在 C++ 里就是“依赖抽象基类的引用/指针，不依赖具体派生类”。**

接口类的析构函数必须是虚的（`virtual ~Shape() = default;`）。原因与上一节相同：调用者可能通过 `unique_ptr<Shape>` 删除对象，析构链必须走到派生类。纯虚析构在语法上允许，但必须提供定义（否则派生类析构时链接不到），本书统一用 `virtual ~Shape() = default;`。

抽象类也可以有数据成员与普通成员函数；纯虚函数只表示“这个操作必须由派生类提供”。不要把“抽象类”理解成“不能有实现”：它可以有构造函数、普通函数、甚至纯虚函数的定义（派生类可以显式调用 `Base::f()`）。**抽象的是“能否实例化”，不是“有没有代码”。**

6.7 构造与析构的顺序

cpp **Derived 继承 Base，并含一个 Member 成员** examples/ch03/ctor_dtor_order/include/ctor_dtor_order.h 第 16-23 行

```cpp
class Derived : public Base {
public:
    Derived();
    ~Derived() override;

private:
    Member member_;
};
```

终端输出 终端输出 ctor-dtor-order-out

```
Base()
Member()
Derived()
~Derived()
~Member()
~Base()
```

构造一个 `Derived` 的顺序是：**基类子对象 → 成员子对象（按声明顺序）→ 派生类构造函数体**；析构顺序完全相反：**派生类析构函数体 → 成员析构（按声明逆序）→ 基类析构。**输出正是 `Base()` → `Member()` → `Derived()`，然后 `~Derived()` → `~Member()` → `~Base()`。

这个顺序有一个实际后果：**在基类构造函数里调用虚函数，不会调用到派生类的重写版本。**因为构造基类时，派生类部分还没构造，对象的动态类型还是基类；虚表指针指向基类的虚表。析构时同理：进入基类析构后，派生类部分已经销毁，虚调用也不会再回到派生类。标准把这条规则写在构造/析构期间的动态类型里；实际写代码时记住一句话：**构造/析构期间不要依赖派生类的虚函数行为。**

成员析构按声明逆序，与成员初始化顺序相反；这与 3.3 节的“按声明顺序初始化”是一对。**初始化顺序与销毁顺序互为逆序**，这是 C++ 对象生命周期的一条基本对称性。

6.8 本机实测：四个读数

实验记录 · **本机实测：虚调用、切片与析构**

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `virtual_basic` 的 stdout 是 `circle=12.56` / `rect=12` / `count=2`：基类指针容器调用到派生实现；
- `slicing` 的 stdout 是 `direct=derived` / `copy=base` / `sizeof_base=8 sizeof_derived=16`：按值传参丢掉派生部分；
- `virtual_dtor` 的 stdout 是 `bad_destroyed=0` / `good_destroyed=1`：非虚析构只走了基类析构，虚析构走完整条链（非虚析构这一半是未定义行为，本机读数只作证据，不作保证）；
- `override_typo` 编译失败，诊断是 `marked 'override', but does not override`，同时有 `-Woverloaded-virtual` 警告。

6.9 把继承变成工程习惯

1. **先问"是不是 is-a"**：派生类必须能替换基类使用（Liskov 替换原则的朴素版本）；只是"想复用代码"不该用继承，用组合。
2. **多态基类必须有虚析构**：只要可能通过基类指针删除，就写 `virtual ~Base() = default;`；不准备被继承就写 `final`。
3. **所有重写都写 `override`**：让编译器检查签名；不要依赖"同名就是重写"。
4. **按引用/指针传递多态对象**：不要按值传基类，也不要把派生对象放进 `vector<Base>`；容器用 `unique_ptr<Base>`。
5. **构造/析构期间不调用虚函数**：基类构造时派生部分还没好，基类析构时派生部分已经没了。

继承层次越深，构造与析构链越长，追踪一次行为要跨过的文件也越多；超过两层的具体继承通常应改成组合。**继承是"类型关系"的声明，不是省代码的工具**，每加一层都要能说清它表达了什么 is-a 关系。

6.10 下一单元从哪继续

下一单元 `6.2 多态的设计代价` 会把这一单元的机制换算成成本：对象里的虚表指针、虚调用的间接跳转、接口类的析构顺序、`dynamic_cast` 的边界，以及"组合优于继承"的判据。这一单元先确保你能写出正确的多态代码；下一单元再判断"这段代码该不该用多态"。

小结 · 这一节要带走的五件事

- 继承表达 is-a；虚函数 + 基类引用/指针才产生动态绑定，`override` 让编译器检查签名。
- 按值传基类会切片：派生部分被静默丢弃，容器要用 `unique_ptr<Base>` 或引用。
- 多态基类的析构必须是虚的；通过非虚析构基类指针删除派生对象是未定义行为。
- 抽象类用纯虚函数规定接口，不能实例化；接口类的析构同样必须是虚的。
- 构造顺序是基类 → 成员 → 派生类，析构顺序相反；构造/析构期间不要依赖派生类虚函数。

自测题 · 自测题（先自己做，再对答案）

1. `Shape` 的 `area()` 是纯虚函数，为什么 `Shape s;` 不能编译，而 `Shape* p` 可以声明？
2. `show_by_value(Shape shape)` 收到一个 `Derived` 对象时，为什么 `shape.name()` 返回 `base`？
3. 非虚析构的基类指针删除派生对象，本机看到 `bad_destroyed=0`；这说明了什么？标准上这属于什么行为？
4. `override` 与 `virtual` 各解决什么问题？为什么本书建议"重写都写 override，不重写不写 virtual"？
5. 构造 `Derived` 时，输出顺序为什么是 `Base()` → `Member()` → `Derived()`？析构为什么反过来？

注 · 自测答案

1. `Shape` 含纯虚函数，是抽象类，不能创建对象；但可以声明指向它的指针/引用，用来绑定派生对象。
2. 按值传参创建了一个独立的 `Shape` 对象，只复制了 `d` 的基类子对象；派生部分与派生虚表都没被复制，所以虚调用只能到 `Shape::name()`。
3. 本机可观察到派生析构没有执行、派生资源没有释放；这属于未定义行为，标准不保证具体结果，不能把 `bad_destroyed=0` 当成可移植的规律。
4. `virtual` 声明"这个函数可以动态绑定"；`override` 声明"我在重写基类虚函数，请检查签名"。重写写 `override` 能让签名错误编译失败；不重写就不写 `virtual`，避免制造新的虚函数。
5. 构造顺序是基类子对象、成员子对象（按声明顺序）、派生类构造函数体；析构按完全相反的顺序执行，所以先派生、再成员、最后基类。

多态的设计代价：虚析构、接口类、组合优于继承

上一单元证明了多态"能用"：基类指针调用虚函数，运行期选到派生实现。这一单元要回答"值不值得用"。虚函数不是免费的：对象里可能多一个虚表指针，调用要经过一次间接跳转，接口类的析构必须是虚的，`dynamic_cast` 是运行期检查而不是零成本转换。更重要的是设计代价：继承表达"是一种"，组合表达"有一个"；用继承复用实现，往往会在第二个需求到来时崩掉。

这一单元把"代价"拆成三类：对象大小、调用方式、设计耦合。对象大小用 `sizeof` 实测；调用方式只讲可观察的机制，不承诺性能数字；设计耦合用组合、`variant` 与 `final` 三个例子对照。

◎ 学习目标 · 看完这一单元，你应该能

- 写出一个只含纯虚函数与虚析构的接口类，并说清调用者为什么只依赖接口；
- 用 `sizeof` 读数说明虚函数给对象增加了什么，以及这个数字为什么不是标准规定；
- 说出虚调用与直接调用的机制差别，不把"有虚函数"当成"一定慢"；
- 用 `dynamic_cast` 做安全的向下转换，并区分指针失败返回空、引用失败抛 `bad_cast`；
- 判断一个复用场景该用继承还是组合，并说出 `final` 与 `variant` 各自解决什么问题。

7.1 先看工程长什么样

这一单元在 `examples/ch03/` 下再加六个多文件小工程。接口类、对象大小、组合、转换、`variant`、`final` 各占一个目录；第 2 章起的硬规定③继续生效。

目录树 **工程：examples/ch03/ 这一单元新增的六个例子**

第 1 步 先建接口类与对象大小对照

```
examples/
└── ch03/
    ├── 上一单元 ： cpp06 的六个例子
    └── interface_shape/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── interface_shape.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── interface_shape.cpp ← 本课新增
        └── interface_shape.json ← 本课新增
```

第 2 步 加上组合与运行期转换

```
examples/
└── ch03/
    ├── interface_shape/ ： 同上
    ├── vtable_size/ ← 本课新增
    ├── composition_over_inheritance/ ← 本课新增
    └── cast_cost/ ← 本课新增
```

第 3 步 最后加上 variant 分派与 final

```
examples/
└── ch03/
    ├── 前四个例子 ： 同上
    ├── variant_dispatch/ ← 本课新增
    └── final_demo/ ← 本课新增
```

第 1 帧的 `interface_shape/` 只有纯虚函数与虚析构，展示"接口类"最小形态；`vtable_size/` 用四个空/非空、有/无虚函数的类型量 `sizeof`。第 2 帧的 `composition_over_inheritance/` 把 `Engine` 作为 `Car` 的成员，而不是让 `Car` 继承 `Engine`；`cast_cost/` 用 `dynamic_cast` 做指针与引用的向下转换。第 3 帧的 `variant_dispatch/` 用 `std::variant` + `std::visit` 实现值语义的分派；`final_demo/` 让一个 `final` 基类被继承，编译失败。

7.2 接口类：只依赖"能做什么"

cpp **接口类：纯虚函数 + 虚析构** examples/ch03/interface_shape/include/interface_shape.h 第 8–13 行

```cpp
class Shape {
public:
    virtual ~Shape();
    virtual std::string name() const = 0;
    virtual double area() const = 0;
};
```

cpp **main: 只通过 Shape 接口使用对象** examples/ch03/interface_shape/src/main.cpp 第 5–14 行

```cpp
int main() {
    {
        auto shapes = make_shapes();
        for (const auto& shape : shapes) {
            std::cout << shape->name() << "=" << shape->area() << "\n";
        }
    }
    std::cout << "done\n";
    return 0;
}
```

终端输出 终端输出 interface-shape-out

```
circle=12.56
square=9
drop circle
drop square
done
```

`Shape` 没有数据成员、没有普通函数，只有三个虚函数：析构、`name`、`area`。调用者只写 `shape->name()` / `shape->area()`，不 include `Circle` 或 `Square` 的头文件；要加一种新形状，只要实现 `Shape` 并在工厂里注册，调用方不用改。**这就是"依赖抽象"的实际收益：变化被关在实现文件里，调用方只面对稳定接口。**

输出里的 `drop circle` 在 `drop square` 之前，是因为 `vector` 析构元素时按逆序进行：先构造的 `Circle` 后析构。`done` 在花括号结束后打印，说明 `shapes` 已经整体销毁；`Shape` 的虚析构让 `unique_ptr<Shape>` 能正确调用派生类析构，再调用 `~Shape()`。

接口类的设计要点：**析构必须是虚的**（否则通过接口指针删除派生对象是未定义行为）；接口里尽量只放纯虚函数，不要把数据成员塞进基类；如果某个操作有默认实现，可以放在基类，但要想清楚它是否依赖派生类的细节。接口类不是"越纯越好"：一个没有任何数据的基类，通常意味着它只表达契约，不表达状态。

7.3 对象大小：虚函数增加了什么

cpp **四个类型：空、含 int、含虚函数、含虚函数与 int** examples/ch03/vtable_size/include/vtable_size.h 第 4–20 行

```cpp
struct Empty {};

struct WithInt {
    int value;
};

struct WithVirtual {
    virtual ~WithVirtual() = default;
    virtual int value() const;
};

struct WithVirtualInt {
    virtual ~WithVirtualInt() = default;
    virtual int value() const;

    int data;
};
```

cpp **main: 四个 sizeof 读数** examples/ch03/vtable_size/src/main.cpp 第 5–11 行

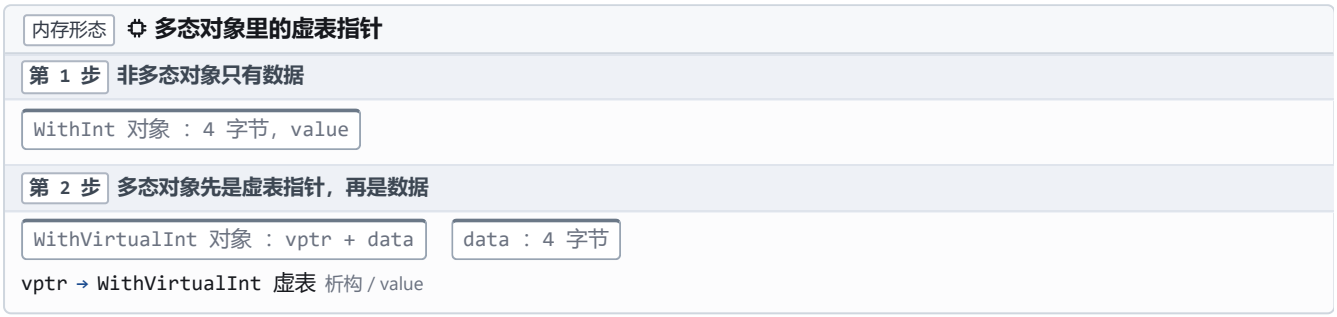
```cpp
int main() {
    std::cout << "empty=" << sizeof(Empty) << "\n";
    std::cout << "int=" << sizeof(WithInt) << "\n";
    std::cout << "virtual=" << sizeof(WithVirtual) << "\n";
    std::cout << "virtual_int=" << sizeof(WithVirtualInt) << "\n";
    return 0;
}
```

终端输出 终端输出 vtable-size-out

```
empty=1
int=4
virtual=8
virtual_int=16
```

`Empty` 是 1 而不是 0：标准要求同类型的两个不同对象有不同地址，空类必须至少占 1 字节。`WithInt` 是 4，正好一个 `int`。`WithVirtual` 是 8，多出来的 8 字节是一个指向虚表的指针（本机 64 位）；`WithVirtualInt` 是 16，因为虚表指针 8 字节 + `int` 4 字节，再按 8 字节对齐补齐到 16。

这些数字是“本机实测”，不是 C++ 标准规定。 标准只要求虚函数能动态绑定，不规定用虚表、也不规定指针大小；换一个 ABI、换一个编译器，布局可能不同。你能带走的结论是：**多态对象通常要比非多态对象多存一份“怎么找到正确函数”的信息**；具体多大，用 `sizeof` 量，不要背数字。



第 1 帧的 `WithInt` 只有数据；第 2 帧的 `WithVirtualInt` 在最前面放了 `vptr`，对象因此从 4 字节变成 16 字节（含对齐）。虚表本身是每个多态类一份，不是每个对象一份；对象里存的是“指向虚表”的指针。

7.4 虚调用的代价：一次间接跳转

虚调用与直接调用的差别在“地址从哪来”：直接调用在编译期就确定函数地址；虚调用要先读对象的 `vptr`，再从虚表里取对应槽位的地址，最后跳过去。**本单元不承诺“虚调用比直接调用慢多少”的具体数字**——那取决于 CPU、缓存、分支预测、优化档；我们只确认机制上的差别：多了一次内存读取与一次间接跳转。

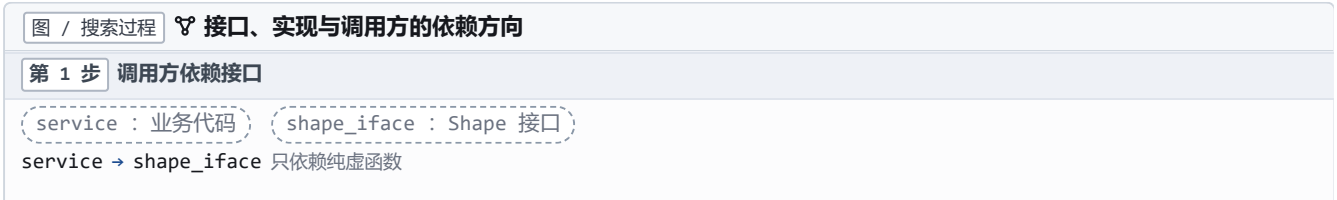
这不意味着“有虚函数就一定慢”。如果一个循环里对同一个对象反复调用虚函数，分支预测会很准；如果虚函数体本身很重（例如做一次 I/O 或复杂计算），那点间接开销可以忽略。反过来，如果虚函数只是返回一个字段，而且被调用几百万次，间接开销就可能显现。**优化前先量，别凭“虚函数慢”的传言改设计。**

更值得警惕的是设计代价：虚函数把“调用哪份实现”推迟到运行期，编译器就不能内联、不能做跨过程的常量传播；接口越抽象，编译器能看到的细节越少。**多态买的是“调用方与实现解耦”，付的是“运行期决策 + 优化空间”**；这笔交易是否划算，取决于变化频率与性能预算。

编译器并非永远无法优化虚调用：如果对象类型在调用点确定（局部对象、`final` 类、去虚化分析），它可以把虚调用改成直接调用甚至内联。链接期优化（LTO）与基于性能分析的优化（PGO）也能提供更多类型信息。**但这些是“可能发生”的优化，不是标准保证**；写代码时不能假设编译器一定去虚化，性能敏感路径要靠测量确认。

接口类的设计也有成本边界：接口只放纯虚函数与虚析构，不放数据成员；数据放实现类，接口保持薄。这样每个实现对象只多一个 `vptr`，不会因为接口里的字段而膨胀。**接口越“胖”，实现越难替换**；把无关的便利函数塞进接口，会逼所有实现都去实现它们，反而降低多态的收益。

`dynamic_cast` 是另一种多态成本：它在运行期检查类型，失败时对指针返回 `nullptr`、对引用抛 `std::bad_cast`。它适合“确实需要按具体类型分支”的边界（例如插件加载、消息分发），不适合在热循环里做类型判断；**能用虚函数表达的行为，不要用 `dynamic_cast` + `if` 模拟**，那会把运行期类型检查散布到调用方，破坏封装，也让新增类型时更容易漏改；新增一个派生类，所有 `if` 分支都要跟着改。



第 2 步 实现依赖接口，调用方不知道实现

circle_impl : Circle 实现　square_impl : Square 实现　service : 不变

circle_impl → shape_iface 实现
square_impl → shape_iface 实现

第 1 帧的箭头从业务代码指向接口，业务代码不依赖任何具体形状；第 2 帧新增实现时，箭头从实现指向接口，业务代码不动。**依赖方向决定改动范围**：如果业务代码直接依赖 `Circle`，新增 `Square` 就要改业务代码；依赖接口，新增实现只改工厂/注册处。

7.5 组合优于继承：`Car` 有一个 `Engine`

cpp **Car 把 Engine 作为成员，而不是继承** examples/ch03/composition_over_inheritance/include/composition_over_inheritance.h 第 6–22 行

```cpp
class Engine {
public:
    void start();
    void stop();

private:
    bool running_ = false;
};

class Car {
public:
    void drive();

private:
    Engine engine_;
    std::string name_ = "car";
};
```

终端输出 终端输出　composition-out

```
engine start
car drive
engine stop
```

`Car` 没有继承 `Engine`，而是持有一个 `Engine engine_` 成员；`drive()` 调用 `engine_.start()` 与 `engine_.stop()`。这就是组合：**“汽车有一个引擎”，不是“汽车是一个引擎”。** 组合的好处是接口可控：`Car` 只暴露 `drive()`，`Engine` 的 `start/stop` 不必成为 `Car` 的公有接口；如果 `Car` 继承 `Engine`，`Car` 就会自动获得 `start/stop`，外部可以把汽车当引擎启动，语义上说不通。

继承复用的陷阱在于“为了省代码而继承”：`class Stack : public std::vector<int>` 看起来省了实现，但 `std::vector` 的所有操作都成了 `Stack` 的公有接口，用户可以下标越界、可以在中间插入、可以删除任意元素，栈的不变量被破坏。**继承会把基类的接口与实现一起带进来；组合只带走你想要的那部分能力。** 判断标准：派生类能不能在**所有**使用基类的地方替换基类，并且行为仍然正确？不能，就不要用 public 继承。

组合也有代价：要多写一层转发函数（`Car::drive` 调用 `engine_.start`），对象里多一个成员。但与“继承带来的接口污染”相比，这点成本通常更可控。工程上的默认策略是：**先组合，确实需要“is-a”且需要运行期多态时，再继承。**

大白话 · **大白话：继承是“是”，组合是“有”**

“苹果是一种水果”用继承；“汽车有一个引擎”用组合。你不会说“汽车是一种引擎”，所以不该让 `Car` 继承 `Engine`。继承会把父类的东西全继承下来，像把整套工具箱都搬进你家；组合只拿你需要的那把螺丝刀。需要“换引擎”时，组合还能把 `Engine` 换成接口/模板参数，继承则把类型关系焊死了。

7.6 `dynamic_cast`：运行期检查，失败有两条路

cpp **Base 与两个派生类** examples/ch03/cast_cost/include/cast_cost.h 第 6–22 行

```cpp
class Base {
public:
    virtual ~Base() = default;
    virtual std::string name() const;
};

class Alpha : public Base {
public:
    std::string name() const override;
};

class Beta : public Base {
public:
    std::string name() const override;
};

std::string describe(Base* base);
```

cpp **main: 指针失败为空，引用失败抛 bad_cast** examples/ch03/cast_cost/src/main.cpp 第 6–24 行

```cpp
int main() {
    Beta beta;
    Base* base = &beta;

    std::cout << "ptr=" << describe(base) << "\n";

    Alpha alpha;
    Base* other = &alpha;
    std::cout << "ptr_null="
              << (dynamic_cast<Beta*>(other) == nullptr ? "yes" : "no") << "\n";

    try {
        Beta& ref = dynamic_cast<Beta&>(*other);
        std::cout << "ref=" << ref.name() << "\n";
    } catch (const std::bad_cast&) {
        std::cout << "ref_caught=bad_cast\n";
    }
    return 0;
}
```

终端输出 终端输出 cast-cost-out

```
ptr=beta
ptr_null=yes
ref_caught=bad_cast
```

`dynamic_cast<Beta*>(base)` 在 `base` 实际指向 `Beta` 时返回 `Beta*`；在 `other` 实际指向 `Alpha` 时返回 `nullptr`，所以 `ptr_null=yes`。引用版本没有“空引用”可返回，转换失败时抛 `std::bad_cast`，被 `catch` 接住，输出 `ref_caught=bad_cast`。**指针版适合“可能失败、失败后走另一条路”；引用版适合“失败就是异常，调用者必须处理”。**

`dynamic_cast` 的前提是源类型必须是多态类型（至少有一个虚函数），因为它要读运行期类型信息；它也不能转走 `const`，不能做非公有/歧义基类的转换。它的成本是一次运行期检查，通常比 `static_cast` 高；**不要用 `dynamic_cast` 做“类型 switch”**：如果代码里排着一串 `if (dynamic_cast<A*>(p)) ... else if (dynamic_cast<B*>(p)) ...`，说明基类接口可能没设计好，或者应该用 `variant` / 访问者模式。

标准原文 · **标准原文：dynamic_cast 失败怎么处理**

> If T is a pointer type, ... The value of a failed cast to pointer type is the null pointer value of the required result type. A failed cast to reference type throws an exception of a type that would match a handler of type std::bad_cast.

大意：`dynamic_cast` 转指针失败时返回空指针；转引用失败时抛出 `std::bad_cast`，可以按这个类型捕获。

出处：N4950 §7.6.1.7「Dynamic cast」（[expr.dynamic.cast]）第 9 段。

⚠ 陷阱 · **用 `dynamic_cast` 串成类型 switch**

你会看到：基类接口只有一个模糊的 `process()`，派生类各自需要不同参数；为了拿到具体类型，代码里排出一串 `dynamic_cast`，每加一种类型就要改这串 if。你会误判成"多态不够灵活，还是得知道具体类型"。真相是：这串转换把"类型集合"从编译期搬到了运行期，新增类型必须改所有分派点。要么把差异提炼成基类虚函数（让实现自己处理），要么用 `variant` + `visit` 把"类型集合"固定在一个地方，让编译器检查漏分支。

7.7 `variant`：值语义的类型集合

`std::variant<Circle, Square>` 是一个"安全的联合体"：它要么存 `Circle`，要么存 `Square`，同一时刻只有一个活动成员。`std::visit` 按当前活动类型调用访问者，不需要虚表，也不需要堆分配。

cpp **variant_dispatch: 同一个 visit 处理两种形状** examples/ch03/variant_dispatch/src/main.cpp 第 6–19 行

```cpp
int main() {
    Shape shape = make_circle(2.0);
    std::cout << "index=" << shape.index() << "\n";
    std::visit([](const auto& item) {
        std::cout << item.name() << "=" << item.area() << "\n";
    }, shape);

    shape = make_square(3.0);
    std::cout << "index=" << shape.index() << "\n";
    std::visit([](const auto& item) {
        std::cout << item.name() << "=" << item.area() << "\n";
    }, shape);
    return 0;
}
```

终端输出 终端输出 variant-dispatch-out

```
index=0
circle=12.56
index=1
square=9
```

`index=0` 表示当前存的是 `variant` 的第一个替代类型 `Circle`，`index=1` 表示第二个 `Square`。`std::visit` 用泛型 lambda 接受任意替代类型，编译器为每种类型生成一份调用；如果新增一种类型，泛型 lambda 自动适配，但如果访问者里有 `if constexpr` 分支漏了类型，编译器在实例化时会报错。**`variant` 把"类型集合"写进了类型系统，新增类型必须改 `variant` 的模板参数，编译器会检查所有 `visit` 是否覆盖。**

`variant` 与继承的取舍：类型集合封闭、以值为主、希望避免堆分配时，用 `variant`；类型集合开放、需要运行期插件式扩展、对象可能很大且共享时，用继承 + 接口。`variant` 也可能"值过大"（存储最大替代类型的大小），并且 `visit` 的返回类型必须对所有替代类型一致；这些是它的代价。第 17 单元会再展开 `variant` 与 `optional` / `expected`。

7.8 `final`：明确关掉继承与重写

cpp **Base 被标记 final** examples/ch03/final_demo/include/final_demo.h 第 4–10 行

```cpp
class Base final {
public:
```

```
    virtual ~Base() = default;
    virtual int value() const;
};

int base_value();
```

cpp 派生类试图继承 final 基类 examples/ch03/final_demo/src/main.cpp 第 3-6 行

```
class Derived : public Base {
public:
    int value() const override;
};
```

编译诊断 本机真实 stderr：不能继承 final 类 final-demo-diag

```
src/main.cpp:3:7: error: cannot derive from 'final' base 'Base' in derived type 'Derived'
    3 | class Derived : public Base {
      |       ^~~~~~~
```

`final` 写在类名后面表示"这个类不能被继承"；写在虚函数后面表示"这个函数不能在派生类里重写"。它的作用不是"优化"，而是**表达设计意图并让编译器执行**：这个类不准备做基类，或者这个虚函数已经是最终实现。编译器看到 `final` 后，可以在某些情况下把虚调用去虚化（devirtualization），但那是优化结果，不是 `final` 的语义保证；**不要把 `final` 当成性能开关，把它当成"此处禁止扩展"的契约。**

`final` 与接口类不矛盾：接口类本身通常不标 `final`（它就是要被实现），但接口里的某个具体实现类可以标 `final`，防止别人再从它派生。一个类标了 `final` 之后，它的析构函数是否虚就不再影响"通过基类指针删除"的安全性——因为它不能被继承，也就不存在派生对象；但作为多态基类时，虚析构仍然必要。去虚化是否发生取决于编译器与优化选项，代码不能依赖它。

7.9 本机实测：四个读数

实验记录 · 本机实测：大小、分派与转换

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `vtable_size` 的 stdout 是 `empty=1` / `int=4` / `virtual=8` / `virtual_int=16`：本机多态对象多一个 8 字节的虚表指针；
- `interface_shape` 的 stdout 是 `circle=12.56` / `square=9` / `drop circle` / `drop square` / `done`：接口指针正确调用派生析构；
- `cast_cost` 的 stdout 是 `ptr=beta` / `ptr_null=yes` / `ref_caught=bad_cast`：指针失败为空，引用失败抛异常；
- `variant_dispatch` 的 stdout 是 `index=0` / `circle=12.56` / `index=1` / `square=9`：`variant` 的活动成员与 `visit` 输出一致。

7.10 下一单元从哪继续

下一章 `8.1 模板入门` 会把"同一份代码适配多种类型"从运行期多态转到编译期：函数模板、类模板、实例化与链接错误。多态用虚表在运行期选实现，模板用编译器在编译期生成实现；两者解决的问题有重叠，但代价与适用场景不同。学完模板，你会更容易判断"这里该用虚函数还是模板"。

小结 · 这一节要带走的五件事

- 接口类只暴露纯虚函数与虚析构；调用方依赖接口，新增实现不改调用方。
- 多态对象通常多存一个虚表指针；`sizeof` 的读数是本机事实，不是标准规定。
- 虚调用多一次间接跳转，但不承诺具体性能差异；优化前先量。
- `dynamic_cast` 转指针失败返回空，转引用失败抛 `std::bad_cast`；不要用它串类型 switch。
- 组合表达"有一个"，继承表达"是一种"；类型集合封闭时 `variant` 可替代运行期多态，`final` 用于明确禁止扩展。

自测题 · 自测题（先自己做，再对答案）

1. 一个只含纯虚函数与虚析构的接口类，为什么析构函数必须是虚的？
2. 本机 `sizeof(Empty)=1` 、 `sizeof(WithVirtual)=8` 、 `sizeof(WithVirtualInt)=16` ，这些数字分别说明什么？哪些是标准规定？
3. `Car` 持有 `Engine` 成员与 `Car` 继承 `Engine` ，在使用上有什么本质差别？
4. `dynamic_cast<Beta*>(p)` 与 `dynamic_cast<Beta&>(*p)` 在失败时分别怎样表现？为什么？
5. `variant` 与继承各自适合什么样的"类型集合"？新增一种形状时，两者各要改哪里？

注 · 自测答案

1. 调用者可能通过 `Shape*` / `unique_ptr<Shape>` 删除派生对象；如果析构不是虚的，只会执行基类析构，派生部分不释放，且这是未定义行为。
2. 空类至少占 1 字节；有虚函数的对象多一个虚表指针（本机 8 字节）；再加一个 `int` 后按 8 字节对齐到 16。1 字节是标准对"不同对象不同地址"的要求，8/16 是本机 ABI 的布局，不是标准规定。
3. 继承会让 `Car` 自动获得 `Engine` 的公有接口，外部可以把汽车当引擎启动；组合只暴露 `Car` 想暴露的接口，`Engine` 可以替换/封装，语义更清楚。
4. 指针版失败返回 `nullptr` ，因为指针可以有空值；引用版失败抛 `std::bad_cast` ，因为引用必须绑定对象，没有"空引用"可返回。
5. `variant` 适合封闭、以值为主的类型集合，新增类型要改 `variant` 的模板参数并检查所有 `visit` ；继承适合开放、运行期扩展的集合，新增类型只需实现接口并在工厂注册，调用方不用改。

第 4 章

模板与泛型

本章 3 个单元 · 页眉从这里之后的页面开始出现

1. 模板入门：函数模板、类模板、实例化 零基础 · 第 8 单元 · 真编译真运行
2. concepts 与约束：从 SFINAE 到 requires 零基础 · 第 9 单元 · 真编译真运行
3. 可变参数模板与完美转发：forward、折叠表达式 零基础 · 第 10 单元 · 真编译真运行

模板入门：函数模板、类模板、实例化

上一章的虚函数在运行期选实现；模板在编译期生成实现。`max_of(3, 9)` 与 `max_of(3.5, 1.5)` 看起来是同一个函数，编译器却为它们生成了两份代码，一份收 `int`，一份收 `double`。这叫**实例化**：模板本身不是函数，是一张"怎么生成函数"的图纸；具体类型填进去，才得到一个真正的函数。

模板是 C++ 标准库的基础：`std::vector<T>`、`std::string`、`std::sort` 都是模板。这一单元只讲入门：函数模板怎么推导、类模板怎么用、为什么定义必须放在头文件、显式实例化是什么、非类型参数怎么用、模板与普通函数重载时谁优先。

◎ 学习目标 · 看完这一单元，你应该能

- 写一个函数模板，说出编译器怎样从实参推导 `T`，以及什么时候需要显式写 `<int>`；
- 写一个类模板，区分"模板"与"实例化后的具体类型"；
- 解释模板定义为什么通常放在头文件，并读懂"定义在 `.cpp` 里"导致的链接错误；
- 用显式实例化与 `extern template` 控制实例化发生在哪个翻译单元；
- 说出非类型模板参数的用途，以及模板与普通函数重载时非模板版本为什么优先。

8.1 先看工程长什么样

这一单元在 `examples/ch04/` 下建六个多文件例子。模板的定义放在头文件里（原因见 8.4），所以 `include/` 会比前几章更"重"；`src/<例名>.cpp` 仍然承担非模板函数或显式实例化，`main.cpp` 只做驱动。

目录树 **工程：examples/ch04/ 这一单元新增的六个例子**

第 1 步 先建函数模板与类模板

```
examples/
└── ch04/
    └── max_template/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── max_template.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── max_template.cpp ← 本课新增
        └── max_template.json ← 本课新增
```

第 2 步 加上"定义藏进 .cpp"的反例与显式实例化

```
examples/
└── ch04/
    ├── max_template/ ：同上
    ├── box_template/ ← 本课新增
    ├── template_in_cpp/ ← 本课新增
    └── explicit_instantiation/ ← 本课新增
```

第 3 步 最后加上非类型参数与重载

```
examples/
└── ch04/
    ├── 前四个例子 ：同上
    ├── non_type_param/ ← 本课新增
    └── template_overload/ ← 本课新增
```

第 1 帧的 `max_template/` 把函数模板定义放进头文件，`main.cpp` 用 `int`、`double`、`string` 三种类型调用；`box_template/` 是类模板。第 2 帧的 `template_in_cpp/` 故意把模板定义藏在 `.cpp` 里，让链接失败；`explicit_instantiation/` 用 `extern template` + 显式实例化把定义留在 `.cpp` 里还能链接。第 3 帧的 `non_type_param/` 用 `FixedArray<T, N>` 展示非类型参数；`template_overload/` 对照普通函数、函数模板与显式特化。

8.2 函数模板：从实参推导 `T`

cpp **函数模板：定义直接放在头文件** examples/ch04/max_template/include/max_template.h 第 6–9 行

```cpp
template <typename T>
T max_of(const T& a, const T& b) {
    return a < b ? b : a;
}
```

cpp **main：三种类型，三次实例化** examples/ch04/max_template/src/main.cpp 第 6–12 行

```cpp
int main() {
    std::cout << "int=" << max_of(3, 9) << "\n";
    std::cout << "double=" << max_of(3.5, 1.5) << "\n";
    std::cout << "string=" << max_of(std::string("apple"), std::string("pear")) << "\n";
    std::cout << "demo=" << demo_name() << "\n";
    return 0;
}
```

终端输出 终端输出 max-template-out

```
int=9
double=3.5
string=pear
demo=max_template
```

`max_of(3, 9)` 的两个实参都是 `int`，编译器推导 `T = int`，生成一份 `int max_of(const int&, const int&)`；`max_of(3.5, 1.5)` 推导 `T = double`；`std::string` 版本同理。**模板不是“一个函数接受任意类型”，而是“为每种用到的类型各生成一个函数”**；生成发生在编译期，运行期没有“类型判断”这回事。

推导规则不总是“看第一个实参”：如果两个实参推导出不同的 `T`（例如 `max_of(3, 4.5)`），推导失败，编译器报 `no matching function`。这时可以显式指定 `max_of<double>(3, 4.5)`，把 `3` 转成 `double`；也可以把两个实参改成同类型。**显式模板实参的作用是“不让编译器猜”，不是“强制转换”**；它仍然要满足参数类型的匹配规则。

模板参数 `T` 也可以出现在返回类型、多个参数、默认实参里；`typename` 与 `class` 在模板参数列表里等价，本书统一用 `typename`。模板可以有多个类型参数（`template <typename Key, typename Value>`），也可以有非类型参数（8.6 节）。模板参数的名字只在模板定义内有效，实例化后不再存在。

推导规则和函数参数的写法强相关。`T max_of(T a, T b)` 按值传参，数组会退化成指针、函数会退化成函数指针、顶层 `const` 会被丢掉；`T max_of(T& a, T& b)` 只接左值，`T` 保留实参类型；本单元的 `const T&` 能接左值也能接临时对象，`T` 仍按实参推导为 `int` / `double` / `std::string`。**想保留数组或函数类型，必须用引用；按值传参一定会退化。**

显式模板实参可以打破推导僵局，但它不是“任意转换”。`max_of<double>(3, 4.5)` 把 `3` 转成 `double`，因为形参是 `const double&`；如果模板参数出现在无法转换的位置（例如要求 `T*`），显式指定也救不了。**先让两个实参推导出同一个 `T`，再考虑显式实参**；显式实参更多用于“指定返回类型”或“指定无法从实参推导的参数”。

模板实例化是**按需**的：一个类模板里如果有十个成员函数，你只调用其中一个，编译器通常只实例化用到的那个（虚函数与某些场景例外）。这让模板库可以暴露大量接口而不付出全部代码体积；也意味着“模板里某段代码从没被实例化”时，它的错误可能一直不出现。**模板的编译错误往往延迟到“第一次真正使用”那一刻**，读错误时要先找到实例化链。

8.3 类模板：`Box<T>` 是一张图纸

cpp **类模板：成员函数定义也在头文件** examples/ch04/box_template/include/box_template.h 第 6–21 行

```cpp
template <typename T>
class Box {
public:
    explicit Box(T value) : value_(value) {}

    const T& get() const {
        return value_;
    }

    void set(const T& value) {
```

```
        value_ = value;
    }

private:
    T value_;
};
```

cpp **main: Box<int> 与 Box<std::string> 是两个具体类型** examples/ch04/box_template/src/main.cpp 第 6–15 行

```cpp
int main() {
    Box<int> number(7);
    number.set(9);
    std::cout << "int=" << number.get() << "\n";

    Box<std::string> text("hi");
    std::cout << "string=" << text.get() << "\n";
    std::cout << "demo=" << demo_name() << "\n";
    return 0;
}
```

终端输出 终端输出 box-template-out

```
int=9
string=hi
demo=box_template
```

`Box<int>` 与 `Box<std::string>` 是**两个不同的类型**：它们的成员函数各有一份实例化，`sizeof` 也可能不同。`Box` 本身不是类型，不能声明 `Box box;`；必须写 `Box<int> box;`。

{} 语法规则 · 类模板成员在类外定义时的写法

- 每个成员定义前都要写 `template <typename T>`；
- 成员名要写成 `Box<T>::get`，不能只写 `get`；
- 定义仍然要放在头文件里，保证使用点可见。

本书的例子把成员函数定义在类内，省掉重复的模板头。**类模板的静态成员、嵌套类型、成员模板也都要在头文件里可见**，否则使用它们的翻译单元无法实例化。

类模板也可以有默认实参：`template <typename T = int> class Box;` 让 `Box<>` 等价于 `Box<int>`。默认实参可以省掉重复写类型的麻烦，但也让"这里到底用了哪个类型"变得不显眼；**接口里用默认实参时，要在文档或别名里写清楚默认是什么。**

C++17 起有类模板实参推导（CTAD）：如果构造函数能推导出模板参数，写 `Box number(7);` 就能得到 `Box<int>`，不必写 `Box<int> number(7);`。CTAD 只从构造函数推导，不读取你心里的意图；`Box text("hi")` 会推导成 `Box<const char*>`，而不是 `Box<std::string>`，因为字符串字面量的类型是 `const char[N]`，按值传参后退化成 `const char*`。**CTAD 方便，但遇到字符串、数组、花括号初始化时要特别小心**；不确定就写全 `Box<std::string>`。

`using IntBox = Box<int>;` 是别名模板/别名声明，它不创建新类型，只是给 `Box<int>` 起个短名字。**别名不会改变类型系统**：`IntBox` 与 `Box<int>` 在重载、特化、`is_same_v` 里完全等价。需要"真正的新类型"时，要用继承或包装类，而不是别名。

8.4 为什么模板定义必须可见

cpp **只有声明，没有定义** examples/ch04/template_in_cpp/include/template_in_cpp.h 第 4–5 行

```cpp
template <typename T>
T twice(T value);
```

cpp **main: 调用 twice(21)** examples/ch04/template_in_cpp/src/main.cpp 第 3–5 行

```cpp
int main() {
    return twice(21);
}
```

编译诊断 本机真实 stderr：模板定义不可见，链接失败 template-in-cpp-diag

```
collect2.exe: error: ld returned 5 exit status
```

`twice` 的定义在 `src/template_in_cpp.cpp` 里，但那个文件没有用到 `twice<int>`，所以编译器不会为它生成 `int twice<int>(int)` 的机器码；`main.cpp` 只看见声明，也无法生成定义。链接器于是找不到 `int twice<int>(int)`，报 `undefined reference`。**这不是"模板不能放 .cpp"，而是"模板的实例化需要定义在调用点可见"**：编译器在调用点才知道 `T = int`，而它手里只有声明，没法生成代码。

解决办法有三种：

1. **把定义放进头文件**（最常见）：每个包含头文件的翻译单元都能在调用点实例化；链接器用弱符号/COMDAT 合并重复实例。
2. **显式实例化**：在一个 `.cpp` 里写 `template int twice<int>(int);`，强制生成这份实例；头文件里用 `extern template` 告诉其他翻译单元"别自己生成，去链接"。
3. **把模板限制在单个 `.cpp` 内使用**：不对外暴露，只在定义它的文件里调用；这适合内部实现细节，不适合库接口。

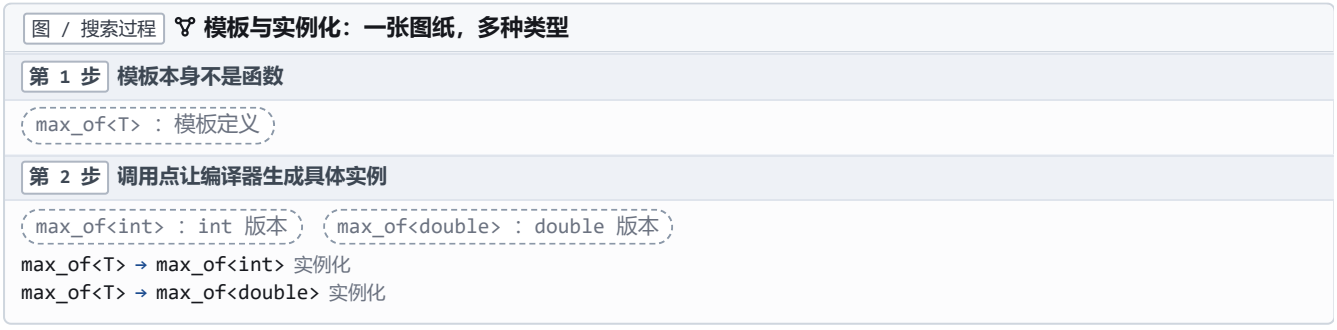
头文件里的模板定义会被多个翻译单元各实例化一份，链接器靠 COMDAT/弱符号把它们合并成一份。**这要求每份实例化的"记号序列"一致**：同一个模板在不同翻译单元里必须来自同一份定义；如果两个 `.cpp` 各写了一个同名模板、函数体不同，链接器可能随便挑一份，程序行为就没有保证。ODR 对模板同样适用，只是它允许"重复定义"而不是"不同定义"。

显式实例化有两种写法：`template int twice<int>(int);` 是**实例化定义**，强制编译器在这里生成代码；`extern template int twice<int>(int);` 是**实例化声明**，告诉编译器"别在这里生成，去别处链接"。前者一个程序里只能有一处，后者可以出现在多个头文件里。**`extern template` 不是"保证有定义"**；如果没有任何一个 `.cpp` 提供显式实例化，链接仍然会失败。

类模板的显式实例化 `template class Box<int>;` 会实例化它的所有非模板成员；如果某个成员对 `int` 不合法，即使你从没调用它，也会在这里报错。**显式实例化类模板前，先确认所有成员对该类型都成立**；否则就只显式实例化需要的成员函数，或者把不满足条件的成员用 `if constexpr` / 约束隔开。



第 1 帧的 `main.cpp` 只看见声明，第 2 帧的调用点要求实例化 `int twice<int>(int)`，但两个翻译单元都没有生成它——链接器只能报 `undefined reference`。**模板的"声明与定义分家"与普通函数相反：普通函数可以把定义藏进 `.cpp`，模板不行，除非你显式实例化。**



第 1 帧的 `max_of<T>` 只是图纸；第 2 帧的 `max_of<int>` 与 `max_of<double>` 才是编译器生成的两份具体函数。实例化由"用到了哪种类型"决定，而不是由模板定义本身决定。

> ⚠ **陷阱 · 模板编译通过、链接失败，以为是编译器 bug**
>
> 你会看到：模板函数在 `.cpp` 里写得好好的，`main.cpp` 也 include 了头文件，编译每个文件都过，链接却报 `undefined reference to 'int twice<int>(int)'`。你会误判成"模板写错了"或"链接器有问题"。真相是：编译器在每个翻译单元独立工作；`main.cpp` 只有声明，生成不了 `int` 版本的机器码；`template_in_cpp.cpp` 没有用到 `int` 版本，也不会主动生成。**模板不是"声明一次、到处链接"的普通函数；它需要定义在实例化点可见，或者被显式实例化。**

8.5 显式实例化与 `extern template`

cpp **头文件：声明 + 抑制隐式实例化** examples/ch04/explicit_instantiation/include/explicit_instantiation.h 第 4–7 行

```cpp
template <typename T>
T triple(T value);

extern template int triple<int>(int);
```

cpp **main：只调用 int 版本** examples/ch04/explicit_instantiation/src/main.cpp 第 5–8 行

```cpp
int main() {
    std::cout << "triple=" << triple(3) << "\n";
    return 0;
}
```

终端输出 终端输出 explicit-instantiation-out

```
triple=9
```

`explicit_instantiation.cpp` 里既有定义，又有 `template int triple<int>(int);`：这行强制编译器在**这个**翻译单元生成 `int` 版本。头文件里的 `extern template int triple<int>(int);` 告诉其他翻译单元"这个实例已经在别处生成了，不要重复生成，链接时去找它"。于是 `main.cpp` 只看见声明也能链接成功。

显式实例化的收益是**减少重复实例化、缩短编译时间**，代价是你要手动维护"哪些类型在哪个 `.cpp` 里实例化"。如果漏写 `extern template`，每个翻译单元都会生成一份，链接器通常会合并（弱符号），但编译时间与目标文件会变大；如果漏写显式实例化定义，就会回到 `undefined reference`。这是工程取舍，不是语法必需。

8.6 非类型模板参数：把值写进类型

cpp **FixedArray<T, N>：N 是编译期常量** examples/ch04/non_type_param/include/non_type_param.h 第 6–19 行

```cpp
template <typename T, int N>
class FixedArray {
public:
    constexpr int size() const {
        return N;
    }

    T& operator[](int index) {
        return data_[index];
    }

private:
    T data_[N] = {};
};
```

cpp **main：FixedArray<int, 3> 的大小是类型的一部分** examples/ch04/non_type_param/src/main.cpp 第 7–13 行

```cpp
int main() {
    FixedArray<int, 3> values;
    values[1] = 42;
    std::cout << "size=" << values.size() << " value=" << values[1] << "\n";
    std::cout << "demo=" << non_type_demo() << "\n";
    return 0;
}
```

终端输出 终端输出 non-type-param-out

```
size=3 value=42
demo=4
```

`FixedArray<int, 3>` 与 `FixedArray<int, 4>` 是不同类型；`N` 在编译期就是常量，所以 `T data_[N]` 可以直接作为数组成员，不需要 `new`。非类型参数可以是整型、枚举、指针、引用、`nullptr_t` 等；C++20 起还允许某些类类型作为非类型参数，但本书不展开。**非类型参数的用途是“把配置写进类型”**：数组长度、维度、策略开关都可以这样做，代价是每种取值生成一份代码。

`FixedArray` 的 `operator[]` 没有边界检查；如果 `index >= N`，是未定义行为。模板不会自动帮你检查运行期下标，除非你用 `std::array::at` 或自己加判断。**模板保证的是“类型与编译期常量正确”，不是“运行期输入安全”。**

非类型参数必须是编译期常量：`FixedArray<int, 4>` 可以，`FixedArray<int, n>`（`n` 是运行期变量）不行。整型非类型参数还允许从常量表达式转换，例如 `FixedArray<int, 4u>` 可能被转成 `int`；但转换必须保留值，不能溢出。**非类型参数让“配置”成为类型的一部分，因此每种配置都是不同类型、各生成一份代码**；配置值很多时，代码体积会明显增加；选非类型参数前，先确认这个配置真的需要在类型里，而不是运行期传一个普通参数就够了。

8.7 模板与重载：非模板优先

cpp **普通函数、函数模板、显式特化** examples/ch04/template_overload/include/template_overload.h 第 7–15 行

```cpp
void show(int value);

template <typename T>
void show(T value) {
    std::cout << "template value " << value << std::endl;
}

template <>
void show(std::string value);
```

cpp **main：三次调用，三种选择** examples/ch04/template_overload/src/main.cpp 第 5–10 行

```cpp
int main() {
    show(7);
    show(2.5);
    show<std::string>("pear");
    return 0;
}
```

终端输出 终端输出 template-overload-out

```
non-template int 7
template value 2.5
template string pear
```

`show(7)` 同时匹配普通函数 `show(int)` 与模板 `show<int>`；重载决议优先选择**非模板**的精确匹配，所以输出 `non-template int 7`。`show(2.5)` 只能匹配模板（普通函数要 `double → int` 转换），输出 `template value 2.5`。`show<std::string>("pear")` 显式指定模板实参，匹配 `std::string` 的显式特化，输出 `template string pear`。

模板特化不是重载：显式特化 `template <> void show(std::string)` 是“为特定类型提供另一份实现”，它仍然属于模板家族；重载决议先选主模板/普通函数，再决定是否用特化。写特化时要保证主模板声明在特化之前，否则编译器可能报“特化在实例化之后”。本书不深入模板元编程，只要求你读懂“非模板优先、特化是例外”这两条。

如果没有普通函数，只有两个模板，重载决议会选“更特殊”的那个：`template <typename T> void show(T)` 与 `template <typename T> void show(T*)`，传指针时后者更特殊。**偏序（partial ordering）比较的是“哪个模板能接受更少的类型”**，不是“哪个写得晚”。普通函数比模板更特殊，是因为它不需要推导；显式特化又比主模板更特殊，因为它指定了具体类型。

函数模板的显式特化容易踩坑：特化必须在使用之前声明，否则编译器可能先用主模板实例化，再看到特化就报“特化在实例化之后”。**更推荐用重载代替函数模板特化**：写一个普通函数 `void show(std::string)`，让重载决议自然选它，比

`template <>` 更直观，也不会遇到声明顺序问题。类模板的特化则常用，因为类没有"重载"这条路。类模板还可以写偏特化，只固定一部分模板参数；函数模板不支持偏特化，只能全特化，这是两者最容易被混淆的差别。

8.8 本机实测：四次实例化与一次链接失败

实验记录 · 本机实测：模板的生成与链接

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `max_template` 的 stdout 是 `int=9` / `double=3.5` / `string=pear`：同一份模板生成了三份实例；
- `box_template` 的 stdout 是 `int=9` / `string=hi`：`Box<int>` 与 `Box<std::string>` 各自实例化；
- `template_in_cpp` 编译通过、链接失败，诊断是 `undefined reference to 'int twice<int>(int)'`；
- `explicit_instantiation` 的 stdout 是 `triple=9`：显式实例化 + `extern template` 让定义留在 `.cpp` 也能链接。

8.9 下一单元从哪继续

下一单元 `8.2 concepts 与约束` 会给模板参数加"门槛"：不是什么类型都能传给 `max_of`，也不是所有类型都该走同一份实现。`requires` 表达式与标准 concepts 把"这个类型必须支持 `<` / 必须可打印"写成编译期约束，让错误在调用点就说清楚，而不是在模板体深处炸出一堆实例化栈。

小结 · 这一节要带走的五件事

- 函数模板按实参推导类型，为每种用到的类型生成一份实例；推导不出时可以显式写 `<T>`。
- 类模板不是类型，`Box<int>` 才是；成员函数与静态成员的定义通常要放在头文件。
- 模板定义必须对实例化点可见；放 `.cpp` 会导致链接期 `undefined reference`，除非显式实例化。
- `extern template` 抑制其他翻译单元的隐式实例化，显式实例化定义负责生成唯一一份。
- 非类型模板参数把编译期常量写进类型；重载时非模板函数优先于模板。

自测题 · 自测题（先自己做，再对答案）

1. `max_of(3, 9)` 与 `max_of(3.5, 1.5)` 调用的函数是同一个吗？编译器分别做了什么？
2. `Box<int>` 与 `Box` 有什么区别？为什么不能声明 `Box box;`？
3. 模板定义放在 `.cpp` 里，为什么编译能过、链接却失败？三种修法各是什么？
4. `extern template int triple<int>(int);` 与 `template int triple<int>(int);` 各写在哪里、各起什么作用？
5. `show(7)` 同时匹配普通函数与模板，为什么选普通函数？`show<std::string>("pear")` 为什么走特化？

注 · 自测答案

1. 不是同一个函数；编译器为 `int` 与 `double` 各生成一份实例。运行期没有类型判断，调用的是两份不同的机器码。
2. `Box` 是类模板（图纸），`Box<int>` 是具体类型（实例化结果）；不能声明 `Box box;`，因为没有指定 `T`，编译器无法生成完整类型。
3. 因为调用点只看见声明，无法实例化；定义所在的 `.cpp` 没有用到该实例，也不会生成。修法：定义放头文件、显式实例化、或把模板限制在单个 `.cpp` 内使用。
4. `extern template` 通常写在头文件，抑制其他翻译单元的隐式实例化；`template int triple<int>(int);` 写在某个 `.cpp`，强制生成该实例。两者配合，定义可以留在 `.cpp`。
5. 重载决议优先选择非模板的精确匹配，所以 `show(int)` 胜出；`show<std::string>` 显式指定模板实参，匹配为 `std::string` 提供的显式特化。

concepts 与约束：从 SFINAE 到 requires

上一单元的模板只问"类型能不能填进来"，不问"这个类型能不能做我要求的事"。`max_of` 要求类型支持 `<`，`dump` 要求类型能遍历；如果没有约束，传错类型时错误会在模板体深处炸开，一屏都是实例化栈。concepts 把"类型必须满足什

么"写成编译期谓词，放在 template 头上；不满足时，编译器在调用点就说"constraints not satisfied"。

这一单元从标准 concepts（`std::integral`、`std::floating_point`、`std::ranges::range`）开始，再写自定义 concept（`requires` 表达式），最后用旧式 `enable_if` 对照。目标是让你能读懂约束失败的真实诊断，并能自己写一个"只接受支持某操作的类型"的模板。

◎ 学习目标 · 看完这一单元，你应该能

- 用 `std::integral` / `std::floating_point` 约束模板参数，并说出约束失败时诊断停在哪；
- 写一个 `requires` 表达式，检查类型是否有某个成员函数、返回类型是否可转换；
- 用多个约束重载同一函数名，解释重载决议怎样选到更匹配的那个；
- 用 `std::ranges::range` 约束容器/视图参数，并说清"满足 range"不等于"元素类型可打印"；
- 读懂 `enable_if` 旧写法与 concepts 的对应关系，知道 concepts 报错为什么更直接。

9.1 先看工程长什么样

这一单元在 `examples/ch04/` 下再加六个多文件例子。约束都写在头文件里的模板头上，`.cpp` 只放非模板辅助函数，`main.cpp` 做调用与打印。

目录树 **工程：examples/ch04/ 这一单元新增的六个例子**

第 1 步 先建标准 concept 与失败对照

```
examples/
└── ch04/
    ├── 上一单元 : cpp08 的六个例子
    └── concept_basic/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── concept_basic.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── concept_basic.cpp ← 本课新增
        └── concept_basic.json ← 本课新增
```

第 2 步 加上约束重载与自定义 concept

```
examples/
└── ch04/
    ├── concept_basic/ : 同上
    ├── concept_error/ ← 本课新增
    ├── concept_overload/ ← 本课新增
    └── concept_compose/ ← 本课新增
```

第 3 步 最后加上 ranges 与旧式 SFINAE

```
examples/
└── ch04/
    ├── 前四个例子 : 同上
    ├── ranges_concept/ ← 本课新增
    └── sfinae_old/ ← 本课新增
```

第 1 帧的 `concept_basic/` 用 `std::integral` 约束 `twice`；`concept_error/` 故意传 `std::string`，让约束失败。第 2 帧的 `concept_overload/` 用 `integral` / `floating_point` 两个约束重载 `kind`；`concept_compose/` 自定义 `Named` concept。第 3 帧的 `ranges_concept/` 用 `std::ranges::range` 接收 `vector` / `list`；`sfinae_old/` 用 `enable_if` 写同样的约束，作为旧写法对照。

9.2 标准 concept：把要求写在 template 头上

cpp **twice：只接受整型** examples/ch04/concept_basic/include/concept_basic.h 第 6-9 行

```cpp
template <std::integral T>
T twice(T value) {
    return value + value;
```

```
9  }
```

cpp main: int 满足约束 examples/ch04/concept_basic/src/main.cpp 第 5–9 行

```cpp
5  int main() {
6      std::cout << "twice=" << twice(21) << "\n";
7      std::cout << "demo=" << concept_demo() << "\n";
8      return 0;
9  }
```

终端输出 终端输出 concept-basic-out

```
twice=42
demo=40
```

`std::integral` 是 `<concepts>` 里的标准 concept，等价于"`std::is_integral_v<T>` 为真"。`template <std::integral T>` 把约束写在模板参数位置；也可以写 `template <typename T> requires std::integral<T>`，两种写法等价。约束是**编译期**的：`T` 不满足时，这个模板根本不进入候选集，不会生成函数体。

`twice` 的函数体只用了 `value + value`，这个操作对整型成立；如果传入 `double`，`std::integral` 不满足，即使 `+` 也能用，模板仍然被拒绝——约束是"我要求整型"，不是"我要求能加"。**约束应该表达接口真正需要的能力**：如果函数只需要"能相加"，用 `std::integral` 太窄；如果需要"整型"，用它就合适。标准库提供了一批常用 concept：`std::integral`、`std::floating_point`、`std::same_as`、`std::convertible_to`、`std::totally_ordered`、`std::ranges::range` 等。

标准原文 · 标准原文：concept 是什么

> A concept is a template that defines constraints on its template arguments.
>
> 大意：concept 是一种模板，用来给它的模板实参定义约束；concept 本身不会被实例化，它在编译期求值为真或假。
>
> 出处：N4950 §13.7.9「Concept definitions」（[temp.concept]）。

concept 求值看的是"替换后的类型"，但模板参数推导会先去掉引用与顶层 `const`：`twice(21)` 推导 `T = int`；传一个 `const int` 变量，按值传参仍然推导 `T = int`。如果参数写成 `const T&`，`T` 也会去掉 `const`。**不要把"concept 对 `T` 为真"误解成"对实参的原始类型逐字为真"**；推导规则先发生，concept 再检查推导结果。

`std::integral` 与 `std::signed_integral` 是包含关系：`signed_integral<T>` 蕴含 `integral<T>`，但不反过来。写两个重载时，`template <std::integral T>` 与 `template <std::signed_integral T>` 对 `int` 都可行，编译器会选后者，因为它的约束更强。**这种"更强约束优先"是 concepts 的重要能力**；它让"通用版本 + 特化版本"可以用约束表达，不必靠标签分发或 `enable_if` 技巧。

标准 concept 也可以组合成自己的 concept：`template <class T> concept Number = std::integral<T> || std::floating_point<T>;`。组合后的 concept 仍然参与包含关系，`Number` 不会自动比 `std::integral` 更强或更弱，除非编译器能证明逻辑包含。**写组合 concept 时，保持定义简单、可读**；过度嵌套会让诊断变长，也会让"谁更特殊"变得难以判断。

9.3 约束失败：诊断停在调用点

cpp concept_error: 把 string 传给只收整型的 twice examples/ch04/concept_error/src/main.cpp 第 5–7 行

```cpp
5  int main() {
6      return static_cast<int>(twice(std::string("x")).size());
7  }
```

编译诊断 本机真实 stderr：constraints not satisfied　concept-error-diag

```
src/main.cpp: In function 'int main()':
src/main.cpp:6:34: error: no matching function for call to 'twice(std::string)'
src/main.cpp:6:34: note: there is 1 candidate
In file included from src/main.cpp:1:
include/concept_error.h:7:3: note: candidate 1: 'template<class T>  requires  integral<T> T twice(T)'
include/concept_error.h:7:3: note: template argument deduction/substitution failed:
include/concept_error.h:7:3: note: constraints not satisfied
In file included from include/concept_error.h:4:
C:/msys64/ucrt64/include/c++/16.1.0/concepts: In substitution of 'template<class T>  requires  integral<T> T
twice(T) [with T = std::__cxx11::basic_string<char>]':
src/main.cpp:6:34:   required from here
C:/msys64/ucrt64/include/c++/16.1.0/concepts:109:13:   required for the satisfaction of 'integral<T>' [with T
= std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >]
C:/msys64/ucrt64/include/c++/16.1.0/concepts:109:24: note: the expression 'is_integral_v<_Tp> [with _Tp =
std::__cxx11::basic_string<char, std::char_traits<char>, std::allocator<char> >]' evaluated to 'false'
```

诊断的前三行就把问题说清了：调用点 `twice(std::string)` 找不到匹配，候选是 `requires integral<T> T twice(T)`，失败原因是 `constraints not satisfied`；后面的 `is_integral_v<...> evaluated to 'false'` 是约束被拒绝的具体证据。**concepts 的收益不是“编译更快”，而是“错误信息指向约束，而不是模板体深处的某一行”。**

这份诊断之所以只有 12 行，是因为清单里加了 `-fdiagnostics-plain-output`（去掉彩色与源码箭头）与 `-fconcepts-diagnostics-depth=1`（限制约束展开深度）；不加这些参数，GCC 会把约束链展开成二十多行。**诊断格式参数只影响“怎么显示”，不影响“是否满足约束”**；本书的基础警告档没有变。

⚠ 陷阱 · 把 `requires` 当成运行期 `if`

你会看到：写了 `requires` 表达式，就以为函数里可以随便用被约束的操作，运行期再判断“类型对不对”。你会误判成“concepts 是运行期反射”。真相是：concept 在编译期求值，约束不满足时模板不会被选中，根本不会生成运行期代码；函数体里也不需要再写 `if (is_integral<T>)`。**约束是“进门前检查”，不是“进门后判断”。**

9.4 约束重载：更匹配的约束胜出

cpp **两个 kind：一个收整型，一个收浮点** examples/ch04/concept_overload/include/concept_overload.h　第 7–15 行

```cpp
template <std::integral T>
std::string kind(T) {
    return "integral";
}

template <std::floating_point T>
std::string kind(T) {
    return "floating";
}
```

cpp **main: int 与 double 各走一个重载** examples/ch04/concept_overload/src/main.cpp　第 7–12 行

```cpp
int main() {
    std::cout << "int=" << kind(7) << "\n";
    std::cout << "double=" << kind(2.5) << "\n";
    std::cout << "demo=" << overload_demo() << "\n";
    return 0;
}
```

终端输出 终端输出　concept-overload-out

```
int=integral
double=floating
demo=integral/floating
```

`kind(7)` 只满足 `std::integral`，`kind(2.5)` 只满足 `std::floating_point`；两个模板的约束互斥，重载决议没有歧义。如果一个类型同时满足两个约束（例如自定义类型同时满足两个 concept），编译器会比较约束的“包含关系”，选择更严格的那个；如果无法比较，就报歧义。**约束不是标签，它参与重载决议**；这就是 concepts 相对 `enable_if` 的最大区别之一——约束可以被编译器比较，而不是只做“能不能替换”的开关。

图 / 搜索过程 **约束与重载候选**

第 1 步 **调用 kind(7)**

kind(7) : int 实参　integral<T> : 满足　floating_point<T> : 不满足

kind(7) → integral<T> 唯一候选

第 2 步 **调用 kind(2.5)**

kind(2.5) : double 实参　integral<T> : 不满足　floating_point<T> : 满足

kind(2.5) → floating_point<T> 唯一候选

第 1 帧的 `int` 只进入 `integral` 候选；第 2 帧的 `double` 只进入 `floating_point` 候选。约束把"这个重载适不适合"从函数体里的 `if` 前移到了候选集构造阶段。

9.5 自定义 concept：用 `requires` 表达式描述操作

cpp **Named: 要求 name() 返回可转成 string 的东西** examples/ch04/concept_compose/include/concept_compose.h 第 7–19 行

```cpp
template <typename T>
concept Named = requires(const T& value) {
    { value.name() } -> std::convertible_to<std::string>;
};

struct Item {
    std::string name() const;
};

template <Named T>
std::string label(const T& value) {
    return value.name();
}
```

终端输出 终端输出　concept-compose-out

```
label=item
```

`requires(const T& value) { ... }` 是一个**要求列表**：它不求值，只检查"这些表达式在给定类型上是否合法、返回类型是否满足约束"。`{ value.name() } -> std::convertible_to<std::string>` 的意思是：`value.name()` 必须合法，且结果能转换成 `std::string`。`Item::name()` 返回 `std::string`，满足；如果某个类型没有 `name()`，`Named` 为假，`label` 不会进入候选集。

`requires` 表达式里可以写多行要求：简单要求（`value.size()`）、类型要求（`typename T::value_type`）、复合要求（带返回类型约束）、嵌套要求（`requires std::integral<T>`）。**要求列表只做语法与类型检查，不执行函数体**；所以 `name()` 里有没有副作用、返回的是不是临时对象，concept 都不知道。约束越宽，模板越通用，但函数体越容易在实例化时才发现问题；约束越窄，错误越早，但复用越少。

四种要求各有分工。简单要求只检查表达式"写不写得出来"，例如 `requires(T value) { value.size(); }`；它不关心 `size()` 返回什么，也不关心调用是否真的会产生结果。类型要求检查某个嵌套类型是否存在，例如 `typename T::value_type`，很多容器用它暴露元素类型。复合要求用 `{ expression } -> constraint` 同时检查"表达式合法"与"结果满足约束"，前面的 `Named` 就是复合要求。嵌套要求写 `requires 条件`，把另一个布尔常量表达式并进来，常用来组合两个 concept。

`requires` 表达式还可以写成"参数列表 + 要求体"，参数列表里的变量只是**占位**，不会真的构造对象，也不会求值。`requires(T value)` 不代表 `T` 必须能默认构造；它只是给要求体一个类型为 `T` 的名字。**不要因为写了 `T value` 就以为 concept 检查了构造**；要检查构造，得写 `requires { T{}; }` 或 `std::default_initializable<T>`。

concept 只能检查"语法与类型"，不能检查"语义"。一个类型可以有 `size()` 但返回负数，可以有 `name()` 但返回空字符串，concept 都认为它满足要求。**concept 是编译期的接口检查，不是行为契约**；行为契约要靠文档、测试与运行期断言。把 concept 当成"自动证明实现正确"会失望；它的价值是把"这个类型有没有这个操作"从模板实例化深处提到候选集构造阶段。

约束可以组合：`std::integral<T> && std::signed_integral<T>` 表示两个条件都要满足；`||` 表示任一满足。约束之间还有**包含关系**（subsumption）：如果约束 A 逻辑上包含约束 B，A 更特殊，重载决议会优先选 A。例如 `template <std::integral T>` 与 `template <std::signed_integral T>`，后者包含前者，传 `int` 时选后者。**包含关系不是"谁写得更长"，而是编译器能证明的原子约束组合**；不要把运行期的 `if` 逻辑套到约束排序上。

标准库已经提供了大量 concept：`std::integral`、`std::floating_point`、`std::same_as`、`std::convertible_to`、`std::totally_ordered`、`std::ranges::range` 等。**优先用标准 concept，再考虑自定义**；标准 concept 经过广泛使用，边界更清楚，诊断也更容易被工具识别。自定义 concept 应聚焦"你的接口真正需要什么"，不要把实现细节写进约束。



第 1 帧的 `Named` 定义必须放在头文件里，和模板定义一样，因为使用点需要它来求值；第 2 帧的 `Named<Item>` 在编译期得到真，`label` 才会被实例化。**concept 不能只在 `.cpp` 里定义，除非使用点也在同一个翻译单元。**

9.6 `std::ranges::range`：约束"能遍历"

cpp **dump: 任何 range 都能进来** examples/ch04/ranges_concept/include/ranges_concept.h 第 7–15 行

```cpp
template <std::ranges::range R>
std::string dump(const R& range) {
    std::string result;
    for (const auto& item : range) {
        result += std::to_string(item);
        result += ',';
    }
    return result;
}
```

cpp **main: vector 与 list 都满足 range** examples/ch04/ranges_concept/src/main.cpp 第 9–16 行

```cpp
int main() {
    std::vector<int> values{4, 5};
    std::list<int> other{6, 7};
    std::cout << "vector=" << dump(values) << "\n";
    std::cout << "list=" << dump(other) << "\n";
    std::cout << "demo=" << range_demo() << "\n";
    return 0;
}
```

终端输出 终端输出 ranges-concept-out

```
vector=4,5,
list=6,7,
demo=1,2,3,
```

`std::ranges::range` 只要求类型有 `begin()` / `end()`（或可被 `std::ranges::begin/end` 取到），不要求它是容器，也不要求元素类型能 `to_string`。`dump` 的函数体用了 `std::to_string(item)`，这要求元素是算术类型；如果传一个 `vector<std::string>`，`range` 约束仍然满足，但函数体实例化时会报 `no matching function for call to to_string(std::string)`。**约束只检查你写出来的要求；函数体里隐含的要求，约束不会自动知道。** 要让 `dump` 接受字符串元素，应该把元素约束也写进 concept（例如 `std::ranges::range R` + `requires` 元素可格式化），或者改用 `std::format` / `std::print`。

`range` 只是范围概念层级的最底层。再往上还有 `std::ranges::input_range`（能单遍读取）、`forward_range`（能多遍向前）、`bidirectional_range`（能后退）、`random_access_range`（能随机访问）、`sized_range`（`size()` 是常数时间）、`borrowed_range`（迭代器可以活得比范围长）等。**约束越往上，算法能用的操作越多**：`std::ranges::sort` 要求 `random_access_range`，`std::ranges::find` 只要求 `input_range`。写泛型算法时，按你实际用到的能力选最弱的 concept；要求过强会把本来能用的类型挡在外面。

`range` 不要求 `size()`，因为有些范围是"惰性"的，算长度需要遍历一遍；`sized_range` 才要求常数时间拿到大小。`std::ranges::begin` / `end` 还能处理数组、C 风格字符串与自由函数 `begin/end`，所以"有 `begin()` 成员"不是唯一条件。**concept 描述的是"能用哪些操作"，不是"像不像容器"**；一个 `std::string_view`、一个 `std::span`、一个 `views::iota` 都能满足 `range`。

约束的位置有三种常见写法：`template <class T> requires C<T>` 把约束写在模板头后面；`template <C T>` 把 concept 直接放在类型参数位置；函数尾部的 `requires C<T>` 适合约束依赖多个参数或返回类型的情况。三种写法语义相近，但可读性与重载决议的呈现不同；**团队里选一种主写法，别在同一文件里混用三种。**

9.7 旧写法对照：`enable_if` 与 concepts

cpp **SFINAE 旧写法：把约束藏进默认模板参数** examples/ch04/sfinae_old/include/sfinae_old.h 第 6–9 行

```cpp
template <typename T, std::enable_if_t<std::is_integral_v<T>, int> = 0>
T old_twice(T value) {
    return value + value;
}
```

cpp **main：旧写法也能选对类型** examples/ch04/sfinae_old/src/main.cpp 第 7–11 行

```cpp
int main() {
    std::cout << "old=" << old_twice(21) << "\n";
    std::cout << "demo=" << old_demo() << "\n";
    return 0;
}
```

终端输出 终端输出 sfinae-old-out

```
old=42
demo=20
```

`enable_if_t<条件, int> = 0` 是 C++11 时代的常见写法：条件为假时，`enable_if_t` 没有 `type`，默认模板参数替换失败，模板从候选集里消失（SFINAE）。它工作，但可读性差：约束藏在默认参数里，错误信息只告诉你"没有匹配的函数"，不会直接说"integral 不满足"；重载之间的约束也无法比较，只能靠"谁替换成功"来选。concepts 把同一件事写成 `template <std::integral T>`，约束在签名上可见，诊断也更直接。**新代码优先用 concepts；旧代码里的 `enable_if` 仍然要能读懂。**

SFINAE 的"替换失败不是错误"只适用于**直接上下文**里的类型与表达式；如果替换成功、但函数体里出错，仍然是硬错误。这也是旧写法难调的原因：约束没写全时，错误会落到函数体深处。`std::void_t` 是另一个旧工具：它把一串类型映射成 `void`，用来探测"某个嵌套类型/成员函数是否存在"；配合 `enable_if` 或特化，能写出"有 `value_type` 就走这条重载"的代码。**这些技巧在标准库实现里仍然大量存在**，读得懂就行，新接口不必再手写。

concept 与 `if constexpr` 的分工也不同：`if constexpr` 是在模板**内部**按类型选分支，两个分支的语法都要能写出来；concept 是在**候选集**阶段决定"这个模板参不参与重载"。能用 concept 表达"这个类型必须支持某操作"时，优先用 concept；只有在函数体内部需要按类型走不同实现、而接口本身仍然接受所有类型时，才用 `if constexpr`。**一个管"能不能进来"，一个管"进来之后走哪条路"**，不要混用。

concept 也会"过度约束"。如果你写 `requires(T t) { t.size(); t[0]; }`，就把"可用下标访问"写成了硬要求，链式容器 `std::list` 即使能遍历也会被挡住。**按算法真正用到的操作写约束**：只遍历就要求 `range`，只读大小就要求 `sized_range`，不要顺手把 `operator[]` 也写进去。约束越贴近需求，模板的适用范围越准确。

读 concept 诊断时，先找 `constraints not satisfied` 这一行，再看它下面列出的"不满足的原子约束"。`is_integral_v<double> evaluated to false` 这类信息会直接指出是哪一条要求没过；如果只看到 `no matching function`，把诊断往下翻，通常会有一行 `because ...` 解释候选被剔除的原因。**诊断长不代表没有信息，关键是把"候选集为什么变成空"找出来。**

9.8 本机实测：四个读数

实验记录 · 本机实测：约束的通过与失败

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`；`concept_error` 额外用 `-fdiagnostics-plain-output -fconcepts-diagnostics-depth=1` 压缩诊断。

- `concept_basic` 的 stdout 是 `twice=42` / `demo=40`：`int` 满足 `std::integral`；
- `concept_error` 编译失败，诊断含 `no matching function`、`constraints not satisfied`、`is_integral_v<...> evaluated to 'false'`；
- `concept_overload` 的 stdout 是 `int=integral` / `double=floating`：两个约束各选一个重载；
- `concept_compose` 的 stdout 是 `label=item`：`Item::name()` 满足 `Named`；
- `ranges_concept` 的 stdout 是 `vector=4,5,` / `list=6,7,`：`vector` 与 `list` 都满足 `std::ranges::range`。

9.9 下一单元从哪继续

下一单元 `8.3 可变参数模板与完美转发` 会把模板参数从"一个类型"扩展到"一包类型"：`template <typename... Ts>`、折叠表达式、`std::forward`。concepts 也会与参数包结合（例如要求每个类型都可打印）；先确保你能读懂单个约束的诊断，再处理"一包类型里哪个不满足"的长诊断。

在工程里，concept 的最大价值往往不是"让模板更强"，而是"让接口意图写出来"。一个新模板如果只写 `typename T`，调用者要从函数体猜需要什么操作；写成 `template <Named T>` 或 `template <std::ranges::range R>`，要求就在签名上。**先写约束，再写函数体**，能逼你想清楚"这个模板到底接受什么、拒绝什么"。

约束也不是越多越好。把实现细节（例如"必须有 `size()`"）写进 concept，会让接口比实际需要更窄；把本该检查的前提漏掉，又会让错误落到函数体。**判断标准是"调用者需要知道的最小契约"**：算法真正依赖的操作写成约束，算法内部的临时实现细节不写。这样既保住复用，又让错误尽早出现；约束写得准，模板才既好用又好懂，调用者也能从签名读出你的意图，而不是从一长串模板报错里猜。

小结 · 这一节要带走的五件事

- concept 是编译期谓词；`template <std::integral T>` 把约束写在模板头上，不满足时模板不进入候选集。
- 约束失败时诊断会指向 `constraints not satisfied` 与具体求值结果；格式参数只影响显示。
- 约束参与重载决议；互斥约束各选一个，重叠约束按包含关系比较，无法比较就报歧义。
- `requires` 表达式检查操作是否合法、返回类型是否满足约束；函数体里的隐含要求不会被自动检查。
- `enable_if` 是旧写法，concepts 可读性与诊断更好；新代码优先 concepts。

自测题 · 自测题（先自己做，再对答案）

1. `template <std::integral T> T twice(T)` 在传入 `double` 时会发生什么？为什么不是"函数体里 `+` 不能用"？
2. 约束失败诊断里的 `constraints not satisfied` 与 `is_integral_v<...> evaluated to 'false'` 各说明什么？
3. 两个重载分别约束 `std::integral` 与 `std::floating_point`，为什么 `kind(7)` 不会歧义？
4. `requires` 表达式里的 `{ value.name() } -> std::convertible_to<std::string>` 检查了哪两件事？
5. `dump` 约束为 `std::ranges::range`，传 `vector<std::string>` 时约束满足，为什么函数体仍可能编译失败？

注 · 自测答案

1. `double` 不满足 `std::integral`，模板不进入候选集，调用点报"没有匹配的函数"；不是函数体里的 `+` 出错，因为模板根本没被实例化。
2. 前者说明这个候选因为约束不满足被排除；后者是约束求值的具体证据：`is_integral_v<double>` 为假，所以 `integral<T>` 不满足。
3. `int` 只满足 `integral`，不满足 `floating_point`，所以只有一个候选，不会歧义。

4. 检查 `value.name()` 这个表达式合法，并检查它的返回类型能转换成 `std::string`；它不执行 `name()` 的函数体。
5. `range` 只要求能遍历，不要求元素能 `to_string`；`dump` 函数体里的 `std::to_string(item)` 对 `std::string` 不成立，实例化时才会报错。约束没有覆盖函数体的全部隐含要求。

可变参数模板与完美转发：forward、折叠表达式

上一单元的模板只有一个 `T`；这一单元处理"一包类型"。`sum_all(1, 2, 3.4)` 的参数个数与类型都可以变，模板要能接住任意多个实参，并且保持每个实参的值类别——左值仍是左值，右值仍是右值。前者靠参数包与折叠表达式，后者靠转发引用（`T&&`）与 `std::forward`。

这一单元的重点不是写"万能函数"，而是理解两条规则：**参数包在一次展开里逐项处理；`T&&` 在推导语境下不是右值引用，而是转发引用。** 把这两条搞错，就会写出"看起来能转发、实际把左值搬空"的包装函数。

◎ 学习目标 · 看完这一单元，你应该能

- 写一个可变参数函数模板，用 `sizeof...(Ts)` 数参数、用折叠表达式求和/打印；
- 解释 `template <typename... Ts>` 与 `Ts... args` 的角色区别；
- 说出 `T&&` 在模板推导中为什么是转发引用，以及引用折叠怎样决定最终类型；
- 解释 `std::forward<T>` 与 `std::move` 的区别，并指出"包装函数里用 move"会怎样搬空左值；
- 用 `emplace_back` 与 `push_back` 的计数说明"少一次临时对象"发生在哪里。

10.1 先看工程长什么样

这一单元在 `examples/ch04/` 下再加六个多文件例子。参数包、折叠、转发、`emplace` 各占一个目录；模板定义仍然在头文件里。

目录树 **工程：examples/ch04/ 这一单元新增的六个例子**

第 1 步 先建参数包与折叠表达式

```
examples/
└── ch04/
    ├── 上一单元 : cpp09 的六个例子
    └── sum_fold/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── sum_fold.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── sum_fold.cpp ← 本课新增
        └── sum_fold.json ← 本课新增
```

第 2 步 加上打印、转发与 move 的对照

```
examples/
└── ch04/
    ├── sum_fold/ : 同上
    ├── print_all/ ← 本课新增
    ├── forward_wrapper/ ← 本课新增
    └── forward_vs_move/ ← 本课新增
```

第 3 步 最后加上 emplace 与 sizeof... 计数

```
examples/
└── ch04/
    ├── 前四个例子 : 同上
    ├── emplace_demo/ ← 本课新增
    └── sizeof_pack/ ← 本课新增
```

第 1 帧的 `sum_fold/` 用 `(... + values)` 折叠求和；`print_all/` 用逗号折叠逐个打印。第 2 帧的 `forward_wrapper/` 用转发引用 + `std::forward` 保持值类别；`forward_vs_move/` 故意把 `forward` 写成 `move`，观察左值被搬空。第 3 帧的 `emplace_demo/` 对照 `push_back` / `emplace_back` 的拷贝/移动计数；`sizeof_pack/` 用 `sizeof...` 数参数个数。

10.2 参数包：`Ts...` 是类型包，`args...` 是值包

cpp **sum_all：任意多个同/异类型实参** examples/ch04/sum_fold/include/sum_fold.h 第 4–7 行

```cpp
template <typename... Ts>
auto sum_all(Ts... values) {
    return (... + values);
}
```

cpp **main：四个 int 进同一个模板** examples/ch04/sum_fold/src/main.cpp 第 5–9 行

```cpp
int main() {
    std::cout << "sum=" << sum_all(1, 2, 3, 4) << "\n";
    std::cout << "demo=" << fold_demo() << "\n";
    return 0;
}
```

终端输出 终端输出 sum-fold-out

```
sum=10
demo=60
```

`typename... Ts` 声明一个**类型参数包**：它可以收零个、一个或多个类型。`Ts... values` 声明一个**函数参数包**：每个实参对应包里的一个类型。调用 `sum_all(1, 2, 3, 4)` 时，`Ts` 推导为 `int, int, int, int`，`values` 是四个 `int`。**包不是数组，也不是 `std::vector`；它在编译期展开，每个元素可以是不同类型**（例如 `sum_all(1, 2.5, 3)` 会得到 `double`，因为 `+` 逐项提升了类型）。

终端输出 终端输出 sizeof-pack-out

```
pack=3
demo=3
```

`sizeof...(Ts)` 返回包里元素个数，`sizeof...(values)` 同理（对值包）。`sizeof...` 是编译期常量，可以用在 `static_assert`、数组长度、模板参数里。参数包也可以递归展开：处理第一个元素，把剩下的包继续传给同一个模板，直到包为空；C++17 的折叠表达式把常见递归写法压缩成一行，本书优先用折叠。

参数包能出现的位置比想象中多：函数参数、模板参数、基类列表、成员初始化列表、`std::tuple` 的构造、花括号初始化、`using` 声明、lambda 捕获。**每出现一次 `...`，就是一次“把包展开成逗号分隔的元素”**；展开的模式必须包含包名，否则编译器不知道按哪个包展开。例如 `std::tuple<Ts...>` 展开类型包，`std::make_tuple(args...)` 展开值包。

空包是合法的，但要看使用场景。`print_all()` 的折叠是逗号折叠，空包展开成“什么也不做”，仍然会打印一个换行；`sum_all()` 用 `(... + values)` 折叠，空包没有初始值，无法形成一个合法表达式，编译失败。**一元折叠对空包不一定有定义**：`&&` 的空折叠是 `true`，`||` 是 `false`，逗号是 `void()`，而 `+`、`*` 之外的很多运算符没有默认单位元。需要支持空包时，用二元折叠给一个初始值，例如 `(0 + ... + values)`。

递归展开要写两个重载：一个处理“至少一个元素”，一个处理空包作为出口。折叠表达式出现后，很多递归可以删掉；但**当每一步的逻辑依赖前一步的结果、或需要提前终止时**，递归仍然更自然。折叠适合“把同一运算套到每个元素”，不适合“元素之间要互相协调”的复杂逻辑。

10.3 折叠表达式：把包“折叠”成一个表达式

cpp **print_all：逗号折叠逐个打印** examples/ch04/print_all/include/print_all.h 第 6–10 行

```cpp
template <typename... Ts>
void print_all(const Ts&... values) {
    ((std::cout << values << ' '), ...);
    std::cout << '\n';
}
```

终端输出 终端输出 print-all-out

```
1 two 3.5
7 2.5 hi
```

`((std::cout << values << ' '), ...)` 是**逗号折叠**：把包里的每个 `values` 依次代入 `std::cout << values << ' '`，用逗号连接，从左到右执行。`(... + values)` 是**二元折叠**：把 `+` 放在每个元素之间，等价于 `1 + 2 + 3 + 4`。折叠有四种形式：一元左折、一元右折、二元左折、二元右折；对 `+` 与 `,` 来说，结合顺序在多数场景下不影响结果，但对 `-`、`/` 这类非结合运算，左折与右折结果不同。

`print_all` 的参数是 `const Ts&...`，所以它接受左值和右值，但不会移动它们；`values` 在折叠里被逐项使用。**折叠表达式不是循环**：它在编译期展开成固定长度的表达式，运行期没有"遍历参数包"的循环变量；包里有几个元素，就展开成几段代码。

调用树 **print_all(1, "two", 3.5) 的展开**

第 1 步 包里有三个元素

```
print_all(1, "two", 3.5) : Ts = int, const char*, double
```

第 2 步 逗号折叠从左到右展开

```
print_all → cout << 1 第一段
print_all → cout << "two" 第二段
print_all → cout << 3.5 第三段
```

第 1 帧的包有三个元素；第 2 帧的折叠把它们展开成三条输出语句，按从左到右的顺序执行。**折叠的"顺序"由运算符决定**：逗号折叠保证从左到右；`+` 折叠不保证求值顺序（但 `+` 本身通常不依赖顺序）。不要在折叠里放有副作用的表达式，除非你确认顺序语义。

折叠的身份值（identity）值得单独记：`&&` 的空一元折叠是 `true`，`||` 是 `false`，逗号是 `void()`；`+`、`*`、`&`、`|` 等运算符没有标准的空折叠结果（除非写二元折叠提供初值）。**"空包能编译"不是对所有运算符都成立**；写通用模板时，要么要求至少一个元素，要么显式提供初值。`static_assert` 配合 `sizeof...(Ts) > 0` 可以把"至少一个"写成编译期要求。

折叠的括号不能省：`(... + values)`、`(values + ...)`、`(init + ... + values)` 是三种不同形式。**省略号在运算符左边还是右边，决定结合方向**；对 `+` 这种结合律成立的运算，结果一样；对 `-`、`/`、`>>` 这类不满足结合律的运算，左折与右折结果不同。不要凭"看起来差不多"选择，先确定你要的是 `((a - b) - c)` 还是 `a - (b - c)`。

`&&` 与 `||` 的折叠保留短路语义：从左到右求值，遇到 `false` / `true` 就停止。**但短路只对逻辑运算符成立**；`+` 折叠会把所有元素都求值，即使前面的结果已经"够用"。需要"遇到第一个不满足就停止"的逻辑，用 `&&` 折叠或显式循环，不要用 `+` 模拟。

10.4 转发引用：`T&&` 不是右值引用

cpp **invoke_all：把实参原样交给目标函数** examples/ch04/forward_wrapper/include/forward_wrapper.h 第 17–21 行

```cpp
template <typename F, typename... Args>
auto invoke_all(F&& function, Args&&... args)
    -> decltype(std::forward<F>(function)(std::forward<Args>(args)...)) {
    return std::forward<F>(function)(std::forward<Args>(args)...);
}
```

cpp **main：左值仍按左值传，右值仍按右值传** examples/ch04/forward_wrapper/src/main.cpp 第 5–15 行

```cpp
int main() {
    Target target;
    invoke_all(take, target);
    std::cout << "lvalue_copies=" << Target::copies
              << " moves=" << Target::moves << "\n";

    invoke_all(take, Target{});
    std::cout << "rvalue_copies=" << Target::copies
              << " moves=" << Target::moves << "\n";
    return 0;
}
```

终端输出 终端输出 forward-wrapper-out

```
lvalue_copies=1 moves=0
rvalue_copies=1 moves=1
```

第一次 `invoke_all(take, target)` 传入左值 `target`，`Args` 推导为 `Target&`，引用折叠后 `Args&&` 变成 `Target&`；`std::forward<Args>(args)` 返回左值，`take(Target)` 按值接收时调用拷贝构造，所以 `lvalue_copies=1 moves=0`。第二次传入临时对象 `Target{}`，`Args` 推导为 `Target`，`Args&&` 是 `Target&&`；`std::forward` 返回右值，`take` 调用移动构造，所以 `moves=1`。**转发引用保留的不只是类型，还有值类别。**

`T&&` 只有在**模板参数推导**语境下才是转发引用；如果 `T` 是具体类型（例如 `void f(int&& x)`），它就是普通右值引用，只能绑定右值。转发引用的两条规则：

- 实参是左值 `U`：`T` 推导为 `U&`，`T&&` 经引用折叠变成 `U&`；
- 实参是右值 `U`：`T` 推导为 `U`，`T&&` 是 `U&&`。

引用折叠规则只有四条：`& & → &`、`& && → &`、`&& & → &`、`&& && → &&`。**左值"传染"**：只要有一个 `&`，结果就是左值引用。这就是 `std::forward<T>` 能"还原"值类别的原因——它只是 `static_cast<T&&>(t)`，真正做判断的是 `T` 的推导结果。

图 / 搜索过程 **实参的值类别怎样穿过转发引用**

第 1 步 左值 target

target ：左值 | Args ：推导为 Target& | Args&& ：折叠为 Target&

target → take 按左值传递

第 2 步 右值 Target{}

Target{} ：右值 | Args ：推导为 Target | Args&& ：Target&&

Target{} → take 按右值传递

第 1 帧的左值在折叠后仍是左值引用，第 2 帧的右值保持右值引用。**转发引用不是"万能引用"这种魔法，它是"推导 + 引用折叠"的结果**；离开模板推导，`T&&` 就只是右值引用。

转发引用有三个常见失效场景。第一，写成 `const T&&` 不是转发引用，它只能绑定右值，而且 `T` 不再按左值/右值推导；第二，类模板里的 `T&&` 成员不是转发引用，因为 `T` 在类实例化时已经确定，不会在每次调用时重新推导；第三，花括号初始化无法推导：`invoke_all(f, {1, 2})` 里的 `{1,2}` 不是一个有类型的表达式，`Args` 推不出来。**看到"转发引用不转发"，先检查它是不是在推导语境里、有没有 `const`、实参是不是花括号列表。**

`std::forward` 只应该用一次。`forward<T>(x)` 把 `x` 还原成 `T&&` 的值类别；如果同一个 `x` 被转发两次，第一次可能已经把它移走，第二次拿到的是 moved-from 对象。**转发是"把值类别传下去"，不是"复制一份"**；需要多次使用同一个参数时，先想清楚所有权，或者只转发给一个目标。

转发引用也常与 `auto&&` 混淆。`auto&& x = expr;` 是转发引用式的推导，能绑定左值也能绑定右值，但 `x` 本身是一个具名变量，表达式 `x` 永远是左值；要把它继续按原值类别传下去，仍然需要 `std::forward<decltype(x)>(x)`。**"变量有名字就是左值"**，这是值类别最基本的规则之一。

10.5 `std::forward` 与 `std::move`：一个保留，一个强制

cpp **两个包装：一个用 move，一个用 forward** examples/ch04/forward_vs_move/include/forward_vs_move.h 第 7–17 行

```cpp
template <typename T>
void bad_wrapper(T&& value) {
    std::string sink(std::move(value));
    (void)sink;
}

template <typename T>
void good_wrapper(T&& value) {
    std::string sink(std::forward<T>(value));
    (void)sink;
}
```

cpp **main: 左值经过 bad_wrapper 后被搬空** examples/ch04/forward_vs_move/src/main.cpp 第 8-18 行

```cpp
int main() {
    std::string bad = "hello";
    bad_wrapper(bad);
    std::cout << "bad=" << (bad.empty() ? "empty" : bad) << "\n";

    std::string good = "hello";
    good_wrapper(good);
    std::cout << "good=" << (good.empty() ? "empty" : good) << "\n";
    std::cout << "demo=" << move_demo() << "\n";
    return 0;
}
```

终端输出 终端输出 forward-vs-move-out

```
bad=empty
good=hello
demo=empty
```

`bad_wrapper(bad)` 传入左值 `std::string`，`T` 推导为 `std::string&`；`std::move(value)` 不管 `T` 是什么，一律转成右值，于是 `std::string sink` 移动构造，`bad` 被搬空，输出 `bad=empty`。`good_wrapper(good)` 用 `std::forward<T>(value)`，`T = std::string&` 时返回左值，`sink` 拷贝构造，`good` 不变，输出 `good=hello`。**`std::move` 是“我确定不要这个对象了”，`std::forward` 是“按调用者原来的值类别传下去”；包装函数不知道调用者要不要保留实参，所以必须用 `forward`。**

`std::move` 与 `std::forward` 都只是类型转换，不搬任何东西：真正搬资源的是移动构造/移动赋值。`std::forward<T>` 的定义就是 `static_cast<T&&>(t)`；`std::move(t)` 等价于 `static_cast<remove_reference_t<T>&&>(t)`。**转发函数里写 `std::move` 是常见的 bug**：它会把左值实参静默搬空，调用者还以为自己的对象还在。

对 `const` 对象用 `std::move` 不会变成移动，而是变成“拷贝”：`const T&&` 不能绑定到 `T&&` 移动构造，编译器退回复制构造。**`move` 只是把表达式变成右值，能不能移动取决于类型有没有可用的移动构造**；对 `const` 对象、没有移动构造的类型、或者被 `const` 成员拖住的类，`move` 可能毫无效果。

moved-from 对象“有效但未指定”：标准不保证它为空，也不保证还能做什么，只保证可以安全析构、可以重新赋值。**不要读 moved-from 对象的值来判断“移动成功没有”**；如果类型没有明确文档化 moved-from 状态，最安全的做法是把它当成“只能销毁或重新赋值”。本单元的例子用 `std::string` 的空状态做演示，是因为标准库对 `string` 的 moved-from 状态有实际保证；换成自定义类型，不能照搬。

对同一个对象自我移动（`x = std::move(x)`）通常是合法的，但结果由类型决定；标准库容器一般能保持有效，自定义类型未必。**移动赋值要处理自赋值**，尤其是在手写资源管理类时；`std::swap` 式的实现通常能自然处理，但不要假设所有类型都安全。

标准原文 · 标准原文：参数包与 forward 的返回

> A template parameter pack is a template parameter that accepts zero or more template arguments.
>
> Returns: static_cast<T&&>(t).

大意：模板参数包接受零个或多个模板实参；`std::forward<T>(t)` 返回 `static_cast<T&&>(t)`，它只做类型转换，不搬对象。

出处：N4950 §13.7.4「Template parameter packs」（[temp.variadic]）与 §22.2.4「Forward」（[forward]）。

⚠ 陷阱 · 在包装函数里用 `std::move` 转发

你会看到：包装函数把参数原样传给下层，测试时用临时对象一切正常；换成左值实参后，调用者的对象莫名其妙变空。你会误判成“下层函数有 bug”或“std::string 有问题”。真相是：包装函数的 `T&&` 是转发引用，左值传进来时 `T` 是 `U&`；你写 `std::move(arg)` 强制把它变成右值，下层按右值搬走了资源。**包装函数要用 `std::forward<T>(arg)`；只有你确实要放弃这个对象时，才用 `std::move`。**

10.6 `emplace_back` 与 `push_back`：少一次临时对象

cpp **emplace_demo：同一个构造参数，两种入容器方式** examples/ch04/emplace_demo/src/main.cpp 第 6–19 行

```cpp
int main() {
    std::vector<Widget> pushed;
    pushed.reserve(4);
    pushed.push_back(Widget("a"));

    std::vector<Widget> emplaced;
    emplaced.reserve(4);
    emplaced.emplace_back("b");

    std::cout << "push copies=" << Widget::copies
              << " moves=" << Widget::moves << "\n";
    std::cout << "size=" << pushed.size() + emplaced.size() << "\n";
    return 0;
}
```

终端输出 终端输出 emplace-demo-out

```
push copies=0 moves=1
size=2
```

`push_back(Widget("a"))` 先构造一个临时 `Widget`，再把它移动进 `vector`，所以 `moves=1`；`emplace_back("b")` 把 `"b"` 直接转发给 `Widget` 的构造函数，在 `vector` 的存储里就地构造，没有临时对象，也不需要移动。**`emplace_back` 的收益是“少一次临时对象 + 移动”，不是“一定更快”**：如果参数本来就是一个已经构造好的对象，`push_back` 可能更清晰；如果容器需要扩容，`emplace_back` 也可能触发元素搬移，收益被扩容成本盖过。本机读数只说明这个例子的情况，不承诺所有场景。

`emplace_back` 也依赖完美转发：它把参数包原样传给元素的构造函数。`vector<Widget>::emplace_back("b")` 里，`const char*` 被转发给 `Widget(std::string)`，先构造 `std::string` 再构造 `Widget`，仍然可能有一次临时 `std::string`；要彻底少一次转换，可以传 `std::string("b")` 或给 `Widget` 加 `const char*` 构造函数。**“emplace 一定比 push 少一次拷贝”是过度简化；看构造链，别看名字。**

`emplace_back` 与花括号初始化配合时还有一个坑：`std::vector<std::vector<int>> v; v.emplace_back({1, 2});` 通常编译失败，因为 `emplace_back` 是模板，无法从 `{1, 2}` 推导出 `std::vector<int>` 的构造参数。要写成 `v.emplace_back(std::initializer_list<int>{1, 2})`，或者直接用 `v.push_back({1, 2})`。**花括号不是表达式，没有类型，模板推导接不住**；这是 `emplace` 与 `push_back` 最常被忽略的差别。

`emplace_back` 用直接初始化构造元素，因此可以调用 `explicit` 构造函数；`push_back` 需要隐式转换，遇到 `explicit` 构造函数就要显式构造临时对象。这也是某些场景下 `emplace_back` 更方便的原因。**但方便不等于更好读**：如果参数已经是一个完整对象，`push_back` 的意图更清楚；`emplace_back` 适合“把构造参数直接交给容器”的场景。

标准库里的 `std::make_unique<T>(args...)`、`std::make_shared<T>(args...)` 也是把参数包转发给构造函数；它们和 `emplace` 一样，参数会按原值类别传给 `T` 的构造函数。**如果你在自己的工厂函数里再包一层，别忘了 `std::forward`**：`template <class T, class... Args> auto make(Args&&... args) { return std::make_unique<T>(args...); }` 会把右值实参当成左值传下去，正确写法是 `std::forward<Args>(args)...`。参数包的转发，本质就是把每个 `args` 单独 forward，再用 `...` 展开。

`std::forward` 只改变值类别，不改变 const：`const` 左值转发后仍是 `const` 左值，不会突然变成可移动的右值。**要移动一个 `const` 对象，必须先去掉 const，但那通常说明设计有问题**；移动意味着“放弃所有权”，而 `const` 对象本来就不该被放弃。转发函数遇到 `const` 实参时，老老实实按 `const` 引用传下去，让下层决定拷贝还是拒绝。

转发的价值不是“性能技巧”，而是让包装层不改变实参的语义：调用方传左值，下层就看到左值；传右值，下层就能移动。包装层越多，越要保证每一层都用 `std::forward` 接力，任何一层漏掉，值类别就在这里退化成左值。

10.7 本机实测：四个读数

实验记录 · 本机实测：折叠、转发与 emplace

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `sum_fold` 的 stdout 是 `sum=10` / `demo=60`：折叠把四个 `int` 求和；
- `print_all` 的 stdout 是 `1 two 3.5` / `7 2.5 hi`：逗号折叠按顺序打印；
- `forward_wrapper` 的 stdout 是 `lvalue_copies=1 moves=0` / `rvalue_copies=1 moves=1`：左值走拷贝，右值走移动；
- `forward_vs_move` 的 stdout 是 `bad=empty` / `good=hello`：用 move 的包装把左值搬空，用 forward 的保留；
- `emplace_demo` 的 stdout 是 `push copies=0 moves=1`：`push_back(Widget("a"))` 有一次移动，`emplace_back("b")` 没有。

10.8 下一单元从哪继续

下一章 `11.1 序列容器` 会把"容器"从模板例子变成日常工具：`vector` 的扩容与迭代器失效、`string` 的短字符串优化、`deque` / `list` 的取舍。参数包与转发会以 `emplace_back`、`vector::emplace` 的形式继续出现；先确保你能解释"为什么左值经过转发包装后没有被搬空"，再去看容器的迭代器规则。

小结 · 这一节要带走的五件事

- `typename... Ts` 是类型包，`Ts... args` 是值包；包在编译期展开，`sizeof...(Ts)` 数元素。
- 折叠表达式把包折叠成一个表达式；逗号折叠从左到右，`+` 折叠不保证求值顺序。
- 模板推导中的 `T&&` 是转发引用：左值推导为 `U&`，右值推导为 `U`；引用折叠决定最终类型。
- `std::forward<T>` 保留值类别，`std::move` 强制右值；包装函数用 forward，不要用 move。
- `emplace_back` 把参数转发给元素构造，少一次临时对象；是否更快取决于构造链与扩容。

自测题 · 自测题（先自己做，再对答案）

1. `template <typename... Ts> void f(Ts... args)` 中，`Ts` 与 `args` 分别是什么？`sizeof...(Ts)` 在 `f(1, 2.5, "x")` 时是多少？
2. `((std::cout << values << ' '), ...)` 是哪种折叠？它按什么顺序执行？
3. 为什么 `invoke_all(take, target)` 里 `Args` 推导为 `Target&`，而 `invoke_all(take, Target{})` 里是 `Target`？
4. `std::move(value)` 与 `std::forward<T>(value)` 在包装函数里有什么不同？用左值实参时各自会发生什么？
5. `emplace_back("b")` 比 `push_back(Widget("b"))` 少做了什么？为什么不能保证一定更快？

注 · 自测答案

1. `Ts` 是类型参数包，`args` 是函数参数包；三个实参对应三个元素，`sizeof...(Ts)` 为 3。
2. 逗号折叠（一元左折的常见形式），从左到右依次执行每个 `std::cout << values << ' '`，最后输出换行。
3. 左值实参让 `Args` 推导为 `Target&`，转发引用的 `&&` 与 `&` 折叠成 `Target&`；右值实参让 `Args` 推导为 `Target`，`Args&&` 是 `Target&&`。
4. `std::move` 一律转成右值，左值实参会被下层移动构造搬空；`std::forward<T>` 按 `T` 的推导结果还原值类别，左值仍是左值，不会被搬空。
5. `emplace_back` 把 `"b"` 直接转发给 `Widget` 的构造函数，在容器存储里就地构造，省掉临时 `Widget` 与一次移动；但如果构造链里还有 `std::string` 临时对象，或者容器扩容搬移元素，成本仍可能发生，所以不能笼统说一定更快。

第 5 章

标准库容器与算法

本章 3 个单元 · 页眉从这里之后的页面开始出现

1. 序列容器：vector、string、deque、list 的取舍 零基础 · 第 11 单元 · 真编译真运行
2. 关联容器与哈希：map、set、unordered_* 零基础 · 第 12 单元 · 真编译真运行
3. 迭代器、算法与 ranges：lambda、views、zip / enumerate 零基础 · 第 13 单元 · 真编译真运行

序列容器：vector、string、deque、list 的取舍

前几章的例子大多用 `std::vector`，但"能装东西"不是容器的全部。`vector` 连续存储、随机访问快，扩容时要把元素搬到新块，迭代器全部失效；`deque` 两端插入便宜，中间插入仍要搬；`list` 每个节点单独分配，插入删除不搬元素，但没有随机访问；`std::array` 大小固定，不碰堆。选错容器，代码可能"能跑但很慢"，也可能在扩容后悄悄用着失效的迭代器。

这一单元用七个小实验把"容量、失效、两端插入、节点搬移、固定大小、越界检查"六件事钉住。性能数字不承诺，只讲可观察的行为与规则；本机 `capacity()` 的序列是 libstdc++ 的实现，不是标准规定。

◎ 学习目标 · 看完这一单元，你应该能

- 说出 `vector` 的 `size()` / `capacity()` / `reserve()` 各表示什么，并解释扩容为什么让指针/迭代器失效；
- 用 `data()` 的前后比较证明一次扩容发生了，并说出失效后继续用旧迭代器的后果；
- 说出 `deque` 与 `list` 在两端/中间插入上的差别，以及它们各自的迭代器稳定性；
- 说出 `std::array` 与 `std::vector` 的存储差别，并按访问模式选容器；
- 解释 `push_back` 与 `emplace_back` 的构造次数差异（上一单元已实测）。

11.1 先看工程长什么样

这一单元在 `examples/ch05/` 下建七个多文件例子。每个例子仍然是 `include/` + 两个 `.cpp` + 清单；容器类型与辅助函数分开，`main.cpp` 只做驱动与打印。

目录树 **工程：examples/ch05/ 这一单元新增的七个例子**

第 1 步 先建 vector 的扩容、失效与 reserve

```
examples/
└── ch05/
    └── vector_growth/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── vector_growth.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── vector_growth.cpp ← 本课新增
        └── vector_growth.json ← 本课新增
```

第 2 步 加上 deque 与 list

```
examples/
└── ch05/
    ├── vector_growth/ : 同上
    ├── vector_invalidate/ ← 本课新增
    ├── reserve_demo/ ← 本课新增
    ├── deque_ends/ ← 本课新增
    └── list_splice/ ← 本课新增
```

第 3 步 最后加上固定数组与调试期越界检查

```
examples/
└── ch05/
    ├── 前五个例子 : 同上
    ├── container_choose/ ← 本课新增
    └── bounds_check/ ← 本课新增
```

第 1 帧的 `vector_growth/` 打印每次 `push_back` 后的 `size/capacity`；第 2 帧的 `vector_invalidate/` 比较扩容前后的 `data()`，`reserve_demo/` 展示预留后指针不变，`deque_ends/` 与 `list_splice/` 展示两端插入与节点搬移。第 3 帧的 `container_choose/` 用 `std::array` 做固定大小求和，`bounds_check/` 故意越界写入，用调试期断言展示"越界不是异常"。

11.2 `vector` 的容量：`size` 与 `capacity` 不是一回事

cpp vector_growth: 八次 push_back 的 size/capacity examples/ch05/vector_growth/src/main.cpp 第 6–17 行

```cpp
int main() {
    std::vector<int> values;
    for (int i = 0; i < 8; ++i) {
        values.push_back(i);
        std::cout << values.size() << "/" << values.capacity() << " ";
    }
    std::cout << "\n";

    auto built = make_values(4);
    std::cout << "built=" << sum_values(built) << "\n";
    return 0;
}
```

终端输出 终端输出 vector-growth-out

```
1/1 2/2 3/4 4/4 5/8 6/8 7/8 8/8
built=6
```

`size()` 是"现在有几个元素"，`capacity()` 是"在不重新分配的前提下最多能放几个"。输出里容量按 `1 → 2 → 4 → 8` 增长：第 3 个元素放不下 2 个的块，`vector` 分配一块更大的（这里是 4 个），把旧元素搬过去，再放新元素。**这是本机 libstdc++ 的增长策略，不是标准规定的倍增**；标准只要求 `push_back` 的摊还复杂度为常数，具体扩多少由实现决定。

`reserve(n)` 请求"至少能放 n 个"，但它不改变 `size()`，只改变 `capacity()`；如果 `n` 小于当前容量，什么也不做。`resize(n)` 才会改变元素个数：变大时补默认值，变小时销毁多余元素。**`reserve` 是给容量提建议，`resize` 是改元素个数**；把两者混用，通常会得到"容量够了但 size 还是 0"的意外。

内存形态 vector 扩容：一块连续存储换成另一块

第 1 步 size=2，capacity=2

vector ：连续块，2 个 int　　vector.data() ：指向旧块

第 2 步 push 第 3 个，容量不够

old ：旧块，2 个 int，已释放　　new ：新块，4 个 int，元素搬过来

vector → new data() 变了，旧指针失效

第 1 帧的 `data()` 指向旧块；第 2 帧扩容后，元素被搬到新块，`data()` 返回新地址。旧块被释放，任何还指向旧块的指针、引用、迭代器都失效——即使旧地址上的字节"看起来还在"，解引用也是未定义行为。

11.3 扩容让迭代器失效

cpp vector_invalidate: push_back 前后比较 data() examples/ch05/vector_invalidate/src/main.cpp 第 6–17 行

```cpp
int main() {
    std::vector<int> values = starting_values();
    const int* before = values.data();
    int first_value = values[0];

    values.push_back(3);

    std::cout << "moved=" << (before != values.data() ? "yes" : "no") << "\n";
    std::cout << "size=" << values.size() << " first=" << values[0]
              << " saved=" << first_value << "\n";
    return 0;
}
```

终端输出 终端输出 vector-invalidate-out

```
moved=yes
size=3 first=1 saved=1
```

`starting_values()` 返回 `{1, 2}`，容量正好是 2；`push_back(3)` 触发扩容，`before != values.data()` 为真，输出 `moved=yes`。程序没有解引用 `before`，而是提前把 `first_value` 存下来，所以输出仍然正确。**这段代码演示的是“扩容发生了”，不是“失效后还能用”**；如果你在 `push_back` 之后写 `*before`，就是未定义行为。

cpp **reserve_demo: 先预留，再 push，指针不变** examples/ch05/reserve_demo/src/main.cpp 第 6-19 行

```cpp
int main() {
    std::vector<int> values;
    values.reserve(8);
    const int* before = values.data();
    for (int i = 0; i < 5; ++i) {
        values.push_back(i);
    }
    std::cout << "capacity_ok=" << (values.capacity() >= 8 ? "yes" : "no") << "\n";
    std::cout << "same=" << (before == values.data() ? "yes" : "no") << "\n";

    auto built = build_with_reserve(4);
    std::cout << "built=" << built.size() << "\n";
    return 0;
}
```

终端输出 终端输出 reserve-demo-out

```
capacity_ok=yes
same=yes
built=4
```

`reserve(8)` 之后容量至少是 8；再 push 5 个元素不会超过容量，所以 `data()` 不变，`same=yes`。`reserve` **能在“你知道大概要放多少”时避免多次扩容**，代价是可能预留过多内存；如果实际元素远少于预留值，浪费的是容量，不是 size。

标准原文 · 标准原文：扩容使所有指针/引用/迭代器失效

> Reallocation invalidates all the references, pointers, and iterators referring to the elements in the sequence, as well as the past-the-end iterator. [Note: If no reallocation happens, they remain valid.]

大意：`vector` 重新分配时，指向元素的引用、指针、迭代器以及尾后迭代器全部失效；没有重新分配时，它们保持有效。

出处：N4950 §24.3.11.5「Modifiers」（[vector.modifiers]），`reserve` 的 Remarks。

⚠ 陷阱 · 扩容后继续用旧迭代器

你会看到：循环里保存了 `auto it = v.begin();`，中间 `v.push_back(...)` 可能扩容，后面继续 `++it` 或 `*it`，程序偶尔输出乱码、偶尔崩溃。你会误判成“迭代器语法用错了”。真相是：扩容把元素搬到新块，旧迭代器指向已释放的内存；标准明确说重新分配会让所有迭代器失效。**在可能扩容的操作之后，重新取 `begin()/end()`，或者先用 `reserve` 把容量定下来；不要保存跨扩容的迭代器。**

不同容器的失效规则不一样，不能把 `vector` 的规则套到所有容器上。`vector` 与 `string`：重新分配时全部失效；没有重新分配时，插入/删除点之后的迭代器、指针、引用失效（插入点之前的仍有效）。`deque`：在两端插入会让所有**迭代器**失效，但指向元素的**引用/指针**仍有效；在中间插入/删除会让全部失效。`list`：插入不使任何迭代器失效，删除只使被删元素的迭代器失效。`array`：大小固定，没有插入/删除，迭代器在对象生命周期内有效。**记住“哪种操作、在哪个位置、让什么失效”**，比背一句“vector 会失效”有用得多。

`erase` 的返回值是“被删元素之后的新迭代器”，这是为了安全地连续删除：`it = v.erase(it);` 而不是 `v.erase(it); ++it;`。后者在删除最后一个元素时会让 `it` 变成 `end()`，再 `++` 就是未定义行为。**删除满足条件的元素，常用 erase-remove 惯用法**：`v.erase(std::remove_if(v.begin(), v.end(), pred), v.end());`；`std::remove_if` 只是

把保留的元素往前挪，并不改变 `size()`，必须再 `erase` 才能真正删除。C++20 的 `std::erase_if(v, pred)` 把这两步封装成一个调用，能用在标准容器上。

`std::vector<bool>` 是特化版本：它把每个 `bool` 打包成一位，`operator[]` 返回的是代理对象，不是 `bool&`。所以 `auto& bit = v[0];` 通常编译失败，`bool* p = &v[0];` 也不行；`auto bit = v[0]` 会得到一个代理值。**需要稳定地址或引用时，用 `std::vector<char>` 或 `std::bitset`**；`vector<bool>` 省空间，但接口与普通 `vector` 不完全一样，泛型代码里要特别小心。

`capacity()` 不会自动缩小：`clear()` 只销毁元素，不释放存储；`shrink_to_fit()` 是"请求"缩小，标准不保证一定执行。**不要为了"省内存"频繁调用 `shrink_to_fit`**，它可能触发重新分配与搬移；如果确实要释放，考虑用 `std::vector<T>(v).swap(v)`（C++11 前的惯用法）或接受容量暂时偏大。`front()` / `back()` 在空容器上是未定义行为，调用前先检查 `empty()`；`at()` 会做边界检查并抛 `std::out_of_range`，但比 `operator[]` 多一次检查。

11.4 `deque`：两端插入，分段存储

cpp **deque_ends: push_front 与 push_back 都不搬已有元素** examples/ch05/deque_ends/src/main.cpp 第 6-17 行

```cpp
int main() {
    std::deque<int> values = make_deque();
    values.push_front(2);
    values.push_front(1);
    values.push_back(5);

    for (int value : values) {
        std::cout << value << " ";
    }
    std::cout << "\nsize=" << values.size() << "\n";
    return 0;
}
```

终端输出 终端输出 deque-ends-out

```
1 2 3 4 5
size=5
```

`deque`（双端队列）在两端插入/删除都是摊还常数时间；它通常由多段连续块拼成，所以支持随机访问，但不是"整块连续"，`data()` 也没有 `vector` 那样的单一指针。`push_front` 在头部加元素，已有元素的迭代器通常不失效（标准对 `deque` 的失效规则比 `vector` 细：在两端插入不会使指向已有元素的引用/指针/迭代器失效，删除可能使被删元素的迭代器失效）。**"通常"不是"永远"**：中间插入/删除仍可能让迭代器失效，具体看操作与实现。

`deque` 适合"头尾都要进出"的队列/滑动窗口；如果你只需要尾部插入 + 随机访问，`vector` 通常更简单、缓存更友好。`deque` 的代价是分段存储带来的间接寻址，以及没有 `reserve`（你无法像 `vector` 那样一次性预留连续空间）。

`deque` 的"分段"通常是由一个指针数组（map）指向若干固定大小的块；块满时再分配新块。因此它的迭代器比 `vector` 迭代器更重（要记录当前块、块内位置、map 位置），随机访问也要多一次间接寻址。**这些是实现细节，标准只规定行为与复杂度**；但选容器时值得知道：`deque` 的"两端快"不是免费的，它的常数因子通常比 `vector` 大。

`vector` 的扩容策略也值得再看一眼：标准只要求 `push_back` 摊还常数时间，没规定每次扩多少；libstdc++ 按 2 倍增长，MSVC 的 `vector` 常见 1.5 倍增长。**`reserve` 的实参是"元素个数"，不是字节数**；`resize(n)` 会真正构造/销毁元素，`reserve(n)` 只改容量。把 `reserve` 当成"预分配对象"是常见误解：它只保证存储够，不保证 `size()` 变化。

11.5 `list`：节点稳定，但没有随机访问

cpp **list_splice: 把一条链表接到另一条末尾** examples/ch05/list_splice/src/main.cpp 第 6-19 行

```cpp
int main() {
    std::list<int> first = first_list();
    std::list<int> second = second_list();
    auto position = first.end();

    first.splice(position, second);

    for (int value : first) {
```

```
        std::cout << value << " ";
    }
    std::cout << "\nfirst=" << first.size()
              << " second=" << second.size() << "\n";
    return 0;
}
```

终端输出 终端输出 list-splice-out

```
1 2 3 4
first=4 second=0
```

`list` 是双向链表，每个元素一个节点；`splice` 把 `second` 的节点直接摘下来接到 `first` 的末尾，不复制元素，`second` 变成空。**`splice` 的复杂度是常数时间（整条链）或与搬移节点数线性（单个/区间）**，而且指向被搬移元素的迭代器仍然有效——节点本身没动，只是换了链表。这是 `list` 相对 `vector` 的核心优势：插入/删除不搬元素，迭代器稳定。

`list` 的代价是每个节点多两个指针（前驱/后继），内存开销比 `vector` 大；节点分散在堆上，遍历时缓存不友好，随机访问是线性时间（没有 `operator[]`）。**`list` 的 `sort()` 是成员函数**，不是 `std::sort`：`std::sort` 要求随机访问迭代器，`list` 只有双向迭代器；成员 `sort` 通常用归并排序，能利用节点拼接。**不要把 `list` 当"插入更快的 vector"**；大多数场景 `vector` 的连续存储和缓存局部性更重要，只有明确的"中间插入/删除 + 迭代器稳定"需求才选 `list`。

`forward_list` 是单向链表，只支持向前遍历，节点更小，插入/删除接口也更"手工"（`insert_after` / `erase_after`）。**没有特殊需求时，`forward_list` 比 `list` 更少见**；先掌握 `vector` 与 `list` 的取舍，再看单向链表。

代价是：`list` 没有随机访问，`std::sort` 不能用（需要随机访问迭代器），要用 `list::sort`；每个节点都有前后指针，内存开销比 `vector` 大；遍历时节点分散，缓存局部性差。**只有"频繁在中间插入/删除，并且需要迭代器稳定"时，`list` 才值得；只是"想少搬元素"而用 `list`，往往会输给 `vector` 的连续存储。**

11.6 `std::array`：固定大小，不碰堆

cpp **container_choose: 大小在编译期固定的数组** examples/ch05/container_choose/src/main.cpp 第 6-14 行

```
int main() {
    std::array<int, 5> values = fixed_values();
    int total = 0;
    for (int value : values) {
        total += value;
    }
    std::cout << "size=" << values.size() << " sum=" << total << "\n";
    return 0;
}
```

终端输出 终端输出 container-choose-out

```
size=5 sum=15
```

`std::array<int, 5>` 的大小是编译期常量，对象直接包含 5 个 `int`，不分配堆内存；`size()` 是编译期常量，`operator[]` 不做边界检查（`at()` 才检查并抛异常）。它适合"元素个数固定、生命周期与对象一致"的场景：查表、固定维度向量、栈上小缓冲。`std::vector` 适合"元素个数运行期才知道、可能增长"的场景。**`array` 不是"更快的 vector"，它是"带容器接口的 C 数组"；不能 `push_back`，也不能改变大小。**

图 / 搜索过程 **按访问模式选容器**

第 1 步 **随机访问为主**

vector：连续存储，O(1) 下标　　array：固定大小，O(1) 下标

第 2 步 **两端进出**

deque：两端摊还 O(1)　　vector：尾部摊还 O(1)，头部 O(n)

第 3 步 中间插入/删除且要迭代器稳定

list ：节点不搬，O(1) 接链　　vector ：中间插入 O(n)，可能扩容

第 1 帧的 `vector` / `array` 适合随机访问；第 2 帧的 `deque` 适合两端进出；第 3 帧的 `list` 适合中间插入且要求迭代器稳定。**选择顺序通常是：先 `vector`，不够再考虑 `deque`，只有明确的节点稳定性/中间插入需求才上 `list`。**

`std::array` 是聚合类型，可以直接用花括号初始化：`std::array<int, 3> a{1, 2, 3};`；`std::get<0>(a)` 与 `a[0]` 都能访问元素，`std::tuple_size_v<decltype(a)>` 在编译期给出长度。它没有构造函数、没有 `push_back`，大小是类型的一部分；`std::array<int, 3>` 与 `std::array<int, 4>` 是不同类型，不能互相赋值。**需要运行期长度就换 `vector`，需要固定长度且不想堆分配就用 `array`**；两者不是"谁替代谁"，而是"长度在编译期还是运行期确定"的两种选择。

`std::array` 作为函数参数时，通常按 `const std::array<T, N>&` 传递，`N` 由模板参数或具体类型给出；如果函数想接受任意长度的 `array`，需要写成模板 `template <std::size_t N> void f(const std::array<int, N>&)`。**`array` 的长度是类型的一部分，不会像 `vector` 那样隐式转换成长度可变的容器**；需要运行期长度时，用 `std::span`（C++20）或指针 + 长度，而不是把 `array` 塞进 `vector` 再复制。

C++17 的类模板实参推导也能用于 `std::array`：`std::array a{1, 2, 3};` 推导为 `std::array<int, 3>`。这很方便，但元素类型必须一致；`std::array a{1, 2.5};` 推导会失败（`int` 与 `double` 冲突），需要显式写 `std::array<double, 2>{1, 2.5}`。**花括号初始化 + CTAD 的组合要看清元素类型**，别让编译器替你猜一个你并不想要的类型。

11.7 调试期边界检查：越界写入的护栏

`operator[]` 不做边界检查，越界是未定义行为；但在调试构建里，libstdc++ 的 `_GLIBCXX_ASSERTIONS` 会给容器的下标访问加断言，把越界变成一次可定位的终止。`bounds_check/` 故意写 `values[values.size()] = 9`，让断言抓住这次越界。

cpp **bounds_check: 下标等于 size()，越界一格** examples/ch05/bounds_check/src/bounds_check.cpp　　第 7–11 行

```cpp
int out_of_bounds_index() {
    auto values = checked_values();
    values[values.size()] = 9;
    return values[0];
}
```

清单把这个例子声明为 `mode: detect`，检测键是 `glibcxx-assertions`：正例用 `-D_GLIBCXX_ASSERTIONS` 编译，本机运行时以退出码 3 终止，stderr 含 `Assertion '__n < this->size()' failed.`；阴性对照用基线参数 `-D_GLIBCXX_NO_ASSERTIONS` 把机制关掉，断言不再出现。**这是 libstdc++ 的调试期护栏，不是标准行为**：换一个标准库、或关掉断言，越界仍然可能是静默破坏或访问冲突。生产代码处理不可信输入时，用 `at()`（标准规定越界抛 `std::out_of_range`）或显式检查，不要把安全寄托在 `_GLIBCXX_ASSERTIONS` 上。

11.8 `string` 与 SSO：只点到

`std::string` 在标准里不是"容器"，但它的接口与 `vector<char>` 很像：连续存储、可扩容、迭代器失效规则类似。主流实现有短字符串优化（SSO）：短字符串直接存在对象内部，不分配堆内存；长字符串才分配。SSO 让短字符串的构造/拷贝更便宜，但也意味着 `data()` 的地址、`capacity()` 的读数依赖实现与字符串长度。**不要依赖 SSO 的具体阈值，也不要把 `data()` 的地址当成长期标识**；需要稳定地址时，用 `std::string` 以外的方案或明确文档化。

`std::string_view` 是"指向一段字符的只读视图"，不拥有数据、不保证以 `\0` 结尾。它适合做只读参数，避免 `const std::string&` 逼调用方构造临时 `string`；但它不能延长数据的生命周期：`std::string_view sv = make_string();` 之后源字符串已经销毁，`sv` 就是悬空视图。**函数参数用 `string_view`，意味着调用方必须保证数据在调用期间活着；函数返回 `string_view` 时，要格外确认它指向的不是局部对象。**

11.9 本机实测：六个读数

实验记录 · 本机实测：容量、失效与两端插入

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `vector_growth` 的 stdout 是 `1/1 2/2 3/4 4/4 5/8 6/8 7/8 8/8`：本机容量按倍增增长；
- `vector_invalidate` 的 stdout 是 `moved=yes`：`push_back` 触发扩容，`data()` 变了；
- `reserve_demo` 的 stdout 是 `capacity_ok=yes` / `same=yes`：预留后 5 次 push 没有扩容；
- `deque_ends` 的 stdout 是 `1 2 3 4 5`：两端插入后顺序正确；
- `list_splice` 的 stdout 是 `1 2 3 4` / `first=4 second=0`：节点被搬走，`second` 变空；
- `bounds_check` 是 `detect` 例子：越界写入被 libstdc++ 断言抓住，退出码 3，stderr 含断言原文。

11.10 下一单元从哪继续

下一单元 `11.2 关联容器与哈希` 会把"按位置访问"换成"按键访问"：`map` / `set` 用比较器维持有序，`unordered_*` 用哈希分桶。`map::operator[]` 会插入、`find` 不会；自定义类型要提供比较或哈希。这一单元先确保你能说清"为什么 `vector` 扩容后不能再用旧迭代器"，下一单元再处理"键怎么比较、怎么哈希"。

小结 · 这一节要带走的五件事

- `size` 是元素个数，`capacity` 是不扩容能放的数量；`reserve` 只改容量，`resize` 改元素个数。
- `vector` 扩容会重新分配并搬移元素，所有指针/引用/迭代器失效；不要跨扩容保存迭代器。
- `deque` 两端插入便宜且支持随机访问；`list` 中间插入/删除不搬节点，但没有随机访问。
- `std::array` 大小编译期固定、不碰堆；需要增长时用 `vector`。
- 默认先选 `vector`；只有明确的两端/中间插入与迭代器稳定性需求，才考虑 `deque` / `list`。

自测题 · 自测题（先自己做，再对答案）

1. `size()` 与 `capacity()` 各表示什么？`reserve(100)` 之后 `size()` 会变成 100 吗？
2. `vector_invalidate` 里 `moved=yes` 说明了什么？如果 `push_back` 之后解引用 `before`，会怎样？
3. `deque` 与 `list` 都能在两端插入，主要差别是什么？`list::splice` 为什么不复制元素？
4. `std::array<int, 5>` 与 `std::vector<int>` 的存储差别是什么？各自适合什么场景？
5. 为什么本书建议"默认先选 vector"，而不是"需要插入就用 list"？

注 · 自测答案

1. `size` 是当前元素个数，`capacity` 是不重新分配时能容纳的元素个数；`reserve(100)` 只保证容量至少 100，`size` 仍是原来的值（通常是 0）。
2. 说明 `push_back` 触发了重新分配，元素被搬到新块，`data()` 变了。解引用 `before` 会访问已释放的旧块，是未定义行为。
3. `deque` 支持随机访问、分段存储，两端插入摊还常数；`list` 是双向链表，不支持随机访问，但节点插入/删除不搬元素，迭代器稳定。`splice` 直接改链表指针，把节点接过去，不复制元素。
4. `array` 直接包含固定数量的元素，不分配堆，大小编译期固定；`vector` 在堆上管理连续存储，可增长。固定大小、小数据用 `array`；运行期大小可变用 `vector`。
5. `vector` 连续存储、随机访问快、常数因子小；`list` 的节点分配与指针跳转成本通常更高。只有"频繁中间插入/删除且需要迭代器稳定"时，`list` 的优势才可能抵消这些成本。

关联容器与哈希：map、set、unordered_*

`vector` 用下标访问；`map` 用键访问。你给一个键，它找回对应的值，不需要自己遍历。`map` / `set` 保持有序（按比较器），查找是对数时间；`unordered_map` / `unordered_set` 用哈希分桶，平均常数时间，但遍历顺序不保证。两者都能去重、都能按"键"找"值"，差别在"有序"与"哈希"这两条底层路线。

这一单元有六个小实验：`map` 的插入与查找、`operator[]` 的副作用、自定义比较器、自定义哈希、`set` 去重、`multimap` 的重复键。每个实验只改一个变量，让你看清“键怎么比较、怎么哈希、什么时候会插入”。

◎ 学习目标 · 看完这一单元，你应该能

- 用 `map` / `set` 插入、查找、遍历，并解释为什么遍历顺序是按键排序的；
- 说清 `operator[]` 与 `find` / `contains` / `at` 的区别，指出哪一行会悄悄插入；
- 给自定义类型提供比较器（有序容器）或哈希 + `operator==`（无序容器）；
- 说出 `unordered_*` 的遍历顺序为什么不可依赖，以及桶/负载因子只作观察；
- 用 `set` 去重、用 `multimap` / `multiset` 保存重复键，并用 `equal_range` 取一组值。

12.1 先看工程长什么样

这一单元在 `examples/ch05/` 下再加六个多文件例子。每个例子仍然 `include/` + 两个 `.cpp` + 清单；容器类型、比较器、哈希放在头文件里，`main.cpp` 只做插入/查找/打印。

目录树 **工程：examples/ch05/ 这一单元新增的六个例子**

第 1 步 先建 map 的插入与副作用

```
examples/
└── ch05/
    ├── 上一单元 ： cpp11 的六个例子
    └── map_basic/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── map_basic.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── map_basic.cpp ← 本课新增
        └── map_basic.json ← 本课新增
```

第 2 步 加上自定义比较器与哈希

```
examples/
└── ch05/
    ├── map_basic/ ： 同上
    ├── map_insert_side_effect/ ← 本课新增
    ├── custom_compare/ ← 本课新增
    └── unordered_hash/ ← 本课新增
```

第 3 步 最后加上 set 与 multimap

```
examples/
└── ch05/
    ├── 前四个例子 ： 同上
    ├── set_unique/ ← 本课新增
    └── multimap_demo/ ← 本课新增
```

第 1 帧的 `map_basic/` 用 `map<string,int>` 存分数，遍历输出按名字排序；`map_insert_side_effect/` 专门展示 `operator[]` 插入。第 2 帧的 `custom_compare/` 用 `std::greater<int>` 让键降序；`unordered_hash/` 给自定义 `Point` 写哈希与 `==`。第 3 帧的 `set_unique/` 去重，`multimap_demo/` 用 `equal_range` 取重复键。

12.2 `map`：键唯一、按比较器排序

cpp **map_basic: 插入、遍历、查找** examples/ch05/map_basic/src/main.cpp 第 7-17 行

```cpp
int main() {
    auto scores = make_scores();
    scores["dave"] = 60;
    for (const auto& [name, score] : scores) {
        std::cout << name << "=" << score << " ";
    }
    std::cout << "\n";
    auto it = scores.find("bob");
```

```
    std::cout << "found=" << (it != scores.end() ? it->second : -1) << "\n";
    return 0;
}
```

终端输出 终端输出 map-basic-out

```
alice=90 bob=75 carol=88 dave=60
found=75
```

`std::map<std::string,int>` 的键是 `std::string`，值 `int`；遍历输出按字符串升序（`alice`、`bob`、`carol`、`dave`），因为 `map` 用 `std::less<Key>` 维持有序。`scores["dave"] = 60;` 插入一个新键；`scores.find("bob")` 返回指向 `bob` 的迭代器，`it->second` 读值。**`map` 的“有序”不是按插入顺序，而是按比较器**；插入顺序不同，遍历顺序仍按比较器。

`map` 的键是唯一的：重复插入同一个键，`insert` 不会覆盖已有值（返回 `pair<iterator,bool>`，`bool` 为假）；要覆盖，用 `operator[]` 或 `insert_or_assign`。`map` 的查找、插入、删除都是对数时间；`set` 是只有键没有值的 `map`，`multimap` / `multiset` 允许重复键。`map` 的迭代器在插入/删除时通常保持稳定（节点树结构，除被删元素外），但不要把“通常”当保证；需要长期保存迭代器时，查标准对该操作的失效规则。

`map` 的元素类型是 `std::pair<const Key, T>`，键是 `const`，不能通过迭代器改键（改了会破坏排序）。`it->second` 可以改值，`it->first` 只能读。**“键不可变”是有序容器的基本不变量**；要“改键”，只能删除旧节点再插入新键，或者用 C++17 的节点句柄 `extract` 把节点取出来（节点的键仍不可变，通常用于转移到另一个容器）。

插入接口要分清：`insert({key, value})` 拷贝/移动构造一个 `pair`，键已存在就丢弃；`emplace(args...)` 就地构造 `pair`，但参数会被转发给 `pair` 的构造函数，容易在隐式转换上踩坑；`try_emplace(key, args...)` 把 `key` 与“值的构造参数”分开，键已存在时不构造值，是插入新键的首选；`insert_or_assign(key, value)` 键存在就赋值、不存在就插入。**C++17 之后，新代码优先 `try_emplace` 与 `insert_or_assign`**，它们把意图写得更清楚，也避免不必要的构造。

`map::at(key)` 在键不存在时抛 `std::out_of_range`；`find` 返回 `end()`；`contains` 返回 `bool`；`count` 返回 0 或 1（`multimap` 可能更多）。**只读查找不要用 `operator[]`**：它的语义是“取或插入”，会改变容器。需要“查不到就用默认值”时，用 `find` + 条件，或 `at` + `try` / `catch`，把“缺失”显式处理；不要用一个会偷偷插入的接口表达“只读”。

12.3 `operator[]` 会插入：`find` 不会

cpp **map_insert_side_effect: 读一个不存在的键也会改变 size** examples/ch05/map_insert_side_effect/src/main.cpp 第 7–19 行

```
int main() {
    auto names = make_names();
    std::cout << "before=" << names.size() << "\n";

    std::string value = names[2];
    std::cout << "value=" << (value.empty() ? "empty" : value)
              << " after=" << names.size() << "\n";

    auto it = names.find(3);
    std::cout << "find3=" << (it == names.end() ? "missing" : "found")
              << " size=" << names.size() << "\n";
    return 0;
}
```

终端输出 终端输出 map-insert-side-effect-out

```
before=1
value=empty after=2
find3=missing size=2
```

`names[2]` 在键 2 不存在时**插入**一个 `{2, ""}`，返回值的引用；所以 `before=1`、`after=2`，`value=empty`。`names.find(3)` 不插入，返回 `end()`，`size` 仍然是 2。**读操作也可能改容器**：`operator[]` 的语义是“取或插入”，不是“只读查找”。如果只想读，用 `at(key)`（不存在抛 `out_of_range`）或 `find`；只想判断存在，用 `contains`（C++20）。

`operator[]` 的插入行为在标准里的写法是 `return try_emplace(x).first->second;`：键不存在就插入一个值初始化的 `mapped_type`，再返回它的引用。对 `int` 来说值初始化为 0；对 `std::string` 是空串。**不要把 `operator[]` 用在 `const map` 上**：它需要修改容器，`const map` 没有这个重载；只读时用 `at` 或 `find`。

标准原文 · **标准原文：`operator[]` 等价于 try_emplace**

> Effects: Equivalent to: return try_emplace(x).first->second;
>
> 大意：`map::operator[]` 在键不存在时插入一个值初始化的映射值，并返回它的引用；因此它可能改变容器大小。
>
> 出处：N4950 §24.4.4.3「Element access」（[map.access]）。

⚠ 陷阱 · **用 `operator[]` 做只读查找**

你会看到：代码里写 `if (m[key] == 0) ...` 判断"键不存在"，结果 `m.size()` 越跑越大，后面遍历多出一堆空值。你会误判成"map 有 bug"或"遍历写错了"。真相是：`operator[]` 在键不存在时插入了默认值；你每判断一次，就多一个元素。**只读用 `find` / `at` / `contains`；要"没有就插入"才用 `operator[]` 或 `try_emplace`。**

`try_emplace` 与 `operator[]` 的差别不只在性能：`operator[]` 要求 `mapped_type` 可默认构造，`try_emplace` 不要求——它把值的构造参数只在"键不存在"时使用。**如果值类型没有默认构造函数，`operator[]` 直接编译失败**；这时必须用 `try_emplace` / `insert` / `insert_or_assign`。插入失败时，`try_emplace` 返回 `{iterator, false}`，迭代器指向已存在的元素，且不会构造新值；需要"插入或读取"时，它比 `operator[]` 更精确。

`at` 与 `operator[]` 的 const 差异也值得注意：`const map` 只能调用 `at` / `find` / `contains`，不能调用 `operator[]`，因为后者可能插入。**把只读接口写成 `const map&`**，编译器就会阻止你误用 `operator[]`；这是用类型系统守住"只读"的简单办法。需要"没有就插入"的接口，明确写成非 `const`，让调用者知道它会改容器。

12.4 自定义比较器：键的顺序由你定

cpp **custom_compare: std::greater<int> 让键降序** examples/ch05/custom_compare/src/main.cpp 第 8–15 行

```cpp
int main() {
    auto ranked = make_ranked();
    for (const auto& [rank, medal] : ranked) {
        std::cout << rank << "=" << medal << " ";
    }
    std::cout << "\n";
    return 0;
}
```

终端输出 终端输出 custom-compare-out

```
3=bronze 2=silver 1=gold
```

`std::map<int, std::string, std::greater<int>>` 把比较器从默认的 `std::less<int>` 换成 `std::greater<int>`，遍历顺序变成降序（3、2、1）。比较器必须是**严格弱序**：对任意 `a`、`b`，`comp(a,b)` 与 `comp(b,a)` 不能同时为真；相等时两者都为假。写自定义比较器时，返回 `a < b` 或 `a > b`，不要写 `<=` / `>=`；否则 `map` 会把相等键当成不同键，或出现未定义行为。

比较器也可以是比较函数对象/ lambda（需要作为类型参数传入，或 C++20 的透明比较器）。**比较器决定"键相等"的定义**：`map` 认为 `!(comp(a,b) || comp(b,a))` 的两个键等价；如果你的比较器忽略大小写，`"Apple"` 与 `"apple"` 就是同一个键。这既是灵活性，也是坑：比较器写错，查找/去重都会跟着错。

严格弱序有两条容易忽略的要求：**不可自反**（`comp(a,a)` 必须为假）与**传递性**（`comp(a,b)` 且 `comp(b,c)` 则 `comp(a,c)`）。写 `comp(a,b) { return a <= b; }` 违反不可自反，`a == b` 时两个方向都为真，`map` 的等价判断失效；写一个依赖外部可变状态的比较器，可能在不同时刻给出不同顺序，破坏容器不变量。**比较器应当是纯函数、稳定、无副作用**；需要"按多个字段排序"时，按优先级依次比较，不要用减法（可能溢出）。

C++14 起可以用透明比较器 `std::less<>` 做异质查找：`std::map<std::string, int, std::less<>>` 可以直接用 `const char*` 查找，不必先构造 `std::string`。**透明比较器的类型是 `std::less<>`，不是**

`std::less<std::string>`；后者要求实参类型一致，异质查找不会启用。自定义比较器也可以提供 `is_transparent` 标记与多个重载，让查找省掉临时对象；这是性能优化，不是正确性要求。

比较器写错时，`map` 可能"看起来能跑"：小数据量下顺序碰巧对，数据一多就出现查找失败、遍历乱序、甚至死循环。**测试比较器时，用相等键、边界键、乱序插入三种情况**；不要只用一组升序数据验证。`std::map` 的正确性依赖比较器，容器自己无法发现"你的 `<` 不满足严格弱序"。

图 / 搜索过程 **有序容器与哈希容器的键约束**

第 1 步 有序容器

map ：需要严格弱序比较器　set ：需要严格弱序比较器

map → less<Key> 默认

第 2 步 无序容器

unordered_map ：需要 hash + operator==　unordered_set ：需要 hash + operator==

unordered_map → hash<Key> 默认，可自定义

第 1 帧的 `map` / `set` 只依赖比较器；第 2 帧的 `unordered_*` 依赖哈希函数与相等比较。**两条路线对键的要求不同**：自定义类型要么提供 `<`（或用比较器），要么提供 `std::hash` 特化/哈希对象与 `==`；只提供其中一个，另一种容器就用不了。

12.5 `unordered_*`：哈希与桶

cpp **Point 与 PointHash** examples/ch05/unordered_hash/include/unordered_hash.h 第 8–19 行

```cpp
struct Point {
    int x;
    int y;

    bool operator==(const Point& other) const;
};

struct PointHash {
    std::size_t operator()(const Point& point) const;
};

std::unordered_map<Point, std::string, PointHash> make_points();
```

cpp **main: 用 Point{3,4} 查找** examples/ch05/unordered_hash/src/main.cpp 第 5–11 行

```cpp
int main() {
    auto points = make_points();
    auto it = points.find(Point{3, 4});
    std::cout << "found=" << (it != points.end() ? it->second : "missing") << "\n";
    std::cout << "size=" << points.size() << "\n";
    return 0;
}
```

终端输出 终端输出　unordered-hash-out

```
found=b
size=2
```

`unordered_map` 用哈希把键分到桶里，查找时先算哈希、定位桶，再在桶内比较相等。自定义 `Point` 必须提供两样东西：哈希函数（`PointHash`）与相等比较（`operator==`）。**哈希相同不代表键相等**：不同键可能落进同一个桶（哈希冲突），容器用 `==` 区分；如果 `==` 与哈希不一致，查找会失败或行为异常。

`unordered_map` 的遍历顺序**不保证**，不要依赖它输出确定的顺序；需要有序输出时，把键取出来排序，或直接用 `map`。`bucket_count()`、`load_factor()` 是可观察的读数，但具体桶数与 rehash 时机由实现决定；`reserve` 可以给个预期元素数，减少 rehash。**哈希容器的"平均 O(1)"是平均情况**；哈希函数很差或冲突很多时，最坏会退化到线性。

自定义类型的哈希要满足两条：相等的键必须得到相同的哈希值；不相等的键尽量分散。只满足第一条、哈希值全相同，程序仍然正确，但所有元素挤在一个桶里，查找退化成线性。**"正确"和"高效"是两件事**：`std::hash` 的默认实现对

整数通常是恒等映射，对字符串有专门算法；自定义类型应把参与 `==` 的字段都纳入哈希，否则相等对象可能哈希不同，容器直接失效。

`rehash` 会改变桶的数量，**通常使所有迭代器失效**，但指向元素的引用/指针仍有效（节点没有搬）。`reserve(n)` 请求"至少能放 n 个元素而不 rehash"；`max_load_factor` 控制"平均每个桶多少元素时扩容"。**不要把 `bucket_count()` 当成元素个数**；它是桶数，元素数用 `size()`。需要稳定迭代器时，不要依赖 `unordered_*` 的 rehash 行为；需要稳定顺序时，直接用 `map` / `set`。

`unordered_map` 的键需要 `std::hash<Key>` 或显式传入哈希对象，值需要 `operator==`。**标准库没有为"任意自定义类型"自动生成哈希**；要么特化 `std::hash`（放在 `namespace std`，遵循特化规则），要么在容器模板参数里传自定义哈希。前者让类型"开箱即用"，后者更局部、更容易测试；库代码里通常选后者，避免污染 `std`。

`unordered_map::operator[]` 与 `map::operator[]` 一样会插入默认值；只读查找用 `find` / `at` / `contains`。无序容器没有 `lower_bound` / `upper_bound` / `equal_range`，因为它不维护顺序；需要这些操作就换 `map` / `set`。**"查找快"不等于"什么都能做"**：无序容器牺牲了顺序与范围查询，换来了平均常数时间的单键查找；选型时先看需不需要按顺序遍历或范围查询。

`unordered_*` 的内存开销通常比 `map` / `set` 更大：每个元素除了键值还要存节点指针/哈希缓存，桶数组本身也占空间。**"哈希表一定更省"是误解**；元素少、键简单时，`map` 的节点开销与 `unordered_map` 的桶开销可能差不多，甚至 `map` 更省。先测量，再选型；不要因为"O(1)"三个字就默认哈希容器更快。

12.6 `set` 去重与 `multimap` 重复键

cpp **set_unique: 重复插入只保留一个** examples/ch05/set_unique/src/main.cpp 第 6–13 行

```cpp
int main() {
    auto values = make_set();
    for (int value : values) {
        std::cout << value << " ";
    }
    std::cout << "\nsize=" << values.size() << "\n";
    return 0;
}
```

终端输出 终端输出 set-unique-out

```
1 2 3
size=3
```

`make_set()` 返回 `{3, 1, 2, 1, 3}`，但 `std::set<int>` 去重并按升序排列，输出 `1 2 3`、`size=3`。`set` 的键就是值，插入重复键不会改变容器；`multiset` 则保留重复。**`set` 的"去重"依赖于比较器**：如果比较器认为两个值等价，它们就是重复的，哪怕字节不同。

cpp **multimap_demo: equal_range 取同一键的所有值** examples/ch05/multimap_demo/src/main.cpp 第 7–16 行

```cpp
int main() {
    auto index = make_index();
    auto [first, last] = index.equal_range("apple");
    int total = 0;
    for (auto it = first; it != last; ++it) {
        total += it->second;
    }
    std::cout << "apple_total=" << total << " size=" << index.size() << "\n";
    return 0;
}
```

终端输出 终端输出 multimap-demo-out

```
apple_total=4 size=3
```

`std::multimap<std::string,int>` 允许同一个键出现多次；`equal_range("apple")` 返回一对迭代器 `[first,last)`，覆盖所有键为 `apple` 的元素。输出 `apple_total=4`（1+3）、`size=3`（三个元素，两个键）。

`multimap` **没有** `operator[]`：同一个键可能对应多个值，取一个值的语义不明确；要访问重复键，用 `equal_range` 或 `find` + 循环。

`equal_range` 在 `map` / `set` 上同样可用，返回的区间长度为 0 或 1；在 `multimap` / `multiset` 上可能更长。它比“`find` 再循环 `++it` 直到键变”更直接，也避免手写边界条件。`lower_bound` / `upper_bound` 分别给出“第一个不小于键”和“第一个大于键”的位置，**对有序容器是二分查找，对无序容器没有这两个接口**；需要范围查询时，这是 `map` / `set` 相对 `unordered_*` 的独特优势。

`set` 只存键，适合“去重 + 有序遍历 + 快速成员检查”；`multiset` 允许重复键，适合“统计每个值的出现次数”或“保留重复记录”。**用** `set` **做去重时，插入返回值告诉你是否真的插入了**：`auto [it, inserted] = s.insert(value);`，`inserted` 为假说明已存在。需要计数时，`map<Value, int>` 通常比 `multiset` 更直接；需要保留全部重复记录时，才用 `multiset`。

`set` / `map` 的迭代器是双向的，不是随机的：不能 `it + 3`，只能 `++` / `--`。`std::next(it, 3)` 可以前进 3 步，但那是线性时间；**不要用** `std::next` **在** `set` **上做“随机访问”**，需要下标就换 `vector`。有序容器的优势是“按顺序查找/遍历”，不是“按位置访问”；把两者混在一起，选型就会错。

`count` 在有序容器上并不是“一次二分就结束”：它先二分找到起点，再逐个统计等价的重复元素，重复很多时耗时随重复个数增长。只关心“有没有”时，用 `contains`（C++20）或 `find`，不要用 `count(...) != 0` 再数一遍。

插入时给一个“提示位置”可以降低查找成本：`m.emplace_hint(hint, key, value)` 告诉容器“新元素大概插在这里”。如果提示正确，插入可以接近常数时间；提示错了，容器会退回到正常查找。**提示是优化，不是正确性要求**；批量插入已知有序的数据时，用 `end()` 作为提示很常见，但不要为了“优化”牺牲代码可读性。先写正确版本，测量后再决定要不要加提示。

最后，`map` / `set` 的键一旦插入就不能原地修改；需要“改键”时，用 `extract` 取出节点、改值（键仍不可变）、再插入，或者删除旧键插入新键。**不要用** `const_cast` **去改键**，那会破坏排序不变量，后续查找与遍历都可能出错；容器的正确性依赖“键在生命周期内不变”，这条约束不能绕过。

C++17 的节点句柄让“在容器之间搬元素而不复制”成为可能：`auto node = m.extract(key);` 取出节点，`m2.insert(std::move(node));` 插入另一个容器；节点的键仍不可变，但值可以改。`extract` **不使其他元素的迭代器失效**，适合“改键”或“合并容器”的场景；`merge` 则把另一个容器的节点直接搬过来，遇到重复键就留下源节点。**这些接口的语义比** `erase` **+** `insert` **更精确**，但用之前要确认两个容器的键类型与比较器兼容。

12.7 本机实测：五个读数

实验记录 · **本机实测：有序、插入副作用与哈希**

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `map_basic` 的 stdout 是 `alice=90 bob=75 carol=88 dave=60` / `found=75`：遍历按字符串升序；
- `map_insert_side_effect` 的 stdout 是 `before=1` / `value=empty after=2` / `find3=missing size=2`：`operator[]` 插入，`find` 不插入；
- `custom_compare` 的 stdout 是 `3=bronze 2=silver 1=gold`：`greater<int>` 让键降序；
- `unordered_hash` 的 stdout 是 `found=b` / `size=2`：自定义哈希与 `==` 能查到 `Point{3,4}`；
- `set_unique` 的 stdout 是 `1 2 3` / `size=3`：重复值被去重并排序；
- `multimap_demo` 的 stdout 是 `apple_total=4 size=3`：`equal_range` 取到两个 `apple`。

12.8 下一单元从哪继续

下一单元 `11.3 迭代器、算法与 ranges` 会把“用下标/键访问”换成“用迭代器访问”：`sort` / `find_if` / `transform` / `accumulate`，lambda 作为谓词，`views` 作为惰性管线。`map` 的迭代器是双向的，`vector` 的是随机的；算法对迭代器类别有要求，这就是为什么 `std::sort` 不能直接排 `list`。先确保你能说清 `map::operator[]` 与 `find` 的差别，再去看算法。

小结 · 这一节要带走的五件事

- `map` / `set` 按键有序、键唯一；`unordered_*` 按哈希分桶，遍历顺序不保证。
- `map::operator[]` 在键不存在时插入默认值；只读用 `find` / `at` / `contains`。
- 有序容器要严格弱序比较器；无序容器要哈希 + `operator==`。
- `set` 去重依赖比较器；`multimap` / `multiset` 用 `equal_range` 访问重复键。
- 哈希容器的平均常数时间不等于最坏常数时间；不要依赖遍历顺序。

自测题 · 自测题（先自己做，再对答案）

1. `map` 的遍历顺序由什么决定？插入顺序会影响它吗？
2. `names[2]` 与 `names.find(2)` 在键不存在时分别做什么？各用什么场景？
3. 自定义类型放进 `std::map` 与 `std::unordered_map`，分别需要提供什么？
4. 为什么 `unordered_map` 的遍历顺序不能依赖？哈希冲突时容器靠什么区分键？
5. `set` 与 `multiset`、`map` 与 `multimap` 的区别是什么？`multimap` 为什么没有 `operator[]`？

注 · 自测答案

1. 由比较器（默认 `std::less<Key>`）决定；插入顺序不影响遍历顺序，只影响插入成本与节点结构。
2. `names[2]` 会插入一个值初始化的映射值并返回引用；`names.find(2)` 返回 `end()`，不插入。只读用 `find` / `at`，需要"没有就插入"才用 `operator[]`。
3. 放进 `map` 需要严格弱序比较器（默认 `<`）；放进 `unordered_map` 需要哈希函数与 `operator==`。
4. 因为桶的数量、rehash 时机、桶内顺序都由实现决定；哈希冲突时容器用 `operator==` 区分同一个桶里的不同键。
5. `set` / `map` 键唯一，`multiset` / `multimap` 允许重复键；`multimap` 同一个键可能对应多个值，`operator[]` 没有明确的"取哪一个值"的语义，所以标准不提供。

迭代器、算法与 ranges：lambda、views、zip / enumerate

容器负责"存"，算法负责"算"，迭代器是两者之间的接口。`std::sort(v.begin(), v.end())` 不认识 `vector`，只认识"随机访问迭代器"；`std::find_if` 不认识 `list`，只认识"能从头走到尾的迭代器"。ranges 把这层接口再往前推一步：你可以写 `v | views::filter(...) | views::transform(...)`，像搭管道一样组合操作，而且求值是惰性的——不遍历就不计算。

这一单元有六个小实验：四个算法、lambda 谓词、惰性 view、`zip` / `enumerate`、range 管线、算法与手写循环的等价结果。重点是"什么时候用算法、什么时候用循环"，以及"view 不拥有数据"这条寿命规则。

学习目标 · 看完这一单元，你应该能

- 用 `sort` / `find_if` / `transform` / `accumulate` 完成常见容器处理，并说出它们要求什么迭代器类别；
- 用 lambda 捕获外部变量作为谓词，并解释按值捕获与按引用捕获的差别；
- 解释 `views::filter` / `transform` 为什么是惰性的，修改源容器后 view 的结果为什么会变；
- 用 `views::zip` / `views::enumerate` 同时遍历多个序列/带下标，并正确解构；
- 说出 view 的悬空风险，知道什么时候必须把源容器放在 view 之前声明。

13.1 先看工程长什么样

这一单元在 `examples/ch05/` 下再加六个多文件例子。算法与 view 都放在 `main.cpp` 或辅助 `.cpp` 里，头文件负责数据与函数声明；第 2 章起的硬规定③继续生效。

目录树 **工程：examples/ch05/ 这一单元新增的六个例子**

第 1 步 **先建排序/查找/变换/累加**

```
examples/
└── ch05/
    ├── 上一单元 ：cpp12 的六个例子
    └── algo_basic/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── algo_basic.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── algo_basic.cpp ← 本课新增
        └── algo_basic.json ← 本课新增
```

第 2 步 **加上 lambda 与惰性 view**

```
examples/
└── ch05/
    ├── algo_basic/ ：同上
    ├── lambda_pred/ ← 本课新增
    └── views_lazy/ ← 本课新增
```

第 3 步 **最后加上 zip/enumerate、管线与手写循环对照**

```
examples/
└── ch05/
    ├── 前三个例子 ：同上
    ├── zip_enumerate/ ← 本课新增
    ├── range_pipeline/ ← 本课新增
    └── algo_manual/ ← 本课新增
```

第 1 帧的 `algo_basic/` 用 `sort` / `find_if` / `transform` / `accumulate` 处理一组数；`lambda_pred/` 用捕获阈值的 lambda 做 `find_if` / `count_if`。第 2 帧的 `views_lazy/` 展示 view 的惰性；`zip_enumerate/` 同时遍历两个序列并带下标。第 3 帧的 `range_pipeline/` 把 filter/transform/take 串成一条管线；`algo_manual/` 用算法与手写循环做同一件事，输出相同。

13.2 四个算法：排序、查找、变换、累加

cpp **algo_basic：四个算法各做一件事** examples/ch05/algo_basic/src/main.cpp 第 7–20 行

```cpp
int main() {
    auto values = make_values();
    std::sort(values.begin(), values.end());
    std::cout << "sorted=";
    for (int value : values) {
        std::cout << value << " ";
    }

    auto found = std::find_if(values.begin(), values.end(),
                              [](int value) { return value > 3; });
    std::cout << "\nfound=" << (found != values.end() ? *found : -1) << "\n";
    std::cout << "sum_doubled=" << sum_after_double(values) << "\n";
    return 0;
}
```

终端输出 终端输出 algo-basic-out

```
sorted=1 2 3 4 5
found=4
sum_doubled=30
```

`std::sort(begin, end)` 要求**随机访问迭代器**：`vector` / `deque` / `array` 满足，`list` 只提供双向迭代器，不能直接用 `std::sort`（要用 `list::sort`）。`std::find_if` 只要求输入迭代器，`list` 也能用；`std::transform` 把每个元素交给函数并写回（或写到另一段），`std::accumulate` 在 `<numeric>` 里，按顺序把元素累加。**算法不关心容器的类型，只关心迭代器能做什么**；这就是"迭代器类别"的实际意义。

`sum_after_double(values)` 在辅助 `.cpp` 里又做了一次 `sort`、`transform`，再 `accumulate`，得到 `2+4+6+8+10=30`。注意 `values` 是按值传进去的，函数内的排序不会影响 `main` 里的 `values`；**算法修改的是“你给它的那个范围”**，按值传参时修改的是副本。

迭代器分成几类：输入、输出、前向、双向、随机访问、连续。类别越高，算法能用的操作越多：`std::find` 只要输入迭代器，`std::reverse` 要双向，`std::sort` 要随机访问。**算法文档里的“要求”不是建议，而是编译期约束**；给 `std::sort` 传 `list` 的迭代器，会在模板实例化深处报错。选算法前先看它要求哪一类迭代器，再确认容器是否满足；这比试错快。

输出迭代器是另一类：它只写不读，典型代表是 `std::back_inserter(v)`。`std::transform(first, last, std::back_inserter(out), f)` 会把结果追加到 `out`，而不是覆盖已有元素；`std::copy`、`std::remove_copy_if` 也常用它。`back_inserter` **会调用** `push_back`**，所以目标容器必须有** `push_back`；`std::array` 没有，就不能用它。输出范围的大小由算法写入的元素个数决定，调用者要保证目标能容纳（或使用插入器）。

`std::sort` 不保证相等元素的相对顺序；需要稳定排序时用 `std::stable_sort`，代价是额外内存或更慢。`std::partial_sort` 只排出前 k 个，`std::nth_element` 只保证第 n 个位置正确、左右分区，适合“找中位数/前 k 小”而不需要全排序。**算法名字里的限定词（stable / partial / nth）就是复杂度与语义的提示**；不要一律用 `sort`，也不要在需要稳定时用 `sort` 然后自己补救。

13.3 lambda 作为谓词：捕获外部状态

cpp **lambda_pred: 捕获 threshold 的谓词** examples/ch05/lambda_pred/src/main.cpp 第 6–15 行

```cpp
int main() {
    auto values = predicate_values();
    int threshold = 3;
    auto first = std::find_if(values.begin(), values.end(),
                              [threshold](int value) { return value > threshold; });

    std::cout << "first=" << (first != values.end() ? *first : -1) << "\n";
    std::cout << "count=" << count_above(values, threshold) << "\n";
    return 0;
}
```

终端输出 终端输出 lambda-pred-out

```
first=4
count=3
```

`[threshold]` 是**按值捕获**：lambda 内部保存一份 `threshold` 的副本，之后外部改 `threshold` 不影响 lambda。`[&threshold]` 是按引用捕获：lambda 内看到的是外部变量本身，外部改了它，lambda 下次调用也会看到新值；代价是 lambda 活得比外部变量久时，引用会悬空。**捕获列表决定“lambda 拥有什么、借用什么”**，不是“写起来方不方便”。

`find_if` 返回第一个满足谓词的迭代器；找不到返回 `end()`。`count_if` 数满足谓词的元素个数。谓词可以是 lambda、函数对象、普通函数；算法只要求“能对元素调用并返回可转成 `bool` 的结果”。lambda 的类型是编译器生成的闭包类型，每个 lambda 表达式类型唯一；这也是为什么 `std::sort` 的第三个参数可以是 lambda，而不需要写一个具名函数对象。

算法按值接收谓词，所以你传进去的 lambda 会被复制一份。**如果 lambda 按引用捕获了外部变量，复制后仍然指向同一个外部变量**；算法执行期间外部变量必须活着。如果 lambda 按值捕获，复制的是闭包成员，谓词内部的修改不会回到你的 lambda（除非传 `std::ref`）。理解“算法复制谓词”这件事，才能解释“为什么我在 lambda 里加计数，外面没变”。

谓词应该是纯的：同样的输入返回同样的结果，不修改外部状态。**有副作用的谓词会让算法行为依赖调用次数与顺序**，而标准只规定算法结果，不规定它内部调用谓词多少次。需要“遍历时做副作用”，用手写循环或 `for_each`；需要“筛选”，用无副作用的谓词。把副作用塞进 `count_if`，代码可能对某个实现正确，换一个标准库就变样。

13.4 `views`：惰性，不拥有数据

cpp **views_lazy: view 在遍历时才计算** examples/ch05/views_lazy/src/main.cpp 第 7–25 行

```cpp
int main() {
    auto values = lazy_values();
    auto view = values
        | std::views::filter([](int value) { return value % 2 == 0; })
        | std::views::transform([](int value) { return value * 10; });

    std::cout << "before=";
    for (int value : view) {
        std::cout << value << " ";
    }

    values[1] = 10;
    std::cout << "\nafter=";
    for (int value : view) {
        std::cout << value << " ";
    }
    std::cout << "\n";
    return 0;
}
```

终端输出 终端输出 views-lazy-out

```
before=20 40
after=100 40
```

第一次遍历 `view` 时，`filter` 挑出偶数 `2, 4`，`transform` 乘 10，输出 `20 40`。然后 `values[1] = 10` 把原来的 2 改成 10（仍是偶数）；第二次遍历同一条 `view`，`filter` 重新读取源容器，输出 `100 40`。**`view` 不缓存结果、不拥有元素；它只是"遍历时怎么取/怎么算"的描述。** 修改源容器后，view 的结果会跟着变——这既是惰性的好处，也是悬空风险的来源。

`view` 的寿命依赖源容器：上面的 `values` 在 `view` 之前声明、之后销毁，所以 `view` 在 `values` 活着时使用是安全的。如果写 `auto view = make_values() | views::filter(...);`，`make_values()` 返回的临时 `vector` 在整条语句结束后销毁，`view` 就指向已销毁的容器；再遍历它是未定义行为。**view 不延长源数据的寿命**；要么把源容器存进一个有名字的变量，要么让 view 只在该变量活着的作用域内使用。

> ⚠ **陷阱** · **把 view 当成容器保存**
>
> 你会看到：`auto view = values | views::filter(...);` 之后，把 `view` 返回/存进成员、过一会儿再遍历，结果偶尔正确、偶尔乱码。你会误判成"views 不稳定"。真相是：view 通常只保存指向源范围的迭代器/指针，不拥有元素；源容器销毁、扩容、修改后，view 可能悬空或结果变化。**view 是"借来看"的，不是"拿走保存"的；需要长期保存就把结果物化到 `vector`（例如 `std::ranges::to<std::vector>()`，C++23）。**

view 对象通常很小，复制一个 view 不会复制元素；它只是复制"怎么遍历"的描述。`filter` 会保存谓词的副本，`transform` 会保存函数的副本，`take` 只保存计数。**如果谓词/函数有状态，复制 view 会复制这些状态**；算法按值接收函数对象时，状态变化可能不会回到你原来的对象——需要共享状态时，用 `std::ref` 包一层，或者把状态放到 view 外部。

C++23 的 `std::ranges::to<std::vector>()` 把 range 物化成容器，例如 `auto v = pipeline | std::ranges::to<std::vector>();`。**物化会立即遍历整条管线**，把结果存下来；之后源容器再变，物化结果不跟着变。需要"快照"时物化，需要"实时视图"时保留 view。两种语义要明确选，不要一会儿当 view 用、一会儿当容器用。

`views::take` 会短路：取够 n 个元素就停止向上游要数据，所以它能用在无限序列上，例如 `views::iota(0) | views::take(5)`。**惰性 + 短路是 view 最实用的组合**；如果先 `filter` 再 `take`，`filter` 可能已经扫描了很多元素才凑够 n 个，这仍然是按需的，只是"需求"由 `take` 决定。

13.5 `zip` 与 `enumerate`：并行遍历与下标

cpp **zip_enumerate：两个序列一起走，一个序列带下标** examples/ch05/zip_enumerate/src/main.cpp 第 6–21 行

```cpp
int main() {
    auto numbers = zip_numbers();
    auto letters = zip_letters();

    std::cout << "zip=";
    for (auto [number, letter] : std::views::zip(numbers, letters)) {
        std::cout << number << letter << " ";
    }

    std::cout << "\nenumerate=";
    for (auto [index, letter] : std::views::enumerate(letters)) {
        std::cout << index << ":" << letter << " ";
    }
    std::cout << "\n";
    return 0;
}
```

终端输出 终端输出 zip-enumerate-out

```
zip=1a 2b 3c
enumerate=0:a 1:b 2:c
```

`views::zip(numbers, letters)` 把两个范围按位置配成 tuple，`auto [number, letter]` 用结构化绑定拆开；遍历长度取两者较短的那个（标准规定 zip 在任一范围结束时停止）。`views::enumerate(letters)` 产生"下标 + 元素"的 tuple，`index` 从 0 开始；它不需要你手写计数器，也不会因为 `continue` 忘加下标。**这两个 view 都是 C++23 才进入标准库的设施**；本机 g++ 已实测可用，换一个较老的编译器可能没有。

`zip` / `enumerate` 返回的是 view，不是容器；它们同样不拥有源数据。`zip` 的迭代器类别取决于两个源范围中最弱的一个；如果其中一个只能输入遍历，`zip` 也只能输入遍历。**组合 view 时，先看每个源范围的迭代器能力，再看算法要求**；不要假设 `zip` 之后还能随机访问。

`enumerate` 的 `index` 默认是 `std::size_t` 或实现定义的无符号类型，参与算术时要注意有符号/无符号混用；需要 `int` 时显式转换，不要把它直接和有符号数比较。`zip` 的元素是引用 tuple，`auto [a, b]` 按值复制元素；要修改源元素，写 `auto&& [a, b]` 或 `auto& [a, b]`。**结构化绑定的 `auto` 与范围 for 的 `auto` 一样，默认是复制**；在 zip 上改元素却没生效，先检查这里漏没漏 `&`。

13.6 range 管线：filter → transform → take

cpp **range_pipeline：三个 view 串成一条管线** examples/ch05/range_pipeline/src/main.cpp 第 7–20 行

```cpp
int main() {
    auto values = pipeline_values();
    auto pipeline = values
        | std::views::filter([](int value) { return value > 2; })
        | std::views::transform([](int value) { return value * 2; })
        | std::views::take(3);

    std::cout << "pipeline=";
    for (int value : pipeline) {
        std::cout << value << " ";
    }
    std::cout << "\n";
    return 0;
}
```

终端输出 终端输出 range-pipeline-out

```
pipeline=6 8 10
```

`values = {1,2,3,4,5,6}`；`filter(>2)` 留下 `3,4,5,6`；`transform(*2)` 变成 `6,8,10,12`；`take(3)` 只取前三个，输出 `6 8 10`。整条管线是**逐元素拉取**的：`take` 要一个元素，才向 `transform` 要，`transform` 再向 `filter`

要，`filter` 再读源容器；不会先建一个中间 `vector`。**这就是惰性管线的价值：少分配、少拷贝，且可以提前停止。**

管线也有边界：`take(3)` 之后的元素不会被计算；如果你需要"结果保存下来"，要把管线物化成容器。`filter` 的谓词与 `transform` 的函数会被反复调用，函数体不要依赖"只调用一次"的假设；有副作用时，顺序与次数由遍历方式决定。

图 / 搜索过程 **range 管线的数据流**

第 1 步 源范围

values : 1 2 3 4 5 6

第 2 步 filter 只放行大于 2 的元素

filter : 3 4 5 6

values → filter 逐元素拉取

第 3 步 transform 与 take

transform : 6 8 10 12　　take3 : 6 8 10

filter → transform 逐元素

transform → take3 逐元素

第 1 帧是源容器；第 2 帧的 `filter` 不建新容器，只在遍历时跳过不满足的元素；第 3 帧的 `transform` 与 `take` 继续按需拉取，最终只计算到第三个结果。**整条管线只在 `for` 循环真正遍历时工作**；没有遍历，就没有计算。

管线里的函数最好无副作用。`filter` / `transform` 在每次遍历时都会重新调用，调用次数取决于上游要多少元素、`take` 何时停止；**你无法从"源容器有 6 个元素"推断谓词一定调用 6 次。**需要"每个元素只处理一次"时，先物化成容器，再在容器上做后续操作；需要"流式处理、提前停止"时，接受按需调用。

`filter` / `transform` 返回的元素类型可以变化：`transform` 可以把 `int` 变成 `std::string`，后面的 `take` 只关心"能不能取"，不关心具体类型。**view 的组合是类型层面的组合**，编译器会为整条管线生成一个具体的 view 类型；如果类型太长，用 `auto` 接住，不要手写。C++23 的 `views::zip` / `enumerate` 也返回 view，可以继续接 `filter` / `transform`。

13.7 算法与手写循环：什么时候用哪个

cpp **algo_manual: 算法与手写循环得到同一个计数** examples/ch05/algo_manual/src/main.cpp 第 5–10 行

```cpp
int main() {
    auto values = manual_values();
    std::cout << "algo=" << count_even(values)
              << " manual=" << manual_count_even(values) << "\n";
    return 0;
}
```

终端输出 终端输出 algo-manual-out

```
algo=3 manual=3
```

`std::count_if` 与手写循环得到相同的 `3`。算法的优势是**意图清楚、少写边界条件**：`count_if` 的名字直接说明"数满足条件的元素"，循环则要读一遍才知道；算法的劣势是**有时不够灵活**：需要提前退出、多个条件、复杂状态时，手写循环可能更直接。判断标准不是"算法更高级"，而是"这段代码的主要读者要一眼看懂什么"。

`std::ranges` 版本的算法可以直接收范围（`std::ranges::sort(values)`），不用写 `begin/end`；本单元仍用迭代器版本，是为了让你看清"算法与容器之间是迭代器"这层关系。两种写法可以混用，但一个项目里最好统一风格。

`std::ranges` 还支持投影（projection）：`std::ranges::sort(people, {}, &Person::age)` 按 `age` 排序，不必写比较 lambda；第三个参数是"取哪个成员参与比较"。**投影让你把"比较什么"与"怎么比较"分开**：`std::ranges::sort(people, std::greater{}, &Person::age)` 按年龄降序。传统 `std::sort` 没有投影，只能写 lambda；新代码可以优先用 ranges 版本。

算法与手写循环的性能通常没有本质差别：编译器会把 `find_if` 内联成类似的循环，ranges 的 view 也能被优化。**选择标准是可读性与正确性，不是"算法一定更快"。**如果手写循环能更清楚地表达"提前退出、多个状态、复杂边界"，就手写；如果算法名字能一眼说明意图，就用算法。把简单循环硬套成算法，反而会让读者多跳一层。

用算法不等于自动安全：`std::remove_if` 只把保留的元素移到前面并返回新逻辑结尾，并不删除元素，必须再接 `erase` 才是“删除”；在遍历中修改容器结构，仍然要遵守迭代器失效规则。**算法负责“怎么处理元素”，不负责“容器的生命周期”**，这两件事要分开想。

13.8 本机实测：六个读数

实验记录 · **本机实测：算法、惰性与 zip**

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `algo_basic` 的 stdout 是 `sorted=1 2 3 4 5` / `found=4` / `sum_doubled=30`；
- `lambda_pred` 的 stdout 是 `first=4` / `count=3`：捕获阈值的谓词生效；
- `views_lazy` 的 stdout 是 `before=20 40` / `after=100 40`：修改源容器后 view 重新计算；
- `zip_enumerate` 的 stdout 是 `zip=1a 2b 3c` / `enumerate=0:a 1:b 2:c`；
- `range_pipeline` 的 stdout 是 `pipeline=6 8 10`：filter/transform/take 串起来；
- `algo_manual` 的 stdout 是 `algo=3 manual=3`：算法与手写循环结果一致。

13.9 下一单元从哪继续

下一章 `14.1 auto、decltype 与结构化绑定` 会把这一单元里的 `auto [number, letter]` 拆开讲：`auto` 怎样推导、为什么有时候会丢掉引用与 `const`、`decltype((x))` 为什么多一层括号。容器与算法会继续作为例子出现，但重点转到“类型推导”——写模板与泛型代码时，这是每天都用的基本功。

算法与 view 的边界要分清：算法通常“立即执行并返回结果/迭代器”，view 通常“描述怎么遍历，遍历时才执行”。`std::ranges::find` 是算法，会立刻扫描；`views::filter` 是 view，不扫描。**名字里带 `views::` 的通常是惰性适配器，名字是动词的通常是算法**；不确定时看它返回的是“结果”还是“范围”。

读标准算法文档时，重点看三件事：要求哪类迭代器、是否修改输入范围、返回什么。`std::sort` 修改范围并返回 `void`；`std::find` 不修改、返回迭代器；`std::transform` 可修改输出、返回输出末尾。**把“是否修改”记错，是算法误用的常见来源**；例如 `std::remove` 并不删除元素，只是把要保留的往前挪，必须配合 `erase`。

读算法返回值也要看清：`std::find` 返回迭代器，`std::count` 返回数量，`std::for_each` 返回函数对象（C++17 起），`std::transform` 返回输出范围的末尾。**返回值不是装饰**：连续操作时，上一个算法的返回值常常是下一个算法的输入；把返回迭代器当成 `bool` 用，是另一个常见错误。

小结 · **这一节要带走的五件事**

- 算法通过迭代器工作：`sort` 要随机访问，`find_if` 只要输入迭代器；`list` 不能直接用 `std::sort`。
- lambda 的捕获列表决定它拥有副本还是借用引用；按引用捕获要保证生命周期。
- `views` 是惰性描述，不拥有数据；修改源容器会改变 view 的结果，源容器销毁会让 view 悬空。
- `views::zip` 并行遍历、`views::enumerate` 带下标，二者都是 C++23 设施，返回 view 而非容器。
- 算法与手写循环都能正确时，选“读者一眼看懂意图”的那个；不要为了用算法而用算法。

自测题 · **自测题（先自己做，再对答案）**

1. 为什么 `std::sort` 不能直接排 `std::list`？应该用什么替代？
2. `[threshold]` 与 `[&threshold]` 两种捕获，在外部修改变量后行为有什么不同？
3. `views_lazy` 里第二次遍历为什么输出 `100 40`，而不是缓存下来的 `20 40`？
4. `views::zip(numbers, letters)` 在其中一个范围更短时会怎样？它返回的是容器还是 view？
5. 为什么 `auto view = make_values() | views::filter(...)` 有悬空风险？安全的写法是什么？

注 · **自测答案**

1. `std::sort` 要求随机访问迭代器，`list` 只提供双向迭代器；用 `list::sort` 成员函数。
2. 按值捕获保存一份副本，外部改动不影响 lambda；按引用捕获借用外部变量，外部改动会被 lambda 看到，但外部变量先销毁时引用悬空。

3. 因为 view 不缓存结果；第二次遍历重新从源容器读取，`values[1]` 已从 2 改成 10，filter 仍放行，transform 得到 100。
4. `zip` 在任一范围结束时停止，长度取较短者；它返回一个 view，不拥有两个源范围的数据。
5. `make_values()` 返回的临时容器在整条语句结束后销毁，view 仍引用它；再遍历就是悬空。安全写法是把源容器存进有名字的变量，保证它活得比 view 长，或把结果物化成容器。

第 6 章

现代语言特性

本章 2 个单元 · 页眉从这里之后的页面开始出现

1. auto、decltype 与结构化绑定 零基础 · 第 14 单元 · 真编译真运行
2. lambda 与函数对象 零基础 · 第 15 单元 · 真编译真运行

auto、decltype 与结构化绑定

`auto` 让编译器替你写类型，`decltype` 让编译器告诉你表达式的类型，结构化绑定让一个聚合对象拆成几个名字。这三件工具都能让代码更短，但短不等于清楚：`auto` 会丢掉顶层 `const` 与引用，`decltype((x))` 比 `decltype(x)` 多一层括号就多一个引用，结构化绑定默认是"绑定到元素"还是"复制元素"，也取决于你写的是 `auto` 还是 `auto&`。

这一单元用六个小实验把这三种推导钉住。每个实验都尽量用 `std::is_same_v` 把类型打印成 0/1，避免靠"看起来像"判断。

◎ 学习目标 · 看完这一单元，你应该能

- 说出 `auto` 推导时丢掉什么、保留什么，并用 `decltype` / `is_same_v` 验证；
- 区分 `auto`、`auto&`、`const auto&`、`auto&&` 在初始化与范围 for 中的行为；
- 解释 `decltype(x)` 与 `decltype((x))` 的结果为什么不同；
- 用结构化绑定拆 `tuple` 与结构体，并通过引用绑定修改原对象；
- 判断什么时候不该用 `auto`，避免把类型藏在代码里。

14.1 先看工程长什么样

这一单元在 `examples/ch06/` 下建六个多文件例子。`auto` / `decltype` 的验证放在 `main.cpp`，辅助函数放 `.cpp`；头文件负责声明。

目录树 **工程：examples/ch06/ 这一单元新增的六个例子**

第 1 步 先建 auto 推导与引用绑定

```
examples/
└── ch06/
    └── auto_deduce/ ← 本课新增
        ├── include/ ← 本课新增
        │   └── auto_deduce.h ← 本课新增
        ├── src/ ← 本课新增
        │   ├── main.cpp ← 本课新增
        │   └── auto_deduce.cpp ← 本课新增
        └── auto_deduce.json ← 本课新增
```

第 2 步 加上 decltype 与结构化绑定

```
examples/
└── ch06/
    ├── auto_deduce/ ：同上
    ├── auto_ref/ ← 本课新增
    ├── decltype_paren/ ← 本课新增
    ├── structured_tuple/ ← 本课新增
    └── structured_struct/ ← 本课新增
```

第 3 步 最后加上范围 for 的三写法

```
examples/
└── ch06/
    ├── 前五个例子 ：同上
    └── auto_loop/ ← 本课新增
```

第 1 帧的 `auto_deduce/` 用 `const int&` 初始化 `auto`，看它丢掉什么；`auto_ref/` 对照 `auto` / `auto&` / `const auto&`。第 2 帧的 `decltype_paren/` 比较 `decltype(x)` 与 `decltype((x))`；`structured_tuple/` 与 `structured_struct/` 拆 tuple 与结构体。第 3 帧的 `auto_loop/` 用三种写法遍历同一个 `vector`，观察哪些写法改到了源元素。

14.2 `auto`：丢掉顶层 `const` 与引用

cpp **auto_deduce：同一个 source，三种声明** examples/ch06/auto_deduce/src/main.cpp 第 7-16 行

```cpp
    const int& source = seed_value();
    auto plain = source;
    const auto& bound = source;
    auto& alias = const_cast<int&>(source);

    std::cout << "plain_is_int=" << std::is_same_v<decltype(plain), int> << "\n";
    std::cout << "bound_is_ref=" << std::is_same_v<decltype(bound), const int&> << "\n";
    std::cout << "same_address=" << (&alias == &source ? "yes" : "no") << "\n";
    return 0;
}
```

终端输出 终端输出 auto-deduce-out

```
plain_is_int=1
bound_is_ref=1
same_address=yes
```

`source` 的类型是 `const int&`；`auto plain = source;` 推导为 `int`，**顶层 `const` 与引用都被丢掉**，`plain` 是一个独立的 `int` 副本。`const auto& bound = source;` 推导为 `const int&`，绑定到 `source`；`auto& alias = const_cast<int&>(source);` 显式去掉 `const` 后绑定，`&alias == &source` 为真。**`auto` 的默认行为是"按值复制"，复制会丢掉引用与顶层 const；要保留引用/const，必须自己写 `&` / `const`。**

`const int& source = seed_value();` 里的 `seed_value()` 返回临时 `int`，`const` 引用延长了它的寿命到 `source` 的作用域；`auto plain = source` 又从 `source` 复制一份。`auto` 推导规则与模板参数推导类似：忽略引用、忽略顶层 const/volatile，数组/函数会退化成指针（`auto` 从数组初始化时得到指针）。**`auto` 不是"自动帮你选最合适的类型"，它有一套固定的去引用/去 const 规则**；不理解规则，就会得到"为什么我改 `plain` 没改到 `source`"的疑问。

`auto` 对花括号初始化有专门的规则：`auto x = {1, 2, 3};` 推导为 `std::initializer_list<int>`，不是 `std::vector<int>`，也不是数组；`auto x{1};` 在 C++17 起推导为 `int`，但 `auto x{1, 2};` 通常编译失败。**看到 `auto` 与花括号一起出现，先确认你想要的是列表、数组还是单个值**；需要容器就写 `std::vector<int> x{1,2,3}`，不要指望 `auto` 替你选。

`auto` 作为函数返回类型时，按值推导，引用与顶层 `const` 都会丢掉：`auto f() { return global_ref; }` 返回的是 `int` 副本，不是引用。要返回引用，写 `auto&` 或 `decltype(auto)`；要返回 `const` 值，`auto` 也帮不了你（顶层 const 会被忽略）。**返回类型是接口的一部分，别让 `auto` 把它藏起来**；库接口尤其应该写清楚返回的是值、引用还是 `const` 引用。

`auto` 还会复制代理对象。`std::vector<bool>` 的 `operator[]` 返回代理引用，`auto bit = v[0];` 得到的是代理对象的副本，不是 `bool&`；`auto&& bit = v[0];` 才能绑定到代理。**遇到"看起来像引用、`auto` 却复制了"的类型，先查它是不是代理**；`vector<bool>`、表达式模板、某些迭代器适配器都有这种行为。

14.3 `auto&`、`const auto&` 与 `auto&&`

cpp **auto_ref：副本、引用、只读引用** examples/ch06/auto_ref/src/main.cpp 第 6-19 行

```cpp
int main() {
    auto values = ref_values();

    auto copy = values[0];
    auto& reference = values[0];
    const auto& constant = values[1];

    reference = 10;
    copy = 99;

    std::cout << "values0=" << values[0] << " copy=" << copy << "\n";
    std::cout << "constant=" << constant << " size=" << values.size() << "\n";
    return 0;
}
```

终端输出 终端输出 auto-ref-out

```
values0=10 copy=99
constant=2 size=3
```

`auto copy = values[0];` 是一个独立的 `int`，改它不影响 `values`；`auto& reference = values[0];` 是引用，`reference = 10` 改到 `values[0]`；`const auto& constant = values[1];` 是只读引用，不能通过它修改，但能读到 2。**`auto&` 要求初始化器是左值**；`auto&&` 是转发引用，既能绑左值也能绑右值，绑左值时 `auto` 推导为 `T&`，绑右值时推导为 `T`。

`auto&&` 在范围 for 里也常见：`for (auto&& item : container)` 能绑定容器返回的左值/代理引用，不会像 `auto` 那样复制；但如果你不打算修改，`const auto&` 更清楚地表达"只读"。**选择顺序：只读用 `const auto&`；要改源元素用 `auto&`；需要转发/绑定任意值类别用 `auto&&`；确实要一份副本才用 `auto`。**

`auto&&` 只有在"需要推导"的语境里才是转发引用；写成 `const auto&&` 就只是右值引用，不能绑左值。`auto&& x = expr;` 会把临时对象的寿命延长到 `x` 的作用域，这和 `const auto&` 的寿命延长类似；但 `auto&&` 还能绑定非 `const` 右值，让你在 `x` 上调用非 `const` 成员。**"能绑定"不等于"该修改"**：绑定一个临时对象后改它，通常没有意义；需要保存结果就按值复制。

`auto&` 要求初始化器是左值，`auto&&` 不要求；但两者都不做隐式类型转换来"凑类型"。`auto& x = 42;` 编译失败，因为 42 是右值；`const auto& x = 42;` 可以，因为 `const` 引用能绑定临时。**`auto` 推导不会帮你加 `const`，也不会帮你把右值变成左值**；类型不匹配时，编译器宁可报错，也不会悄悄复制出一个新对象。

选择写法时，先问"我要不要改源对象、要不要复制、要不要接受临时值"。只读且可能接受临时：`const auto&`；要改且源是左值：`auto&`；要转发或处理代理：`auto&&`；明确要一份独立副本：`auto`。**四种写法不是风格偏好，而是语义选择**；写错一种，行为就不同。

14.4 `decltype`：括号会改变结果

cpp **decltype_paren: x 与 (x) 的类型不同** examples/ch06/decltype_paren/src/main.cpp 第 6–15 行

```cpp
int main() {
    int x = global_value();
    const int cx = x;

    std::cout << "x_is_int=" << std::is_same_v<decltype(x), int> << "\n";
    std::cout << "paren_is_ref=" << std::is_same_v<decltype((x)), int&> << "\n";
    std::cout << "const_is_const=" << std::is_same_v<decltype(cx), const int> << "\n";
    std::cout << "value=" << x << "\n";
    return 0;
}
```

终端输出 终端输出 decltype-paren-out

```
x_is_int=1
paren_is_ref=1
const_is_const=1
value=3
```

`decltype(x)` 看的是**名字/表达式被声明的类型**，`x` 是 `int`，结果是 `int`；`decltype((x))` 看的是**表达式的值类别**，`(x)` 是一个左值表达式，结果是 `int&`。**多一层括号，不是"多此一举"，而是从"名字的类型"切到"表达式的类型"**；这是 `decltype` 最容易踩的坑。

`decltype(cx)` 是 `const int`，因为 `cx` 声明为 `const int`；`decltype` 不会像 `auto` 那样丢掉顶层 `const`。这也是为什么返回类型推导里 `decltype(auto)` 与 `auto` 不同：`auto` 按值推导，`decltype(auto)` 保留表达式的引用/const。写模板库时，这两个工具经常一起出现；应用代码里，优先用 `auto` / `auto&`，只在需要精确类型时用 `decltype`。

`decltype` 不求值它的操作数：`decltype(f())` 只分析 `f` 的返回类型，不会真的调用 `f`。这让它适合写"只关心类型、不产生副作用"的代码，也适合在模板里推导返回类型。**`decltype` 的操作数是一个"未求值表达式"**；如果表达式里有副作用，副作用不会发生。需要"制造一个类型但不构造对象"时，标准库用 `std::declval<T>()`，它也只出现在未求值语境里。

`decltype(auto)` 是“用 `decltype` 的规则推导返回类型”：`decltype(auto) f() { return x; }` 返回 `int`，因为 `x` 是名字；`decltype(auto) g() { return (x); }` 返回 `int&`，因为 `(x)` 是左值表达式。**返回类型里多一对括号，可能让函数从“返回值”变成“返回引用”**；这是 `decltype(auto)` 最容易出错的地方。写返回引用时，优先显式写 `T&`，只在泛型转发场景用 `decltype(auto)`。

`decltype` 也常用于表达式类型检查：`static_assert(std::is_same_v<decltype(a + b), int>);`。它看的是表达式的“声明类型 + 值类别”，不是运行期值；`decltype(i++)` 是 `int`（后置 `++` 返回右值），`decltype(++i)` 是 `int&`（前置 `++` 返回左值）。**同一个变量，不同的表达式形式，`decltype` 结果不同**；写类型断言时要把表达式写完整。

图 / 搜索过程 **auto 与 decltype 的推导路径**

第 1 步 auto 从初始化器按值推导

source : const int& | auto_plain : int，去掉 const 与 &

第 2 步 decltype 看名字或表达式

decltype_x : int，名字的类型 | decltype_paren : int&，左值表达式的类型

第 1 帧的 `auto` 把 `const int&` 变成 `int`；第 2 帧的 `decltype(x)` 保留名字的 `int`，`decltype((x))` 因为 `(x)` 是左值而得到 `int&`。**同一个 `x`，写法不同，类型不同；这不是编译器不一致，是规则不同。**

14.5 结构化绑定：把聚合拆成名字

cpp **structured_tuple: 拆 tuple，并用引用改回原对象** examples/ch06/structured_tuple/src/main.cpp 第 7-16 行

```cpp
int main() {
    auto record = make_record();
    auto& [number, name, score] = record;

    number = 8;
    std::cout << "number=" << number << " name=" << name
              << " score=" << score << "\n";
    std::cout << "tuple0=" << std::get<0>(record) << "\n";
    return 0;
}
```

终端输出 终端输出 structured-tuple-out

```
number=8 name=seven score=7.5
tuple0=8
```

`make_record()` 返回 `tuple<int,string,double>`；`auto& [number, name, score] = record;` 把三个元素绑定成三个名字，`number = 8` 直接改到 `record` 的第 0 个元素，所以 `tuple0=8`。**结构化绑定不是“创建三个副本”，而是“给元素起名字”**；写 `auto [number, name, score]` 会复制整个 tuple，改 `number` 不会影响 `record`；写 `auto&` 才是引用绑定。

cpp **structured_struct: 结构体也能拆** examples/ch06/structured_struct/src/main.cpp 第 5-12 行

```cpp
int main() {
    Config config = make_config();
    auto& [retries, host] = config;

    retries = 5;
    std::cout << "retries=" << config.retries << " host=" << host << "\n";
    return 0;
}
```

终端输出 终端输出 structured-struct-out

```
retries=5 host=localhost
```

`Config` 是两个公有成员的结构体；`auto& [retries, host] = config;` 按声明顺序把成员绑定到名字，`retries = 5` 改到 `config.retries`。结构化绑定对 `tuple`、`pair`、数组、以及“所有非静态数据成员都是公有”的类都适用；对

私有成员不起作用。**绑定顺序是成员声明顺序，不是初始化列表顺序，也不是你写名字的顺序**；这是它与"随便起三个变量"最大的差别。

数组也能结构化绑定：`int a[3] = {1,2,3}; auto& [x, y, z] = a;` 把三个元素绑定成名字，`x` 就是 `a[0]`。数组绑定要求元素个数与名字个数一致，不能少写或多写；编译器在编译期检查。**这是"按位置绑定"，不是"按值复制"**；写 `auto` 而不是 `auto&` 时，会复制整个数组（数组不能按值返回，但可以按值绑定生成一份副本）。

结构化绑定的名字不是普通变量，而是一组"绑定到子对象"的别名；`decltype(name)` 得到的是子对象的类型（不是某种引用类型），不能对这些名字做 `sizeof...(name)` 之类的包操作。绑定本身不创建新的对象，所以取地址得到的是子对象的地址；修改绑定就是修改原对象（当绑定是引用时）。**把绑定当成"解包出来的变量"容易误解**；它更像"给已有子对象起了个短名字"。

C++20 起，结构化绑定的名字可以被 lambda 捕获（C++17 不允许直接捕获）。**跨标准版本写代码时要注意这条差异**；如果项目还在 C++17，先把绑定拆成普通变量再捕获，或者用初始化捕获把需要的值复制进 lambda。绑定名字的生命周期与所在作用域一致，lambda 捕获引用时仍然要检查原对象是否活着。



第 2 帧的三个名字没有各自的存储，它们分别指向 tuple 的三个元素；所以 `number = 8` 之后 `tuple0=8`。如果把 `auto&` 换成 `auto`，第 2 帧就会变成"三个独立副本"，改名字不改 tuple。

14.6 范围 for 里的三种写法

cpp **auto_loop：副本、引用、只读引用遍历同一个 vector** examples/ch06/auto_loop/src/main.cpp 第 6–25 行

```cpp
int main() {
    auto values = loop_values();

    int copy_sum = 0;
    for (auto copy : values) {
        copy *= 10;
        copy_sum += copy;
    }

    for (auto& reference : values) {
        reference += 1;
    }

    std::cout << "copy_sum=" << copy_sum << "\n";
    for (const auto& value : values) {
        std::cout << value << " ";
    }
    std::cout << "\n";
    return 0;
}
```

终端输出 终端输出 auto-loop-out

```
copy_sum=60
2 3 4
```

第一段 `for (auto copy : values)` 每次复制一个元素，`copy *= 10` 改的是副本，`copy_sum` 累加 10+20+30=60；第二段 `for (auto& reference : values)` 改的是源元素，三个元素各加 1；第三段 `const auto&` 只读输出 `2 3 4`。**范围 for 的 `auto` 默认也是按值复制**；如果循环体里要修改容器元素，必须写 `auto&`；如果元素很大，写 `const auto&` 避免复制。

范围 for 的底层是 `begin()` / `end()`，所以它要求范围可遍历；`auto&&` 在范围 for 里也常用（尤其元素是代理引用时）。**`auto` 在范围 for 里不是“自动引用”**；这是初学者最常见的误解之一。看到循环体里改元素却没生效，先检查是不是漏了 `&`。

范围 for 遍历临时对象时，临时对象的寿命会被延长到整个循环：`for (auto value : make_vector())` 是安全的，`make_vector()` 的返回值在循环结束才销毁。**但延长的是“范围对象”的寿命，不是元素内部的引用/指针**；如果容器里存的是指针，指针指向的对象仍然可能已经销毁。判断安全性要看“循环体访问的到底是什么”，不能只看范围 for 的语法。

结构化绑定在范围 for 里同样默认复制：`for (auto [key, value] : map)` 每次复制一个 `pair`，改 `value` 不会改 map；要改值写 `for (auto& [key, value] : map)`，只读写 `for (const auto& [key, value] : map)`。**`map` 的键是 `const`，即使写 `auto&` 也不能通过绑定改键**；这是有序容器保持不变量的一部分。

14.7 什么时候不该用 `auto`

`auto` 适合“类型在右边很明显、写出来很啰嗦”的场景：迭代器、lambda 类型、模板返回的复杂类型、结构化绑定。它不适合“类型本身就是信息”的场景：接口返回值、数值类型（`int` / `double` / `size_t`）、有隐式转换风险的表达式。**`auto x = f();` 把 `f` 的返回类型藏起来了，读代码的人必须跳到 `f` 的声明才能知道 `x` 是什么**；如果这个类型影响溢出、精度、生命周期，就应该写出来。

另外，`auto` 与 `const`/引用一起用时，规则要写清楚：`const auto&` 是只读引用，`auto&` 是可写引用，`auto` 是副本。不要为了少打几个字把 `const` 省掉；`auto` 不会自动帮你加 `const`。**短的前提是清楚，不是“让编译器猜”。**

`auto` 还会把“数值宽度”藏起来：`auto n = v.size();` 的 `n` 是 `std::size_t`（无符号），`auto i = 0;` 的 `i` 是 `int`；把两者比较时，编译器会做无符号转换，`i = -1` 可能变成巨大正数。**涉及大小、下标、循环计数时，显式写 `std::size_t` 或 `int`，别让 `auto` 把符号性藏起来**；这是 `auto` 在算法代码里最常见的坑之一。

`auto` 作为函数返回类型会隐藏接口信息：调用者看不到返回值是 `int` 还是 `size_t`，是值还是引用（`auto` 永远返回值，除非写 `auto&` / `decltype(auto)`）。**公共接口、库 API、跨模块边界，尽量写清返回类型**；内部辅助函数、迭代器、lambda 类型可以用 `auto`。判断标准是“读调用点的人需不需要知道具体类型”，需要就别藏。

`auto` 与 `std::function` 一起用时也要注意：`auto f = [](int x) { return x; };` 得到闭包类型，零开销；`std::function<int(int)> f = [](int x) { return x; };` 得到类型擦除的包装，可以重新赋值成别的可调用对象，但有间接调用开销。**先用 `auto` 接 lambda，确需“统一类型 / 运行期替换”再上 `std::function`**；下一单元会展开这两种选择。

最后记住：`auto` 的推导规则和模板参数推导同源。写函数模板时，`template <class T> void f(T)` 与 `void f(auto)` 在参数推导上是一回事；理解 `auto` 的“去引用、去顶层 const、数组退化”，就等于理解了模板参数推导的一大半。**`auto` 不是新语法糖，而是模板推导在变量声明上的应用**；把两者放在一起学，收益最大。

还有一个反向陷阱：如果函数返回 `const std::string&`，写 `auto name = f();` 会复制一份，之后改 `name` 不影响源对象；想保留引用必须写 `const auto&`。**“自动推导”不等于“保持原样”**，这正是 `auto` 与 `decltype` 最核心的差别：前者按值推导，后者保留表达式的类型与值类别。

团队规范可以更细：容器大小、下标、协议字段宽度，写 `std::size_t` 或固定宽度类型；金额、坐标这类对精度敏感的量，写 `double` 或定点类型；只有“类型在右边一眼可见”时才用 `auto`。**把 `auto` 当默认、把显式类型当例外，通常比反过来更容易维护**；评审时看到 `auto x = f();`，先问 `f` 的返回类型会不会影响溢出、精度或生命周期，会的话就写出来。这条规则听起来保守，但它把“读代码的人要跳几次”压到了最低。

14.8 本机实测：六个读数

实验记录 · 本机实测：推导结果

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `auto_deduce` 的 stdout 是 `plain_is_int=1` / `bound_is_ref=1` / `same_address=yes`：`auto` 去引用去 const，`const auto&` 保留引用；
- `auto_ref` 的 stdout 是 `values0=10 copy=99` / `constant=2 size=3`：引用改源，副本不改；
- `decltype_paren` 的 stdout 是 `x_is_int=1` / `paren_is_ref=1` / `const_is_const=1`：括号改变 `decltype` 结果；
- `structured_tuple` 的 stdout 是 `number=8` / `tuple0=8`：引用绑定改回 tuple；
- `structured_struct` 的 stdout 是 `retries=5 host=localhost`：结构体成员绑定生效；
- `auto_loop` 的 stdout 是 `copy_sum=60` / `2 3 4`：副本循环不改源，引用循环改源。

14.9 下一单元从哪继续

下一单元 `14.2 lambda 与函数对象` 会把这一单元里的 `auto` 和捕获列表放在一起：lambda 的类型是编译器生成的闭包类型，`auto` 是保存它的常用方式；捕获列表决定闭包对象里有什么，`mutable` 决定能不能改按值捕获的成员。`std::function` 则用类型擦除把不同 lambda 装进同一个类型。

读别人的 `auto` 代码时，可以用编译器确认类型：写一个 `static_assert(std::is_same_v<decltype(x), 期望类型>)`，让编译期告诉你推导结果；或者用调试器看变量类型。**不要靠"猜"来确认 `auto`**；推导规则固定，但人脑容易忽略引用、`const`、数组退化这些细节。用一次 `static_assert`，比读十遍代码更可靠。

`auto` 与 `decltype` 的组合也常用于写泛型辅助函数：先用 `decltype(auto)` 保留表达式的引用/值类别，再用 `std::forward` 把值类别传下去。**推导工具不是"让代码变短"的装饰，而是泛型编程的基础设施**；先把 `auto` 的默认规则、`decltype` 的括号规则记牢，再组合使用，才不会在模板里制造"看起来一样、实际类型不同"的 bug。

小结 · 这一节要带走的五件事

- `auto` 按值推导，丢掉引用与顶层 `const`；要引用写 `auto&`，要只读引用写 `const auto&`。
- `auto&&` 是转发引用，既能绑左值也能绑右值；范围 for 里的 `auto` 默认复制元素。
- `decltype(x)` 看名字的类型，`decltype((x))` 看表达式的值类别；多一层括号可能多一个 `&`。
- 结构化绑定给元素起名字；`auto&` 绑定到原对象，`auto` 复制，绑定顺序按成员声明。
- `auto` 适合类型冗长且明显的场景；类型本身是接口信息时，写出来更清楚。

自测题 · 自测题（先自己做，再对答案）

1. `const int& source = 7; auto plain = source;` 中 `plain` 的类型是什么？它和 `source` 是同一块存储吗？
2. `auto&` 与 `const auto&` 在范围 for 里分别能不能修改源元素？各适合什么场景？
3. `decltype(x)` 与 `decltype((x))` 对 `int x` 分别得到什么类型？为什么？
4. `auto& [a, b] = record;` 与 `auto [a, b] = record;` 修改 `a` 时，`record` 会不会变？
5. 什么时候应该避免用 `auto`？举一个类型信息很重要的例子。

注 · 自测答案

1. `plain` 是 `int`（去掉了引用与顶层 const），它是 `source` 的副本，不是同一块存储；改 `plain` 不影响 `source`。
2. `auto&` 能修改源元素，适合需要就地更新；`const auto&` 不能修改，适合只读遍历，避免复制大对象。
3. `decltype(x)` 是 `int`（名字的声明类型）；`decltype((x))` 是 `int&`，因为 `(x)` 是左值表达式，`decltype` 对表达式保留引用。
4. `auto& [a, b]` 绑定到 `record` 的元素，改 `a` 会改 `record`；`auto [a, b]` 复制元素，改 `a` 不影响 `record`。
5. 当类型本身承载接口信息（如 `int` / `double` / `size_t`、函数返回类型影响精度或溢出）时，应写出类型；`auto` 适合迭代器、lambda、模板返回类型等冗长且显而易见的场景。

lambda 与函数对象

lambda 看起来像“临时写一个函数”，其实是编译器在你写下方括号的位置生成一个**类**，再创建一个该类的对象。捕获列表决定这个类有哪些数据成员，函数体变成它的 `operator()`。理解这一点，捕获的“按值还是按引用”、`mutable` 为什么能改副本、泛型 lambda 为什么能收不同类型，全都能顺下来。

这一单元用六个多文件例子拆开闭包：先看捕获的值何时复制，再看引用捕获的生命周期风险，然后用 `mutable` 改副本，用泛型 lambda 生成多个 `operator()`，最后把不同可调用对象装进 `std::function`，并和老的 `std::bind` 对照。

◎ 学习目标 · 看完这一单元，你应该能

- 说出 lambda 表达式生成的是什么样的对象，为什么两个写法相同的 lambda 类型也不同；
- 区分按值捕获与按引用捕获，指出复制发生在什么时候；
- 解释引用捕获为什么会悬垂，以及“在本作用域内调用”为什么暂时安全；
- 用 `mutable` 修改按值捕获的副本，并说明它不会改到外面的变量；
- 写泛型 lambda，理解每个实参类型会实例化一份 `operator()`；
- 用 `std::function` 统一存放 lambda 与函数对象，并说出它的代价；
- 读懂 `std::bind`，并在新代码里优先选择 lambda。

15.1 先看工程长什么样

这一单元在 `examples/ch06/` 下新增六个例子。每个例子的头文件只放声明，辅助函数放在 `.cpp`，`main.cpp` 负责构造 lambda 并打印读数。这样每个例子仍是两个以上源文件、至少一个头文件，和后面的大作业保持同一种组织方式。

目录树 **工程：examples/ch06/ 这一单元新增的六个例子**

第 1 步 先看捕获与生命周期

```
examples/
└── ch06/
    ├── lambda_capture/ ← 本课新增
    │   ├── include/ ← 本课新增
    │   │   └── lambda_capture.h ← 本课新增
    │   ├── src/ ← 本课新增
    │   │   ├── main.cpp ← 本课新增
    │   │   └── lambda_capture.cpp ← 本课新增
    │   └── lambda_capture.json ← 本课新增
    ├── capture_ref_dangle/ ← 本课新增
    └── mutable_counter/ ← 本课新增
```

第 2 步 加上泛型 lambda 与 std::function

```
examples/
└── ch06/
    ├── 前三个例子 ： 同上
    ├── generic_lambda/ ← 本课新增
    └── function_wrapper/ ← 本课新增
```

第 3 步 最后加上 std::bind 对照

```
examples/
└── ch06/
    ├── 前五个例子 ： 同上
    └── bind_old/ ← 本课新增
```

第 1 帧的三个例子都在 `main.cpp` 里就地写 lambda：`lambda_capture/` 对照按值与按引用，`capture_ref_dangle/` 把引用捕获限制在对象存活期内，`mutable_counter/` 修改按值捕获的副本。第 2 帧把 lambda 放进函数：`generic_lambda/` 用 `auto` 参数写两个调用点，`function_wrapper/` 用 `std::function` 接收 lambda 和函数对象。第 3 帧的 `bind_old/` 用 `std::bind` 完成与 lambda 相同的绑定，方便对照。

15.2 lambda 生成的是闭包对象，不是函数

写下一个 lambda 表达式，标准规定它产生一个**独有的、没有名字的类类型**，叫闭包类型；表达式本身创建一个该类型的对象，叫闭包对象。函数体成为这个类的 `operator()`。**“独有的、没有名字”有两层意思**：第一，两个写法完全相同的 lambda，类型也不一样；第二，你没法直接写出这个类型，只能用 `auto`、模板参数或 `std::function` 接住它。

cpp **lambda_capture：按值捕获与按引用捕获** examples/ch06/lambda_capture/src/main.cpp 第 8–13 行

```cpp
    auto by_value = [value]() { return value; };
    auto by_ref = [&value]() { return value; };

    value = 20;
    std::cout << "value_capture=" << by_value()
              << " ref_capture=" << by_ref() << "\n";
```

终端输出 终端输出 lambda-capture-out

```
value_capture=10 ref_capture=20
```

`by_value` 的方括号里写 `value`，闭包对象里就多一个 `int` 成员，初始化时复制当时的 10；之后 `value = 20` 改的是外面的变量，闭包里的副本仍是 10，所以输出 `value_capture=10`。`by_ref` 写 `&value`，闭包里存的是引用（实现上通常是一个指针），调用时读的是 `value` 当前的值，所以输出 `ref_capture=20`。**按值捕获的复制发生在 lambda 创建时，不是调用时**；这是理解后面所有捕获行为的时间点。

标准原文 · 标准原文：闭包类型与按值捕获的成员

> The type of a lambda-expression (which is also the type of the closure object) is a unique, unnamed non-union class type, called the closure type …
>
> For each entity captured by copy, an unnamed non-static data member is declared in the closure type. The declaration order of these members is unspecified.

大意：lambda 表达式产生一个独有的、无名的类类型；每个按值捕获的实体，都会在闭包类型里声明一个无名的非静态数据成员。

出处：N4950 §7.5.5.2「Closure types」（[expr.prim.lambda.closure]）与 §7.5.5.3「Lambda capture」（[expr.prim.lambda.capture]）。

按值捕获的成员类型还有细节：如果被捕获的是一个引用，成员类型是“被引用对象的类型”，不是引用；如果捕获的是函数，成员是函数引用。这些规则保证了闭包对象可以复制、可以移动。**闭包对象本身是一个普通对象**：可以按值传给函数、可以放进容器、可以返回；它的“函数”身份只体现在 `operator()` 上。

一个不捕获任何东西的 lambda 还可以转换成函数指针，因为它的 `operator()` 不依赖任何成员。捕获了东西的 lambda 不能这样转换，因为函数指针没有地方存那些成员。**这也是为什么回调接口如果只收函数指针，就没法直接传一个有状态的 lambda**；要么改成模板/`std::function`，要么把状态放到别处。

捕获默认值 `[=]` / `[&]` 是“让编译器按使用情况自动捕获”：`[=]` 默认按值，`[&]` 默认按引用。**默认捕获看起来省事，但会把“到底捕获了什么”藏起来**；读到 lambda 时，你必须扫一遍函数体才知道它依赖哪些变量。工程代码更推荐显式列出捕获，尤其是跨作用域保存的 lambda；默认引用捕获 `[&]` 在异步/回调场景最容易悬垂。

类成员函数里的 lambda 捕获 `this` 时，捕获的是对象指针，不是对象副本；`[this]` 后访问成员等价于通过 `this->` 访问。如果对象在 lambda 调用前销毁，`this` 悬垂；C++17 起可以用 `[*this]` 按值捕获整个对象（要求可复制），避免悬垂。**`[=]` 在成员函数里默认捕获的是 `this` 指针，不是成员的副本**；这是另一个容易误判的点。

初始化捕获（`[owner = std::move(ptr)]`）解决了两件事：把资源移进闭包，以及让捕获的名字与外部不同。**需要 lambda 拥有资源时，用初始化捕获把所有权移进去**；这比捕获裸引用安全，也比 `std::function` 包装更直接。捕获列表是闭包的“成员声明”，写法越明确，闭包的寿命与所有权越清楚，读代码的人也越容易发现潜在的悬垂。

15.3 引用捕获：闭包不延长对象寿命

引用捕获省掉了复制，也让闭包能改到外面的变量。代价是闭包不拥有那个对象，**对象的寿命必须比闭包长**。标准在引用捕获的条文里明确提醒：这种捕获是否有效，取决于被引用对象的寿命，而不是引用本身的寿命。

cpp **capture_ref_dangle：在本作用域内，引用捕获是安全的** examples/ch06/capture_ref_dangle/src/main.cpp 第 6–10 行

```cpp
    int local = scoped_value();
    auto safe = [local]() { return local; };
    auto borrowed = [&local]() { return local; };

    std::cout << "safe=" << safe() << " borrowed=" << borrowed() << "\n";
```

终端输出 终端输出 capture-ref-out

```
safe=7 borrowed=7
```

这个例子里 `borrowed` 在 `local` 还活着时调用，所以输出 7，没有任何问题。**但把同一个 lambda 返回出去，情况就变了**：`local` 在函数返回时销毁，闭包里的引用指向一块已经结束生命周期的存储；再调用 `borrowed()` 就是未定义行为。它可能打印出旧值 7，可能打印垃圾，也可能直接崩溃——**三种结果都不算"正确"，因为标准没有规定。**

⚠ 陷阱 · 引用捕获逃出作用域就是悬垂

设想一个函数 `make_bad()`：它声明局部变量 `int local = 7;`，然后 `return [&local]() { return local; };`。返回的闭包对象里存着一个指向 `local` 的引用；`local` 是栈上的自动变量，函数一返回就销毁。调用这个闭包是未定义行为。本机某次运行可能仍然打印 7，那只是那块栈内存还没被覆盖，**不能当成"这样写也行"的证据。**

正确做法只有两种：按值捕获（`[local]`，闭包自己存一份），或者让被引用对象的寿命确实长于闭包（例如捕获堆上的对象、捕获 `shared_ptr`、或让闭包在对象销毁前用完）。`[&]` 默认引用捕获在"马上调用"的场景很方便，但只要 lambda 会被存起来、返回、传给异步任务，就要逐个检查它引用的东西还活着没有。

C++14 起可以用初始化捕获把"搬进闭包"写得更明确：`[owner = std::move(ptr)]` 创建一个新成员，用 `std::move(ptr)` 初始化；`[&ref = obj]` 创建一个引用成员。**初始化捕获让捕获的名字和外面的名字可以不同，也让移动捕获成为可能。**例如把 `std::unique_ptr` 移进闭包，闭包就拥有了对象，寿命问题自然消失；这比捕获裸引用安全得多。

图 / 搜索过程 **两种捕获的存储与寿命**

第 1 步 按值捕获，闭包自己有一份

`local : int, 值 7` `closure_safe : 成员 int = 7`

`local` → `closure_safe` 创建时复制，之后互不影响

第 2 步 按引用捕获，闭包指向外面

`closure_borrowed : 保存对 local 的引用` `local 销毁 : 引用悬垂，再调用是未定义行为`

`local` → `closure_borrowed` 调用时读同一块存储

第 1 帧的闭包可以安全地活过 `local`；第 2 帧的闭包只是"借看" `local`，`local` 一销毁，借用就失效。**按值捕获买的是寿命，按引用捕获买的是"能看到最新值、能改原对象"**；两者不能同时免费得到。

15.4 `mutable`：改的是闭包里的副本

默认情况下，lambda 的 `operator()` 是 `const` 成员函数；按值捕获的成员在函数体里是只读的。想修改这份副本，要在参数列表后写 `mutable`，它去掉 `operator()` 的 `const`。注意：`mutable` **改的是闭包对象里的成员，不是外面的原变量。**

cpp **mutable_counter：每次调用都把副本加一** examples/ch06/mutable_counter/src/main.cpp 第 6–10 行

```cpp
    int start = counter_start();
    auto next = [start]() mutable { return ++start; };

```

```
    std::cout << "first=" << next() << " second=" << next()
              << " outer=" << start << "\n";
```

终端输出 终端输出 mutable-counter-out

```
first=1 second=2 outer=0
```

`next` 创建时复制 `start` 的 0；第一次调用把成员从 0 加到 1 并返回 1，第二次从 1 加到 2 并返回 2；外面的 `start` 一直是 0，所以 `outer=0`。**闭包对象有状态，状态在多次调用之间保留**；这也是 lambda 能当计数器、累加器的原因。

如果把 `mutable` 去掉，`return ++start;` 会编译失败，因为 `operator()` 是 `const`，不能改成员。标准的规定是：`operator()` 声明为 `const` 当且仅当 lambda 没有写 `mutable`（也没有显式对象参数）。**`mutable` 不是"让捕获变成引用"**，它只解除 `operator()` 的 `const`；按值捕获仍然是副本，外面的变量仍然不受影响。

内存形态 **mutable 闭包：成员在闭包对象里**

第 1 步 **创建闭包，复制 start**

`start : int = 0 (main 的局部变量)` `next.start : int = 0 (闭包成员，独立存储)`

第 2 步 **两次调用后**

`next.start : int = 2` `start : int = 0 (没有被改)`

`next` → `next.start` operator() 改的是成员

第 1 帧的两块存储互不相干；第 2 帧只改了闭包成员。**看到"lambda 里的计数器每次都从零开始"，先检查闭包对象是不是每次都被重新创建**：临时写一个 `[count = 0]() mutable { return ++count; }` 并马上调用，闭包用完就销毁，下一次又建一个新的，状态自然不保留。

15.5 泛型 lambda：一个写法，多份 `operator()`

C++14 允许 lambda 的参数写 `auto`。这不是"运行期看类型"，而是编译器把闭包类型的 `operator()` 做成一个**函数模板**；每个不同的实参类型，实例化一份具体的 `operator()`。所以一个泛型 lambda 可以同时处理 `int` 和 `std::string`，但每一份实例仍是强类型的。

cpp **generic_lambda: 同一个 lambda 写法，两种实参类型** examples/ch06/generic_lambda/src/generic_lambda.cpp 第 5–17 行

```
std::string describe_int(int value) {
    auto describe = [](const auto& item) {
        return std::to_string(item);
    };
    return describe(value);
}

std::string describe_text(const std::string& value) {
    auto describe = [](const auto& item) {
        return std::string(item);
    };
    return describe(value);
}
```

终端输出 终端输出 generic-lambda-out

```
int=7
text=hi
```

两个函数里各写了一个 `describe`，函数体都只有一句返回。`describe_int` 用 `int` 调用，实例化出 `operator()(const int&)`；`describe_text` 用 `std::string` 调用，实例化出 `operator()(const std::string&)`。**它们是同一个 lambda 写法的两个实例，不是同一个函数**；如果某个类型不支持函数体里的操作（例如 `std::to_string` 不接受的类型），编译在实例化时才报错。

`const auto&` 在这里让实参按只读引用传入，避免复制字符串。泛型 lambda 也可以写 `auto&`、`auto&&`，规则与函数模板的参数推导一致：`auto` 按值、`auto&` 只接左值、`auto&&` 是转发引用。**泛型 lambda 本质上是"少写一个模板**

头”的简写，理解函数模板的推导，就理解它。

泛型 lambda 还有一个常见用途：给标准算法提供比较器。`std::sort(v.begin(), v.end(), [](const auto& a, const auto& b) { return a.key < b.key; })` 对元素类型不敏感，算法内部也不用为每种元素写一个新函数。**代价是每换一种元素类型就多一份实例代码**，编译时间和二进制体积都会增加；这是模板化的常规取舍，不是 lambda 特有的问题。

15.6 `std::function`：把不同可调用对象装进同一个类型

lambda 的类型各不相同，函数指针、函数对象也各有各的类型。如果接口需要“存一个可调用对象，稍后调用”，就需要类型擦除。`std::function<R(Args...)>` 就是这样一种容器：它可以装函数指针、lambda、函数对象，只要调用签名匹配。

cpp **function_wrapper: 接口只认签名** examples/ch06/function_wrapper/include/function_wrapper.h 第 7–7 行

```cpp
```

cpp **function_wrapper: lambda 与函数对象都能传进去** examples/ch06/function_wrapper/src/main.cpp 第 6–14 行

```cpp
int main() {
    auto add_one = [](int value) { return value + 1; };
    std::cout << "lambda=" << apply_twice(add_one, 3) << "\n";

    struct Doubler {
        int operator()(int value) const { return value * 2; }
    };
    std::cout << "functor=" << apply_twice(Doubler{}, 3) << "\n";
    return 0;
```

终端输出 终端输出 function-wrapper-out

```
lambda=5
functor=12
```

`apply_twice` 把传入的函数调用两次：`add_one` 先算 3+1=4，再算 4+1=5；`Doubler` 先算 3×2=6，再算 6×2=12。`std::function` **不关心你传的是 lambda 还是类**，它只检查“能不能用 `int` 调用、返回能不能转成 `int`”。这正是类型擦除的价值：接口只需要写一次签名，调用方可以用任何满足签名的东西。

cpp **function_wrapper: apply_twice 的实现** examples/ch06/function_wrapper/src/function_wrapper.cpp 第 3–5 行

```cpp
int apply_twice(const std::function<int(int)>& function, int value) {
    return function(function(value));
}
```

`function(function(value))` 先调一次得到中间值，再用中间值调一次。`std::function` 的 `operator()` 会转去调用它内部保存的目标；如果 `std::function` 是空的，调用会抛 `std::bad_function_call`。这个例子里传的都是有目标的对象，所以不会走到那条路径。

代价也要说清楚：`std::function` **通常要做间接调用，可能还要堆分配**（小对象可能放在内部缓冲区，是否放、放多大由实现决定）；拷贝 `std::function` 可能拷贝它保存的闭包。作为对比，模板参数 `template<class F> int apply_twice(F f, int)` 可以完全内联，没有类型擦除的开销，但会让每个 `F` 生成一份 `apply_twice`，而且 `F` 的类型必须写在头文件里。**要“统一类型、运行期替换”用 `std::function`；要“零开销、编译期确定”用模板**，这是工程上最常见的分界。

15.7 `std::bind`：老办法，仍能读懂

在 lambda 普及之前，绑定参数常用 `std::bind`。它接收一个可调用对象和一串参数，返回一个新的可调用对象；参数里的 `std::placeholders::_1`、`_2` 表示“调用时再填”。

cpp **bind_old: 把二元函数绑定成只差一个参数** examples/ch06/bind_old/src/bind_old.cpp 第 9–12 行

```cpp
int bind_demo() {
    auto add_ten = std::bind(add, 10, std::placeholders::_1);
```

```
    return add_ten(5);
}
```

cpp **bind_old: lambda 与 bind 得到同样结果** examples/ch06/bind_old/src/main.cpp 第 8-11 行

```cpp
int main() {
    auto lambda = [](int value) { return value + 10; };
    std::cout << "lambda=" << lambda(5) << " bind=" << bind_demo() << "\n";
    return 0;
```

终端输出 终端输出 bind-old-out

```
lambda=15 bind=15
```

`std::bind(add, 10, _1)` 把 `add` 的第一个参数固定为 10，第二个参数留到调用时填；`add_ten(5)` 等价于 `add(10, 5)`，结果是 15。lambda 版本直接写 `value + 10`，读起来更直白。

`std::bind` 的规则比 lambda 多：它默认按值保存绑定参数，要传引用得用 `std::ref` / `std::cref`；嵌套 bind、成员函数指针、重载函数都要额外处理；占位符的编号也容易写错。**C++14 之后，绝大多数 `std::bind` 都能用 lambda 更清楚地表达**，尤其是初始化捕获出现以后。读旧代码要认识 `std::bind`，写新代码优先 lambda；C++20 还提供了 `std::bind_front` 处理"只固定前几个参数"的常见情形。

编译器视角 · **编译器视角：`std::bind` 的返回类型很长**

`std::bind` 的返回类型是一个由实现定义的类型，通常带有 `_Bind`、`_Placeholder` 之类的模板细节。它不能直接写成变量类型，所以老代码里常见 `auto` 或 `std::function` 接住。lambda 的闭包类型同样没有名字，但它的"接口"就是函数体，读代码时不需要穿过 bind 的模板去看参数怎么绑定。**这也是"可读性"在类型层面的体现**：同样的 15，lambda 版本一眼能看懂，bind 版本要先在脑子里展开参数映射。

15.8 本机实测：六个读数

实验记录 · **本机实测：捕获、`mutable`、泛型与类型擦除**

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `lambda_capture` 的 stdout 是 `value_capture=10 ref_capture=20`：按值捕获停在 10，按引用捕获看到改后的 20；
- `capture_ref_dangle` 的 stdout 是 `safe=7 borrowed=7`：闭包在 `local` 存活期内调用，两种捕获都读到 7；逃出作用域会悬垂，本单元不把它写成"能跑"的例子；
- `mutable_counter` 的 stdout 是 `first=1 second=2 outer=0`：`mutable` 改的是闭包成员，外面不变；
- `generic_lambda` 的 stdout 是 `int=7` / `text=hi`：同一写法实例化出 `int` 与 `string` 两份 `operator()`；
- `function_wrapper` 的 stdout 是 `lambda=5` / `functor=12`：`std::function` 同时接住 lambda 与函数对象；
- `bind_old` 的 stdout 是 `lambda=15 bind=15`：两种绑定方式结果一致，lambda 更短。

15.9 下一单元从哪继续

下一单元进入异常处理：函数怎么把"失败"报告给调用者，`try` / `catch` 怎么接，栈展开时局部对象怎么按 RAII 销毁，`noexcept` 又承诺了什么。这一单元的 lambda 会继续出现——标准库的谓词、回调、错误处理回调都常写成 lambda；而 `std::function` 的拷贝与调用开销，也会在异常安全的讨论里再次遇到。

小结 · **这一节要带走的四件事**

- lambda 生成独有的无名闭包类；按值捕获在创建时复制，按引用捕获只是借看，闭包活过对象就悬垂。
- `mutable` 去掉 `operator()` 的 `const`，改的是按值捕获的成员；泛型 lambda 的 `operator()` 是函数模板，每个实参类型实例化一份。
- `std::function` 用类型擦除统一签名，灵活但有间接调用/可能的堆分配；能编译期确定时优先模板。
- `std::bind` 能读懂即可；新代码用 lambda（尤其初始化捕获）表达更清楚。

自测题 · 自测题（先自己做，再对答案）

1. `int x = 1; auto f = [x]() { return x; }; x = 2;` 调用 `f()` 返回几？如果把 `[x]` 改成 `[&x]` 呢？
2. 为什么返回 `[&local]` 的 lambda 是危险的？按值捕获为什么能避免？
3. `mutable` 的作用是什么？它会让按值捕获变成按引用捕获吗？
4. 泛型 lambda `[](const auto& v) { ... }` 对 `int` 和 `std::string` 各生成什么？
5. `std::function<int(int)>` 能装下哪些东西？它相比模板参数有什么代价？

注 · 自测答案

1. 返回 1：按值捕获复制的是创建时的 1；改成 `[&x]` 后返回 2，因为引用读到的是改后的值。
2. `local` 在函数返回时销毁，闭包里的引用指向已结束生命周期的存储，再调用是未定义行为；按值捕获让闭包自己存一份副本，寿命随闭包。
3. `mutable` 去掉闭包 `operator()` 的 `const`，允许修改按值捕获的成员；它不会把副本变成引用，外面的变量仍然不变。
4. 对 `int` 实例化 `operator()(const int&)`，对 `std::string` 实例化 `operator()(const std::string&)`；函数体里的操作必须对每个类型都合法。
5. 能装函数指针、lambda、函数对象等所有满足 `int(int)` 调用的目标；代价是类型擦除后的间接调用、可能的堆分配，以及拷贝时可能复制闭包。

第 7 章

错误处理与资源

本章 2 个单元 · 页眉从这里之后的页面开始出现

1. 异常、noexcept 与异常安全 零基础 · 第 16 单元 · 真编译真运行
2. optional、variant 与 expected 零基础 · 第 17 单元 · 真编译真运行

异常、noexcept 与异常安全

函数发现"这件事做不成"时，有两种报告方式：返回一个错误码，或者抛出异常。错误码要求调用者每次都检查，漏一次就可能拿着无效结果继续算；异常把"成功路径"和"失败路径"分开，错误会沿着调用栈向上找 `catch`，中间的函数不必层层转交。代价是：控制流会突然跳走，路上必须把已经构造的对象清理干净。

这一单元讲三件事：异常怎么抛、怎么接，栈展开时析构函数做了什么；`noexcept` 承诺了什么，为什么它会影响 `vector` 扩容时选移动还是拷贝；以及"异常安全"的三个等级，怎么用 copy-and-swap 写出强保证。

◎ 学习目标 · 看完这一单元，你应该能

- 写 `throw` / `try` / `catch`，并说出异常对象按值抛出、按引用捕获的理由；
- 画出栈展开路径，解释局部对象按什么顺序析构；
- 说明析构函数默认为什么是 `noexcept`，以及析构中抛异常为什么危险；
- 用 `noexcept` 标记移动构造，并解释 `vector` 扩容为什么偏爱不抛的移动；
- 区分基本保证、强保证与不抛保证，用 copy-and-swap 实现强保证；
- 判断什么时候用异常、什么时候用错误码或 `expected`。

16.1 先看工程长什么样

这一单元在 `examples/ch07/` 下建六个例子，仍然保持"两个 `.cpp` + 一个 `.h`"的多文件结构：异常类型与辅助函数放头文件/实现文件，`main.cpp` 负责触发与打印。`dtor_throw/` 是反例，它故意在析构函数里抛异常，靠编译器的 `-Werror=terminate` 在编译期拦下。

目录树 **工程：examples/ch07/ 这一单元新增的六个例子**

第 1 步 先看抛出、栈展开与按引用捕获

```
examples/
└── ch07/
    ├── throw_basic/ ← 本课新增
    │   ├── include/ ← 本课新增
    │   │   └── throw_basic.h ← 本课新增
    │   ├── src/ ← 本课新增
    │   │   ├── main.cpp ← 本课新增
    │   │   └── throw_basic.cpp ← 本课新增
    │   └── throw_basic.json ← 本课新增
    ├── stack_unwind/ ← 本课新增
    └── catch_by_ref/ ← 本课新增
```

第 2 步 加上 noexcept 移动与析构反例

```
examples/
└── ch07/
    ├── 前三个例子 ： 同上
    ├── noexcept_move/ ← 本课新增
    └── dtor_throw/ ← 本课新增
```

第 3 步 最后加上强保证

```
examples/
└── ch07/
    ├── 前五个例子 ： 同上
    └── strong_guarantee/ ← 本课新增
```

第 1 帧的 `throw_basic/` 演示最小抛出与捕获，`stack_unwind/` 用带打印的 `Tracer` 看析构顺序，`catch_by_ref/` 对照按值与按引用捕获。第 2 帧的 `noexcept_move/` 数 `vector` 扩容时的移动/拷贝次数，`dtor_throw/` 是编译期反例。第 3 帧的 `strong_guarantee/` 用 copy-and-swap 对照"先改再抛"的基本保证。

16.2 抛出与捕获：错误沿调用栈向上找

最小用法只有三块：可能失败的函数 `throw` 一个异常对象；调用方用 `try` 包住可能抛的语句；`catch` 按类型接住。异常一旦抛出，当前函数剩余语句不再执行，控制流开始向上找匹配的 `catch`。

cpp **throw_basic：正常路径与异常路径** examples/ch07/throw_basic/src/main.cpp 第 6–15 行

```cpp
int main() {
    try {
        std::cout << "result=" << checked_div(12, 3) << "\n";
        int bad = checked_div(1, 0);
        std::cout << "result=" << bad << "\n";
    } catch (const std::invalid_argument& error) {
        std::cout << "caught=" << error.what() << "\n";
    }
    return 0;
}
```

终端输出 终端输出 throw-basic-out

```
result=4
caught=division by zero
```

`checked_div(12, 3)` 返回 4，打印 `result=4`；第二次调用分母为 0，`checked_div` 在除法之前 `throw std::invalid_argument("division by zero")`，`int bad = ...` 这条语句没有完成，后面的 `std::cout << "result="` 也不会执行；控制流跳到 `catch`，打印异常对象的 `what()`。**注意 `throw` 之后函数里剩下的代码不会运行**，哪怕它只是 `return`。

`std::invalid_argument` 的构造函数接收一个字符串，`what()` 返回它。抛出的异常对象在标准里叫"异常对象"，它是一份**按值保存的副本**：`throw` 表达式里的对象会被复制（或移动）到一块由运行时管理的存储，`catch` 拿到的是这块存储的引用。**按值抛出、按引用捕获**是标准库和工程代码的常规组合：按值抛出保证异常对象活过抛出函数的作用域，按引用捕获避免再复制、也避免切片。

{} 语法规则 · try / catch 的基本形状

形状是 `try { ... } catch (const SomeError& error) { ... }`：`try` 包住可能抛的语句，`catch` 的参数写异常类型。`catch` 的顺序要从"最具体"到"最一般"：`catch (const std::exception&)` 能接住所有派生自 `std::exception` 的异常，如果把它写在前面，后面的具体类型就永远轮不到。**catch 只按类型匹配，不看值**；同一个 `try` 里可以有多个 `catch`，第一个匹配的胜出。

`throw;`（不带表达式）只能在 `catch` 里用，表示"把当前异常继续往上抛"。如果 `catch` 里不想处理、只想记录一下，可以写 `throw;` 而不是 `throw error;`；后者会再复制一份，还可能因为静态类型变化而切片。**重新抛出用 `throw;`，这是保持异常对象原样的方式。**

`catch (...)` 能接住任何异常，但它拿不到异常对象，通常只用于"必须保证不泄漏"的边界（例如线程入口、回调边界）里做清理并重新抛出。**不要用 `catch (...)` 吞掉异常**：程序看起来继续跑了，实际问题被藏起来，后面的状态可能已经不可信。

16.3 栈展开：离开的每一层都要清理

异常从抛出点向上传播时，会依次退出沿途的函数。每退出一层，这一层已经构造完成的自动对象都要按**构造的逆序**析构。标准把这个过程叫"栈展开"（stack unwinding）。它不是"跳转"，而是一层一层地退，析构函数该跑的都跑。

cpp **stack_unwind：抛出前构造两个 Tracer** examples/ch07/stack_unwind/src/stack_unwind.cpp 第 14–18 行

```cpp
void run_with_throw() {
    Tracer outer("outer");
    Tracer inner("inner");
    throw std::runtime_error("boom");
}
```

cpp **stack_unwind：catch 之后程序继续** examples/ch07/stack_unwind/src/main.cpp 第 6–14 行

```cpp
int main() {
```

```
    try {
        run_with_throw();
    } catch (const std::runtime_error& error) {
        std::cout << "caught=" << error.what() << "\n";
    }
    std::cout << "done\n";
    return 0;
}
```

终端输出 终端输出 stack-unwind-out

```
ctor=outer
ctor=inner
dtor=inner
dtor=outer
caught=boom
done
```

`outer` 先构造，`inner` 后构造；抛出后 `inner` 先析构，`outer` 后析构，正好与构造顺序相反。两个析构都发生在 `catch` 之前，然后才打印 `caught=boom`，最后 `main` 继续执行到 `done`。**异常不是"程序结束"**；只要有人接住，程序就从 `catch` 之后继续。

调用树 **栈展开：从抛出点到 catch**

第 1 步 调用栈上有 main、run_with_throw 与两个局部对象

```
run_with_throw() : 构造 outer，再构造 inner
main() → run_with_throw() 普通调用
```

第 2 步 throw 之后逐层退出

```
throw std::runtime_error : 抛出点
inner : 析构，打印 dtor=inner
outer : 析构，打印 dtor=outer
run_with_throw() ⇒ main() 异常传到 catch
```

第 3 步 catch 接手，main 继续

```
main() : catch 打印 caught=boom
main() : 打印 done，正常返回
```

第 2 帧的析构顺序是栈展开的核心：**后构造的先析构，先构造的后析构。**如果 `run_with_throw` 里还开了文件、锁、堆内存，这些资源的释放都写在析构函数里，栈展开就会自动完成清理——这就是 RAII 与异常能配合的原因。反过来说，如果资源是裸 `new` 出来的、析构函数又没管，栈展开不会替你 `delete`，泄漏就发生了。

标准原文 · 标准原文：栈展开

> As control passes from the point where an exception is thrown to a handler, objects are destroyed by a process, specified in this subclause, called stack unwinding.
>
> Each object with automatic storage duration is destroyed if it has been constructed, but not yet destroyed, since the try block was entered. ... The objects are destroyed in the reverse order of the completion of their construction.

大意：异常从抛出点走向处理器的过程中，自 `try` 块进入以来构造完成、尚未销毁的自动对象都会被销毁，顺序与构造完成的顺序相反。

出处：N4950 §14.3「Constructors and destructors」（[except.ctor]）。

"构造完成"很关键：如果对象的构造函数自己抛了，这个对象不算构造完成，它的析构不会运行；但它的基类与已完成的成员子对象会按规则清理。这也是为什么构造函数里分配资源后，后续步骤再抛异常时，要用 RAII 成员而不是裸指针。

16.4 按引用捕获：避免切片，也避免复制

`catch` 的参数写 `Error` 还是 `const Error&`，差别不只是"少复制一次"。如果抛出的实际类型是 `AppError`（`Error` 的派生类），`catch (Error error)` 会把异常对象**切片**成 `Error` 子对象，派生类部分丢失，虚函数也按 `Error` 的版本调用。

cpp **catch_by_ref：同一个异常，两种捕获方式** examples/ch07/catch_by_ref/src/main.cpp 第 6–19 行

```cpp
    try {
        throw_app();
    } catch (Error error) {
        std::cout << "by_value=" << error.code() << "\n";
    }

    try {
        throw_app();
    } catch (const Error& error) {
        std::cout << "by_ref=" << error.code()
                  << " what=" << error.what() << "\n";
    }
    return 0;
}
```

终端输出 终端输出 catch-by-ref-out

```
by_value=0
by_ref=7 what=app failed
```

`throw_app()` 抛的是 `AppError`，它的 `code()` 覆盖为 7。第一次 `catch (Error error)` 按值接住，异常对象被复制成 `Error`，`error.code()` 调用的是 `Error::code()`，返回 0；第二次 `catch (const Error& error)` 按引用接住，动态类型仍是 `AppError`，`code()` 返回 7，`what()` 返回 `"app failed"`。**切片不只发生在函数传参，也发生在按值捕获异常**；这是"按引用捕获"最硬的理由。

按引用捕获还避免了复制异常对象。异常对象可能很大，复制可能再次抛异常（虽然少见），也可能有副作用；`const &` 既只读又不复制，是默认选择。只有当你确实需要一份独立副本、并且不关心动态类型时，才按值捕获——这种场景很少。

> ⚠ **陷阱** · **catch 顺序写反，具体异常永远接不到**
>
> 如果 `try` 里抛出 `AppError`，而先写 `catch (const Error& error)`、后写 `catch (const AppError& error)`，第一个处理器就把所有 `Error` 及其派生类接走了，后面的 `AppError` 处理器永远到不了，有的编译器会给 unreachable 警告。`catch` 按书写顺序匹配，第一个能接住的胜出。**从派生到基类排列**：先 `AppError`，再 `Error`，最后 `std::exception`。反过来写不会编译失败，但具体类型的处理器变成死代码，行为与你的预期不同。

16.5 `noexcept`：承诺不抛，影响容器选择

`noexcept` 是函数接口的一部分：它承诺"这个函数不会让异常逃出去"。如果函数真的抛了，程序不会正常传播异常，而是调用 `std::terminate` 直接结束。**它不是优化提示，而是行为契约**；写上去就要保证做到。

`noexcept` 最重要的实际影响之一是移动构造。`std::vector` 扩容时要把旧元素搬到新缓冲区；如果移动构造可能抛异常，搬到一半失败就会留下"旧缓冲区有一部分已被移走、新缓冲区还没建好"的烂摊子，连基本保证都难维持。所以标准库用 `std::move_if_noexcept` 做选择：**移动构造是 `noexcept` 就移动，否则如果有拷贝构造就拷贝。**

cpp **noexcept_move：一个 noexcept 移动，一个可能抛的移动** examples/ch07/noexcept_move/src/noexcept_move.cpp 第 15–30 行

```cpp
SafeMove::SafeMove(SafeMove&& other) noexcept : value(other.value) {
    ++moves;
}

int UnsafeMove::moves = 0;
int UnsafeMove::copies = 0;

UnsafeMove::UnsafeMove(int value) : value(value) {}
```

```
23
24  UnsafeMove::UnsafeMove(const UnsafeMove& other) : value(other.value) {
25      ++copies;
26  }
27
28  UnsafeMove::UnsafeMove(UnsafeMove&& other) noexcept(false) : value(other.value) {
29      ++moves;
30  }
```

cpp **noexcept_move：同样装四个元素，统计搬运方式** examples/ch07/noexcept_move/src/noexcept_move.cpp 第 32–49 行

```
32  void build_vectors() {
33      std::vector<SafeMove> safe;
34      for (int i = 0; i < 4; ++i) {
35          safe.emplace_back(i);
36      }
37
38      std::vector<UnsafeMove> unsafe;
39      for (int i = 0; i < 4; ++i) {
40          unsafe.emplace_back(i);
41      }
42
43      std::cout << "safe_moves=" << SafeMove::moves
44                << " safe_copies=" << SafeMove::copies << "\n";
45      std::cout << "unsafe_moves=" << UnsafeMove::moves
46                << " unsafe_copies=" << UnsafeMove::copies << "\n";
47      std::cout << "capacities=" << safe.capacity() << ","
48                << unsafe.capacity() << "\n";
49  }
```

终端输出 终端输出 noexcept-move-out

```
safe_moves=3 safe_copies=0
unsafe_moves=0 unsafe_copies=3
capacities=4,4
```

两个 `vector` 都装了 4 个元素，都发生了两次扩容（1→2、2→4），一共搬运 1+2=3 个旧元素。`SafeMove` 的移动构造标了 `noexcept`，3 次全是移动；`UnsafeMove` 的移动构造写的是 `noexcept(false)`，`move_if_noexcept` 判断"移动可能抛、拷贝可用"，于是 3 次全走拷贝。**`noexcept` 在这里直接决定了性能路径**：移动通常只是搬几个指针/整数，拷贝可能要分配内存、复制全部数据。

这里的 `capacities=4,4` 与"两次扩容"是 libstdc++ 的本机实测结果，不是标准规定。标准只要求 `vector` 的 `push_back` / `emplace_back` 均摊常数时间，具体扩容倍数由实现决定；换一个标准库，容量序列可能不同。**但"移动是否 `noexcept` 会影响扩容选移动还是拷贝"是标准库算法的要求，换实现也成立。**



第 2 帧的移动路径快，但如果移动到一半抛异常，`vector` 很难恢复；第 3 帧的拷贝路径慢，可一旦某个拷贝失败，旧缓冲区还完整，容器仍能保持基本保证。**标准库选的是"先保证正确，再谈快"**；你的移动构造如果确实不会抛，就应当标 `noexcept`，把快路径打开。

写 `noexcept` 要诚实：移动构造通常只搬指针/整数，标 `noexcept` 没问题；如果移动里要分配内存、调用可能抛的函数，就不能标。**标了却抛，程序直接 `terminate`，比不标更糟。**析构函数默认就是 `noexcept`，原因相同：栈展开时析

构再抛，没法同时处理两个异常。

16.6 析构函数为什么不能抛

标准规定，析构函数默认带 `noexcept(true)`（除非基类或成员的析构可能抛，隐式声明才会变成可能抛）。如果析构函数里真的让异常逃出去，`std::terminate` 会被调用。更危险的是"栈展开过程中析构又抛"：运行时正在处理一个异常，又来了第二个，C++ 没有"异常嵌套传播"的机制，只能终止。

cpp **dtor_throw：析构里抛异常，编译器直接拦下** examples/ch07/dtor_throw/src/dtor_throw.cpp 第 5-7 行

```cpp
Bad::~Bad() {
    throw std::runtime_error("dtor boom");
}
```

编译诊断 编译诊断 dtor-throw-diag

```
src/dtor_throw.cpp: In destructor 'Bad::~Bad()':
src/dtor_throw.cpp:6:5: error: 'throw' will always call 'terminate' [-Werror=terminate]
    6 |     throw std::runtime_error("dtor boom");
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
src/dtor_throw.cpp:6:5: note: in C++11 destructors default to 'noexcept'
cc1plus.exe: some warnings being treated as errors
```

这个例子在清单里加了 `-Werror=terminate`，所以 GCC 的 `-Wterminate` 警告变成错误，编译失败。诊断会指出"`throw` 总会调用 `terminate`"，并提醒"C++11 起析构函数默认是 `noexcept`"。即使不加这个开关，代码也"能编译"，但一运行就会终止；**编译期拦下比运行期崩溃好**，这也说明编译器能识别这类必然终止的写法。

编译器视角 · 本机诊断：`-Wterminate` 抓的是"必然终止"

GCC 的 `-Wterminate` 不需要运行就能看出：在一个 `noexcept` 函数（包括默认 `noexcept` 的析构）里写 `throw`，异常一定逃不出去，必然走到 `std::terminate`。它给的是警告而不是错误，因为标准没有禁止这种写法——标准只是规定运行时会发生什么。工程上通常把这类警告升级为错误（`-Werror=terminate`），让问题停在编译期。

如果确实需要"析构里的清理动作可能失败"，正确做法不是让异常逃出去，而是在析构内部捕获并记录/忽略，或者提供一个显式的 `close()` / `commit()` 让调用者在对象销毁前处理失败。**析构函数的职责是释放资源，不是报告业务错误**；把可能失败的逻辑放到显式接口里，是 RAII 设计的常见边界。

`noexcept(false)` 的析构函数可以写，但只在极特殊场景（例如析构需要调用可能抛的第三方接口，且你能保证不在栈展开中销毁）才考虑；即便写了，也要在内部 `try` / `catch` 兜住。**默认让析构 `noexcept`，把失败前移到显式接口**，这是更安全的工程惯例。

16.7 异常安全：基本、强、不抛

"异常安全"问的不是"会不会抛"，而是"抛了之后对象还能不能用"。工程上常分三档：

1. **基本保证**：异常后对象仍处于有效状态，可以继续使用或安全销毁，但具体内容可能已改变；
2. **强保证**：操作要么完全成功，要么回到操作前的状态，像事务一样；
3. **不抛保证**：操作承诺不抛异常（通常标 `noexcept`）。

`vector::push_back` 通常提供强保证（扩容失败时原容器不变），`swap` 通常提供不抛保证。自己写的多步修改，如果中途可能抛，就要想清楚停在哪一档。

cpp **strong_guarantee：先改再抛，还是先做完再换** examples/ch07/strong_guarantee/src/strong_guarantee.cpp 第 5-19 行

```cpp
void Buffer::append_strong(int value, bool fail) {
    std::vector<int> next = data_;
    next.push_back(value);
    if (fail) {
        throw std::runtime_error("strong failed");
    }
    data_.swap(next);
}

```

```cpp
void Buffer::append_basic(int value, bool fail) {
    data_.push_back(value);
    if (fail) {
        throw std::runtime_error("basic failed");
    }
}
```

cpp **strong_guarantee：同一个失败，两种残留** examples/ch07/strong_guarantee/src/main.cpp 第 6–26 行

```cpp
int main() {
    Buffer buffer;

    try {
        buffer.append_strong(1, true);
    } catch (const std::runtime_error&) {
        std::cout << "strong_caught\n";
    }
    std::cout << "strong_size=" << buffer.size() << "\n";

    try {
        buffer.append_basic(1, true);
    } catch (const std::runtime_error&) {
        std::cout << "basic_caught\n";
    }
    std::cout << "basic_size=" << buffer.size() << "\n";

    buffer.append_strong(2, false);
    std::cout << "final_size=" << buffer.size() << "\n";
    return 0;
}
```

终端输出 终端输出 strong-guarantee-out

```
strong_caught
strong_size=0
basic_caught
basic_size=1
final_size=2
```

`append_strong` 先把 `data_` 复制到 `next`，往 `next` 追加，失败就抛；此时 `data_` 完全没动，所以 `strong_size=0`。`append_basic` 直接往 `data_` 追加，再抛；异常后 `data_` 里已经多了一个 1，所以 `basic_size=1`。两者都"没有崩溃、对象仍可析构"，但强保证把"失败"变成了"什么都没发生"。

内存形态 **copy-and-swap：先在新副本上做完，再一次换过来**

第 1 步 **复制原数据**

`data_ : [ ]` `next : [ ] (data_ 的副本)`

第 2 步 **在副本上追加，然后失败**

`next : [1]` `data_ : [ ] (完全没动)` `throw : append_strong 抛出`

第 3 步 **成功路径才交换**

`data_ : [1, 2]` `next : [ ] (换走旧内容，析构时释放)`

第 2 帧的 `data_` 没有被碰过，所以强保证成立；第 3 帧的 `swap` 只交换内部指针，通常不抛，于是"交换"这一步不会破坏保证。**copy-and-swap 的代价是每次操作都复制整个容器**，适合"写少、要求强保证"的场景；如果数据很大、失败又很常见，就要权衡性能，可能退而求其次使用基本保证，或在操作前先做检查。

`swap` 本身是否 `noexcept` 取决于容器与分配器；标准库容器在同分配器下通常提供不抛的 `swap`，这是 copy-and-swap 能收尾的前提。**如果你的类型没有不抛的 `swap`，最后一步也可能失败**，强保证就不完整；实现时要检查这一点。



选择哪一档，取决于接口承诺与调用方需求：资源管理类的移动/交换操作应尽量做到不抛；修改多个字段的业务操作，如果调用方需要"重试"，强保证更有用；日志、清理这类"尽力而为"的操作，基本保证加上明确的返回值就够了。

16.8 异常还是错误码

异常适合"真正的异常"：文件打不开、内存耗尽、网络断开、参数违反前置条件。它们不常发生，发生后当前函数通常无法继续，必须由上层决定怎么办。异常的优势是不会被忽略——调用者不写 `catch`，异常就一路向上，直到有人处理或程序终止。

错误码适合"预期内的失败"：解析到文件末尾、查找键不存在、用户输入不合法。这些失败是正常流程的一部分，调用者通常就在附近处理。`std::optional`、`std::expected` 与错误码把失败放在返回值里，调用者必须显式检查，控制流是直的，没有隐藏跳转。

不要用异常做普通控制流：在一个循环里用 `throw` 表示"找到结尾"，会让读代码的人很难看出循环的正常出口，也会让性能不可预测。反过来，也不要用错误码掩盖真正的异常：如果内存分配失败、 invariant 被破坏，继续返回一个"空值"让上层猜，问题会更难查。

C++ 标准库自己两种都用：`vector::at` 越界抛 `std::out_of_range`，`map::find` 用 `end()` 表示没找到；`std::stoi` 抛异常，`std::from_chars` 用错误码。**接口设计时先问"这个失败常见吗、调用方能在本地处理吗"**，答案决定用哪种报告方式；下一单元的 `optional` / `variant` / `expected` 会把"不用异常的错误处理"展开。

16.9 本机实测：六个读数

实验记录 · 本机实测：抛出、展开、捕获与保证

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `throw_basic` 的 stdout 是 `result=4` / `caught=division by zero`：抛出后剩余语句不执行，`catch` 接住；
- `stack_unwind` 的 stdout 依次是 `ctor=outer`、`ctor=inner`、`dtor=inner`、`dtor=outer`、`caught=boom`、`done`：析构按构造逆序；
- `catch_by_ref` 的 stdout 是 `by_value=0` / `by_ref=7 what=app failed`：按值捕获切片，按引用保留动态类型；
- `noexcept_move` 的 stdout 是 `safe_moves=3 safe_copies=0` / `unsafe_moves=0 unsafe_copies=3` / `capacities=4,4`：`noexcept` 移动走移动，可能抛的移动退回复制；
- `dtor_throw` 在 `-Werror=terminate` 下编译失败，诊断指出 `throw` 必然调用 `terminate`；
- `strong_guarantee` 的 stdout 是 `strong_caught` / `strong_size=0` / `basic_caught` / `basic_size=1` / `final_size=2`：强保证回到起点，基本保证留下部分修改。

16.10 下一单元从哪继续

下一单元换一种报告失败的方式：`std::optional` 表示"可能有值、可能没有"，`std::variant` 表示"几种类型中的一种"，`std::expected` 表示"成功值或错误值"。它们把失败放进返回类型，让调用者在类型系统里看到"这里可能没有结果"，而不是靠文档或约定。异常仍然会在真正异常的场景使用；两者不是替代关系，而是分工。

小结 · 这一节要带走的四件事

- 异常按值抛出、按引用捕获；`catch` 从具体到一般排列，重新抛出用 `throw;`。
- 栈展开按构造逆序销毁局部对象，RAII 让清理自动发生；构造函数自己抛时，该对象不算构造完成。
- 析构默认 `noexcept`，异常逃出会 `terminate`；`noexcept` 移动会让 `vector` 扩容选择移动而不是拷贝。
- 异常安全分基本/强/不抛；copy-and-swap 用"副本上做完再交换"实现强保证，代价是复制。

自测题 · 自测题（先自己做，再对答案）

1. 抛出 `AppError` 后，`catch (Error error)` 与 `catch (const Error& error)` 调用虚函数 `code()` 的结果为什么不同？
2. 栈展开时，局部对象按什么顺序析构？构造函数抛异常时，这个对象自己的析构会运行吗？
3. 为什么析构函数默认是 `noexcept`？如果析构里真的抛了，会发生什么？
4. `noexcept` 移动构造如何影响 `std::vector` 扩容？`UnsafeMove` 为什么走了拷贝？
5. 强保证与基本保证的差别是什么？copy-and-swap 为什么能提供强保证，代价是什么？

注 · 自测答案

1. 按值捕获会把 `AppError` 切片成 `Error` 子对象，虚调用落到 `Error::code()`，返回 0；按引用捕获保留动态类型，调用 `AppError::code()`，返回 7。
2. 按构造完成的逆序析构，后构造的先析构；构造函数自己抛异常时，该对象不算构造完成，它自己的析构不运行，但已完成的基类/成员子对象会按规则清理。
3. 因为栈展开时不能再抛第二个异常，析构默认承诺不抛；异常真的逃出析构会调用 `std::terminate`，程序直接终止。
4. `vector` 扩容用 `move_if_noexcept`：移动构造 `noexcept` 就移动，否则有拷贝构造就拷贝。`UnsafeMove` 的移动是 `noexcept(false)`，所以扩容选择拷贝，`copies=3`。
5. 基本保证只要求对象仍有效，内容可能已改变；强保证要求失败后回到操作前。copy-and-swap 先在副本上完成修改，成功后才 `swap`，失败时原对象未动；代价是每次都要复制整个容器。

optional、variant 与 expected

上一单元的异常适合"函数没法继续"的失败。但有些情况不是异常：查字典可能没有这个键，解析可能失败，一个格子可能暂时没有值。这些"没有"是正常结果，不该用异常来表达，也不该用 `-1`、空字符串、`nullptr` 这类哨兵值硬凑。C++17 的 `std::optional`、`std::variant` 与 C++23 的 `std::expected` 把"可能没有""几种之一""值或错误"写进类型，让编译器提醒调用者处理。

这一单元用六个例子拆开这三种类型：先看 `optional` 怎么表示"没有值"，再用一个编译错误说明它不能直接当 `int` 用；然后看 `variant` 的 `index()` 与 `std::visit`，用 `overloaded` 把多个分支写成一组重载；最后用 `expected` 表示"值或错误"，并用 `and_then` 把多步操作串起来。

学习目标 · 看完这一单元，你应该能

- 用 `optional<T>` 表示"可能有值、可能没有"，并正确选择 `has_value` / `value_or` / `*`；
- 说出 `optional` 与"错误"的边界，不把错误信息塞进一个空 `optional`；
- 用 `variant` 存"几种类型之一"，读 `index()`，用 `std::visit` 分派；
- 用 `overloaded` 辅助器把多个 lambda 合成一个访问器，并知道漏掉分支会怎样；
- 用 `expected<T, E>` 表示"值或错误"，用 `and_then` / `transform` 串联；
- 根据"没有值 / 出错 / 多选一"选择正确的类型。

17.1 先看工程长什么样

这一单元在 `examples/ch07/` 下新增六个例子。`optional_basic/`、`variant_visit/`、`expected_basic/` 各展示一种类型的最小用法；`optional_misuse/` 是编译期反例；`variant_overload/` 展示 `overloaded` 访问器；`expected_chain/` 展示多步错误传播。结构仍是"两个 `.cpp` + 一个 `.h`"。

目录树 **工程：examples/ch07/ 这一单元新增的六个例子**

第 1 步 先看 optional 与 variant 的基本用法

```
examples/
└── ch07/
    ├── optional_basic/ ← 本课新增
    │   ├── include/ ← 本课新增
    │   │   └── optional_basic.h ← 本课新增
    │   ├── src/ ← 本课新增
    │   │   ├── main.cpp ← 本课新增
    │   │   └── optional_basic.cpp ← 本课新增
    │   └── optional_basic.json ← 本课新增
    ├── optional_misuse/ ← 本课新增
    └── variant_visit/ ← 本课新增
```

第 2 步 加上 overloaded 与 expected

```
examples/
└── ch07/
    ├── 前三个例子 ： 同上
    ├── variant_overload/ ← 本课新增
    └── expected_basic/ ← 本课新增
```

第 3 步 最后加上 expected 的链式调用

```
examples/
└── ch07/
    ├── 前五个例子 ： 同上
    └── expected_chain/ ← 本课新增
```

第 1 帧的 `optional_basic/` 用查分数演示“有/没有”，`optional_misuse/` 直接把 `optional<int>` 赋给 `int`，让编译器拦下；`variant_visit/` 用 `index()` 和 `visit` 处理三种类型。第 2 帧的 `variant_overload/` 用 `overloaded` 写分支，`expected_basic/` 用 `expected<int, string>` 表示成功或错误。第 3 帧的 `expected_chain/` 把解析、校验、计算串成一条链，错误自动短路。

17.2 `optional`：有值或没有值

`std::optional<T>` 要么包含一个 `T`，要么为空。它自己管理内部值的构造与析构，不需要堆分配；标准明确说，实现不得用动态内存来放这个值。**它表达的是“可能没有”，不是“出错了”**：没有值本身就是一个正常结果，调用者应该决定“没有”意味着什么。

cpp **optional_basic: 查到与查不到** examples/ch07/optional_basic/src/main.cpp 第 5–14 行

```cpp
int main() {
    auto found = find_score("Ada");
    auto missing = find_score("Bob");

    std::cout << "has=" << found.has_value()
              << " score=" << found.value_or(-1) << "\n";
    std::cout << "has=" << missing.has_value()
              << " fallback=" << missing.value_or(-1) << "\n";
    return 0;
}
```

终端输出 终端输出 optional-basic-out

```
has=1 score=90
has=0 fallback=-1
```

`find_score("Ada")` 返回包含 90 的 `optional`，`has_value()` 为真，`value_or(-1)` 返回 90；`find_score("Bob")` 返回 `std::nullopt`，`has_value()` 为假，`value_or(-1)` 返回备用值 -1。**`value_or` 不会改原对象**，它只是“有值就返回，没有就返回你给的默认值”。

访问内部值有几种方式，安全程度不同：

- `has_value()` / `operator bool`：先问有没有；

- `value()`：有值就返回，没有就抛 `std::bad_optional_access`；
- `value_or(default)`：没有值时返回默认值；
- `*it` / `it->`：不做检查，空的时候是未定义行为。

`value()` 比 `*` 安全，但也会抛异常；如果你的代码路径不允许异常，就先 `has_value()` 再解引用，或者用 `value_or`。`optional` 的 `operator*` 不检查，是因为它被设计成"你已经知道有值"时的零开销访问；把检查责任交给调用者，代价是调用者必须真的检查。

`optional` 不能直接存引用：`std::optional<T&>` 不是标准类型。需要"可能没有的引用"时，用 `T*`（空指针表示没有）、`std::reference_wrapper<T>` 的 `optional`，或者 `std::optional<std::reference_wrapper<T>>`。**这不是标准库的疏忽，而是"引用必须绑定到对象"的语义决定的**；引用一旦存在就必须指向某物，无法表示"没有"。需要可空引用时，指针更直接。

`optional` 也可以参与比较：两个 `optional` 都有值时比较内部值，空 `optional` 与有值对象比较时按"空小于有值"的规则；`optional<T> == std::nullopt` 用来判断空。**这些比较规则在泛型代码里很有用**，但不要用它们替代 `has_value()` 的可读性；明确写出"有没有值"比依赖隐式比较更清楚。

C++23 还给 `optional` 加了 monadic 操作：`and_then`、`transform`、`or_else`，语义与 `expected` 的对应操作类似——有值时继续，空时跳过。**如果你的标准库支持，可以用它们把"有值才处理"的链拉平**；不支持就手写 `if (opt)`，语义一样。不要把 monadic 操作当成"必须用的新风格"，可读性优先。

标准原文 · **标准原文：optional 的存储与状态**

> An optional object is an object that contains the storage for another object and manages the lifetime of this contained object, if any. The contained object may be initialized after the optional object has been initialized, and may be destroyed before the optional object has been destroyed. The initialization state of the contained object is tracked by the optional object.

大意：`optional` 对象包含另一对象的存储并管理它的生命周期；内部值可以在 `optional` 之后初始化、之前销毁，是否初始化由 `optional` 自己跟踪。

出处：N4950 §22.5.1「In general」（[optional.general]）。

`optional` 也可以按值返回大对象，但要注意复制/移动成本：`return std::optional<Big>{...}` 会把 `Big` 构造进 `optional` 的存储；如果返回 `optional<Big>` 的副本，`Big` 也要复制。C++17 的强制复制消除不覆盖 `optional` 内部，所以大对象场景要考虑移动或 `optional<Big>` 的原地构造（`emplace`）。**`optional` 省掉的是"堆分配 + 空指针"，不是所有复制。**

17.3 `optional` 的边界：别把错误塞进去

`optional` 只有"有/没有"两种状态，没有地方放错误原因。如果把"解析失败""文件打不开""权限不足"都压成一个空 `optional`，调用者只知道"没有"，不知道"为什么"，也没法决定是重试、提示用户还是换路径。**这是 `optional` 最常见的误用**：把"错误"降级成"缺失"。

cpp **optional_misuse: optional 不能直接当 int 用** examples/ch07/optional_misuse/src/main.cpp 第 3-6 行

```cpp
int main() {
    int value = maybe_score();
    return value;
}
```

编译诊断 编译诊断 optional-misuse-diag

```
src/main.cpp: In function 'int main()':
src/main.cpp:4:28: error: cannot convert 'std::optional<int>' to 'int' in initialization
    4 |     int value = maybe_score();
      |                 ~~~~~~~~~~~^~
      |                            |
      |                            std::optional<int>
```

这个编译错误是好事：`std::optional<int>` 没有到 `int` 的隐式转换，编译器不允许你把“可能没有”悄悄当成“一定有”。**类型系统在这里替你踩了刹车**；如果你用 `int` 加 `-1` 当哨兵，编译器什么都不会说，`-1` 到底是“没有”还是“真实的 -1”只能靠约定。

设计上可以这样分：

- 只是“可能没有”，调用者不关心原因：用 `optional<T>`；
- 没有值就是错误，调用者需要原因：用 `expected<T, E>`，`E` 放错误码或错误信息；
- “没有”与“错误”是两件事，且都要区分：用 `variant<None, T, Error>` 或自定义结果类型。

不要用 `optional<T>` 表示“出错了”，除非错误确实不需要原因、也不需要分类。`optional` 的名字就是“可选的值”，不是“可能失败的结果”；名字会误导维护者，也会让错误处理丢失信息。

17.4 `variant`：几种类型之一

`std::variant<Ts...>` 是一个类型安全的联合：它在任意时刻持有 `Ts...` 中的一个类型的值，或者在没有合适值时进入“无值”状态（`valueless_by_exception`，只在赋值/构造抛异常时出现）。它记住当前是哪一个类型，`index()` 返回这个下标，`std::get<T>` / `std::get<I>` 取对应值，取错类型会抛 `std::bad_variant_access`。

cpp **variant_visit: 三种类型、三个下标** examples/ch07/variant_visit/src/main.cpp 第 5–14 行

```cpp
int main() {
    Value number = 7;
    Value text = std::string("hi");
    Value real = 2.5;

    std::cout << "index=" << number.index() << " render=" << render(number) << "\n";
    std::cout << "index=" << text.index() << " render=" << render(text) << "\n";
    std::cout << "index=" << real.index() << " render=" << render(real) << "\n";
    return 0;
}
```

终端输出 终端输出 variant-visit-out

```
index=0 render=int:7
index=1 render=text:hi
index=2 render=real:2.500000
```

`Value` 是 `variant<int, std::string, double>`，三个类型按声明顺序对应下标 0、1、2。`number.index()` 是 0，`text.index()` 是 1，`real.index()` 是 2；`render` 用 `std::visit` 根据当前类型调用对应的分支。**`index()` 是运行期读数，类型是编译期信息**；两者配合，才能既知道“现在是什么”，又让编译器检查分支是否齐全。

`variant` 的大小通常等于“最大备选类型的大小 + 标签/对齐开销”，具体布局由实现决定。它不会为每个类型都分配一块存储，也不做堆分配；**这是它与“基类指针 + 多态”的差别**：`variant` 的值语义、无堆分配，但备选类型在编译期固定，运行期不能增加新类型。

cpp **variant_visit: 用 visit 分派，用 if constexpr 选分支** examples/ch07/variant_visit/src/variant_visit.cpp 第 5–16 行

```cpp
std::string render(const Value& value) {
    return std::visit([](const auto& item) -> std::string {
        using T = std::decay_t<decltype(item)>;
        if constexpr (std::is_same_v<T, int>) {
            return "int:" + std::to_string(item);
        } else if constexpr (std::is_same_v<T, std::string>) {
            return "text:" + item;
        } else {
            return "real:" + std::to_string(item);
        }
    }, value);
}
```

`std::visit` 把 `variant` 当前持有的值传给访问器；这里的访问器是泛型 lambda，`item` 的类型随分支变化。`std::decay_t<decltype(item)>` 去掉引用与 `const`，得到 `int` / `std::string` / `double`，再用 `if constexpr` 在

编译期选择分支。**`if constexpr` 不满足条件的分支不会被实例化**，所以 `std::to_string(item)` 不会对 `std::string` 报错；如果写成普通 `if`，三个分支都要能编译，`std::to_string(std::string)` 就会失败。

`std::visit` 要求访问器对每个备选类型都可调用；漏掉一个类型会编译失败，而不是运行期崩。
这是 `variant` 相对"void + 手写标签"的最大好处*：类型清单在编译期已知，编译器能检查你是否处理了所有情况。泛型 lambda 自动覆盖所有类型；如果你用一组具体重载，就要显式覆盖每个类型（下一节的 `overloaded`）。

17.5 `variant` + `overloaded`：把分支写成重载

泛型 lambda + `if constexpr` 适合"分支逻辑相似"的场景；如果每个类型要写不同的函数体，用一组重载更清楚。`std::visit` 需要一个可调用对象，而多个 lambda 不能直接合成一个；`overloaded` 辅助器用继承把多个 lambda 的 `operator()` 聚到一起。

cpp **variant_overload: 把多个 lambda 的 operator() 聚成一个** examples/ch07/variant_overload/include/variant_overload.h 第 9-17 行

```cpp
template <class... Ts>
struct overloaded : Ts... {
    using Ts::operator()...;
};

template <class... Ts>
overloaded(Ts...) -> overloaded<Ts...>;

std::string classify(const Value& value);
```

cpp **variant_overload: 三段逻辑各写各的** examples/ch07/variant_overload/src/variant_overload.cpp 第 3-15 行

```cpp
std::string classify(const Value& value) {
    return std::visit(overloaded{
        [](int number) {
            return number < 10 ? std::string("small-int") : std::string("large-int");
        },
        [](const std::string& text) {
            return "text:" + text;
        },
        [](double number) {
            return "real:" + std::to_string(number);
        },
    }, value);
}
```

终端输出 终端输出 variant-overload-out

```
small-int
large-int
text:hi
real:2.500000
```

`overloaded` 继承三个 lambda 类型，并用 `using Ts::operator()...;` 把三个 `operator()` 都引入作用域；调用时按参数类型重载决议，选出匹配的那一个。**这不是运行期查表，是编译期生成一个带三个重载的类**；`std::visit` 仍然按 `variant` 的当前类型调用。

漏掉一个备选类型时，`std::visit` 会编译失败，因为访问器对那个类型不可调用。这是好事：**新增一个备选类型，所有 `visit` 点都会被编译器找出来**，你不得不处理新情况。如果用的是"整数标签 + switch"，新增类型时很容易漏掉某个 `switch`，运行期才暴露。

`overloaded` 的 deduction guide（`overloaded(Ts...) -> overloaded<Ts...>;`）让 `overloaded{lambda1, lambda2}` 自动推导出类型；没有它，模板参数无法从构造实参推导。**C++20 起聚合类可以有隐式推导指引，但显式写出来兼容性更好、意图也更清楚**；这段辅助器在大量 C++17/20 代码里都能见到。



第 1 帧的 `optional` 不需要堆；第 2 帧的 `variant` 用标签记录当前类型，存储大小至少能放下最大的备选类型。**标签是运行期状态，备选类型清单是编译期信息**；两者缺一不可。实际布局（bool 放哪、标签多大、对齐怎么排）由实现决定，标准只规定行为。

`variant` 有一个特殊状态：`valueless_by_exception()`。当赋值/构造过程中抛异常，导致原值已销毁、新值没构造成功时，`variant` 可能进入"无值"状态；此时 `index()` 返回 `variant_npos`，`visit` 会抛 `std::bad_variant_access`。**这是异常安全的边界，不是正常状态**；正常情况下不要依赖它，但读代码时要知道它的存在，尤其是对 `variant` 做赋值/交换的代码。

`std::visit` 的访问器对所有备选类型必须返回"兼容"的类型；如果不同分支返回 `int` 与 `std::string`，返回类型无法统一，编译失败。**需要不同分支返回不同类型时，让它们返回 `std::variant` 或 `std::any`，或者把公共逻辑抽出来**；不要指望 `visit` 自动做类型擦除。`visit` 的返回类型由所有分支共同决定，这也是它能在编译期检查分支完整性的原因。

`variant` 的备选类型不能重复：`variant<int, int>` 不合法，因为 `get<int>` 无法区分两个 `int`。需要"同类型但语义不同"时，用包装结构体（`struct Meters { int value; };` 与 `struct Seconds { int value; };`）或不同的标签类型。**类型系统靠"类型不同"区分分支；重复类型会让 `visit` 与 `get` 失去意义。**

17.6 `expected`：值或错误

`std::expected<T, E>` 要么包含一个 `T`，要么包含一个 `unexpected<E>`。它和 `optional` 的区别是：**空的时候有错误对象**，调用者能读到原因；它和异常的区别是：错误在返回类型里，调用者必须面对，不需要 `try` / `catch`，也没有栈展开成本。

cpp **expected_basic: 成功读 value，失败读 error** examples/ch07/expected_basic/src/main.cpp 第 5–14 行

```cpp
int main() {
    auto good = parse_positive(5);
    auto bad = parse_positive(-1);

    std::cout << "ok=" << good.has_value()
              << " value=" << good.value() << "\n";
    std::cout << "ok=" << bad.has_value()
              << " message=" << bad.error() << "\n";
    return 0;
}
```

终端输出 终端输出 expected-basic-out

```
ok=1 value=5
ok=0 message=negative
```

`parse_positive(5)` 返回包含 5 的 `expected`，`has_value()` 为真，`value()` 返回 5；`parse_positive(-1)` 返回 `std::unexpected(std::string("negative"))`，`has_value()` 为假，`error()` 返回 `"negative"`。**`value()` 在错误状态下会抛 `std::bad_expected_access<E>`**，所以读值前也要检查；`error()` 在成功状态下是未定义行为（标准要求先检查）；不确定时先看 `has_value()`，不要靠"应该不会成功"的假设。

标准原文 · 标准原文：expected 保存值或错误

> An `expected<T, E>` object holds an object of type `T` or an object of type `unexpected<E>` and manages the lifetime of the contained objects.

大意：`expected<T, E>` 持有 `T` 或 `unexpected<E>` 之一，并管理内部对象的生命周期。

出处：N4950 §22.8.1「In general」（[expected.general]）。

`expected` 的错误类型 `E` 是接口设计的一部分。用 `std::error_code` 能表达分类与系统错误，用 `std::string` 能带上下文但每次失败可能分配，用一个小的 `enum class` 最轻量。**不要把 `E` 设计成"什么都能装"的大对象**：`expected` 的存储要能放下 `T` 和 `E` 中较大的那个，错误类型太大，成功路径也要背这块空间；错误类型应当是"可枚举、可比较、可复制"的小对象。

`expected<void, E>` 用于"成功但没有返回值"的操作，例如 `close()`、`flush()`；成功时用 `expected<void, E>{}`，失败时返回 `unexpected`。它没有 `value()` 可读，通常用 `has_value()` 或直接 `if (!result)` 判断。**这比返回 `bool` + 输出参数更清楚**，因为错误原因在类型里，不会被忽略。

17.7 `expected` 链：错误自动短路

多步操作如果每步都可能失败，手写 `if` 嵌套会很深。`expected` 的 `and_then`（继续一个返回 `expected` 的操作）与 `transform`（对成功值做变换）把链拉平；任何一步返回 `unexpected`，后面的步骤都跳过，错误原样传下去。

cpp **expected_chain: 解析、校验、计算三步** examples/ch07/expected_chain/src/expected_chain.cpp 第 5-25 行

```cpp
std::expected<int, std::string> parse_number(const std::string& text) {
    int value = 0;
    const char* begin = text.data();
    const char* end = begin + text.size();
    auto [ptr, error] = std::from_chars(begin, end, value);
    if (error != std::errc{} || ptr != end) {
        return std::unexpected(std::string("not a number"));
    }
    return value;
}

std::expected<int, std::string> require_positive(int value) {
    if (value <= 0) {
        return std::unexpected(std::string("must be positive"));
    }
    return value;
}

std::expected<double, std::string> halve(int value) {
    return value / 2.0;
}
```

cpp **expected_chain: 一条链处理三种输入** examples/ch07/expected_chain/src/main.cpp 第 5-20 行

```cpp
int main() {
    auto good = parse_number("8")
                    .and_then(require_positive)
                    .and_then(halve);
    auto negative = parse_number("-3")
                        .and_then(require_positive)
                        .and_then(halve);
    auto text = parse_number("x")
                    .and_then(require_positive)
                    .and_then(halve);

    std::cout << "ok=" << good.has_value() << " value=" << good.value() << "\n";
    std::cout << "error=" << negative.error() << "\n";
    std::cout << "error=" << text.error() << "\n";
    return 0;
}
```

终端输出 终端输出 expected-chain-out

```
ok=1 value=4
error=must be positive
error=not a number
```

`parse_number("8")` 得到 8，`require_positive` 通过，`halve` 得到 4.0，所以 `value=4`；`parse_number("-3")` 解析成功但校验失败，`require_positive` 返回 `unexpected("must be positive")`，后面的 `halve` 不再执行；`parse_number("x")` 一开始就失败，返回 `unexpected("not a number")`。**`and_then` 的成功值类型可以变化**：`parse_number` 与 `require_positive` 都是 `expected<int, string>`，`halve` 返回 `expected<double, string>`，链的最终类型是 `expected<double, string>`。

`and_then` 与 `transform` 的分工：`and_then` 接一个"返回 `expected` 的函数"，用于可能失败的一步；`transform` 接一个"普通函数"，只在成功时变换值，不会引入新错误。`or_else` 用于错误分支的恢复或转换。**这三个是 C++23 的 monadic 操作**，让错误处理像流水线；如果编译器/标准库不支持，可以手写 `if`，语义一样，只是更长。



第 1 帧的"没有"是正常结果；第 2 帧的"错误"需要原因；第 3 帧的"多种形态"不是错误，而是数据本身的可能性。**先把问题归到这三类，再选类型**，比"先选一个容器再想办法塞"要稳。

17.8 三者怎么选

- 只关心"有没有值"，不关心原因，缺省值合理：`optional<T>`；
- 关心失败原因，调用者需要分支处理，且不想用异常：`expected<T, E>`；
- 值本身可以是几种类型之一，每种都是正常情况：`variant<Ts...>`；
- 失败是真正的异常、当前层无法处理：抛异常，让上层 `catch`；
- 失败极常见、性能敏感、且调用者就在附近处理：错误码或 `expected`，避免异常开销。

`optional` 不适合表达错误，`expected` 不适合表达"多选一"，`variant` 不适合表达"成功/失败"（虽然可以，但语义不如 `expected` 直接）。类型选对了，调用者的代码自然清楚；类型选错，后面会靠注释和约定补洞。

还有一种常见混用：用 `optional<T>` 表示"查不到"，用 `expected<T, E>` 表示"查到了但处理失败"。把两种失败塞进同一层，调用者就分不清该重试还是该报错；**能区分的失败，尽量在类型上区分**，这比在错误信息里写一句"可能是没找到"可靠得多。

17.9 本机实测：六个读数

实验记录 · 本机实测：三种类型的真实输出

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `optional_basic` 的 stdout 是 `has=1 score=90` / `has=0 fallback=-1`：有值读值，无值走 `value_or`；
- `optional_misuse` 编译失败，诊断是 `cannot convert 'std::optional<int>' to 'int' in initialization`：类型系统阻止"悄悄当 int 用"；
- `variant_visit` 的 stdout 是 `index=0 render=int:7` / `index=1 render=text:hi` / `index=2 render=real:2.500000`：下标与类型一一对应；
- `variant_overload` 的 stdout 是 `small-int` / `large-int` / `text:hi` / `real:2.500000`：`overloaded` 按类型选分支；
- `expected_basic` 的 stdout 是 `ok=1 value=5` / `ok=0 message=negative`：成功读值，失败读错误；
- `expected_chain` 的 stdout 是 `ok=1 value=4` / `error=must be positive` / `error=not a number`：错误在链上短路。

17.10 下一单元从哪继续

下一单元进入编译期：`constexpr` 让函数在编译期求值，`consteval` 强制立即求值，`static_assert` 把条件变成编译错误，`if constexpr` 让模板只实例化命中的分支。这一单元的 `variant` 访问器已经用了 `if constexpr`；下一单元会把"编译期能做什么、不能做什么"系统讲清楚。

小结 · 这一节要带走的四件事

- `optional<T>` 表示"有值或没有"，用 `has_value` / `value_or` / `*` 访问；它不携带错误原因，别拿它当错误类型。
- `variant<Ts...>` 是类型安全的联合，`index()` 给下标，`std::visit` 分派；漏掉备选类型会编译失败。
- `overloaded` 用继承聚合多个 lambda 的 `operator()`，让 `visit` 的分支像重载一样书写。
- `expected<T, E>` 表示"值或错误"，`and_then` / `transform` 把多步操作串成短路链；按"没有值 / 出错 / 多选一"选类型。

自测题 · 自测题（先自己做，再对答案）

1. `optional<int>` 为空时，`value()`、`value_or(-1)`、`*it` 分别是什么行为？
2. 为什么 `std::optional<int>` 不能直接赋给 `int`？这个限制有什么好处？
3. `variant<int, string, double>` 的 `index()` 返回什么？`std::visit` 的访问器漏掉一个类型会怎样？
4. `overloaded` 辅助器解决了什么问题？没有它，`std::visit` 还能用吗？
5. `optional`、`expected`、`variant` 各适合表达什么？各举一个场景。

注 · 自测答案

1. `value()` 抛 `std::bad_optional_access`；`value_or(-1)` 返回 -1；`*it` 是未定义行为（空 `optional` 不做检查）。
2. `optional` 没有到 `T` 的隐式转换，编译器不允许把"可能为空"当成"一定有值"；好处是强制调用者先检查，避免空值被静默使用。
3. `index()` 返回当前备选类型在模板参数列表中的下标（从 0 开始）；访问器漏掉一个类型会编译失败，因为 `visit` 要求对每个备选类型都可调用。
4. 它把多个 lambda 的 `operator()` 聚合到一个类里，让 `visit` 用一个对象接收多个分支；没有它也可以用泛型 lambda + `if constexpr`，或手写一个带多个 `operator()` 的类。
5. `optional` 表示"可能没有值"（查字典没找到）；`expected` 表示"值或错误"（解析失败并带原因）；`variant` 表示"几种正常类型之一"（一个格子可以是数字、文本或实数）。

第 8 章

编译期与系统 I/O

本章 2 个单元 · 页眉从这里之后的页面开始出现

1. constexpr 与 consteval 零基础 · 第 18 单元 · 真编译真运行
2. 文件、格式化与生成器 零基础 · 第 19 单元 · 真编译真运行

constexpr 与 consteval

普通函数在运行时执行；`constexpr` 函数多了一种可能——如果实参是编译期常量，编译器可以在编译期把它算完，直接把结果写进程序。`consteval` 更进一步：它**必须**在编译期求值，运行期调用直接编译失败。`static_assert` 把条件放进编译期，不成立就报错；`if constexpr` 让模板只实例化命中的分支。

这一单元用六个例子拆开编译期计算：先看同一个 `constexpr` 函数怎样既在编译期算、又在运行期算，再用 `consteval` 把"必须编译期"钉死；然后用 `static_assert` 检查类型与大小，用 `if constexpr` 写只在某个类型下合法的分支；最后看一个 `consteval` 的运行期调用反例，以及 C++20 起能在编译期使用的 `std::vector`。

◎ 学习目标 · 看完这一单元，你应该能

- 写 `constexpr` 函数与变量，说出"编译期求值"与"运行期调用"是两条路径；
- 用 `consteval` 强制立即求值，并读懂"不是常量表达式"的编译错误；
- 用 `static_assert` 在编译期检查条件，理解它检查的是当前目标平台；
- 用 `if constexpr` 在模板里丢弃不匹配的分支，知道非模板环境下它仍会检查两个分支；
- 说出编译期计算的边界：不能有未定义行为、不能读运行期输入、受实现限制；
- 知道 C++20 起 `constexpr` 函数里可以用 `std::vector` 等容器做临时计算。

18.1 先看工程长什么样

这一单元在 `examples/ch08/` 下建六个例子。`constexpr` / `consteval` 函数放在头文件里（使用点需要看到定义，才能求值），`.cpp` 提供普通函数包装，`main.cpp` 打印结果。`constexpr_error/` 是反例：运行期实参传给 `consteval`，编译失败。

目录树 **工程：examples/ch08/ 这一单元新增的六个例子**

第 1 步 先看 constexpr、consteval 与 static_assert

```
examples/
└── ch08/
    ├── constexpr_fact/ ← 本课新增
    │   ├── include/ ← 本课新增
    │   │   └── constexpr_fact.h ← 本课新增
    │   ├── src/ ← 本课新增
    │   │   ├── main.cpp ← 本课新增
    │   │   └── constexpr_fact.cpp ← 本课新增
    │   └── constexpr_fact.json ← 本课新增
    ├── consteval_check/ ← 本课新增
    └── static_assert_demo/ ← 本课新增
```

第 2 步 加上 if constexpr 与编译期反例

```
examples/
└── ch08/
    ├── 前三个例子 ： 同上
    ├── if_constexpr/ ← 本课新增
    └── constexpr_error/ ← 本课新增
```

第 3 步 最后加上 constexpr 容器

```
examples/
└── ch08/
    ├── 前五个例子 ： 同上
    └── constexpr_container/ ← 本课新增
```

第 1 帧的 `constexpr_fact/` 用阶乘对照编译期与运行期，`consteval_check/` 强制立即求值，`static_assert_demo/` 在编译期检查 `sizeof`。第 2 帧的 `if_constexpr/` 用 `if constexpr` 分派类型，`constexpr_error/` 故意让运行期实参进 `consteval`。第 3 帧的 `constexpr_container/` 在编译期建一个 `std::vector` 并求和。

18.2 `constexpr`：能编译期算，也能运行期算

`constexpr` 函数没有承诺"一定在编译期执行"，它只是**允许**在常量表达式中使用。同一个 `factorial`，用字面量调用时可以常量求值，用运行期变量调用时就在运行期算。编译器可以选择在编译期折叠它，但语义上两条路径都合法。

cpp **constexpr_fact：循环版的阶乘** examples/ch08/constexpr_fact/include/constexpr_fact.h 第 4–12 行

```cpp
constexpr int factorial(int value) {
    int result = 1;
    for (int i = 2; i <= value; ++i) {
        result *= i;
    }
    return result;
}

static_assert(factorial(5) == 120, "factorial(5) should be 120");
```

cpp **constexpr_fact：一个编译期，一个运行期** examples/ch08/constexpr_fact/src/main.cpp 第 7–12 行

```cpp
int main() {
    constexpr int compiled = factorial(5);
    std::cout << "compiled=" << compiled
              << " runtime=" << runtime_factorial(6) << "\n";
    return 0;
}
```

终端输出 终端输出 constexpr-fact-out

```
compiled=120 runtime=720
```

`constexpr int compiled = factorial(5);` 要求 `factorial(5)` 是常量表达式，编译器在编译期算出 120；`runtime_factorial(6)` 的参数 6 虽然是字面量，但它经过普通函数调用，函数体在运行期执行，结果是 720。**同一个 `constexpr` 函数，用在哪里决定它在哪算**；`constexpr` 不是"把这个函数搬到编译期执行"的指令，而是"它具备在编译期执行的能力"。

`constexpr` 函数体在 C++14 起可以包含循环、局部变量、多个 `return`，比 C++11 的"单表达式"宽松得多。它仍然有约束：不能有未定义行为，不能调用非 `constexpr` 的函数（在常量求值路径上），不能读运行期输入。**这些约束只在"常量求值"时检查**；运行期调用时，它就是一个普通函数，可以有指针、可以分配内存（只要不要求在编译期算）。

`constexpr` 变量要求初始化器是常量表达式：`constexpr int size = 10;` 可以，`constexpr int size = runtime_value();` 不行。`constexpr` **变量隐含** `const`，而且必须在编译期确定值；如果只是"运行期不再改"，用 `const` 就够了，不必强行 `constexpr`。

标准原文 · **标准原文：constexpr 与 consteval 的声明**

> The `constexpr` specifier shall be applied only to the definition of a variable or variable template or the declaration of a function or function template. The `consteval` specifier shall be applied only to the declaration of a function or function template. A function or static data member declared with the `constexpr` or `consteval` specifier is implicitly an inline function or variable.

大意：`constexpr` 只能用于变量/变量模板的定义或函数/函数模板的声明；`consteval` 只能用于函数/函数模板的声明；带这两个说明符的函数或静态数据成员隐含 `inline`。

出处：N4950 §9.2.6「The `constexpr` and `consteval` specifiers」（[dcl.constexpr]）。

"隐含 `inline`"很实用：把 `constexpr` 函数定义在头文件里，被多个翻译单元包含也不会重复定义。**这也是 `constexpr` 工具函数通常直接写在头文件里的原因**；模板函数同理，二者都靠 `inline` 或模板规则避免 ODR 冲突。

18.3 `consteval`：必须编译期算

`consteval` 声明的是"立即函数"（immediate function）：每一次调用都必须在编译期求值，结果是一个常量。它比 `constexpr` 更严格——`constexpr` 可以运行期调用，`consteval` 不行。**需要"这个值必须在编译期确定"时用它**，例如

生成固定表、检查配置、在编译期做断言。

cpp **consteval_check: 立即函数与静态断言** examples/ch08/consteval_check/include/consteval_check.h 第 4–10 行

```cpp
consteval int square(int value) {
    return value * value;
}

static_assert(square(4) == 16);

int constant_square();
```

cpp **consteval_check: 运行期函数里的常量调用** examples/ch08/consteval_check/src/consteval_check.cpp 第 3–5 行

```cpp
int constant_square() {
    return square(5);
}
```

终端输出 终端输出 consteval-check-out

```
constant_square=25
square4=16
```

`constant_square()` 是普通函数，在运行期被调用；但它里面的 `square(5)` 实参是常量，`consteval` 要求立即求值，编译器在编译期算出 25，运行期只是返回这个常量。`square(4)` 出现在 `main` 里，同样在编译期算出 16。**`consteval` 的限制在“实参必须是常量表达式”，不在“调用点必须在常量上下文”**；只要实参是常量，运行期函数里也能调用。

如果实参是运行期变量，`consteval` 调用就无法成立，编译器会报“不是常量表达式”。下一节的 `constexpr_error/` 会看到真实诊断。**这正是 `consteval` 的用途：把“必须在编译期”写成类型系统能检查的约束**，而不是靠注释提醒。

`consteval` 函数不能声明为变量，也不能取地址后运行期调用；它的每一次调用都必须在编译期完成。`constexpr` 函数则可以取地址、可以运行期调用。**二者不是“新代替旧”**：`constexpr` 是“可以编译期”，`consteval` 是“必须编译期”，按需求选。

`constexpr` 与 `const` 容易混：`const int n = runtime_value();` 只表示“运行期不再改”，初始化器不要求常量；`constexpr int n = 42;` 要求编译期常量，可以用于数组长度、模板实参。**需要“编译期常量”时写 `constexpr`，只是“只读”写 `const`**；反过来，把 `const` 当 `constexpr` 用，会在需要常量表达式的地方编译失败。

`constinit` 是 C++20 的另一个工具：它要求变量“在编译期完成初始化”，但不要求变量本身是 `const`，也不要求初始化器是常量表达式（只要初始化本身能静态完成）。**`constinit` 解决的是“静态初始化顺序”问题**，不是“运行期只读”；它常与全局/静态变量一起用，避免动态初始化的顺序不确定。本书只在边界里点到，不展开静态初始化顺序的细节。

`constexpr` 也可以出现在构造函数、析构函数、`operator[]` 等成员函数上，让对象在编译期构造/使用。C++20 起 `constexpr` 的限制进一步放宽（例如可以调用 `new` / `delete`，只要在常量求值内配对）；C++23 起 `constexpr` 函数里能用更多标准库设施。**标准版本越高，能放进编译期的代码越多**；写库时若要兼容旧标准，先查目标标准支持哪些 `constexpr` 设施。

`constexpr` 函数在运行期调用时就是普通函数，不要以为标了 `constexpr` 就“一定在编译期执行”。判断是否编译期求值，看调用上下文：`constexpr` 变量初始化、`static_assert`、模板实参、`consteval` 调用等要求常量表达式的地方才会强制编译期；普通运行期调用不会。**“能编译期”与“已编译期”是两件事**，性能分析时别把 `constexpr` 当成优化保证。

18.4 `static_assert`：条件不成立就编译失败

`static_assert(条件, 消息)` 在编译期检查条件；条件为假时，编译器报错并显示消息。它不需要运行程序，也不会产生运行期开销。条件必须是常量表达式，消息在 C++17 起可以省略，但**建议总是写**，因为它是给未来维护者看的线索。

cpp **static_assert_demo: 检查大小与关系** examples/ch08/static_assert_demo/include/static_assert_demo.h 第 6–10 行

```cpp
constexpr std::size_t int_bytes = sizeof(int);
constexpr std::size_t long_long_bytes = sizeof(long long);

```

```
9   static_assert(int_bytes == 4, "this unit assumes 32-bit int");
10  static_assert(long_long_bytes >= int_bytes, "long long must not be smaller than int");
```

终端输出 终端输出 static-assert-out

```
int_bytes=4 long_long_bytes=8
```

第一个断言检查"本机的 `int` 是 4 字节"，第二个检查"`long long` 不小于 `int`"。**第一条不是标准保证**：C++ 只规定 `int` 至少 16 位，具体大小由实现决定；本书的本机是 4 字节，所以这个断言在本书工具链上成立，换一个平台可能直接编译失败。第二条同样是平台事实的检查；标准只保证 `long long` 至少 64 位（通常 8 字节），不保证一定不小于 `int` 的字节数（虽然实践中成立）。

`static_assert` 常用来检查模板参数、类型布局、常量配置：`static_assert(std::is_copy_constructible_v<T>)`、`static_assert(sizeof(Header) == 16)`。**把"运行时才发现"的约定前移到编译期**，是 C++ 模板库的常见手法；标准库大量用 `static_assert` 给出比模板报错更清楚的消息。

`static_assert` 与 `if constexpr` 的区别：前者检查一个条件，不成立就终止编译；后者在模板里选择分支，不满足条件的分支被丢弃。**`static_assert` 是"必须成立"，`if constexpr` 是"按类型选路"**；二者经常一起用：在某个分支里 `static_assert` 检查该分支的前提。

18.5 `if constexpr`：只实例化命中的分支

模板函数里，普通 `if` 的两个分支都会被编译；如果某个分支对当前类型不合法，即使运行期不会走到，也会编译失败。`if constexpr` 的条件必须是常量表达式；在模板中，条件为假的分支**不会被实例化**，因此可以写"只对某些类型合法"的代码。

cpp **if_constexpr：一个模板，三种类型** examples/ch08/if_constexpr/include/if_constexpr.h 第 7–16 行

```
7   template <class T>
8   std::string kind_of(const T& value) {
9       if constexpr (std::is_integral_v<T>) {
10          return "int:" + std::to_string(value);
11      } else if constexpr (std::is_floating_point_v<T>) {
12          return "real:" + std::to_string(value);
13      } else {
14          return "text:" + value;
15      }
16  }
```

终端输出 终端输出 if-constexpr-out

```
int:7
real:2.500000
text:hi
```

`kind_of(7)` 让 `T = int`，只有第一个分支参与编译；`kind_of(2.5)` 让 `T = double`，只有第二个分支；`kind_of("hi")` 让 `T = std::string`，走 `else`。**如果写成普通 `if`，`std::to_string(value)` 对 `std::string` 会报错，`"text:" + value` 对 `int` 也会报错**；`if constexpr` 把不匹配的分支在实例化时丢掉，才让三种类型共存于一个模板。

谁能看见谁 **if constexpr 的实例化视野**

第 1 步 T = int

kind_of<int> ：条件 is_integral_v<int> 为真

命中分支 ：return "int:" + std::to_string(value)

丢弃分支 ：浮点与 text 分支不实例化

第 2 步 T = std::string

kind_of<std::string> ：条件 is_integral_v 为假，浮点为假

命中分支 ：return "text:" + value

丢弃分支 ：两个 to_string 分支不实例化

第 1 帧只生成 `int` 版本，第 2 帧只生成 `std::string` 版本；**每个 `T` 只看到自己那条路。**这也是 `if constexpr` 与运行时多态的区别：它在编译期决定生成哪些代码，运行期没有分支判断，也没有虚调用。

需要提醒：**在非模板函数里，`if constexpr` 的丢弃分支仍然会被完整检查。**标准只在模板实例化时允许"不实例化丢弃的语句"；普通函数里写 `if constexpr (false) { ... }`，里面的代码仍然要语法正确、名字可查。想真正"按条件排除代码"，要么放进模板，要么用预处理宏。**`if constexpr` 不是 `#if` 的替代品**：前者是类型/常量分支，后者是文本替换；把平台相关、语法不同的代码塞进 `if constexpr`，通常得不到你想要的结果。

`if constexpr` 的条件必须是常量表达式，不能依赖运行期输入；条件为假的分支如果在模板里被丢弃，里面的 `static_assert(false)` 也可能不触发。**不要把 `static_assert(false)` 放在模板的未命中分支里当"哨兵"**，除非你确认它不会被实例化；需要"这个类型永远不支持"时，用 `= delete` 或约束，而不是依赖分支丢弃。

18.6 编译期计算的边界：不是所有东西都能算

`constexpr` / `consteval` 的常量求值有一条硬边界：**不能有未定义行为。**除零、越界、空指针解引用、有符号整数溢出，在运行期是"未定义"，在常量求值里则会让这个表达式"不是常量表达式"，编译失败。标准要求编译器诊断这种失败；实际消息可能很长，但根因通常是某一步不满足常量求值规则。

cpp **constexpr_error: 运行期实参不能进 consteval** examples/ch08/constexpr_error/src/constexpr_error.cpp 第 3–5 行

```cpp
int runtime_square(int value) {
    return square(value);
}
```

编译诊断 编译诊断 constexpr-error-diag

```
src/constexpr_error.cpp: In function 'int runtime_square(int)':
src/constexpr_error.cpp:4:18: error: call to consteval function 'square(value)' is not a constant expression
    4 |     return square(value);
      |            ~~~~~~^~~~~~~
src/constexpr_error.cpp:4:19: error: 'value' is not a constant expression
    4 |     return square(value);
      |                   ^~~~~
```

诊断会指出"调用 `consteval` 函数 `square(value)` 不是常量表达式"，并进一步指出 `value` 不是常量表达式。**这不是 `consteval` 的缺陷，而是它的承诺**：它只接受编译期能确定的值。要让运行期变量参与，就把函数改成 `constexpr`（允许运行期调用），或者把调用移到运行期路径。

另一个边界是"不能读运行期输入"：`constexpr` 函数不能调用 `std::cin`、不能读文件、不能取系统时间。**编译期计算的输入只能来自源码里的常量、模板参数、`sizeof`、类型特征等编译期信息**；这也是它可预测、可缓存的原因。

编译期计算还受实现限制：递归深度、步数、内存都有上限，超过会报"常量表达式过大/过深"。标准没有规定具体上限，由编译器决定；**写 `constexpr` 算法时要避免无界递归**，能写成循环就写循环，既省编译期资源，也更容易读。不同编译器版本对边界写法的接受程度也可能不同，跨平台项目要给编译期代码留出保守余量。

18.7 `constexpr` 容器：编译期也能用 `vector`

C++20 起，`constexpr` 函数里可以构造 `std::vector`、`std::string` 等动态容器，只要整个常量求值过程在结束时把分配的内存释放干净。这让"编译期建表、编译期解析、编译期校验"变得自然；下面的 `sum_squares` 在编译期建一个 `vector`，平方求和，再用 `static_assert` 检查结果。

cpp **constexpr_container: 编译期 vector** examples/ch08/constexpr_container/include/constexpr_container.h 第 6–19 行

```cpp
constexpr int sum_squares(int count) {
    std::vector<int> values;
    for (int i = 1; i <= count; ++i) {
        values.push_back(i);
    }

    int total = 0;
    for (int value : values) {
        total += value * value;
    }
```

```
    return total;
}

static_assert(sum_squares(3) == 14);
```

终端输出 终端输出 constexpr-container-out

```
sum3=14 sum4=30
```

`static_assert(sum_squares(3) == 14)` 要求在编译期完成整个 `vector` 的构造、`push_back`、遍历与析构；编译器在常量求值环境里分配临时存储，求值结束后释放。`sum_squares(3)` 在 `main` 里是常量表达式，得到 14；`container_sum(4)` 是运行期调用，得到 $1^2+2^2+3^2+4^2=30$。**同一个函数，两条路径都成立**；区别只是编译期路径不能依赖运行期输入。

编译期 `vector` 不是"把容器放进二进制常量区"，它只是常量求值期间的临时对象；求值结束后，结果是一个整数 14。**别把"编译期能建 vector"误解成"运行时零成本容器"**；运行期该分配还是要分配，`constexpr` 只影响常量求值那一次。

图 / 搜索过程 **constexpr 的两条路径**

第 1 步 源码里的常量实参

`factorial(5)` ：实参是字面量 | 编译器 ：尝试常量求值 | 结果 ：120 写进常量/指令

第 2 步 运行期变量实参

`factorial(n)` ：n 来自输入 | 编译器 ：生成普通函数代码 | 运行期 ：循环计算，返回结果

第 3 步 consteval 只留第一条

`square(value)` ：value 不是常量 | 编译器 ：拒绝，报"不是常量表达式"

第 1 帧走编译期，第 2 帧走运行期，第 3 帧被 `consteval` 拒绝。**`constexpr` 是"两扇门都开"，`consteval` 是"只开编译期这扇门"**；选哪个取决于你是否允许运行期调用。

18.8 本机实测：六个读数

实验记录 · 本机实测：编译期与运行期

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `constexpr_fact` 的 stdout 是 `compiled=120 runtime=720`：同一个函数，编译期与运行期各算一次；
- `consteval_check` 的 stdout 是 `constant_square=25` / `square4=16`：`consteval` 只接受常量实参，结果在编译期确定；
- `static_assert_demo` 的 stdout 是 `int_bytes=4 long_long_bytes=8`：两个 `static_assert` 在本机成立；
- `if_constexpr` 的 stdout 是 `int:7` / `real:2.500000` / `text:hi`：每个类型只实例化命中的分支；
- `constexpr_error` 编译失败，诊断指出 `consteval` 调用不是常量表达式；
- `constexpr_container` 的 stdout 是 `sum3=14 sum4=30`：编译期与运行期都能用 `vector` 计算。

18.9 下一单元从哪继续

下一单元离开"纯计算"，进入文件与输出：`<filesystem>` 遍历目录、`<format>` 拼格式串、`std::generator` 逐次产出、`ifstream` / `ofstream` 读写文件。编译期计算会在那里再次出现——格式串检查、路径常量、生成器的惰性状态，都需要区分"编译期确定"与"运行期变化"。

小结 · 这一节要带走的四件事

- `constexpr` 函数既能在编译期求值，也能在运行期调用；`consteval` 只接受常量实参，必须立即求值。
- `static_assert` 在编译期检查条件，条件不成立就编译失败；它检查的是当前目标平台，不是"所有平台都成立"。
- `if constexpr` 在模板中只实例化命中的分支，让不同类型共用一份模板；非模板环境仍会检查两个分支。
- 编译期计算不能有未定义行为、不能读运行期输入，并受实现的深度/步数限制；C++20 起可在 `constexpr` 里用 `vector`。

自测题 · 自测题（先自己做，再对答案）

1. `constexpr int f(int);` 的函数，在什么情况下会在编译期算，什么情况下在运行期算？
2. `consteval` 与 `constexpr` 的核心区别是什么？运行期实参传给 `consteval` 会怎样？
3. `static_assert(sizeof(int) == 4)` 在什么情况下会失败？这说明了 `static_assert` 检查的是什么？
4. `if constexpr` 在模板里解决了什么问题？在非模板函数里，丢弃分支还会被检查吗？
5. 为什么 `constexpr` 函数里不能有未定义行为？C++20 起 `constexpr` 里能用 `vector` 吗？

注 · 自测答案

1. 实参是常量表达式、且处于需要常量求值的上下文（如 `constexpr` 变量初始化、`static_assert`、模板实参）时，编译器在编译期算；实参是运行期变量或处于普通运行期调用时，在运行期算。
2. `constexpr` 允许编译期也允许运行期；`consteval` 必须在编译期求值。运行期实参传给 `consteval` 会编译失败，诊断指出"不是常量表达式"。
3. 在 `int` 不是 4 字节的平台上会失败；它检查的是当前编译目标的事实，不是标准对所有平台的保证。所以写这种断言时要清楚它绑定了平台。
4. 它让模板只实例化与类型匹配的分支，避免其他分支对当前类型不合法而报错；在非模板函数里，丢弃分支仍然会被完整检查，不能当作 `#if` 使用。
5. 常量求值要求每一步都有确定结果，未定义行为会让表达式不再是常量表达式，编译器必须诊断；C++20 起可以在 `constexpr` 函数里使用 `std::vector`，只要求值结束时释放所有分配。

文件、格式化与生成器

程序最终要和外界打交道：读目录、写文件、把结果格式化给人看、把一大串数据分次产出。这一单元把四件事放在一起：`<filesystem>` 负责路径与目录遍历，`<fstream>` 负责文件读写，`<format>` / `<print>` 负责把值变成字符串，`std::generator` 负责"要一个算一个"的惰性序列。

这一单元的例子都在本机真跑：`filesystem_walk/` 建目录、遍历、删除；`file_io/` 写三行再读回来；`format_basic/` 拼格式串；`print_lib/` 用 `std::println`，需要额外的链接库与干净的环境变量；`generator_basic/` 与 `generator_pipeline/` 展示协程生成器与管线。

学习目标 · 看完这一单元，你应该能

- 用 `std::filesystem::path` 拼路径，用 `recursive_directory_iterator` 遍历目录；
- 用 `ifstream` / `ofstream` 读写文本文件，解释 Windows 文本模式为什么会多出字节；
- 用 `std::format` 写替换字段、补零、对齐、进制与浮点格式；
- 说出 `std::print` 与 `std::format` 的链接差异，以及本机为什么需要 `-lstdc++exp` 与干净 `PATH`；
- 用 `std::generator` 写惰性序列，用 `co_yield` 逐次产出；
- 把两个生成器接成管线，并解释"惰性"带来的内存与求值顺序差异。

19.1 先看工程长什么样

这一单元在 `examples/ch08/` 下新增六个例子。文件与目录操作放在 `.cpp` 里，`main.cpp` 只打印结果；`print_lib/` 的清单额外声明 `libs` 与 `env`，`filesystem_walk/` 与 `file_io/` 也把 `PATH` 钉到干净的 MSYS2 路径，避免 Git

Bash 环境里的 DLL 干扰，其余例子用默认环境。`filesystem_walk/` 与 `file_io/` 会在当前目录创建临时文件/目录，结束时删除，重复运行结果一致。

目录树 **工程：examples/ch08/ 这一单元新增的六个例子**

第 1 步 先看目录遍历、格式化与文件读写

```
examples/
└── ch08/
    ├── filesystem_walk/ ← 本课新增
    │   ├── include/ ← 本课新增
    │   │   └── filesystem_walk.h ← 本课新增
    │   ├── src/ ← 本课新增
    │   │   ├── main.cpp ← 本课新增
    │   │   └── filesystem_walk.cpp ← 本课新增
    │   └── filesystem_walk.json ← 本课新增
    ├── format_basic/ ← 本课新增
    └── file_io/ ← 本课新增
```

第 2 步 加上 print 与 generator

```
examples/
└── ch08/
    ├── 前三个例子 ： 同上
    ├── print_lib/ ← 本课新增
    └── generator_basic/ ← 本课新增
```

第 3 步 最后加上生成器管线

```
examples/
└── ch08/
    ├── 前五个例子 ： 同上
    └── generator_pipeline/ ← 本课新增
```

第 1 帧的 `filesystem_walk/` 建 `fs_demo/` 并递归遍历，`format_basic/` 拼三种格式，`file_io/` 写读一个文本文件。第 2 帧的 `print_lib/` 用 `std::println`，清单带 `-lstdc++exp` 与干净 `PATH`；`generator_basic/` 用 `co_yield` 产出一个整数序列。第 3 帧的 `generator_pipeline/` 把两个生成器接起来，先产出 0..4，再平方。

19.2 `<filesystem>`：路径、目录与遍历

`std::filesystem::path` 表示路径，可以用 `/` 拼接（Windows 上也能用，库会转成合适的分隔符），可以取文件名、扩展名、父目录。遍历目录用 `directory_iterator`（只一层）或 `recursive_directory_iterator`（递归）。**遍历顺序没有标准保证**，要稳定输出就得自己排序。

cpp **filesystem_walk: 建目录、递归遍历、删除** examples/ch08/filesystem_walk/src/filesystem_walk.cpp 第 8–27 行

```cpp
std::vector<std::string> walk_demo() {
    namespace fs = std::filesystem;
    const fs::path root = "fs_demo";

    std::error_code ignored;
    fs::remove_all(root, ignored);
    fs::create_directories(root / "sub");

    std::ofstream(root / "a.txt") << "alpha\n";
    std::ofstream(root / "sub" / "b.txt") << "beta\n";

    std::vector<std::string> names;
    for (const auto& entry : fs::recursive_directory_iterator(root)) {
        names.push_back(fs::relative(entry.path(), root).generic_string());
    }
    std::sort(names.begin(), names.end());

    fs::remove_all(root, ignored);
    return names;
}
```

终端输出 终端输出 filesystem-walk-out

```
count=3
entry=a.txt
entry=sub
entry=sub/b.txt
```

`create_directories` 会连同缺失的父目录一起建；`recursive_directory_iterator` 会进入 `sub/`，所以一共看到三项：`a.txt`、`sub/`、`sub/b.txt`。`fs::relative(entry.path(), root)` 把绝对/相对路径转成"相对 `root` 的路径"，`generic_string()` 统一用 `/` 作为分隔符，便于跨平台比较。**排序是为了让输出可复现**：标准不规定迭代顺序，不排序就可能在不同文件系统上得到不同顺序。

`remove_all(root, ignored)` 接收一个 `std::error_code&`，不抛异常，把错误写进 `ignored`；第一次调用是为了清掉上次可能残留的目录，第二次是清理本例创建的东西。**文件系统操作可能失败**（权限、占用、路径不存在），工程代码要么用 `error_code` 重载检查，要么用会抛异常的版本并 `try` / `catch`；本例用前者，避免把"清理失败"升级成异常。

`path` 的 `/` 运算符是拼接，不是除法；`root / "sub"` 得到 `fs_demo/sub`，`root / "sub" / "b.txt"` 得到 `fs_demo/sub/b.txt`。`path` 还提供 `filename()`、`extension()`、`parent_path()`、`stem()`；**路径是平台相关的字符串，但 `path` 的拼接与分解是标准化的**，比手写字符串拼接更安全。

路径的"字符串编码"是跨平台的坑：Windows 的 `path` 内部用宽字符，`string()` 返回本地编码，`u8string()` 返回 UTF-8；Linux 通常直接用字节串。**把 `path` 转成 `std::string` 再交给别的接口，可能丢失非 ASCII 字符**；需要跨平台传递时，明确用 `u8string()` 或统一用 `path` 对象。`generic_string()` 只统一分隔符，不统一编码；不要把它当成"跨平台字符串"的万能解。

标准原文 · 标准原文：filesystem 管什么

> Subclause 31.12 describes operations on file systems and their components, such as paths, regular files, and directories.

大意：`<filesystem>` 描述对文件系统及其组成部分（路径、普通文件、目录）的操作。

出处：N4950 §31.12.1「In general」（[fs.general]）。

`recursive_directory_iterator` 默认不跟随符号链接；需要跟随时在构造时传 `directory_options::follow_directory_symlink`，但要小心循环链接。**遍历时目录可能被别的进程修改**，标准不保证你在遍历期间看到一致快照；需要一致性时，先收集再处理，或接受"边遍历边变化"的现实。

19.3 文件读写：文本模式与字节数

`std::ofstream` 写文件，`std::ifstream` 读文件，`std::getline` 逐行读。流的析构会关闭文件；但**删除文件前要确保流已经关闭**，否则 Windows 会报"文件被另一个进程使用"。本例用一对花括号限制流的作用域，出了作用域就关闭，再删除。

cpp **file_io: 先写三行** examples/ch08/file_io/src/file_io.cpp 第 6–11 行

```cpp
FileReport write_and_read() {
    const std::string path = "io_demo.txt";
    {
        std::ofstream out(path);
        out << "alpha\nbeta\ngamma\n";
    }
```

文件写完后，内层作用域结束，`ofstream` 析构关闭文件；接着读回内容，读完再让 `ifstream` 关闭，最后删除文件。

cpp **file_io (续)：读回来、关闭后删除** examples/ch08/file_io/src/file_io.cpp 第 13–28 行

```cpp
    FileReport report{0, "", "", std::filesystem::file_size(path)};
    {
        std::ifstream in(path);
        std::string line;
        while (std::getline(in, line)) {
```

```
            if (report.lines == 0) {
                report.first = line;
            }
            report.last = line;
            ++report.lines;
        }
    }

    std::filesystem::remove(path);
    return report;
}
```

终端输出 终端输出 file-io-out

```
lines=3
first=alpha
last=gamma
bytes=20
```

内容看起来是 `alpha`、`beta`、`gamma` 三行，逻辑字符是 5+1+4+1+5+1=17 个；但 `file_size` 报 20。**原因是 Windows 的文本模式**：`std::ofstream` 默认以文本模式打开，写入时把每个 `\n` 转成 `\r\n`，三个换行各多一个字节，17+3=20。`std::getline` 读的时候会按行切分并去掉行尾的 `\r`，所以 `first` / `last` 仍是干净的字符串。**同一个程序在 POSIX 上通常报 17**；要精确控制字节，就用 `std::ios::binary` 打开，或接受文本模式的平台差异。

这也是"文本"与"二进制"的边界：文本模式负责换行转换，二进制模式原样读写。**如果你写的是自定义二进制格式、图片、序列化数据，必须用 `binary`**；否则 Windows 会把数据里的 `\n` 改掉，读回来就对不上。文本文件则可以用文本模式，让平台处理换行习惯。

`std::getline` 返回流引用，放在 `while` 条件里会隐式转 `bool`：读到一行返回真，遇到 EOF 或错误返回假。**循环结束后要区分"正常读完"与"读出错"**，可以检查 `in.eof()` / `in.bad()`；本例只关心行数与内容，省略了这层检查。工程代码处理外部文件时，建议在读完后确认流状态，避免把"读到一半失败"当成"文件就这么短"。

打开文件后还要检查流是否可用：`std::ofstream out(path); if (!out) { /* 处理失败 */ }`。流构造失败默认不抛异常，后续写入会静默失败，对象本身却仍然存在；**"文件没写成"比"程序崩溃"更难发现**。写外部路径前，先确认父目录存在，需要时用 `std::filesystem::create_directories` 建好，再打开文件。

删除文件用 `std::filesystem::remove(path)`；如果文件不存在，它返回 `false`，不抛异常；如果删除失败（权限/占用），会抛 `filesystem_error`。**本例在流关闭后删除，所以不会撞上 Windows 的占用错误**；这也是为什么要把 `ifstream` 放进内层花括号。

19.4 `<format>`：类型安全的格式串

`std::format` 用 `{}` 作为替换字段，按参数顺序填值；`{}` 里可以写格式说明：`{:03d}` 表示十进制、至少 3 位、不足补零，`{:.2f}` 表示两位小数，`{:>6}` 表示右对齐、宽度 6，`{:#x}` 表示带 `0x` 前缀的十六进制。**类型由参数决定，格式串只描述"怎么显示"**，不需要像 `printf` 那样手写 `%d` / `%f` 并保证类型匹配。

cpp **format_basic：补零、对齐、浮点、进制** examples/ch08/format_basic/src/format_basic.cpp 第 5–15 行

```
std::string format_line() {
    return std::format("name={} score={:03d} pi={:.2f}", "Ada", 90, 3.14159);
}

std::string format_right() {
    return std::format("|{:>6}|", "ok");
}

std::string format_hex() {
    return std::format("{:#x}", 255);
}
```

终端输出 终端输出 format-basic-out

```
name=Ada score=090 pi=3.14
|    ok|
hex=0xff
```

`score={:03d}` 把 90 显示成 `090`；`pi={:.2f}` 把 3.14159 四舍五入到两位小数 `3.14`；`|{:>6}|` 把 `ok` 放在宽度 6 的区域右对齐，左边补四个空格；`{:#x}` 把 255 显示成 `0xff`。**格式说明写在冒号后面，类型由实参推导**；如果格式与类型不匹配（例如对字符串用 `d`），`std::format` 会抛 `std::format_error`（或编译期检查失败）。

C++20 的 `std::format` 对**字面量格式串**做编译期检查：格式串是常量表达式时，编译器能发现未知的格式说明、参数个数不匹配等问题；运行期拼接出来的格式串只能用 `std::vformat`，检查推迟到运行期。**这是"类型安全"的一半**：参数类型由模板推导，格式串在编译期尽量检查；剩下的一半是运行期的值检查（例如宽度是否为负）。

`std::format` 返回 `std::string`，可以继续 `+`、可以写文件、可以交给日志库。它不直接输出，所以适合"先拼好再决定去哪"；需要直接输出时用下一节的 `std::print`。

`std::format` 的格式串在编译期检查，这个"编译期"有边界：只有字面量、且类型可被 `format_string` 推导时才检查；运行期拼接的字符串要用 `std::vformat`，检查推迟到运行期，格式错误抛 `std::format_error`。**不要把用户输入直接当格式串**，那等于把格式串注入交给运行期；需要动态格式时，用 `vformat` 并准备好捕获异常。

自定义类型默认不能直接格式化，需要特化 `std::formatter<MyType>`；标准库已经为整数、浮点数、字符串、指针、`chrono` 类型等提供了格式说明。格式串里的字段数量必须与实参匹配，多了少了都会在编译期报错，这也是它比 `printf` 安全的地方：**参数与占位符由同一套类型信息绑定，不靠人肉对齐。**

19.5 `<print>`：直接输出，但本机要额外链接

`std::print` / `std::println` 把格式串直接写到标准输出（或指定文件），语法与 `std::format` 一致：`std::println("sum={}", 3 + 4)` 打印 `sum=7` 并换行。**它比 `std::cout <<` 少写很多 `<<`，也比 `printf` 类型安全**；代价是本机工具链需要额外链接 `-lstdc++exp`，运行时还需要干净的 MSYS2 `PATH`。

cpp **print_lib: println 直接输出** examples/ch08/print_lib/src/print_lib.cpp 第 5–8 行

```cpp
int print_sum(int left, int right) {
    std::println("sum={}", left + right);
    return left + right;
}
```

cpp **print_lib: print 不自动换行** examples/ch08/print_lib/src/main.cpp 第 5–9 行

```cpp
int main() {
    int sum = print_sum(3, 4);
    std::print("done={}\n", sum);
    return 0;
}
```

终端输出 终端输出 print-lib-out

```
sum=7
done=7
```

`std::println` 自动换行，`std::print` 不换行，需要自己写 `\n`。这个例子的清单里写了：

- `libs: ["-lstdc++exp"]`：不加时链接器报 `undefined reference to std::__open_terminal` / `std::__write_to_terminal`；
- `env.PATH` 固定为 `C:/msys64/ucrt64/bin;C:/msys64/usr/bin;C:/Windows/System32`：本机运行 `std::print` 时需要干净的 MSYS2 环境，否则可能以 3221225785 退出。

这两条都是本机工具链事实，不是标准要求。标准只规定 `std::print` 的行为；某个实现把它放在哪个库、运行时依赖什么环境，是平台细节。正文把清单里的 `libs` / `env` 明确写出来，是为了让例子可复现；换到 Linux + libstdc++ 的新版本，可能直接链接通过，不需要这些设置。

`std::print` 的格式化失败会抛 `std::format_error`；输出失败（例如管道断开）会设置流状态或抛异常，具体看重载。**它不会像 `printf` 那样因为格式符与参数不匹配而打印垃圾**；类型不匹配在编译期或运行期被拦住，这是它相对 C 风格输出的核心优势。

`std::print` 的缓冲行为与 `std::cout` 类似：不保证每次调用立刻刷新，程序崩溃时缓冲区里的内容可能丢失。**需要“立刻可见”时，用 `std::flush` 或 `std::endl`（对 `cout`）**；日志场景尤其要注意，别让最后一行日志留在缓冲区里。输出到管道/文件时，缓冲策略还可能与终端不同；调试输出不要假设“打印了就一定写出去了”。

19.6 `std::generator`：要一个，算一个

`std::generator<T>` 是 C++23 的协程生成器：函数里用 `co_yield` 产出一个值，调用方用范围 for 逐个取。**它不是一次性算完再返回容器**；每次迭代请求下一个值，生成器从上次暂停的地方继续，产出一个就停。这样内存里只需要当前值，不需要把整个序列存下来。

生成器是惰性的：调用函数只创建协程帧，函数体要等到第一次迭代才开始执行，运行到第一个 `co_yield` 就暂停。生成器对象销毁时，协程帧也跟着销毁，局部对象按析构规则清理；如果生成器内部持有引用，要保证被引用的对象活得比遍历更久。**惰性省内存，也带来生命周期约束**，不要在返回生成器的函数里引用局部变量。

cpp **generator_basic: co_yield 逐个产出** examples/ch08/generator_basic/src/generator_basic.cpp 第 3–7 行

```cpp
std::generator<int> count_up(int limit) {
    for (int i = 0; i < limit; ++i) {
        co_yield i;
    }
}
```

cpp **generator_basic: 范围 for 逐个拉取** examples/ch08/generator_basic/src/main.cpp 第 5–13 行

```cpp
int main() {
    int sum = 0;
    for (int value : count_up(4)) {
        std::cout << value << " ";
        sum += value;
    }
    std::cout << "\nsum=" << sum << "\n";
    return 0;
}
```

终端输出 终端输出 generator-basic-out

```
0 1 2 3
sum=6
```

`count_up(4)` 返回一个生成器对象；范围 for 第一次迭代让生成器运行到第一个 `co_yield 0`，拿到 0 后暂停；第二次迭代从暂停点继续，到 `co_yield 1`，以此类推；循环结束（`i == 4`）时生成器函数返回，迭代结束。**控制流在“生成器”和“调用方”之间来回切换**，而不是先跑完整个循环。

调用树 **generator：调用方与协程的往返**

第 1 步 范围 for 请求第一个值

```
main() ：开始遍历 count_up(4)
count_up() ：运行到 co_yield 0，暂停
main() ：拿到 0，打印
```

第 2 步 请求下一个值

```
main() ：下一次迭代
count_up() ：从暂停点恢复，运行到 co_yield 1
main() ：拿到 1，打印
```

第 3 步 生成器返回

```
count_up() ：i 到 4，循环结束，函数返回
main() ：迭代结束，打印 sum
```

第 1、2 帧的往返是"拉取式"的：调用方要一个，生成器才算一个。**这与返回 `vector` 的函数相反**：`vector` 版本会先把所有元素算完、存进内存，再交给调用方；生成器版本把"计算"和"消费"交错进行，适合大序列、无限序列、或计算代价高的元素。

生成器是**单遍**的：迭代器只能向前，不能回头，也不能复制生成器对象后各自独立遍历（生成器对象本身可移动，但底层协程状态是单一的）。**需要多次遍历就重新调用生成函数**；需要随机访问就用容器。生成器的价值在"流式"，不在"可重复"。

19.7 生成器管线：把两个生成器接起来

生成器可以作为参数传给另一个生成器：`squares(numbers(5))` 里，`numbers(5)` 产出 0..4，`squares` 逐个读取并平方，再 `co_yield` 出去。**外层每要一个平方数，内层才算一个原数**；两层都是惰性的，没有中间容器。

cpp **generator_pipeline: 两个生成器串联** examples/ch08/generator_pipeline/src/generator_pipeline.cpp 第 3–13 行

```cpp
std::generator<int> numbers(int limit) {
    for (int i = 0; i < limit; ++i) {
        co_yield i;
    }
}

std::generator<int> squares(std::generator<int> source) {
    for (int value : source) {
        co_yield value * value;
    }
}
```

cpp **generator_pipeline: 消费最终序列** examples/ch08/generator_pipeline/src/main.cpp 第 5–13 行

```cpp
int main() {
    int count = 0;
    for (int value : squares(numbers(5))) {
        std::cout << value << " ";
        ++count;
    }
    std::cout << "\ncount=" << count << "\n";
    return 0;
}
```

终端输出 终端输出 generator-pipeline-out

```
0 1 4 9 16
count=5
```

`squares` 接收 `std::generator<int>` 是按值传递，取得内层生成器的所有权；范围 for 从 `source` 拉一个值，平方后 `co_yield`，再等下一次请求。**如果内层是无限序列，外层也可以无限产出**，调用方用 `break` 或 `take` 限制数量；这正是生成器适合"惰性管线"的原因。

图 / 搜索过程 **文件 → 解析 → 生成器 → 输出**

第 1 步 读取输入

file：磁盘上的文本　ifstream：逐行读入

第 2 步 解析成记录

parse：一行变成一个记录　generator：co_yield 记录，暂停

第 3 步 消费与输出

main：范围 for 拉取　format/print：格式化并写出

第 2 帧的生成器把"解析"和"消费"解耦：解析器不需要知道消费者要多少条，消费者也不需要等全部解析完。**惰性让内存占用与输入规模脱钩**；代价是调用链更长、调试时栈帧会来回切换，且生成器只能单遍消费。

生成器与 `views` 的区别：`std::views` 是范围适配器，组合的是"视图"，通常也是惰性的；`std::generator` 是协程，能把"有状态的、需要暂停/恢复的计算"写成普通循环。**简单的过滤/变换优先用 `views`；需要协程式暂停、或把复杂控制流写成顺序代码时用 `generator`。**两者都能避免中间容器，选择看表达力与习惯。

19.8 本机实测：六个读数

实验记录 · 本机实测：目录、文件、格式与生成器

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `filesystem_walk` 的 stdout 是 `count=3` / `entry=a.txt` / `entry=sub` / `entry=sub/b.txt`：递归遍历看到三项，排序后输出稳定；
- `file_io` 的 stdout 是 `lines=3` / `first=alpha` / `last=gamma` / `bytes=20`：三行逻辑字符 17 个，Windows 文本模式把三个 `\n` 写成 `\r\n`，所以文件是 20 字节；
- `format_basic` 的 stdout 是 `name=Ada score=090 pi=3.14` / `| ok|` / `hex=0xff`：补零、对齐、浮点、进制都按格式说明输出；
- `print_lib` 的 stdout 是 `sum=7` / `done=7`：`println` 换行，`print` 不换行；清单必须带 `-lstdc++exp` 与干净 `PATH`；
- `generator_basic` 的 stdout 是 `0 1 2 3` / `sum=6`：`co_yield` 逐个产出；
- `generator_pipeline` 的 stdout 是 `0 1 4 9 16` / `count=5`：两个生成器串联，外层按需拉取。

19.9 下一单元从哪继续

下一单元是最后一章：多文件工程、ODR 与收尾。这一单元已经出现多个翻译单元、头文件声明、`libs` / `env` 清单；下一单元会把这些组织成一个小型项目，讲清楚"什么能放头文件、什么必须放 `.cpp`、重复定义与 ODR 怎么发生"，并用一个完整的收尾程序把前面的容器、智能指针、异常、格式化串起来。

小结 · 这一节要带走的四件事

- `<filesystem>` 的 `path` 用 `/` 拼路径，`recursive_directory_iterator` 递归遍历；迭代顺序不保证，输出前要排序。
- 文本模式在 Windows 会把 `\n` 写成 `\r\n`，`file_size` 因此变大；删除文件前先让流离开作用域关闭。
- `std::format` 用 `{}` 与格式说明拼字符串；`std::print` 直接输出，但本机需要 `-lstdc++exp` 与干净 `PATH`。
- `std::generator` 用 `co_yield` 惰性产出，可与另一个生成器串成管线；单遍消费，需要重复遍历就重新生成。

自测题 · 自测题（先自己做，再对答案）

1. `recursive_directory_iterator` 的遍历顺序有标准保证吗？为什么例子要先 `sort`？
2. `file_io` 写的内容逻辑上是 17 个字符，为什么 `file_size` 报 20？换到 Linux 可能报多少？
3. `std::format` 与 `std::print` 的分工是什么？本机使用 `std::print` 需要哪些额外设置？
4. `std::generator` 与返回 `std::vector` 的函数在求值时机上有什么不同？
5. 生成器管线 `squares(numbers(5))` 中，外层每要一个值，内层会做什么？生成器为什么不能重复遍历？

注 · 自测答案

1. 没有标准保证；不同文件系统、不同实现的迭代顺序可能不同。先收集再 `sort`，输出才可复现。
2. 文本模式把三个 `\n` 各转成 `\r\n`，多出 3 个字节，17+3=20；Linux 上通常不做转换，报 17。
3. `std::format` 返回字符串，适合先拼好再决定输出到哪；`std::print` 直接写标准输出。本机需要 `libs: ["-lstdc++exp"]` 和干净的 MSYS2 `PATH`，否则链接或运行会失败。
4. `vector` 版本先把所有元素算完并存进内存，再返回；生成器版本在调用方请求时才计算下一个值，计算与消费交错，内存只保留当前值。
5. 外层请求一个平方数时，内层从上次暂停处恢复，算出一个原数交给外层；生成器的协程状态是单份的，迭代器只能向前，所以不能像容器那样重复遍历。

第 9 章

工程与收尾

本章 1 个单元 · 页眉从这里之后的页面开始出现

1. 多文件工程、ODR 与收尾 零基础 · 第 20 单元 · 真编译真运行

多文件工程、ODR 与收尾

一个 `.cpp` 能跑，不代表工程能维护。真实项目由多个翻译单元组成：头文件放接口，源文件放实现，构建脚本把它们编译、链接成一个程序。多文件带来的第一个硬规则是 ODR（单一定义规则）：同一个函数或变量，在整个程序里只能有一个定义；头文件被多个 `.cpp` 包含时，写错位置就会重复定义。

这一单元是 C++ 分册的收尾。先用 `multi_file/` 看"声明在头、定义在源"的标准分工，再用 `odr_violation/` 看头文件里定义非 `inline` 函数的后果；然后用 `inline_var/`、`anonymous_ns/`、`template_header/` 处理三个常见边界；最后用 `final_app/` 把接口、实现、测试与 `build.sh` 串成一个可维护的小程序。

◎ 学习目标 · 看完这一单元，你应该能

- 说出翻译单元、声明、定义、链接之间的关系；
- 解释 ODR 为什么禁止在头文件里定义非 `inline` 函数，以及链接期会看到什么；
- 用 include guard（或 `#pragma once`）保护头文件，知道什么该放头、什么该放源；
- 用 `inline` 变量在头文件里定义跨翻译单元共享的变量；
- 用匿名命名空间给 `.cpp` 里的辅助函数内部链接，避免符号冲突；
- 把模板定义放在头文件，理解实例化点为什么需要看到定义；
- 用一条 `g++` 或 `build.sh` 构建程序与测试，并按审查清单检查一个多文件工程。

20.1 先看工程长什么样

这一单元在 `examples/ch09/` 下建六个例子，最后一个 `final_app/` 是一个完整的小工程：两个头文件、三个实现文件、一个测试文件、一个构建脚本。前五个例子分别聚焦一个规则，避免把"多文件"一次性塞进一个例子。

目录树 **工程：examples/ch09/ 这一单元的六个例子**

第 1 步 先看标准的多文件分工与 ODR 反例

```
examples/
└── ch09/
    ├── multi_file/ ← 本课新增
    │   ├── include/ ← 本课新增
    │   │   ├── math_ops.h ← 本课新增
    │   │   └── report.h ← 本课新增
    │   ├── src/ ← 本课新增
    │   │   ├── main.cpp ← 本课新增
    │   │   ├── math_ops.cpp ← 本课新增
    │   │   └── report.cpp ← 本课新增
    │   └── multi_file.json ← 本课新增
    ├── odr_violation/ ← 本课新增
    └── inline_var/ ← 本课新增
```

第 2 步 加上匿名命名空间与模板头文件

```
examples/
└── ch09/
    ├── 前三个例子 ： 同上
    ├── anonymous_ns/ ← 本课新增
    └── template_header/ ← 本课新增
```

第 3 步 最后加上综合程序、测试与构建脚本

```
examples/
└── ch09/
    ├── 前五个例子 ： 同上
    └── final_app/ ← 本课新增
        ├── include/task.h ← 本课新增
        ├── include/report.h ← 本课新增
        ├── src/task.cpp ← 本课新增
        ├── src/report.cpp ← 本课新增
        ├── src/main.cpp ← 本课新增
        ├── test/tests.cpp ← 本课新增
        └── build.sh ← 本课新增
```

第 1 帧的 `multi_file/` 是"声明与定义分家"的正面样例，`odr_violation/` 是头文件里定义非 `inline` 函数的反例，`inline_var/` 展示跨翻译单元共享的 `inline` 变量。第 2 帧的 `anonymous_ns/` 用匿名命名空间隔离两个 `.cpp` 里的同名 helper，`template_header/` 把模板定义放在头文件里。第 3 帧的 `final_app/` 把任务清单的接口、实现、格式化、测试与构建脚本组织成一个可运行的小工程。

20.2 翻译单元：编译器一次看到的东西

预处理之后，一个 `.cpp` 加上它包含的所有头文件，形成一个**翻译单元**（translation unit）。编译器一次处理一个翻译单元，生成一个目标文件；链接器再把多个目标文件与库合在一起。**头文件本身不参与编译**，它只是被 `#include` 复制进翻译单元；所以头文件里写定义，等于在每个包含它的翻译单元里都写了一遍。

cpp multi_file: 头文件只放声明 examples/ch09/multi_file/include/math_ops.h 第 1–7 行

```cpp
#ifndef CPP20_MATH_OPS_H
#define CPP20_MATH_OPS_H

int add(int left, int right);
int multiply(int left, int right);

#endif
```

cpp multi_file: main 只调用接口 examples/ch09/multi_file/src/main.cpp 第 6–12 行

```cpp
int main() {
    int sum = add(3, 4);
    int product = multiply(3, 4);
    std::cout << "sum=" << sum << " product=" << product
              << " text=" << describe(sum + product) << "\n";
    return 0;
}
```

终端输出 终端输出 multi-file-out

```
sum=7 product=12 text=result:19
```

`math_ops.h` 里只有函数声明，没有函数体；`math_ops.cpp` 里才有 `add` / `multiply` 的定义。`main.cpp` 包含头文件后，编译器知道"有这两个函数、签名是什么"，但不知道函数体在哪；链接器把 `main.o` 与 `math_ops.o` 接起来，才找到定义。**声明让编译通过，定义让链接通过**；缺声明是编译错误，缺定义是链接错误。

`report.h` 与 `report.cpp` 重复同样的分工：`std::string describe(int)` 的声明在头文件，定义在 `report.cpp`。**一个头文件对应一组相关的接口**，不要把所有声明塞进一个巨大的 `all.h`；头文件越具体，依赖越清楚，改一个模块时重编译的范围越小。

谁能看见谁 翻译单元与链接：声明与定义各在哪

第 1 步 main.cpp 的翻译单元

main.cpp + math_ops.h + report.h ： 看到 add / multiply / describe 的声明	main.cpp ： 生成 main.o，引用这三个符号



第 1 帧的 `main.o` 只有“引用”，没有定义；第 2 帧的实现文件提供定义；第 3 帧的链接器把引用与定义配对。**这就是多文件工程的最小模型**：编译按翻译单元并行，链接按符号全局配对。

20.3 ODR：同一个定义只能出现一次

ODR 有两条最常用的规则：**一个翻译单元里，任何可定义实体只能有一个定义；整个程序里，非 `inline`、非模板的函数或变量只能有一个定义。**头文件里的普通函数定义被两个 `.cpp` 包含，就同时踩中两条：每个翻译单元里有一份，整个程序里有多份。

cpp **odr_violation: 头文件里的非 inline 定义** examples/ch09/odr_violation/include/odr_violation.h 第 1–3 行

```cpp
int twice(int value) {
    return value * 2;
}
```

cpp **odr_violation: 同一个头文件被包含两次** examples/ch09/odr_violation/src/main.cpp 第 1–8 行

```cpp
#include "odr_violation.h"
#include "odr_violation.h"

int other_twice(int value);

int main() {
    return other_twice(2);
}
```

编译诊断 编译诊断 odr-violation-diag

```
In file included from src/main.cpp:2:
include/odr_violation.h:1:5: error: redefinition of 'int twice(int)'
    1 | int twice(int value) {
      |     ^~~~~
In file included from src/main.cpp:1:
include/odr_violation.h:1:5: note: 'int twice(int)' previously defined here
    1 | int twice(int value) {
      |     ^~~~~
```

这个头文件既没有 include guard，函数也不是 `inline`；`main.cpp` 包含它两次，同一个翻译单元里出现两份 `twice` 的定义，编译器直接报 `redefinition of 'int twice(int)'`。**这是 ODR 在同一翻译单元里的表现**：不需要链接，编译期就能抓住。

如果没有“包含两次”，而是让两个 `.cpp` 各包含一次，编译都能过；链接器会发现两个目标文件都导出 `twice`，报 `multiple definition of 'twice(int)'`。**链接器能抓到的只是“同名符号重复”**；如果两个定义分别放在不同翻译单元、又因为 `inline` 或模板被允许重复，链接器不一定能判断它们是否“相同”，这就是 ODR 违反常常“无诊断”的原因。

标准原文 · 标准原文：ODR 的两条硬规则

> No translation unit shall contain more than one definition of any definable item.
>
> For any definable item D with definitions in multiple translation units, if D is a non-inline non-templated function or variable ... the program is ill-formed; a diagnostic is required only if the definable item is attached to a named module and a prior definition is reachable at the point where a later definition occurs.

大意：一个翻译单元里，任何可定义实体只能有一个定义；如果一个非 `inline`、非模板的函数或变量在多个翻译单元里都有定义，程序是病态的（ill-formed）。标准只在特定模块场景要求诊断，普通多文件工程的重复定义常常由链接器发现。

出处：N4950 §6.3「One-definition rule」（[basic.def.odr]）第 2 段与第 14 段。

"ill-formed, no diagnostic required"不是"可以随便写"，而是"程序行为没有保证，编译器/链接器可以不报错"。**ODR 违反是未定义行为的一种**：可能链接失败，可能链接成功但调用到其中一个定义，也可能在不同优化级别下表现不同。工程上不能依赖"本机没报错"来判断合规。

修复方式很直接：头文件只放声明；函数定义移到 `.cpp`；如果确实要在头文件里定义（模板、`inline` 函数、`constexpr` 函数），就加 `inline` 或确保它是模板。**"能在头文件里定义"的前提是允许重复定义，并且每份定义完全一致**；普通函数不满足这个前提。

20.4 头文件卫生：guard、放什么、不放什么

头文件的第一道防线是 include guard：`#ifndef` / `#define` / `#endif` 包住整个文件，保证同一个翻译单元里只展开一次。`#pragma once` 更短，主流编译器都支持，但它不是标准；**跨编译器、跨工具链的工程优先用标准 guard**，或者两者都用（guard 为主，`#pragma once` 作为优化）。

头文件里应该放：

- 函数、类的**声明**；
- 类定义、枚举、类型别名、`constexpr` / `inline` 函数与变量；
- 模板定义（实例化点需要看到）；
- 必要的 `#include`，且尽量用前向声明减少依赖。

头文件里不应该放：

- 普通函数的定义（除非 `inline` / `constexpr` / 模板）；
- 普通变量的定义（除非 `inline` / `constexpr`）；
- `using namespace`（会污染所有包含者的命名空间）；
- 实现细节、平台相关的宏、内部 helper（放 `.cpp` 的匿名命名空间）。

"头文件是接口，源文件是实现"：接口要稳定、最小、可读；实现可以改，只要接口不变，调用方就不需要重编译（理想情况下）。这也是为什么 `math_ops.h` 只写两行声明，而把函数体放进 `math_ops.cpp`。

20.5 `inline` 变量：头文件里的共享变量

C++17 起，变量也能声明为 `inline`。`inline` 变量允许在多个翻译单元里出现相同定义，链接器把它们合并成一个实体；**取地址在哪个翻译单元都得到同一个地址**。这让"头文件里定义一个全局配置/计数器"变得合法，不必再写 `extern` 声明 + 一个 `.cpp` 定义。

cpp **inline_var: 头文件里的 inline 变量** examples/ch09/inline_var/include/inline_var.h 第 4–7 行

```cpp
inline int shared_counter = 0;

const int* address_in_module();
int bump_in_module();
```

cpp **inline_var：两个翻译单元看到同一个变量** examples/ch09/inline_var/src/main.cpp 第 5–13 行

```cpp
int main() {
    ++shared_counter;
    int after = bump_in_module();
    const int* here = &shared_counter;
    const int* there = address_in_module();
    std::cout << "same=" << (here == there ? 1 : 0)
              << " counter=" << after << "\n";
    return 0;
}
```

终端输出 终端输出 inline-var-out

```
same=1 counter=2
```

`main.cpp` 里 `++shared_counter` 把它从 0 加到 1；`bump_in_module()` 在另一个翻译单元里再加一次，返回 2；`&shared_counter` 在 `main.cpp` 与 `inline_var.cpp` 里取到的地址相同，所以 `same=1`。**这不是“两个同名变量碰巧值一样”，而是链接器把它们合并成了同一个实体**；`inline` 变量的地址一致性是标准要求。

`inline` 变量与 `static` 变量的区别：`static` 全局变量在每个翻译单元里各有一份（内部链接），地址不同、值也各自独立；`inline` 变量全程序一份。**需要“全局唯一”用 `inline`，需要“每个文件私有”用匿名命名空间或 `static`**；不要用 `inline` 表达“每个文件一份”，那是反的。

20.6 匿名命名空间：只在本翻译单元可见

`.cpp` 里的辅助函数如果不需要被别的翻译单元看到，放进匿名命名空间：`namespace { ... }`。匿名命名空间里的名字具有**内部链接**，每个翻译单元各有一份，不会与别的 `.cpp` 里的同名函数冲突。它比 `static` 函数更现代、对类型也适用（`static` 只能用于函数和变量，不能用于类定义）。

cpp **anonymous_ns：第一个 TU 的私有 helper** examples/ch09/anonymous_ns/src/anonymous_ns.cpp 第 3–11 行

```cpp
namespace {
int helper() {
    return 1;
}
}

int first_value() {
    return helper();
}
```

第二个翻译单元 `second.cpp` 里也有一个 `helper()`，返回 2；两个 `helper` 在各自的匿名命名空间里，链接器看到的是两个内部链接符号，不会冲突。`main` 调用 `first_value()` 与 `second_value()`，分别得到 1 和 2：

终端输出 终端输出 anonymous-ns-out

```
first=1 second=2
```

匿名命名空间是“文件私有的工具箱”：实现细节放进去，头文件只暴露必要的接口。这样改 helper 不会影响别的翻译单元，也不会制造符号冲突；代价是每个翻译单元各有一份代码，不能跨文件复用（需要复用时才放进头文件/公共模块）。

匿名命名空间与 `static` 的另一个差别是链接性规则的细节：匿名命名空间的名字具有内部链接，且它的成员可以被模板参数等使用；`static` 全局函数在 C++ 里仍可用，但现代代码更倾向匿名命名空间。**两者都不要放在头文件里**：头文件里的匿名命名空间会给每个包含者各生成一份，容易造成“看起来共享、实际各一份”的误解。

20.7 模板与头文件：实例化点要看到定义

模板不是普通函数：编译器在**实例化点**才生成具体代码，而实例化点需要看到模板定义。如果定义藏在 `.cpp` 里，调用方只看到声明，编译器无法生成 `twice<int>`，链接器就报 `undefined reference`。**所以模板定义通常放头文件**，除非你显式实例化并在头文件里 `extern template` 控制。

cpp **template_header：模板定义与包装声明** examples/ch09/template_header/include/template_header.h 第 6–12 行

```cpp
template <class T>
T twice(T value) {
    return value + value;
}

int twice_int(int value);
std::string twice_text(const std::string& value);
```

cpp **template_header：两个包装函数各自实例化** examples/ch09/template_header/src/main.cpp 第 5–9 行

```cpp
int main() {
    std::cout << "int=" << twice_int(4)
              << " text=" << twice_text("hi") << "\n";
    return 0;
}
```

终端输出 终端输出 template-header-out

```
int=8 text=hihi
```

`twice_int` 与 `twice_text` 是两个普通函数，定义在 `template_header.cpp` 里；它们各自调用 `twice<int>` 与 `twice<std::string>`，实例化发生在那个 `.cpp` 里，而模板定义在头文件里可见。**“模板定义放头文件”不是说所有实例都必须在使用点生成**；你也可以在一个 `.cpp` 里显式实例化，让链接器找到，但那时要确保实例化覆盖所有调用类型。

`extern template` 可以抑制某些翻译单元的重复实例化：在头文件里写 `extern template class Box<int>;`，在一个 `.cpp` 里写 `template class Box<int>;`，其他翻译单元就不再生成。**这是编译期/链接期的优化手段，不改变 ODR 要求**；模板的每份实例化仍然必须一致。

20.8 构建：从一条命令到 `build.sh`

本机没有 make/cmake，构建就是一个 `g++` 命令：`-std=c++23`、警告、`-Iinclude`、所有 `.cpp`、`-o` 输出。**漏一个 `.cpp` 就是 `undefined reference`；多一个重复定义就是 `multiple definition`**；链接命令里的源文件清单，就是工程的“参与编译的翻译单元”清单。

`final_app/build.sh` 把两条命令固定下来：一条构建程序，一条构建测试。两条都带 `-Iinclude`，都列全 `src/task.cpp` 与 `src/report.cpp`；测试多一个 `test/tests.cpp`，程序多一个 `src/main.cpp`。

bash **final_app：一条脚本构建程序与测试** examples/ch09/final_app/build.sh 第 1–7 行

```bash
#!/usr/bin/env bash
set -euo pipefail

g++ -std=c++23 -Wall -Wextra -Wpedantic -O0 -g -Iinclude \
    src/main.cpp src/task.cpp src/report.cpp -o final_app.exe
g++ -std=c++23 -Wall -Wextra -Wpedantic -O0 -g -Iinclude \
    test/tests.cpp src/task.cpp src/report.cpp -o final_app_test.exe
```

`set -euo pipefail` 让脚本在任一命令失败时立刻退出，不会出现“编译失败但旧的可执行文件还在跑”的假成功。**先编译再运行**是构建脚本的基本顺序；更完整的脚本还会清理旧产物、区分 debug/release、把目标文件放到 `build/`，本单元只保留最小可用版本。

图 / 搜索过程 **接口 → 实现 → 测试 → 构建**

第 1 步 接口

task.h：Task 与 TaskList 的声明　　report.h：render 的声明

第 2 步 实现

task.cpp：add / complete / size / done_count　　report.cpp：format 组装输出

第 3 步 测试与构建

tests.cpp ：调用接口，断言行为，返回 0/1　　build.sh ：两条 g++，分别产出程序与测试

第 1 帧的接口只暴露"能做什么"，第 2 帧的实现负责"怎么做"，第 3 帧的测试只依赖接口、不依赖实现细节。**测试通过接口验证行为**，所以实现可以替换，只要接口与行为不变；`build.sh` 把"编译哪些文件"固定下来，避免手工命令漏文件。

20.9 `final_app`：一个能维护的小程序

`final_app` 是一个任务清单：`TaskList` 保存任务，`add` 追加，`complete(id)` 标记完成，`render` 输出清单与统计。接口在 `task.h` / `report.h`，实现在 `task.cpp` / `report.cpp`，`main.cpp` 只做驱动，`tests.cpp` 只做行为断言。

cpp **final_app: 按 id 标记完成** examples/ch09/final_app/src/task.cpp　　第 9–17 行

```cpp
bool TaskList::complete(int id) {
    for (auto& task : tasks_) {
        if (task.id == id) {
            task.done = true;
            return true;
        }
    }
    return false;
}
```

cpp **final_app: 用 format 拼清单** examples/ch09/final_app/src/report.cpp　　第 5–13 行

```cpp
std::string render(const TaskList& list) {
    std::string output;
    for (const auto& task : list.tasks()) {
        output += std::format("[{}] {} {}\n",
                              task.done ? 'x' : ' ', task.id, task.title);
    }
    output += std::format("summary={}/{}\n", list.done_count(), list.size());
    return output;
}
```

cpp **final_app: 三个任务，完成一个** examples/ch09/final_app/src/main.cpp　　第 6–18 行

```cpp
int main() {
    TaskList list;
    list.add("write docs");
    list.add("fix bug");
    list.add("ship");

    list.complete(2);

    std::cout << "tasks=" << list.size()
              << " done=" << list.done_count() << "\n";
    std::cout << render(list);
    return 0;
}
```

终端输出 终端输出　　final-app-out

```
tasks=3 done=1
[ ] 1 write docs
[x] 2 fix bug
[ ] 3 ship
summary=1/3
```

`add` 自动分配递增 id，`complete(2)` 把第二个任务标记完成，`render` 用 `std::format` 逐行拼出 `[x]` / `[ ]`，最后一行是完成数/总数。**接口与实现分开后，`main` 只关心"加任务、完成任务、打印"，不关心 `TaskList` 内部用 `vector` 还是别的容器**；以后换成 `std::list` 或数据库，只要接口不变，`main` 不用改。

测试只调用接口，检查三件事：初始大小为 3、`complete(2)` 成功且完成数为 1、对不存在的 id 返回 `false`。任何一条不成立，`ok` 变假，程序以非零退出：

cpp **final_app 的测试：只依赖接口** examples/ch09/final_app/test/tests.cpp 第 5–20 行

```cpp
int main() {
    TaskList list;
    list.add("a");
    list.add("b");
    list.add("c");

    bool ok = list.size() == 3;
    ok = ok && list.complete(2);
    ok = ok && list.done_count() == 1;
    ok = ok && !list.complete(99);

    std::cout << (ok ? "PASS" : "FAIL")
              << " size=" << list.size()
              << " done=" << list.done_count() << "\n";
    return ok ? 0 : 1;
}
```

终端输出 终端输出 final-app-test-out

```
PASS size=3 done=1
```

`ok = ok && ...` 的写法保证前面的检查失败后不再执行后面的调用；如果想看每一步的结果，可以拆成多个 `if` 或输出中间值。**测试的返回值很重要**：`return ok ? 0 : 1` 让 CI 或 `build.sh` 能通过退出码判断失败，而不是靠人眼看输出。`build.sh` 里用 `set -e`，测试失败会让脚本以非零退出，构建就"红"了。

20.10 收尾：审查清单

把这一册的内容压成一份可执行的清单，提交前逐条过一遍：

- **接口**：头文件只有声明、类型、模板与 `inline` 定义；没有普通函数/变量的定义；有 include guard。
- **实现**：`.cpp` 里的 helper 放匿名命名空间；不把实现细节泄漏到头文件。
- **链接**：构建脚本列全所有 `.cpp`；新增文件后同步更新脚本；没有重复定义的全局符号。
- **资源**：裸 `new` / `delete` 是否被 RAII 取代；容器与智能指针是否表达了所有权；析构是否 `noexcept` 安全。
- **错误**：异常路径是否清理资源；是否区分"没有值"与"错误"；`noexcept` 是否诚实。
- **模板**：定义是否对实例化点可见；是否用 `static_assert` / concepts 给出清楚约束；是否避免无界递归。
- **测试**：是否只依赖接口；失败是否返回非零；是否覆盖边界（空容器、重复操作、不存在的 id）。
- **构建**：警告是否清零；`-Wall -Wextra -Wpedantic` 下是否干净；Debug/Release 是否有明确区分。

这份清单不是形式：每一条都对应前面单元里的一个真实坑。多文件工程的质量，往往不取决于你写了多少高级特性，而取决于接口是否清楚、资源是否有着落、失败是否被处理、构建是否可重复。

清单适合在提交前跑一遍，也适合在重构前跑一遍：先确认接口与所有权，再动实现；先让测试变红，再让它变绿。**代码审查时，清单上的每一条都能对应到一个具体问题**，比"看起来没问题"更可靠。把这份清单放进仓库的 `CONTRIBUTING.md`，下次换人维护时，他不必重新摸索这套工程约定；约定写下来，才算真正交付。如果只能记住三件事：头文件只放声明，所有权写进类型，失败必须被检查。先照着做一遍，再把它改成适合你们团队的版本；这份清单来自本册二十个单元的真实返工记录，不是从别处抄来的通用模板。

20.11 本机实测：七个读数

实验记录 · 本机实测：多文件、ODR 与综合程序

工具链：g++ 16.1.0，标准 C++23，固定 `-Wall -Wextra -Wpedantic -O0 -g`。

- `multi_file` 的 stdout 是 `sum=7 product=12 text=result:19`：三个 `.cpp` 链接后接口可用；
- `odr_violation` 编译失败，诊断是 `redefinition of 'int twice(int)'`：头文件里的非 `inline` 定义在同一翻译单元里重复；
- `inline_var` 的 stdout 是 `same=1 counter=2`：两个翻译单元取到同一个 `inline` 变量地址；
- `anonymous_ns` 的 stdout 是 `first=1 second=2`：两个 `.cpp` 的同名 helper 互不冲突；
- `template_header` 的 stdout 是 `int=8 text=hihi`：模板定义在头文件，两个包装函数分别实例化；
- `final_app` 的 stdout 是 `tasks=3 done=1` / 三行清单 / `summary=1/3`：综合程序按接口工作；
- `final_app_test` 的 stdout 是 `PASS size=3 done=1`，退出码 0：测试通过。

20.12 下一站：算法与项目

C++ 分册到这里结束。你已经走过工具链、对象生命周期、继承与多态、模板、容器与算法、现代语法、错误处理、编译期、系统 I/O 与多文件工程；接下来是算法分册与项目实战分册。算法分册会用这些容器与迭代器做数据结构与算法分析；项目实战分册会把 `final_app` 这种"接口 + 实现 + 测试 + 构建"的结构扩展成真正的多模块项目。

下一步不是再学一个新语法，而是把已经会的东西组合起来：先读现有代码，再改一个小功能，再写一个测试，再让它通过。能维护，才算真的会。

小结 · 这一节要带走的五件事

- 翻译单元 = 一个 `.cpp` 加它包含的头文件；编译按翻译单元，链接按符号全局配对。
- ODR 要求同一翻译单元内一个定义、全程序非 `inline` 非模板实体一个定义；头文件只放声明、模板与 `inline` 定义。
- `inline` 变量跨翻译单元是同一实体，地址一致；匿名命名空间给 `.cpp` 内部链接，避免同名冲突。
- 模板定义必须对实例化点可见，通常放头文件；`extern template` 可控制实例化位置。
- 多文件工程用构建脚本列全源文件，测试只依赖接口并通过退出码报告结果。

自测题 · 自测题（先自己做，再对答案）

1. 声明与定义的区别是什么？为什么"编译通过"不代表"链接通过"？
2. 为什么头文件里不能定义普通函数？`inline` 函数和模板为什么可以？
3. `inline` 变量与 `static` 全局变量在多个翻译单元里的行为有什么不同？
4. 匿名命名空间解决了什么问题？为什么不能把它放进头文件？
5. `final_app` 的测试为什么只 `#include "task.h"`，而不包含 `task.cpp`？`build.sh` 为什么要把测试源文件也列进编译命令？

注 · 自测答案

1. 声明告诉编译器"名字与类型"，定义提供实体（函数体/存储）。编译只需要声明；链接需要定义，缺定义会 `undefined reference`。
2. 头文件会被多个翻译单元包含，普通函数定义会在每个翻译单元各出现一次，违反 ODR；`inline` 函数与模板允许重复定义，但要求每份定义一致，链接器会合并。
3. `inline` 变量全程序一个实体，各翻译单元地址相同；`static` 全局变量每个翻译单元各一份，地址与值都独立。
4. 它给 `.cpp` 内的 helper 内部链接，避免与其他翻译单元的同名符号冲突；放进头文件会让每个包含者各生成一份，容易造成"看似共享、实际各一份"的误解。
5. 测试只依赖接口，包含 `task.h` 就能声明 `TaskList`；`task.cpp` 是定义，必须由链接器与测试目标文件一起链接，所以 `build.sh` 要把它列进源文件清单。